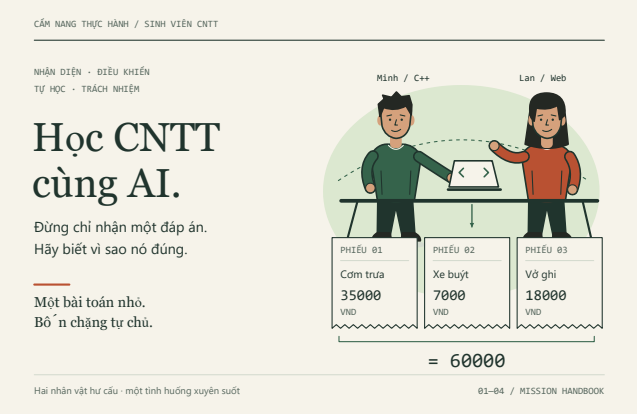
AI có thể viê´t code. *Bạn cầ`n hiểu vì sao.*



Cùng Minh và Lan xây một chiê´c ví nhỏ — và giữ phầ`n suy nghĩ vê` phía mình.

Bản tái thiê´t kê´ 30/10/2026 · Dữ liệu & nhân vật hư cấ´u · Không phải quy định chính thức của trường · Đọc và thực hành ngoại tuyê´n

---

BẢN ĐỒ / ĐỌC TRƯỚC KHI VÀO CHI TIẾT

Bản đồ` một trang

Một câu. Cẩm nang tự học cho sinh viên năm 1–2 Công nghệ thông tin, Đại học Thủy lợi: dùng AI để làm bài tập lập trình mà **không giao phầ`n suy nghĩ cho công cụ**.

Không phải là quy định chính thức của trường, giáo trình học phầ`n, quảng cáo công cụ hay bảng giá. Minh và Lan, mọi hội thoại, phiê´u khảo sát và mọi con sô´ chi phí là **hư cấ´u hoặc giả định** để học. Rubric trong sách là rubric luyện tập.

Bố´n chặng và dự án tổng kê´t

Chặng	Sau chặng này, bạn làm được gì	Gợi ý xê´p lịch	Sản phẩm tự làm
01 · Bước vào thê´ giới	Phân biệt model / provider / application / producer; tính được	6 mục ≈ 3 ca 45 phút	Một bản nhận diện có phầ`n "chưa biê´t", một

Chặng	Sau chặng này, bạn làm được gì	Gợi ý xế´p lịch	Sản phẩm tự làm
AI	chi phí một việc; chọn công cụ theo việc, không theo danh tiế´ng		prompt đủ ngữ cảnh, một phép tính chi phí
02 · Trở thành người điề`u khiển AI	Điề`u khiển quá trình qua **bảy Mission**: đọc môi trường, hỏi đúng, debug C++, debug Web, giao Agent có giới hạn, đọc diff, nghiệm thu	7 Mission ≈ 4 ca 45 phút	ViSinhVien chạy được, hai lỗi đã sửa, bảng test, diff
03 · Đừng để AI suy nghĩ thay bạn	Viế´t dự đoán **trước** khi hỏi; dùng phản ví dụ nhỏ bác bỏ kế´t luận tự tin; giải thích lại không cầ`n mở chat	7 mục ≈ 3 ca 45 phút	Bảng trace, kế´t quả tự quan sát, bài giải thích 6 phút
04 · Sử dụng AI có trách nhiệm	Xử lý **bảy hồ` sơ sự cố´**: key bị lộ, dữ liệu cá nhân, Agent vượt phạm vi, gói thư viện, bài làm hộ, mã không rõ giấ´y phép, khai báo thiế´u	7 hồ` sơ ≈ 4 ca 45 phút	Biên bản sự cố´, phiế´u bằ`ng chứng, bản khai báo AI
FINAL BOSS	Ghép bố´n chặng thành một bản nộp có bằ`ng chứng: **làm được — biế´t là đúng — giải thích được**	Bronze 60–90; Silver 90–120; Gold +30–45 phút	Code bản sao, hai bản vá, test, diff, khai báo AI, bảo vệ 8 phút

Gợi ý xế´p lịch chỉ là phép tính từ đơn vị 20–25 phút cho một mục ở **phầ`n Mở đầ`u**. Đây **không** phải yêu cầ`u của học phầ`n và không phải thước đo tố´c độ.

Vào thẳng chỗ cầ`n

Bảy Mission — Chương 2: 01 môi trường · 02 hỏi đúng · 03 debug C++ · 04 debug Web · 05 Agent có giới hạn · 06 đọc diff · 07 nghiệm thu

Bảy hồ` sơ sự cố´ — Chương 4: 01 key trên GitHub · 02 dữ liệu cá nhân · 03 Agent vượt phạm vi · 04 gói thư viện · 05 bài làm hộ · 06 giấ´y phép · 07 khai báo thiế´u · phòng khám SQL/XSS/C++

Bộ công cụ: 11 mẫu prompt P01–P11 · từ điển tra nhanh · thỏa thuận học tập và mẫu khai báo AI · demo ViSinhVien

Một hợp đồ`ng số´ liệu chạy xuyên suố´t

Ba khoản giả **35000 + 7000 + 18000 = 60000**; tiề`n nguyên VND từ **1 đế´n 1000000000**; tố´i đa **500 khoản**; danh sách rỗng trả **0**; tổng cực đại **500000000000**. Cùng một hợp đồ`ng, bố´n cách kiểm khác nhau:

Bộ kiểm	Sô´ ca	Trạng thái trong gói
Demo C++ đúng `vi-du/nam-1-cpp/vi_sinh_vien.cpp --test`	9	đã chạy, đạt
Demo dữ liệu Web `vi-du/nam-2-web/test_data.cjs`	34	đã chạy, đạt
Đê` Bronze `vi-du/final-boss/bronze.cpp`	9	**5 đạt, 4 lỗi chủ đích** — bạn phải sửa
Đê` Silver `vi-du/final-boss/test_silver.cjs`	12	**6 đạt, 6 lỗi chủ đích** — bạn phải sửa

Bô´n bộ **không cộng gộp** và không thay nhau. `loi-bo-sot.cpp` in ra 42000 và không có `--test`; đó là đô` dùng học, không phải bài đã kiểm.

Nê´u bạn chỉ có 60 phút

Đọc trang này → làm **1.6 MINI CHALLENGE** → xem **Mission 03** → đọc **Hô` sơ 01**. Bỏ qua được phâ`n còn lại mà không hỏng mạch.

Nê´u bạn là giảng viên

Dùng **bảng rubric 100 điểm của Final Boss** làm thang châ´m luyện tập, **thỏa thuận học tập** làm phiê´u nộp đâ`u kỳ, và **bảy hô` sơ Chương 4** làm tình huô´ng thảo luận 15 phút. Câu hỏi biê´n thể ở **mục 8 Final Boss** dùng được cho vâ´n đáp.

Đã kiểm chứng tới đâu — và phâ`n nào chưa

Đã chạy trên máy biên tập: 46 phép kiểm câ´u trúc nội dung; 16 phép kiểm ví dụ thực thi (Node 24, Python 3.14, g++ 11.4 trong WSL); 80 phép kiểm trong trình duyệt Chrome 156, gô`m 16 hình SVG, ba cỡ màn hình, bản in PDF 159 trang và xác nhận **không gửi yêu câ`u ra mạng**. Nhật ký giữ cả lâ`n thâ´t bại — xem **báo cáo châ´t lượng**.

Chưa kiểm: chỉ một trình duyệt trên một hệ điê`u hành; PDF chưa soát từng trang; giá, quota và quyê`n lợi của các dịch vụ AI chưa xác minh độc lập; **chưa thử với lớp sinh viên thật**; lộ trình Gold chưa vận hành Agent thật. Nguô`n pháp luật giữ mô´c đô´i chiê´u 29/10/2026.

Gặp một con sô´ không có ở trang này, hãy tìm nhãn **HƯ CÂ´U**, **GIẢ ĐỊNH** hoặc **chưa kiểm** cạnh nó. Không có nhãn nào thì coi là điê`u câ`n hỏi lại.

Cẩm nang tự học và tra cứu dành cho sinh viên năm 1–2 Khoa Công nghệ thông tin, Trường Đại học Thủy lợi.

Một bữa trưa, một câu hỏi nhỏ

Sau bữa trưa, Minh mở máy: “Hôm nay mình đã chi bao nhiêu?” Bạn học năm nhất, biết biến và vòng lặp C++, nhưng thường muốn hỏi AI ngay khi thấy lỗi. Lan học năm hai, đã gặp Web, OOP, cấu trúc dữ liệu và cơ sở dữ liệu; bạn dựng được giao diện nhưng còn lúng túng khi lỗi đi qua nhiều tệp. Hai bạn cùng làm **ViSinhVien**, ứng dụng ghi những khoản chi nhỏ.

Nhân vật, hội thoại và dữ liệu đều hư cấu, không phải khảo sát sinh viên TLU hay bảng giá thực tế.

Khoản chi	Nhóm	Số nguyên lưu trong code	Hiển thị
Cơm trưa	Ăn uống	`35000`	35.000 đ
Xe buýt	Đi lại	`7000`	7.000 đ
Vở ghi	Học tập	`18000`	18.000 đ
Tổng đối chiếu		`60000`	**60.000 đ**

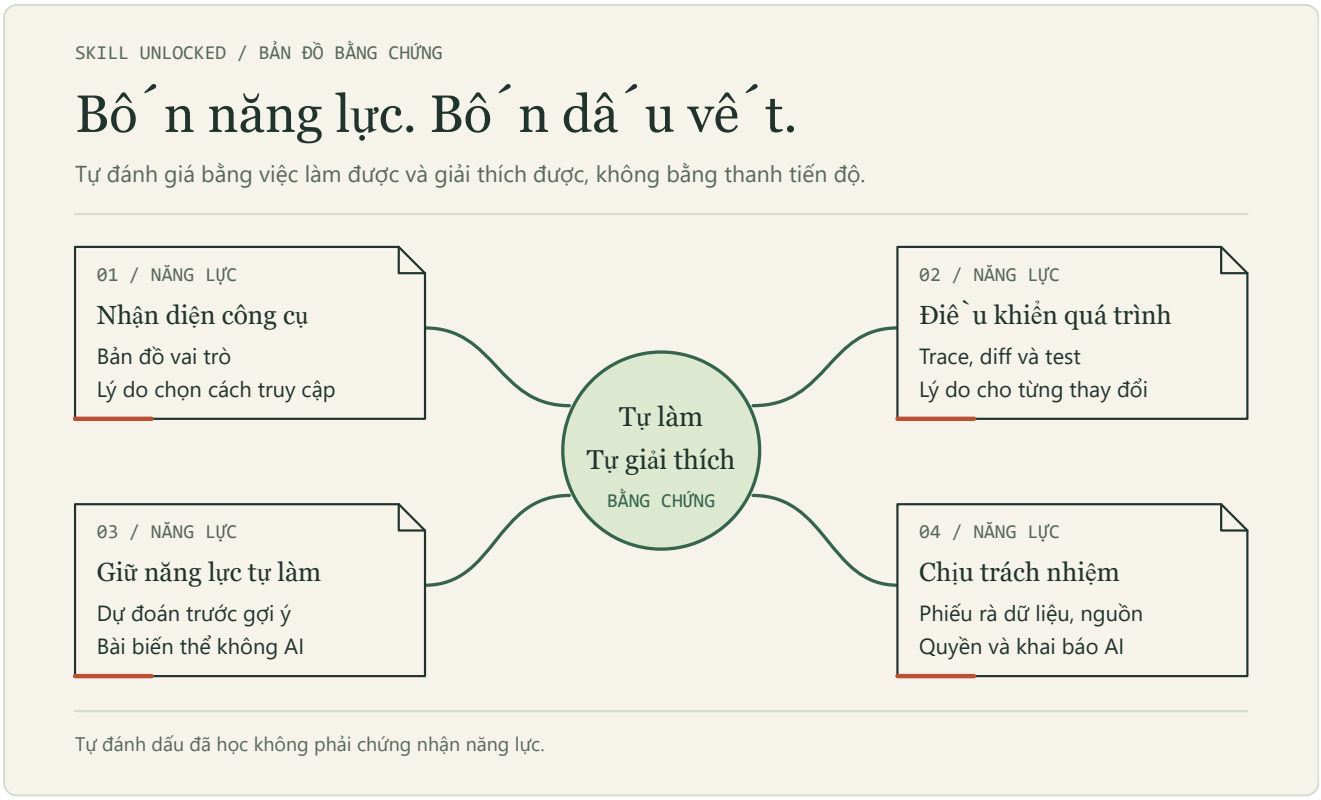
Minh gặp bản C++ kết thúc ở `42000`; Lan muốn biết tiền đi từ ô nhập đến tổng thế nào. Bạn sẽ cùng hai người dự đoán, kiểm tra một thay đổi nhỏ, rồi giải thích vì sao nó đúng. Không cần mua dịch vụ AI để bắt đầu.

Hai lộ trình, một cách học

Theo Minh — C++ trước: nhận diện công cụ ở Chương 1; đọc vòng lặp, trace và sửa lỗi bỏ khoản cuối ở Chương 2; tự làm biến thể ở Chương 3; chuẩn bị bản nộp trung thực ở Chương 4. Chưa học Web/SQL thì để lại, không cần cài thêm cho nhánh C++.

Theo Lan — lần theo Web: sau Chương 1, theo `amount` từ biểu mẫu qua xử lý đến hiển thị/lưu trữ ở Chương 2; tự giải thích kiểu dữ liệu và làm biến thể ở Chương 3; rà quyền, dữ liệu, nguồn ở Chương 4. Khi Lan cùng Minh xem C++, đó là đọc chéo, không đổi nhánh của Minh.

Không cần hoàn thành cả hai ngay. Một hàm tự giải thích được có giá trị hơn cả ứng dụng chép lại mà chưa hiểu.



Bốn năng lực: nhận diện công cụ, điều khiển quá trình, tự làm và chịu trách nhiệm; mỗi năng lực gắn với bằng chứng. Xem hình lớn

Bổ sung biên tập — bản đồ chọn việc luyện tiếp, không phải chứng nhận tự động. Bảng mục tiêu bên dưới cũng trình bày bằng chữ các bằng chứng cần giữ.

Hợp đồng chung của ViSinhVien

`amount` là **số nguyên VND từ 1 đến 1.000.000.000 (`1e9`), kể cả hai đầu**. Tối đa **500 khoản**; danh sách rỗng hợp lệ, tổng bằng **0**. Từ chối khoản bằng 0, số âm, số thập phân (như 1.5) hoặc vượt trần. Dấu chấm trong "35.000 đ" chỉ để hiển thị.

Hai nhánh dùng cùng tiền mẫu nhưng **khác cấu trúc dữ liệu**:

- C++ dùng `std::vector<long long>` chứa `{35000, 7000, 18000}`: chỉ có các số tiền, chưa có tên, nhóm, mã.
- Web dùng danh sách đối tượng `Expense` có **bốn trường**:

```json
{"id": 1, "title": "Cơm trưa", "amount": 35000, "category": "an-uong"}
```

`id` nhận diện khoản; `title` là tên; `amount` là tiề̀n; `category` là mã nhóm, không phải nhãn hiển thị. Ô nhập Web trả chuỗi: chuyển đổi không thay kiểm tra hợp lệ. Hàm tính tổng cũng không thay kiểm đủ bản ghi khi đọc/lưu.

Tổng cực đại `500000000000` nằ̀m trong miề̀n số nguyên an toàn của JavaScript và biểu diễn được bằ̀ng `long long`. Đây là quy ước học tập, không phải thiế́t kế́ cho mọi hệ thố́ng tài chính; đổi giới hạn/đơn vị tiề̀n thì phải xét lại.

Web chỉ có HTML, JavaScript, `localStorage`, **không backend/đăng nhập**. SQLite là minh họa Python riêng dùng `:memory:`, không phải cơ sở dữ liệu của Web. Không nhập sao kê, chi tiêu thật hay thông tin cá nhân.

Mục tiêu đi cùng bằ̀ng chứng

Đây là mục tiêu cẩm nang, **không thay CLO hoặc thang điểm học phầ̀n**. Chọn một dòng để tự kiểm, không đánh dấ́u chỉ vì đã đọc hế́t.

Mục tiêu	Chỗ học	Bằ̀ng chứng do bạn tạo
Phân biệt model, provider, ứng dụng	Chương 1	Bảng nhận diện công cụ, ghi điề̀u chưa xác định.
Chọn cách dùng hợp túi tiề̀n	Chương 1	Tính lại chi phí từ giả định, nêu giới hạn so sánh.
Giao việc nhỏ, đủ ngữ cảnh	Chương 1–2	Yêu cầ̀u có mục tiêu, dữ liệu, phạm vi, tiêu chí kiểm tra.
Đọc lỗi, diff, kiểm thử	Chương 2	Trace lỗi `42000/60000`; kiểm rỗng/một phầ̀n tử; theo `amount`.
Học chủ động	Chương 3	Bài biế́n thể tự làm khi không mở AI.
Giải thích phầ̀n mình làm	Chương 3	Trả lời “vì sao?”, “đổi yêu cầ̀u thì sao?”, ghi chỗ còn vướng.
Bảo vệ dữ liệu, rà mã/phụ thuộc	Chương 4	Quyế́t định về̀ dữ liệu gửi, điểm hiển thị, truy vấ́n SQL, gói đề̀ xuấ́t.
Khai báo, kiểm quyề̀n, xử lý sự cố́	Chương 4	Bản khai đúng hỗ trợ, câu hỏi cho giảng viên, thứ tự xử lý key lộ.

Cách đọc và tự học

Trước buổi chia sẻ: đọc mở đầ̀u, phầ̀n nhận diện công cụ và mang một câu hỏi thật; chưa cầ̀n cài phầ̀n mề̀m. **Trong buổi:** dự đoán trước khi xem chạy mẫu/đáp án, ghi chỗ hiểu sai thay vì chỉ chụp màn hình.

Sau buổi: chọn nhiệm vụ vừa sức trong khoảng 20–25 phút: đọc → dự đoán → thử → đổi chiếu → giải thích. Ghi ba dòng: “Mình đã hiểu… / Mình chưa chắc… / Lần tới mình sẽ…”. Thời lượng là gợi ý; chưa xong thì giữ đầu vào, lỗi và bước đang làm, không chấm mình theo tốc độ.

Trong bản HTML, dùng mục lục, tìm mục, chỉnh cỡ chữ và in khi có; mở **Đáp án** sau khi thử. Giữ cả thư mục sách để đọc offline cùng hình và ví dụ. Nguồn ngoài và dịch vụ AI cần mạng; mô phỏng tại chỗ không gọi AI. Không có tài khoản vẫn học bằng đề, trace và phiếu đáp minh họa.

Phân biệt **hội thoại mô phỏng**, **trace**, **đầu ra kỳ vọng** với **kết quả thực sự chạy**. Bản có lỗi chủ đích được giữ để học, không phải demo đúng cần sửa tùy tiện. Tổng trung gian `42000` chưa chứng minh lỗi; phải biết đã duyệt đến đâu. Thiếu môi trường thì ghi **chưa chạy**, không chép kỳ vọng làm quan sát.

Mở ví dụ nào trước?

Mở gốc `AI-Student-Handbook` khi theo lệnh Chương 2. Giữ `vi-du/` làm tham khảo; tự tạo thư mục `bai-lam/` (chưa có sẵn) theo Chương 2, mục 2.1, rồi sao chép nhánh sang đó trước khi sửa.

Mở tệp	Mục đích và điều kiện
C++ có lỗi chủ đích	Trace tổng thiếu; C++17, không có `--test`. Chưa có compiler thì đọc trace trước.
C++ đúng, có kiểm thử	Đổi chiếu sửa lỗi và giới hạn; cần trình biên dịch C++17 như g++.
Web ViSinhVien	Thử khoản giả bằng trình duyệt hiện đại; giữ cùng `data.js`, `app.js`.
Kiểm thử dữ liệu Web	Kiểm logic, không chứng minh DOM/lưu trữ; cần Node.js hiện đại, không cần `npm install`.
SQLite riêng	Truy vấn tham số hóa với fixture riêng; Python 3 có `sqlite3`, không phải backend Web.

`localStorage` lưu trong trình duyệt, không trên máy chủ. Quyền lưu khi mở tệp cục bộ tùy trình duyệt; demo có thể báo lỗi và dùng bộ nhớ tạm, mất thay đổi khi kết thúc phiên. Không dùng để giữ dữ liệu lâu dài.

Nguồn và lời hẹn trước khi học tiếp

Bản nguồn có mốc đối chiếu sản phẩm/pháp lý **29/10/2026**, không bảo đảm còn áp dụng khi bạn đọc. Tra nguồn cuối chương, kiểm ngày, đúng sản phẩm và điều kiện tài khoản. Có hướng dẫn môn mới hơn hoặc điều chưa rõ thì hỏi người phụ trách; lời AI hay một liên kết không thay kiểm chứng.

> **Thỏa thuận học tập:** AI có thể giải thích, tạo giả thuyết, đề xuất code và giúp kiểm tra. Bạn quyết định mục tiêu, phần tự làm, phạm vi chia sẻ và bằng chứng. “AI làm xong” chưa có nghĩa “mình học xong”.

Tiếp tục với thỏa thuận có mẫu điền, glossary hoặc thư viện prompt.

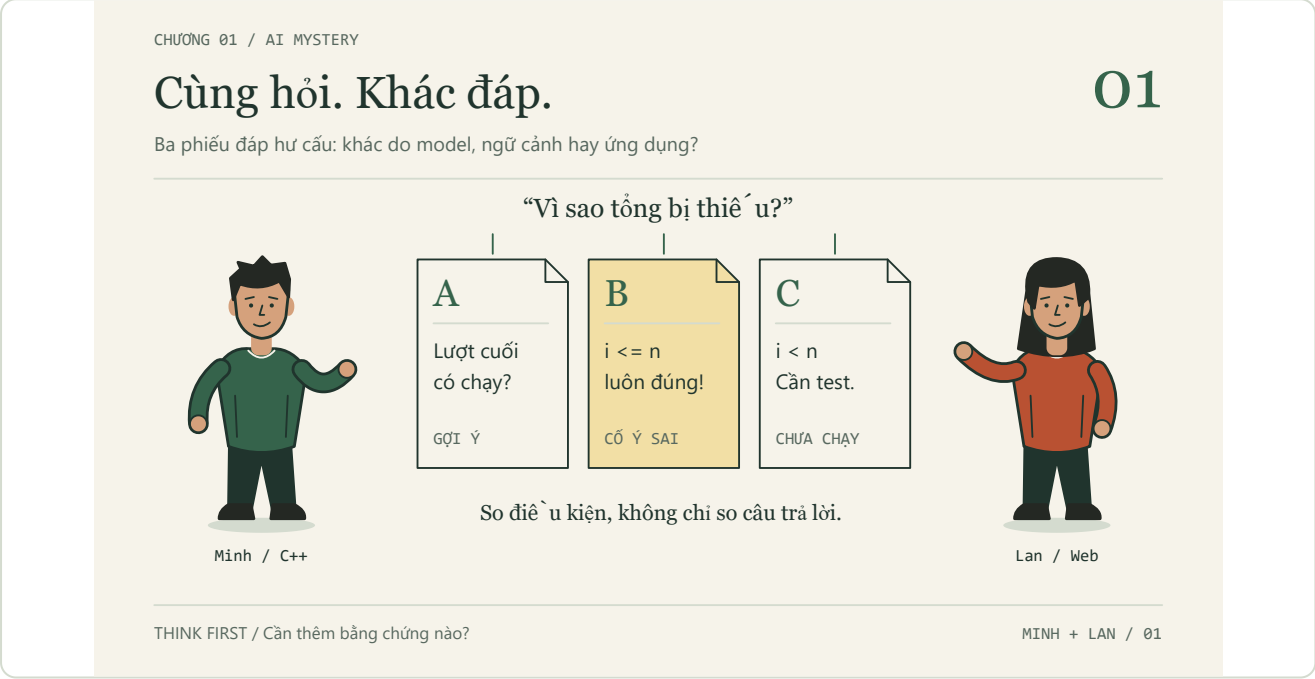
Ghi chú biên tập: kế thừa câu chuyện, dữ liệu, hai nhánh, mục tiêu, hướng dẫn đọc và thỏa thuận nguồn. Bổ sung biên tập: cảnh bữa trưa, bản đồ năng lực, cách lưu bằng chứng, quy ước bản sao. Cẩm nang không phải quy định TLU, quảng cáo hay tư vấn pháp lý.

01

CHƯƠNG 01 / MINH & LAN

Bước vào thế giới AI

Cùng một câu hỏi, ba câu trả lời. Bạn cần biết mình đang dùng gì trước khi quyết định tin điều gì.



Tình huống và nhân vật hư cấu · Đọc, dự đoán rồi tự kiểm chứng. Xem hình lớn

Minh mở ba giao diện AI, dán cùng đoạn C++ và hỏi cùng một câu. Nơi thứ nhất hỏi ngược lại. Nơi thứ hai đưa cách sửa rất chắc chắn. Nơi thứ ba sửa khác, nhắc phải kiểm thử. Minh nhìn ba cửa sổ: “Nếu đều là AI, tại sao chúng không nói giống nhau? Mình nên tin cái nào?”.

Bổ sung biên tập — tình huống mở và ba thẻ trả lời hoàn toàn HƯ CẤU, không đại diện sản phẩm, không phải hội thoại hay benchmark thực. Minh năm nhất học C++; Lan năm hai học OOP/DSA/CSDL/Web. Hai nhân vật hư cấu không đại diện khảo sát TLU.

Hai bạn làm **ViSinhVien**, bài tập chi tiêu bằng dữ liệu GIẢ. Minh học mảng và vòng lặp; Lan làm Web với HTML, JavaScript, `localStorage`. Không có backend, đăng nhập, ngân hàng hay tài chính thật. Trợ lý AI giúp học; ViSinhVien không tích hợp AI.

Ba khoản GIẢ trên giấy của Minh là:

Khoản chi GIẢ	Sô´ tiê`n nguyên VND
Cơm trưa	35000
Xe buýt	7000
Vở ghi	18000
Tổng cầ`n có	60000

Đây là **snippet có lỗi chủ đích** để đọc và đặt câu hỏi, không phải yêu cầ`u làm hỏng demo đúng. Các sô´ tiê`n trong snippet đã được giả định hợp lệ; chương trình chưa làm nhiệm vụ nhập và xác thực dữ liệu.

C++

```
#include <iostream>
#include <vector>

int main() {
    std::vector<long long> amounts = {35000, 7000, 18000};
    long long total = 0;
    int n = static_cast<int>(amounts.size());
    for (int i = 0; i < n - 1; ++i) {
        total += amounts[i];
    }
    std::cout << "Tong chi tieu: " << total << " VND\n";
    return 0;
}
```

Minh dán nguyên snippet vào từng giao diện, rô`i dùng cùng câu hỏi:

Prompt / Ghi chú

```
Tôi mới học vòng lặp C++. Tôi cần cộng tất cả khoản chi GIẢ:
35000, 7000, 18000 phải ra 60000, nhưng chương trình đang cho 42000.
Hãy gợi ý một bước để tôi tự tìm lỗi, chưa viết lại cả chương trình.
```

Thẻ trả lời A — HƯ CÂ´U, không gă´n với sản phẩm

> Khi `n = 3`, chỉ sô´ của khoản Vở ghi là bao nhiêu? Hãy thay chỉ sô´ đó vào điê`u kiện `i < n - 1` và quyê´t định xem vòng lặp có cộng khoản này không.

Thẻ trả lời B — HƯ CÂ´U, không gă´n với sản phẩm

> Vòng lặp thiê´u một lượt. Bạn cứ đổi thành `i <= n` ; như vậy sẽ cộng đủ và luôn an toàn. Không cầ`n thử thêm vì lỗi này rấ´t đơn giản.

Thẻ trả lời C — HƯ CÂ´U, không phải báo cáo đã chạy

> Điề`u kiện đang bỏ phầ`n tử cuố´i. Với yêu cầ`u cộng mọi khoản, hướng sửa là `i < n` , không phải `i <= n` . Tôi chưa thực thi chương trình; hãy kiểm tra danh sách rỗng, một khoản và ba khoản sau khi tự sửa.

Lan chưa chọn "AI thắ´ng cuộc". Cô lấ´y bút khoanh yêu cầ`u **gợi ý một bước**, rồ`i gạch chân chữ **luôn an toàn** trên thẻ B. Một câu trả lời phải được xét cả độ đúng, cách giúp người học và bằ`ng chứng, chứ không chỉ tố´c độ xuấ´t hiện.

Trước khi đọc tiế´p, hãy tự cộng ba khoản, chọn một thẻ để bắ´t đầ`u học và viế´t lý do. Nế´u chưa biế´t C++, bạn vẫn có thể nhận ra rằ`ng một kế´t luận về` chương trình cầ`n cách kiểm tra. Sau mục 1.5, hãy chấ´m lại ba thẻ bằ`ng rubric 10 điểm ở đó (rubric viế´t cho Đề` A–C, nhưng năm tiêu chí dùng được cho thẻ).

**Đầ`u ra của chương:** một bản nhận diện công cụ có phầ`n "chưa biế´t", một prompt đủ context, một phép tính chi phí có đơn vị và một quyế´t định dùng công cụ có điề`u kiện. Không cầ`n đăng ký, cài đặt, tạo key, gọi API hoặc trả tiề`n để hoàn thành.

1.1. AI MYSTERY / AI MAP — Giải bí ẩn, đọc bản đồ` AI

Từ 42000 đế´n câu hỏi "mình đang dùng cái gì?"

`35000 + 7000 = 42000` ; thêm `18000` mới được `60000` . **42000 có thể là tổng trung gian đúng** hoặc tổng của hai khoản. Nó chỉ là sai lệch khi yêu cầ`u cộng đủ ba khoản mà chương trình kế´t thúc ở đó. Một con số´ đứng riêng chưa chẩn đoán được lỗi.

Snippet có `n = 3` , chỉ số´ hợp lệ 0, 1, 2. Cận `i < n - 1` , tức `i < 2` , chặn khoản Vở ghi ở chỉ số´ 2. `static_cast<int>` chuyển kích thước sang số´ nguyên có dấ´u để biểu diễn cận; tố´i đa 500 khoản vừa trong miề`n đang dùng.

Kết quả dự đoán từ snippet, không phải log thực thi của lần biên tập:

```
Prompt / Ghi chú
```

```
Tong chi tieu: 42000 VND
```

Chuỗi không dấu nhằm giảm lỗi hiển thị ở một số terminal. Trace đầy đủ, ca rỗng, kiểu có dấu/không dấu và executable cũ được học ở **Chương 2 · Trở thành người điều khiển AI, mục 2.3 — Mission 03: Debug C++**. Chương này mới nhận diện vấn đề và loại bằng chứng cần có.

▼ Đáp án mở đầu — Chọn thẻ để học, không chọn giọng nói tự tin · mở sau khi tự thử

Nên bắt đầu học với Thẻ A, vì nó giữ đúng yêu cầu gợi ý một bước. Thẻ C dùng để đổi chiếu sau khi tự sửa: nó sửa đúng, nói rõ chưa chạy nhưng tiết lộ nhiều hơn Minh yêu cầu. Loại Thẻ B: `i <= n` cho phép chỉ số `n` ngoài danh sách; truy cập đó trong C++ dẫn tới hành vi không xác định.

Cận `i < n` duyệt các chỉ số hợp lệ và chặn chỉ số bằng kích thước. Sửa cận **chưa bổ sung toàn bộ kiểm tra dữ liệu**; Minh còn phải giải thích và thử biến thể. Ba thẻ hư cấu không chứng minh thương hiệu nào tốt hơn.

Model, provider, application và producer: bốn câu hỏi khác nhau

"Mình dùng ChatGPT" thường nêu sản phẩm Minh mở, chưa xác định model. **AI**, trí tuệ nhân tạo, gồm cả nhận diện và phát hiện bất thường. Chương tập trung vào **AI tạo sinh**, tạo văn bản hoặc mã. **LLM**, mô hình ngôn ngữ lớn, thường nằm trong trợ lý; không phải mọi AI đều là chatbot. Khả năng nhận ảnh, âm thanh, tệp tùy model và ứng dụng.

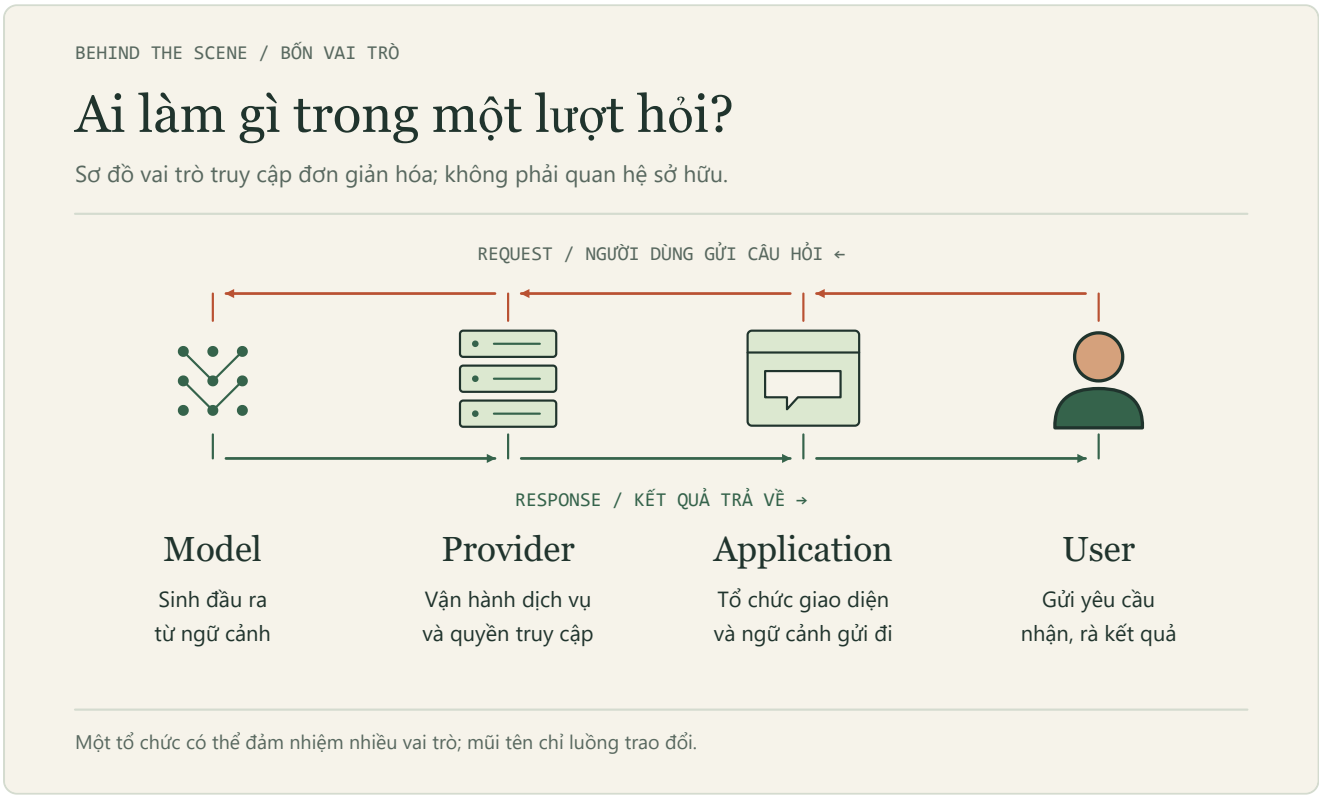
Model là mô hình tính toán đã được huấn luyện để xử lý đầu vào và tạo đầu ra. GPT, Claude, Gemini có thể chỉ họ mô hình, chưa phải phiên bản. Tài liệu định danh đáng tin hơn đoán qua văn phong; lời giải mạch lạc không bảo đảm đúng.

Provider là bên cung cấp dịch vụ truy cập đến model trong đường đang xét: tiếp nhận, xác thực, áp dụng điều kiện; có thể vận hành trực tiếp hoặc làm trung gian. **Producer** xây dựng, huấn luyện hoặc phát hành model. Hai vai trò có thể cùng tổ chức nhưng không bắt buộc.

Application là nơi thao tác: trang chat, ứng dụng điện thoại/máy tính hoặc trợ lý lập trình. Nó tổ chức lịch sử, chọn thông tin, có thể nối công cụ/tìm kiếm và hiển thị bản sửa. **User** đặt mục tiêu, quyết định dữ

liệu được phép gửi và kiểm tra phâ`n mình sử dụng.

Vai trò	Câu hỏi phải trả lời	Cách ghi nê´u thiê´u bă`ng chứng
Model	Thành phâ`n nào tạo đâ`u ra? Định danh nào được công bô´?	Chỉ biê´t họ model; chưa xác định phiên bản hoặc model từng lượt
Provider	Ai cung câ´p đường truy cập đê´n model cho ứng dụng?	Chưa xác định provider của đường truy cập
Application	Tôi mở và thao tác ở phâ`n mê`m nào?	Ghi tên quan sát được, không suy ra model từ đó
Producer	Ai phát triển hoặc phát hành model?	Chưa đủ dữ kiện; không gán cho nhóm viê´t giao diện
User	Ai giao việc và kiểm tra trước khi dùng kê´t quả?	Minh hoặc Lan trong tình huô´ng này



Sơ đô` vai trò model, provider, application và user; mũi tên biểu diễn đường truy cập, không phải quyê`n sở hữu.
Xem hình lớn

AI MAP — Bản đô` vai trò. Một tổ chức có thể giữ nhiê`u vai trò; mỗi ô không nhâ´t thiê´t là một công ty hay máy chủ riêng. Producer là câu hỏi bổ sung vê` bên phát triển, không phải một tâ`ng bă´t buộc trong mọi request.

Bản nguô`n dùng ví dụ **Claude qua Amazon Bedrock**: Anthropic là bên phát triển Claude, AWS cung câ´p đường truy cập qua Bedrock trong tình huô´ng đó. Vì vậy, nhãn “hỗ trợ Claude” không tự chứng minh kê´t nô´i trực tiê´p tới Anthropic. Nguô`n chính thức gô´c là tài liệu Anthropic và AWS, liệt kê cuô´i

chương. **Mốc bộ nguồn là 29/10/2026; các thông tin dịch vụ chưa được xác minh độc lập trong lần biên tập này, cần tra lại khi áp dụng.**

Tương tự, Gemini có thể được nhắc ở lớp ứng dụng hoặc họ model. “Mở ứng dụng Gemini”, “gọi Gemini API” và “chọn một model thuộc họ Gemini” không đồng nghĩa. Điều quan trọng không phải thuộc hết tên, mà là đặt tên vào đúng vai trò.

API và key không phải một bộ não tải về máy

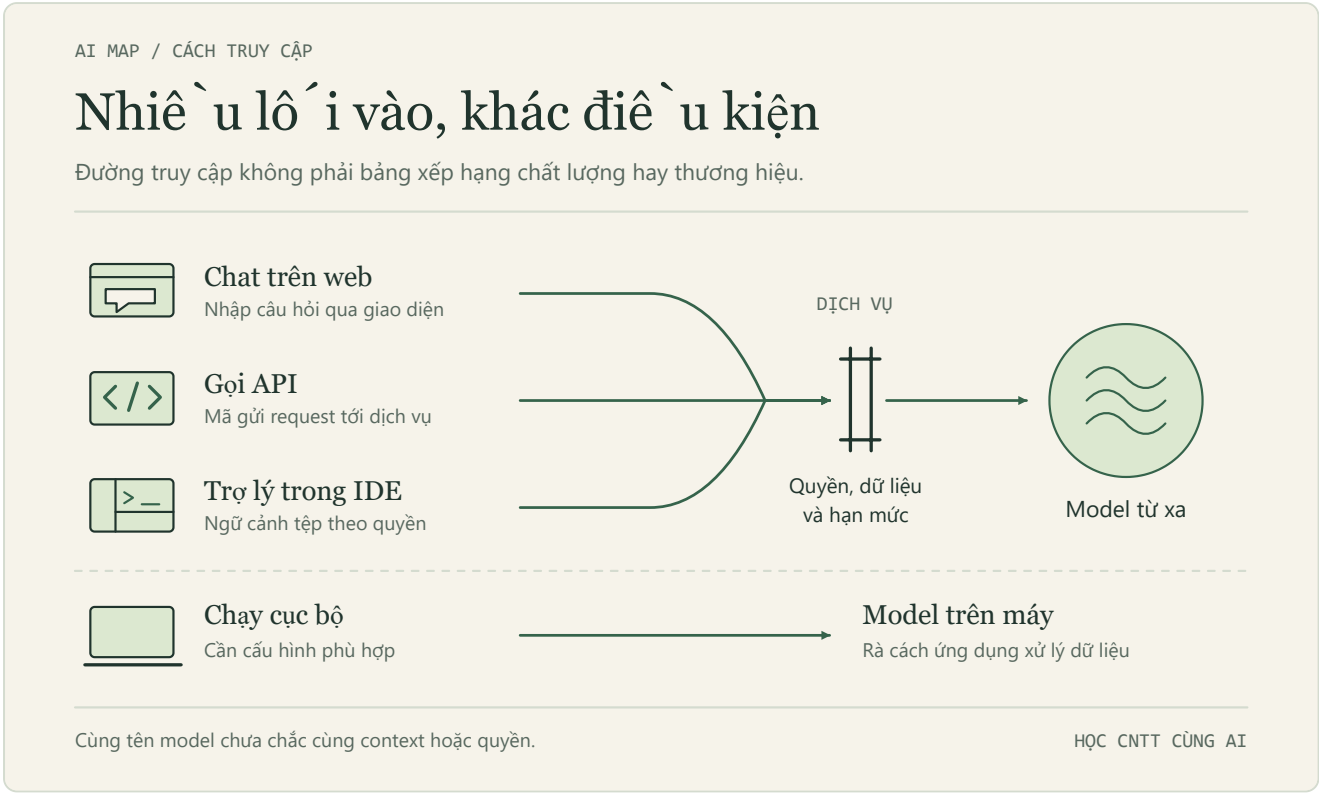
API là giao diện để các phần mềm giao tiếp theo quy tắc xác định. Trong chat, Minh gõ và gửi. Với API, chương trình chuẩn bị request theo định dạng dịch vụ; request có thể chỉ định model. API không phải model mới và cũng không chỉ dùng cho AI.

API key là một dạng thông tin xác thực để dịch vụ nhận biết quyền truy cập; có dịch vụ dùng cơ chế xác thực khác. Có key không có nghĩa sở hữu model, được dùng mọi phiên bản, quota vô hạn hoặc miễn phí. Quyền và hóa đơn còn phụ thuộc tài khoản, dự án và điều kiện dịch vụ.

> **SECURITY ALERT — Khóa không phải dữ liệu thực hành.** Không đưa key thật vào chat, ảnh chụp, nhóm lớp hoặc kho mã công khai. Không nhúng khóa bí mật vào JavaScript phía trình duyệt: mã được gửi đến người dùng có thể bị đọc. ViSinhVien không cần key; đừng tạo bí mật chỉ để hoàn thành bài nhập môn.

Nếu gặp key trong một bài hướng dẫn, hãy dùng placeholder thay vì sao chép khóa thật. Nếu nghi khóa đã lộ, thu hồi hoặc vô hiệu hóa qua dịch vụ trước, rồi kiểm tra mức dùng và xử lý bản sao. Xóa một dòng khỏi tệp mới nhất không làm các bản sao biến mất. Quy trình sự cố đầy đủ nằm ở **Chương 4 · Sử dụng AI có trách nhiệm, mục 4.1 — Hồ sơ 01: Khóa API xuất hiện trên GitHub**.

Những đường vào AI, không phải những bậc thang danh giá



Bản đồ các cách truy cập AI qua ứng dụng chat, trợ lý lập trình, chương trình dùng API hoặc mô hình cục bộ; không xếp hạng thương hiệu. Xem hình lớn

AI MAP — Bản đồ đường truy cập. Chọn đường phù hợp nhiệm vụ và điều kiện, không mặc định API hay agent là cấp học cao hơn chat.

Một luồng giản lược là **người dùng → ứng dụng chuẩn bị thông tin → provider tiếp nhận → model tạo đầu ra → ứng dụng hiển thị → người dùng kiểm tra**. Hệ thống thực có thể thêm trung gian, công cụ và nhiều lượt. Model chạy cục bộ có thể không cần provider bên ngoài nhận câu hỏi. Ngược lại, ứng dụng cài trên máy vẫn có thể gửi dữ liệu lên đám mây.

Ứng dụng đôi khi được gọi là **wrapper**, phần mềm bao quanh model/API. "Chỉ là cái vỏ" bỏ qua giá trị của việc chọn đúng tệp, giữ phạm vi và cho người học duyệt thay đổi. Nhưng cũng đừng suy mọi wrapper đều đọc được dự án, chạy code hay tìm web. Nếu dùng Qoder, cần phân biệt Qoder IDE, Qoder Desktop và Qoder CLI; tên cùng họ không chứng minh chúng có cùng giao diện, quyền hay quy trình. (Theo tài liệu sản phẩm Qoder; phần này chưa được kiểm chứng độc lập, cần tra lại ở Chương 2, mục 2.1.)

Cùng câu hỏi không đồng nghĩa cùng đầu vào

Ba thẻ hư cấu không xác định nguyên nhân khác biệt. Trong hệ thống thật, cùng câu gõ có thể kèm lịch sử, tài liệu, chỉ dẫn hoặc công cụ khác. Một nơi làm gia sư, nơi khác ưu tiên sinh mã. Giới hạn đầu ra, cấu hình và tính không tất định cũng ảnh hưởng.

Minh điề̀u tra theo ba bước: **ghi nhãn ứng dụng/chế̂́ độ/model → liệt kê context và công cụ → ghi phầ̀n chưa biế́t**. Nhãn "Tự động" chỉ cho phép ghi "chưa xác định model thực sự xử lý lượt này". Chatbot tự giới thiệu không đủ làm bằ̀ng chứng duy nhấ́t. Sang Chương 2, "chế́ độ" sẽ cụ thể thành Ask (chỉ đọc, trả lời) và Agent (được sửa tệp, chạy lệnh); "context" được gọi là ngữ cảnh.

Bổ sung biên tập — nhận diện trên giấ́y. Ứng dụng GIẢ "Gia sư ViSinhVien" nói dùng Gemini API nhưng không công bố́ model cụ thể. Điề̀n bố́n vai trò; rồ̀i thay mô tả bằ̀ng "hỗ trợ Claude". Điề̀u gì không còn biế́t được?

▼ **Đáp án nhận diện — Giữ chỗ trố́ng đúng chỗ · mở sau khi tự thử**

Application là Gia sư ViSinhVien; theo mô tả của chính ứng dụng, đường truy cập là Gemini API — nế́u đúng là gọi trực tiế́p thì provider là Google, nhưng ứng dụng cũng có thể đi qua trung gian, nên ghi "provider theo tự mô tả, chưa xác minh"; biế́t họ Gemini, chưa biế́t model cụ thể. Nhóm viế́t ứng dụng không vì thế́ là bên huấ́n luyện model. Nế́u chỉ còn "hỗ trợ Claude", provider và phiên bản đề̀u chưa xác định. Thiế́u bằ̀ng chứng thì giữ chỗ trố́ng.

1.2. BEHIND THE SCENE — Phía sau một câu trả lời

Prompt là lời giao việc; context là thông tin thực sự nhận được

Lan từng hỏi: "Làm app ví sinh viên chuyên nghiệp". Câu trả lời HƯ CẤ́U trong tình huố́ng biên tập đề̀ xuấ́t đăng nhập, máy chủ và đồ̀ng bộ thanh toán. Nghe đầ̀y đủ, nhưng sai phạm vi. Minh thì viế́t "sửa vòng lặp này" mà quên dán code. Cả hai đang mong hệ thố́ng biế́t những điề̀u chưa gửi.

Prompt là yêu cầ̀u hoặc chỉ dẫn đưa cho hệ thố́ng. **Context**, ngữ cảnh của một lượt, gồ̀m thông tin thực sự được nhận: prompt, lịch sử được chọn, tài liệu, code và kế́t quả công cụ. Theo cách dùng phổ biế́n, prompt là một phầ̀n của context. Tài liệu đang mở trước mặ́t bạn chưa chắ́c đã nằ̀m trong context của model.

Hãy sửa cách hỏi theo thứ tự có lý do:

1. **Nêu mục tiêu trước triệu chứng.** "Cộng tấ́t cả khoản hợp lệ" cho biế́t 42000 có phải sai hay không; riêng "ra 42000" chưa đủ.
2. **Đưa phầ̀n nhỏ có liên quan.** Với lỗi cận, cầ̀n khai báo dữ liệu, kiểu tổng và vòng lặp; không cầ̀n cả ổ đĩa.

3. **Đặt kỳ vọng cạnh hiện tượng.** Ba khoản phải ra 60000, snippet dự đoán ra 42000; nếu chưa chạy, ghi đúng là dự đoán.
4. **Giữ ràng buộc và giới hạn trợ giúp.** Số nguyên VND, tối đa 500 khoản, gợi ý trước; tránh nhận một bản viết lại vượt bài học.
5. **Yêu cầu cách kiểm tra.** Hỏi test và bằng chứng thay vì chỉ yêu cầu "chắc chắn 100%".

Prompt tốt cần đủ dữ kiện, không cần dài hoặc đầy lời tôn xưng. "Hãy đóng vai chuyên gia hàng đầu" không thay thế code và kết quả mong đợi. Ngược lại, cắt mất hợp đồng để tiết kiệm vài token có thể khiến bạn tốn nhiều lượt sửa sai yêu cầu.

Brief chung: giúp hai người học giữ cùng một bài

Brief là bản mô tả ngắn việc cần làm. Minh và Lan giữ bản hiện hành rồi thay dòng "việc lần này", thay vì kể lại dự án theo trí nhớ. Đây là mẫu có thể dùng độc lập nếu bạn dán kèm code liên quan:

Prompt / Ghi chú

```
DỰ ÁN: ViSinhVien, bài tập chỉ dùng dữ liệu GIẢ.
NGƯỜI HỌC: Minh năm 1 học C++; Lan năm 2 học OOP/DSA/CSDL/Web.
MỤC TIÊU: lưu danh sách khoản chi và tính tổng mọi khoản hợp lệ.
DỮ LIỆU CHUNG: mỗi khoản có tên; amount là số nguyên VND.
Nhánh C++ chỉ lưu phần tiền, không lưu tên khoản trong vector.
HỢP ĐỒNG: 0 < amount <= 1000000000; tối đa 500 khoản.
DANH SÁCH RỖNG: hợp lệ, tổng bằng 0; không tạo khoản amount = 0.
MẪU GIẢ: Cơm trưa 35000; Xe buýt 7000; Vở ghi 18000.
KỲ VỌNG: tổng 60000 VND; snippet lỗi bỏ khoản cuối cho 42000.
C++: vector<long long> chỉ mô hình phần tiền; tổng tối đa 500000000000.
WEB: Expense có id, title, amount, category; HTML + JS + localStorage.
PHẠM VI: không backend, đăng nhập, ngân hàng, API AI hoặc dữ liệu thật.
VIỆC LẦN NÀY: đọc snippet đính kèm, tìm nguyên nhân bỏ khoản cuối.
CÁCH GIÚP: gợi ý một bước trước, chưa viết lại chương trình.
NẾU THIẾU: chỉ ra thông tin cần bổ sung, không tự thêm tính năng.
ĐẦU RA: vị trí nghi vấn, giải thích vừa sức, test và kết quả mong đợi.
BẰNG CHỨNG: phân biệt suy luận, kỳ vọng và kết quả đã thực thi.
```

Minh chỉ cần mô hình tiền `vector<long long>` khi học vòng lặp; tên ba khoản trên giấy giúp hiểu ý nghĩa. Bản Web của Lan dùng `Expense{id,title,amount,category}`: `id` phân biệt khoản GIẢ, `title` là tên hiển thị, `amount` là tiền, `category` là nhóm. `id` không phải mã sinh viên. Mảng tiền C++ không tự trở thành mô hình đầy đủ bốn trường của Web.

Liên hệ OOP là phân biệt trách nhiệm dữ liệu, xác thực và tính tổng; không bắt buộc thêm một lớp chỉ để có chữ "hướng đối tượng". Với DSA, duyệt tổng một lần có thời gian O(n), phần nhớ phụ cho tổng là O(1). Với CSDL, Lan có thể xét thuộc tính và ràng buộc trên giấy. Đổi góc học không cho phép tự đưa máy chủ hay CSDL vào bản Web; minh họa SQLite `:memory:` trong `vi-du/minh-hoa-sqlite.py` (Chương 4, mục 4.8.1) là bài riêng.

Hợp đồ̀ng dữ liệu: bằng chứng bắt đầ̀u trước khi chạy

Một **hợp đồ̀ng** nói đầ̀u vào nào được chấ́p nhận và đầ̀u ra nào được yêu cầ̀u. Ở đây, mỗi khoản tồ̀n tại phải có `amount` nguyên, lớn hơn 0, không quá 1000000000; danh sách có tố́i đa 500 khoản. Rỗng vẫn hợp lệ vì không có khoản nào vi phạm. Tổng 0 không đồ̀ng nghĩa có một khoản chi 0.

Từ hợp đồ̀ng, tính được tổng lớn nhấ́t:

```
Prompt / Ghi chú

  500 × 1000000000 = 500000000000 VND
```

C++ cầ̀n kiểu chứa được mức đó. Kiểu số́ nguyên có dấ́u 32 bit thường gặp chỉ đạt khoảng 2,1 tỷ; không đủ chứa 500 tỷ. Snippet dùng `long long` cho tiề̀n và tổng. Không suy mọi `int` trên mọi hệ thố́ng đề̀u giố́ng nhau; lý do lựa chọn là miề̀n dữ liệu, không phải nhãn "tố́i ưu" của AI.

Trong JavaScript, `Number` biểu diễn chính xác các số́ nguyên trong miề̀n **safe integers**, số́ nguyên an toàn, từ `-9007199254740991` đế́n `9007199254740991`. Tổng 500000000000 nằ̀m trong miề̀n đó. Từng khoản nguyên và mọi tổng trung gian không vượt trầ̀n đang xét nên phù hợp với phép cộng của bài. Vượt miề̀n an toàn thì không còn được bảo đảm biểu diễn riêng từng số́ nguyên.

Điề̀u này **không** có nghĩa "dùng Number là dữ liệu tự hợp lệ". `amount` vẫn phải thỏa `Number.isSafeInteger(amount)` như trong `data.js`, lớn hơn 0 và không quá 1000000000. Chuỗi `"7000"` không phải số́ 7000. Ô nhập Web có thể trả chuỗi; dữ liệu trong `localStorage` cũng được lưu dưới dạng chuỗi, cầ̀n phân tích và kiểm tra cấ́u trúc khi đọc lại. Chỉ kiểm tra ở giao diện không bảo vệ được mọi dữ liệu đã lưu.

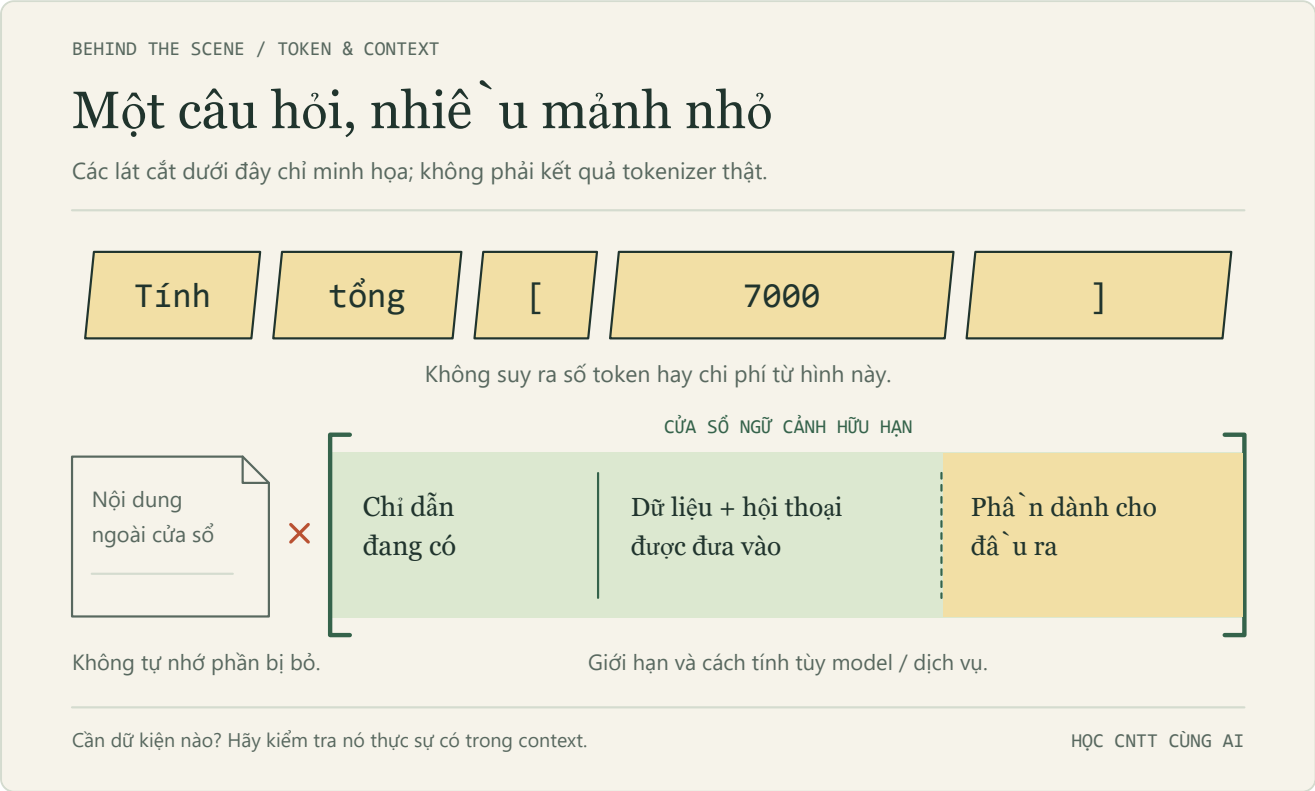
Bảng dưới là **kỳ vọng theo hợp đồ̀ng, chưa phải kế́t quả đã chạy**:

Ca kiểm tra	Dữ liệu hoặc thao tác GIẢ	Kỳ vọng
Rỗng	Không có khoản nào	Tổng 0 VND
Một khoản	7000	Tổng 7000 VND
Ba khoản mẫu	35000, 7000, 18000	Tổng 60000 VND
Chạm trầ̀n	500 khoản, mỗi khoản 1000000000	Tổng 500000000000 VND
Sai giá trị	0, số́ âm, 1.5 hoặc lớn hơn 1000000000	Từ chố́i khoản sai, không âm thầ̀m làm tròn
Vượt số́ lượng	Thêm khoản thứ 501	Từ chố́i thêm, giữ danh sách hợp lệ trước đó

Ba ca đầu kiểm tra một phần hành vi tổng. Ba ca sau yêu cầu cả xác thực dữ liệu. **Đổi cận vòng lặp không tự giải quyết validation.** Nếu snippet chỉ cộng dữ liệu đã hợp lệ, hãy nói giới hạn ấy thay vì tuyên bố nó đáp ứng toàn bộ bảng.

Token và context window: không phải số từ hay trí nhớ vô hạn

Token là đơn vị dữ liệu hệ thống xử lý. Một từ có thể thành nhiều token; dấu câu, khoảng trắng, code, ngôn ngữ và tokenizer đều ảnh hưởng. Không có quy tắc "một từ tiếng Việt bằng một token". Đếm khoảng trắng của sách không tính được hóa đơn.



Các khối dữ liệu minh họa token và vùng context hữu hạn; cách chia khối không phải kết quả của tokenizer thật.
Xem hình lớn

BEHIND THE SCENE — Khối chỉ minh họa chia/chọn thông tin, không tính token thật. Context không phải toàn bộ ổ đĩa.

Context window giới hạn lượng thông tin xử lý trong một lần theo dịch vụ; thường còn có giới hạn đầu ra riêng. Cửa sổ lớn không bảo đảm sử dụng chính xác mọi chi tiết hoặc cho phép gửi vô hạn.

Lan dán lịch sử có cả yêu cầu backend đã bỏ và nhiều log thừa. Brief hiện hành bị chìm. Hội thoại dài còn có thể bị ứng dụng cắt, tóm tắt hoặc chọn lại; "đã nói từ đầu" chưa bảo đảm còn nguyên trong context.

Hãy giữ brief hiện hành, hàm liên quan, dữ liệu GIẢ và ghi phần thay đổi. Model không tự biết tệp vừa sửa nếu chưa nhận bản mới. Trợ lý được quyền đọc dự án có thể lấy thêm nội dung; cần kiểm quyền và

tệp thực sự được dùng, không đoán qua giao diện.

Bổ sung biên tập — chọn context. Với lỗi 42000, chọn giữa brief, hàm tổng, ba khoản GIẢ, sao kê thật và bài giảng không liên quan. Giải thích một phần phải loại.

▼ Đáp án chọn context — Đủ để kiểm tra, không đủ để lộ dữ liệu · mở sau khi tự thử

Chọn brief, hàm cùng khai báo cần hiểu và ba khoản GIẢ. Loại sao kê vì riêng tư, bài giảng vì gây nhiễu. Thiếu kiểu dữ liệu hoặc đoạn gọi hàm thì bổ sung đúng phần đó, không gửi cả ổ đĩa.

Training, inference và web search: ba việc không thay thế nhau

Training, huấn luyện, dùng dữ liệu và thuật toán để điều chỉnh tham số model. **Inference**, suy luận theo nghĩa vận hành ở đây, dùng model đã có để tạo đầu ra từ đầu vào. Minh gửi ba khoản thì hệ thống có thể dùng chúng trong context; điều đó không tự chứng minh model vừa được huấn luyện lại và sẽ nhớ ba khoản ấy cho mọi người dùng.

Nhưng "không huấn luyện lại ngay" cũng không đồng nghĩa "không lưu dữ liệu". Lịch sử chat, bộ nhớ của ứng dụng, log vận hành và việc dùng nội dung để cải thiện dịch vụ là các vấn đề khác nhau. Cần đọc chính sách và cài đặt đúng sản phẩm. Không dùng hiểu biết về inference để thay cho kiểm tra quyền riêng tư.

Web search là truy xuất thông tin qua công cụ kết nối web, nếu sản phẩm có và lượt đó dùng nó. Hỏi "ưu đãi hôm nay là gì?" không tự bật tìm web. Câu trả lời có thể dựa vào kiến thức cũ. Ngược lại, có tìm kiếm cũng chưa bảo đảm nguồn phù hợp hoặc tóm tắt đúng.

Với ba khoản tiền, phép cộng và kiểm tra code là căn cứ phù hợp. Với tuổi được sử dụng dịch vụ, cần điều khoản chính thức còn áp dụng. Minh không cần tìm web để cộng 60000; Lan không được dùng phép suy luận nghe hợp lý để thay cho điều kiện ưu đãi.

Hallucination và trích dẫn: chuyển sự tự tin thành câu hỏi kiểm chứng

Hallucination, thường gọi ảo giác AI, là nội dung sai hoặc không có căn cứ được trình bày như đúng. Ví dụ HƯ CẤU: "JavaScript tự lưu mọi biến vào localStorage", một hàm C++ không tồn tại, hoặc lời "đã chạy test" khi chỉ đọc văn bản. Tên gọi này không giải thích mọi nguyên nhân lỗi, nhưng nhắc ta không coi câu chữ là bằng chứng.

Thay vì yêu cầu "đừng sai", Minh hỏi chỉ số nào được duyệt, ca một khoản cho gì, và câu nào là dự đoán. Lan hỏi tệp nào ghi lưu trữ, khi nào đọc lại và JSON hỏng được xử lý ra sao. Nếu công cụ nói đã chạy, cần lệnh, dữ liệu, đầu ra, môi trường và khả năng đổi chiếu. Một vài test qua vẫn không chứng minh đúng mọi đầu vào; test logic JavaScript không tự kiểm chứng DOM hay lưu/nạp trên trình duyệt.

Với trích dẫn, làm lần lượt: **khoanh claim → mở nguồn → tìm đoạn hỗ trợ → kiểm phạm vi và ngày → ghi giới hạn**. Liên kết có thật chưa chắc hỗ trợ kết luận. "Đủ điều kiện nộp hồ sơ" khác "chắc chắn được duyệt"; điều khoản API khác điều khoản ứng dụng chat. Ngày truy cập không phải ngày phát hành. Không tìm được căn cứ thì ghi **chưa xác minh**, không bù bằng một đoạn giải thích tự tin hơn.

1.3. AI WALLET — Hiểu túi tiền trước khi bấm gửi

AI WALLET / MÔ PHỎNG CỤC BỘ

50 request tốn bao nhiêu?

Toàn bộ giá và tỷ giá dưới đây là giả định học tập, không phải báo giá hay hóa đơn của bạn. Đổi 50 thành 1000 rồi tính nhẩm hệ số tăng.

Số request (yêu cầu gửi dịch vụ)	Token vào / request	Token ra / request
50	20000	5000

USD / triệu token vào	USD / triệu token ra	Tỷ giá giả định VND/USD
0.30	2.50	25000

0.925 USD ≈ 23125 VND · chưa phí khác

Chưa tính công cụ, cache, thuế và phí. Budget alert không mặc nhiên là trần chặn chi tiêu.

Ba hình thức dùng, ba cách đọc quyền lợi

Minh thấy nút nâng cấp; Lan nghe "API rẻ hơn". Trước khi so tiền, hai bạn cần biết đang so cái gì. Một sản phẩm miễn phí đủ giúp hiểu vòng lặp có thể là lựa chọn tốt hơn cả hai phương án trả tiền.

Hình thức	Cách hiểu nhập môn	Câu hỏi trước khi dùng
Free	Một phạm vi dùng không thu tiề̀n theo điề̀u kiện	Model, tính năng, quota, dữ liệu và điề̀u kiện tài khoản nào?
Subscription — thuê bao	Trả định kỳ cho quyề̀n lợi của một gói/sản phẩm	Bao gồ̀m gì, giới hạn gì, gia hạn khi nào, khoản nào ngoài gói?
API pay-as-you-go	Chương trình gọi dịch vụ, phí theo lượng dùng và bảng giá	Đơn vị input/output, công cụ, xác thực và giới hạn chi phí nào?

Quota là hạn mức có đơn vị và chu kỳ: số́ lượt, tố́c độ hoặc lượng dùng không thể đọc lẫn nhau. Free không có nghĩa vô hạn; trả tiề̀n cũng không bảo đảm mọi yêu cầ̀u được phục vụ không giới hạn.

Quyề̀n dùng ứng dụng và API phải được kiểm tra riêng. Đừng lấ́y biên lai thuê bao chat làm bằ̀ng chứng đã trả mọi API. Cũng không khẳng định "thuê bao không bao giờ có API": bản nguồ̀n dẫn tài liệu OpenAI vềề credit ChatGPT/API như một ngoại lệ cầ̀n đọc theo gói đủ điề̀u kiện. **Credit** là giá trị hoặc lượng dùng được cấ́p, có thể có thời hạn và phạm vi. Claim vềề gói tích hợp này mang mố́c nguồ̀n **29/10/2026, chưa xác minh độc lập ở lầ̀n biên tập này; cầ̀n tra lại**, không phải lời hứa quyề̀n lợi hiện hành.

Bài toán GIẢ ĐỊNH: tách input khỏi output

Lan muố́n dự toán 50 request. **Request** là một yêu cầ̀u gửi đế́n dịch vụ; một lầ̀n bấ́m của người dùng có thể dẫn đế́n nhiề̀u request phía sau. Ta cố́ định kích thước và đơn giá để học phép tính, không mô tả hóa đơn của một hãng.

Toàn bộ giá, tỷ giá và lượng dùng dưới đây là GIẢ ĐỊNH. Không dùng chúng để mua dịch vụ hoặc dự đoán giá thật. Chưa có phí riêng của công cụ, cache, lưu trữ hay cơ chế́ xử lý đặc thù.

Đại lượng	Giá trị GIẢ ĐỊNH
Input mỗi request	20000 token
Output mỗi request	5000 token
Số́ request ban đầ̀u	50
Giá input	0.30 USD / 1000000 token
Giá output	2.50 USD / 1000000 token
Tỷ giá	25000 VND / USD

Trước khi xem đáp án, viết sáu dòng: tổng input, tổng output, tiền input, tiền output, tổng USD, tổng VND. Giữ đơn vị ở từng dòng. Phải chia cho một triệu vì đơn giá được niêm yết theo triệu token, không phải theo một token hoặc nghìn token.

Prompt / Ghi chú

```
Tổng input  = input mỗi request × số request
Tổng output = output mỗi request × số request
Tiền input  = tổng input / 1000000 × giá input
Tiền output = tổng output / 1000000 × giá output
Tổng USD    = tiền input + tiền output
Tổng VND    = tổng USD × tỷ giá giả định
```

▼ Đáp án AI WALLET — 50 request và kiểm tra chéo · mở sau khi tự thử

Bước	Phép tính	Kết quả
Gom input	20000 × 50	1000000 token
Gom output	5000 × 50	250000 token
Tính tiền input	1000000 / 1000000 × 0.30	0.30 USD
Tính tiền output	250000 / 1000000 × 2.50	0.625 USD
Cộng hai khoản	0.30 + 0.625	0.925 USD
Quy đổi sau cùng	0.925 × 25000	23125 VND

Kết quả là **0.925 USD = 23125 VND, chưa phí**. Không làm tròn 0.925 thành 0.93 ở giữa rồi gọi con số VND sau đó là kết quả chính xác. Thuế, chênh lệch tỷ giá thanh toán, phí ngân hàng và các dịch vụ tính riêng nếu có chưa nằm trong mô hình.

Kiểm tra theo mỗi request: input là `20000 / 1000000 × 0.30 = 0.006 USD`; output là `5000 / 1000000 × 2.50 = 0.0125 USD`. Một request tốn `0.0185 USD`; nhân 50 cũng được `0.925 USD`. Hai hướng tính trùng nhau giúp phát hiện nhầm triệu với nghìn.

Output ít token hơn nhưng đắt hơn ở bài này vì đơn giá output cao hơn. Không gộp 1250000 token rồi nhân một giá chung. Tiền phụ thuộc cả lượng và đơn giá của từng loại, không chỉ tổng số token.

Từ 50 lên 1000 request: con sô´ nhỏ có thể đổi nghĩa

Giữ nguyên tâ´t cả giả định, tăng sô´ request từ 50 lên 1000, tức tăng **20 lâ`n**, không phải 1000 lâ`n. Tổng input thành 20000000 token; output thành 5000000 token. Tiê`n input là 6.00 USD, output là 12.50 USD.

```
Prompt / Ghi chú

1000 request × 0.0185 USD = 18.50 USD
0.925 USD × 20           = 18.50 USD
18.50 USD × 25000        = 462500 VND, chưa phí
```

Như vậy, với `20000/5000` token input/output mỗi request, **×50 cho 0.925 USD = 23125 VND**, còn **×1000 cho 18.50 USD = 462500 VND**. Tính tuyê´n tính đúng vì ta giữ đơn giá và cơ câ´u cô´ định. Dịch vụ thật có thể đổi bậc giá, model, chê´ độ hoặc phí công cụ; khi đó phải lập lại bảng.

Để so sánh, một **thuê bao GIẢ ĐỊNH 20 USD/tháng** tương đương **500000 VND/tháng** theo cùng tỷ giá. Chưa thể kê´t luận API tương đương nhưng rẻ hơn: quota, tính năng và điê`u kiện có thể khác. Nê´u Minh không câ`n mua thuê bao, chi tiê`n cho API không tự trở thành "tiê´t kiệm". Phương án 0 VND phải có mặt trong bảng lựa chọn.

Những token không nă`m trong câu bạn vừa gõ

Lan tưởng input chỉ là "sửa hàm tổng". Ứng dụng có thể gửi thêm brief, lịch sử, tệp và **tool output**, kê´t quả công cụ như log test, vào lượt kê´ tiê´p. Lịch sử đã gửi không mặc định miễn phí; cache và lưu phiên có cách tính riêng.

Một sô´ model ghi nhận/tính phí **reasoning token**, token xử lý suy luận. Cách tính vào usage, output hoặc giới hạn tùy model/provider, không tự nă`m trong ví dụ này. Dự toán thật câ`n bộ đê´m/ước tính phù hợp rô`i đô´i chiê´u usage khi được phép dùng, không dựa vào đê´m từ.

Agent có thể đọc mã → sửa → test → đọc lỗi → lặp lại. "Sửa cho xong" có thể sinh nhiê`u request. Lan giới hạn một hàm, một đê` xuâ´t, đâ`u ra đủ kiểm tra và điểm dừng để xem lại. Khi bă´t đâ`u lượt mới, cô dùng brief cập nhật, không bỏ ràng buộc để tiê´t kiệm.

Budget alert không phải hard cap

Budget alert thông báo đạt ngưỡng; **hard cap** phải có cơ chê´ chặn dùng/chi theo phạm vi dịch vụ định nghĩa. Email ở 80% không chứng minh dừng ở 100%; sô´ ghi trong ô "budget" không tự là câ`u dao.

Bản nguô`n dẫn Gemini API billing và Google Cloud Budgets, **mô´c 29/10/2026, chưa xác minh độc lập, câ`n tra lại**. Không mặc định mọi dịch vụ có cùng nút/cơ chê´. Kể cả cap được hỗ trợ, độ trễ, request

đang xử lý và ngoại lệ có thể ảnh hưởng mức vượt.

Chưa hiểu thì chưa bật thanh toán. Nếu sau này được phép dùng, cần biết nơi xem usage, quota/giới hạn thực có, số bước agent, cách dừng và xử lý khóa. Prompt "đừng tiêu quá" không thay cơ chế tài khoản. Bài này tính trên giấy, không bật thanh toán để thử cảnh báo.

1.4. FREE FIRST — Học trước, chỉ mua khi biết mình thiếu gì

Quyền không mua là một phần của kế hoạch

Nhìn thẻ A, Minh thấy mình cần giấy, kiến thức vòng lặp và có thể trình biên dịch sẵn có, chưa cần agent sửa kho mã. Lan cũng học được Web cục bộ không tích hợp AI. **Free first** ưu tiên phương án không thu phí đủ dùng, không bỏ qua điều kiện dịch vụ.

Hết quota thì chia nhỏ câu hỏi, chờ chu kỳ sau, đọc tài liệu hoặc nhờ bạn. Không lách giới hạn bằng nhiều tài khoản, dùng key nhặt được hay thử thanh toán vì tò mò. Chưa có công cụ hợp lệ thì luyện bằng thẻ HƯ CẤU.

Trước nâng cấp, điền: "Tôi thiếu _____; gói giải quyết bằng _____; tôi đo lợi ích qua _____". Chưa điền được thì chưa có lý do rõ để mua. Trích vài trang đôi khi đủ thay việc mua context lớn.

> **Quyền không mua.** Khả năng trả tiền không đo năng lực CNTT. Đây không phải chuẩn đầu ra, yêu cầu chi tiêu hay quy định TLU. Ngân sách AI bằng 0 vẫn hợp lệ.

300000 VND cho 15 tuần: trần kế hoạch, không phải cap kỹ thuật

Lan thử một kế hoạch **GIẢ ĐỊNH 300000 VND cho học kỳ 15 tuần**. Đây không phải khuyến nghị mức chi cho mọi sinh viên. Mục đích là thấy một quyết định hàng tháng có thể vượt kế hoạch cả kỳ.

Giai đoạn GIẢ ĐỊNH	Dự trù	Điều kiện để quyết định
Tuần 1–5: học nền tảng	0 VND	Tài liệu và công cụ miễn phí hợp lệ đủ dùng
Tuần 6–10: một nhu cầu rõ	Tối đa 100000 VND	Có lợi ích đo được, đủ điều kiện dùng
Tuần 11–15: hoàn thiện bài	Tối đa 100000 VND	Đánh giá lại nhu cầu, không gia hạn theo quán tính

Giai đoạn GIẢ ĐỊNH	Dự trù	Điề̀u kiện để quyế́t định
Dự phòng	100000 VND	Có thể giữ nguyên, không phải mục tiêu phải tiêu hế́t
Tổng trả̀n cá nhân	300000 VND	Kế́ hoạch trên giấ́y, không chặn hóa đơn tự động

Thuê bao giả định 500000 VND/tháng vượt cả kế́ hoạch; nế́u 1000 request dồ̀n trong cùng học kỳ, 462500 VND chưa phí cũng vượt trả̀n 300000 VND. Mức 23125 VND ban đầ̀u chưa chứng minh Lan cầ̀n API, đủ tuổi hay quản lý khóa được.

Trước chi, ghi nhiệm vụ, phương án miễn phí đã thử, trả̀n tiề̀n và ngày đánh giá lại. Biế́t ngày/cách dừng tự gia hạn **trước khi mua**, không giả định nút hủy giố́ng nhau. Sau một tuầ̀n, đo lỗi đã hiểu hoặc thao tác tiế́t kiệm, không đế́m câu AI sinh.

Bổ sung biên tập — cuố́i tuầ̀n thứ năm. Lan chưa thấ́y chức năng cầ̀n trả tiề̀n. Giữ 100000 VND hay mua để "tạo động lực"? Ghi tiêu chí đánh giá lại.

▼ Đáp án kế́ hoạch — Tiề̀n chưa tiêu không phải phầ̀n học bị thiế́u · mở sau khi tự thử

Giữ dự trù; đánh giá lại sau khi làm Đề̀ B (mục 1.5) bằ̀ng context đủ dùng, giữ phạm vi và khả năng tự kiểm. Chỉ mua nế́u giải quyế́t hạn chế́ thật, đủ điề̀u kiện, trong ngân sách. Không cầ̀n tiêu hế́t dự phòng.

Ưu đãi sinh viên: đủ điề̀u kiện nộp không có nghĩa chắ́c chắ́n nhận

Bản nguồ̀n dẫn hướng dẫn GitHub Education với các điề̀u kiện như đang theo học thuộc diện đủ điề̀u kiện, có tài khoản cá nhân, chứng minh tình trạng sinh viên và **từ 13 tuổi**. Bản nguồ̀n cũng ghi quyề̀n lợi GitHub Pro và Copilot Student cho người học đã xác minh theo điề̀u kiện. **Đây là nội dung được ghi nhận theo bộ nguồ̀n ngày 29/10/2026, chưa được kiểm chứng độc lập trong lầ̀n biên tập này; tên gói, tuổi, quyề̀n lợi và điề̀u kiện phải tra lại.**

Không dùng một hướng dẫn cũ để tự đổi tên thành "Copilot Pro hiện tại", không chố́t quota từ trí nhớ. **Completion** là gợi ý hoàn thành mã; **chat** là trao đổi; **agent** có thể tổ chức nhiề̀u bước. Quyề̀n lợi ở một hình thức không bảo đảm mọi hình thức dùng đề̀u vô hạn. Chương trình ưu đãi có thể gồ̀m nhiề̀u bên với điề̀u khoản riêng, không mặc định cấ́p API key miễn phí cho mọi dịch vụ.

Để đọc một ưu đãi, Lan tách năm câu hỏi: ai đủ điều kiện; cần bằng chứng nào; quyền lợi cụ thể là gì; kéo dài bao lâu; hết ưu đãi hoặc đến kỳ gia hạn thì điều gì xảy ra. Email trường có thể hỗ trợ chứng minh, không phải vé tự được duyệt. Email TLU không tự bảo đảm hồ sơ thành công.

Bài tập chỉ yêu cầu đọc thông tin, không nộp hồ sơ hoặc tải giấy tờ. Nếu sau này đăng ký, dùng quy trình chính thức và chỉ cung cấp giấy tờ cần thiết ở đúng nơi. Không gửi giấy tờ vào chatbot hoặc nhóm lớp để "nhờ duyệt hộ".

Tuổi, dữ liệu và tài khoản phải được xét trước giá

Điều kiện tuổi của một chương trình không áp dụng tự động cho chương trình khác. Bản nguồn ghi điều khoản Gemini API yêu cầu **18 tuổi trở lên** với Gemini API và Google AI Studio. **Claim này mang mốc nguồn 29/10/2026, chưa xác minh độc lập; cần đọc lại điều khoản chính thức trước khi dùng.** Không suy từ mốc 13 của một chương trình giáo dục rằng đủ điều kiện dùng mọi API. Khi chưa đáp ứng hoặc chưa rõ điều kiện, chọn bài trên giấy, không tìm cách vượt giới hạn.

Cũng theo nội dung bản nguồn dẫn điều khoản Gemini API ở mốc đó, dịch vụ unpaid thuộc phạm vi áp dụng có thể dùng nội dung để cải thiện sản phẩm và có người xem xét; paid có nhiều trường hợp không dùng nội dung để cải thiện nhưng có thể tồn tại chương trình/tier đóng góp dữ liệu. **Các chi tiết này chưa xác minh độc lập và không được trình bày như cam kết hiện hành.** Bài học bền vững là phải xem đúng sản phẩm, cấp dịch vụ, lựa chọn dữ liệu và ngoại lệ.

Không lấy điều khoản API gán nguyên cho ứng dụng chat. Không rút thành "trả phí là riêng tư tuyệt đối", "không training là không lưu" hoặc "mọi provider giống nhau". Với ViSinhVien, chỉ dùng ba khoản GIẢ; không có lý do đưa sao kê, số tài khoản hoặc lịch sử chi tiêu thật lên AI. `localStorage` cũng không phải kho bí mật hay sao lưu chắc chắn: dữ liệu có thể mất khi xóa dữ liệu trình duyệt, không tự đồng bộ thiết bị hoặc tạo đăng nhập.

Nếu dùng dịch vụ, cần tài khoản hợp lệ, mật khẩu riêng và xác thực nhiều yếu tố khi được hỗ trợ. Không mua tài khoản lậu hoặc chia sẻ thuê bao/key mà bỏ qua điều khoản và quyền riêng tư. Người bán hoặc người dùng chung có thể đọc lịch sử, thu hồi quyền, làm phát sinh phí. Giá rẻ không bù được việc mất kiểm soát; điều kiện sử dụng phải được kiểm tra trước phép so sánh giá.

1.5. AI TOOL BATTLE — Đối chiếu công cụ theo việc, không theo danh tiếng

Ma trận tác vụ: "hợp việc nào?" đứng trước "mạnh nhất?"

Minh thích thẻ A vì được tự suy nghĩ; Lan thích công cụ có thể chỉ đúng tệp liên quan. Không có mâu thuẫn: hai người đang ưu tiên tác vụ khác nhau. Giỏi dịch không tự chứng minh giỏi kiểm tra code; viết mã nhanh

không chứng minh biết chính sách mới.

Năm lăng kính để chọn là **provenance** — nguồn gốc/căn cứ; **test** — cách kiểm tra; **context** — thông tin cần nhận; **privacy** — điều kiện dữ liệu; **cost** — tổng chi phí. "Bằng chứng" của debug có thể là dòng code và test; "bằng chứng" của tuổi dùng dịch vụ phải là điều khoản đúng phạm vi.

Tác vụ	Context cần có	Provenance và test	Privacy và cost
Suy luận logic	Code, chỉ số, mục tiêu cộng mọi khoản của Minh	Bám cận vòng lặp; tự xét rỗng, một khoản, ba khoản	Dữ liệu GIẢ; gợi ý ngắn trên phương án free phù hợp
Code hoặc refactor	Brief, hàm liên quan, cấu trúc Expense của Lan	Đọc thay đổi; kiểm mẫu, biên và dữ liệu sai; không thêm backend	Không key; xem quyền đọc tệp, số bước và phí lặp
Đọc tài liệu dài	Đặc tả và đoạn ghi chú được phép	Trích đúng đoạn, tìm mâu thuẫn, so brief hiện hành	Không gửi tài liệu hạn chế; trích đoạn trước khi mua context lớn
Dịch và diễn giải	Đoạn lỗi/tài liệu, trình độ người học	Giữ tên API, số liệu và phủ định; không đổi số nguyên thành số thực	Loại thông tin riêng; không cần agent nhiều bước
Tìm thông tin mới	Câu hỏi về ưu đãi/giá, đúng sản phẩm và phạm vi	Nguồn chính thức hiện hành, đoạn hỗ trợ, ngày truy cập	Không gửi giấy tờ; xét phí tìm kiếm nếu có

Ma trận không xếp hạng sản phẩm thật. Không có web thì không dùng làm nguồn duy nhất cho chính sách hiện tại. Không chạy code vẫn có thể gợi ý, nhưng cần kiểm thử ở nơi khác. Chọn công cụ có phần việc mình kiểm chứng được, thay vì giao hết chỉ vì nó nhận làm.

Benchmark A–C: chuẩn bị một phép thử công bằng vừa đủ

Benchmark là phép thử có kiểm soát để so mức phù hợp. Ba đề kế thừa nguồn, **không có điểm sản phẩm điền sẵn**. Không có công cụ được phép dùng thì phân tích trên giấy, ghi là luyện đánh giá, không phải benchmark thực.

Giữ cùng brief, code, dữ liệu, đầu ra và giới hạn trợ giúp; nên mở trao đổi mới để giảm ảnh hưởng lịch sử. Ghi ứng dụng, nhãn model/chưa xác định, chế độ, ngày, tính năng bật. Khác biệt web/chạy code phải được ghi riêng, không quy tất cả cho model.

Làm theo thứ tự **gửi đề → giữ nguyên câu trả lời → tự kiểm → chấm → ghi hỏi bổ sung và giới hạn kết luận**. Có điều kiện thì thử lại vì đầu ra có thể đổi. Vài đề không chứng minh "tốt nhất cho mọi người".

Đề A — Giải thích logic C++ cho Minh

Dán **nguyên snippet đầu chương và brief mục 1.2**; chữ "xem đầu chương" không tự gửi tài liệu cho ứng dụng.

Prompt / Ghi chú

```
Với snippet C++ đính kèm, hãy gợi ý một bước trước khi nêu cách sửa.
Sau đó giải thích bằng chỉ số vì sao tổng ba khoản GIẢ bị thiếu.
Nêu kết quả mong đợi sau sửa cho: rỗng; một khoản 7000;
ba khoản 35000, 7000, 18000. Không truy cập quá danh sách.
Phân biệt dự đoán với kết quả đã chạy; chưa viết lại toàn bộ bài.
```

Chấm cả độ đúng và cách giúp học: viết ngay toàn bài có thể đúng kết quả nhưng sai mức trợ giúp. Lời "đã chạy" cần bằng chứng.

Đề B — Refactor Web mà không đổi sản phẩm

Dùng brief và **bản rút gọn cho đề bài** dưới đây, chỉ cộng khi đầu vào hợp lệ. Không khẳng định demo `data.js` thiếu kiểm tra; không thay snippet vào demo đúng.

JavaScript

```
function totalExpenses(expenses) {
  return expenses.reduce((sum, expense) => sum + expense.amount, 0);
}
```

Prompt / Ghi chú

```
Hãy đề xuất cách bổ sung xác thực cho bản rút gọn đính kèm.
Expense có id, title, amount, category; amount là số nguyên VND,
0 < amount <= 1000000000; danh sách tối đa 500 khoản; rỗng tổng 0.
Giữ HTML + JavaScript + localStorage, không backend hoặc đăng nhập.
Phân biệt kiểm tra khi thêm, khi nạp và trách nhiệm hàm tính tổng.
Nêu cách xử lý JSON hỏng và các test; chưa thêm thư viện hay tính năng.
```

Chấm hợp đồng và hành vi, không thưởng số lớp hoặc tên hàm giống mẫu. Kiểm form rồi tin mọi dữ liệu đọc lại là chưa đủ.

Đề C — Đọc và kiểm tra nguồn cho Lan

Cung cấp brief và đoạn dữ kiện sau, kèm liên kết để người trả lời biết giới hạn nguồn:

Prompt / Ghi chú

```
ĐOẠN ĐỐI CHIẾU TỪ BẢN NGUỒN, MỐC 29/10/2026:
GitHub Education yêu cầu đáp ứng điều kiện người học, tài khoản cá nhân
và bằng chứng tình trạng sinh viên; bản nguồn ghi tuổi tối thiểu 13.
Email trường không tự bảo đảm được duyệt. Chưa xác minh độc lập lại.
Nguồn: https://docs.github.com/en/education/explore-the-benefits-of-teaching-and-learning-with-github-education/github-education-for-students/apply-to-github-education-as-a-student

Hãy tóm tắt cho Minh, phân biệt đủ điều kiện nộp với chắc chắn được duyệt.
Email TLU có tự bảo đảm ưu đãi không? Đoạn này cho phép kết luận gì
về quyền sử dụng một API khác? Nếu không truy xuất web thực,
chỉ phân tích đoạn được cung cấp, không tuyên bố đã kiểm tin mới.
```

Đề C cố ý có mốc cũ và giới hạn xác minh. Một câu trả lời tốt không "làm mới" tài liệu bằng cách gắn ngày hiện tại. Nếu có truy xuất thật, phải ghi trang đã đọc và claim được hỗ trợ; việc đó là bằng chứng bổ sung, không xóa mốc của đoạn được cung cấp.

▼ Đáp án benchmark A–C — Chuẩn đối chiếu, không phải kết quả đo · mở sau khi tự thử

A: `i < n - 1` bỏ khoản cuối; sửa thành `i < n`, không dùng `i <= n`. Ba kỳ vọng sau sửa là **0, 7000, 60000**, không phải log. Ca một khoản phơi lộ việc bỏ mất khoản duy nhất; rỗng một mình chưa đủ. Trace/thực thi xem mục 2.3; validation là nhiệm vụ riêng.

B: kiểm cấu trúc danh sách, tối đa 500 khoản; từng `amount` thỏa `Number.isSafeInteger`, lớn hơn 0, không quá 1000000000. Kiểm khi thêm và nạp lại; xử lý JSON hỏng có thông báo, không âm thầm mất dữ liệu. Có thể xác thực ở biên hoặc đầu hàm tổng nhưng phải rõ hợp đồng. Không thêm backend/tài khoản/thư viện lớn. Test logic chưa kiểm chứng giao diện/lưu trữ.

C: email TLU không bảo đảm duyệt; điều kiện và bằng chứng khác kết quả xét hồ sơ. Đoạn nguồn không chứng minh quyền mọi API hay điều khoản hiện tại. Không truy xuất mới thì giữ mốc gốc, ghi cần tra lại; không suy tuổi sang dịch vụ khác.

Rubric 10 điểm: dùng sau khi có bằng chứng

Rubric là bảng tiêu chí chấm. Chấm từng đề A, B hoặc C theo năm tiêu chí, mỗi tiêu chí 0–2; tổng tối đa **10 điểm cho một đề**. Đây là cách chuyển năm lăng kính thành đánh giá đầu ra, không phải thuật toán xếp hạng công cụ.

Tiêu chí	0 điểm	1 điểm	2 điểm
Đúng trọng tâm	Sai bài hoặc sai kết luận chính	Đúng một phần, thiếu quan trọng	Đúng tác vụ và kết quả chính
Giữ ràng buộc/context	Bỏ hoặc tự đổi yêu cầu	Giữ phần lớn, cần nhắc	Giữ yêu cầu liên quan, nhận ra dữ kiện thiếu
Bằng chứng/provenance	Bịa nguồn, bịa chạy test hoặc không căn cứ	Có căn cứ nhưng chưa rõ/đủ	Suy luận, test hoặc nguồn phù hợp; nêu giới hạn
Dễ học và kiểm tra	Khó hiểu, không có cách tự kiểm	Hiểu được một phần	Vừa sức, giúp tự làm bước tiếp
An toàn và tiết kiệm	Đòi dữ liệu thật/key hoặc mở rộng tốn kém	Có nhắc nhưng chưa cụ thể	Giữ dữ liệu GIẢ, phạm vi nhỏ, tránh chi phí vô ích

Một đề nghị đưa key bí mật vào Web phải bị loại khỏi phương án triển khai đến khi sửa, dù tổng điểm khác cao. Điểm không bù được vi phạm điều kiện bắt buộc. Nếu chưa kiểm nguồn hoặc chưa chạy code, ghi phần đó là chưa kiểm, không tự cộng điểm như đã có bằng chứng.

Phiếu benchmark để trống

Trường ghi nhận	Kết quả của bạn
Thử công cụ thật hay luyện đánh giá trên giấy?	_____
Ứng dụng, model công bố, provider biết được, chế độ	_____
Ngày thử, nguồn nhận diện, tính năng bật	_____
Brief, code và dữ liệu dùng chung	_____
Điểm A /10; B /10; C /10	_____
Căn cứ cho điểm và phần chưa kiểm	_____
Thời gian, số lượt hỏi lại	_____
Chi phí quan sát được hoặc “chưa xác định”	_____
Chọn cho tác vụ nào; không chọn cho tác vụ nào	_____

Chỉ chấm khi có bằng chứng; bài trên giấy không là kết quả model. Không phân bổ thuê bao thành giá request tùy tiện. Minh ưu tiên đúng, dễ hiểu, miễn phí hơn nhiều mã không giải thích nổi.

1.6. MINI CHALLENGE — 20 phút để nhận diện, ước chi và quyết định

Chuẩn bị một tờ giấy, không chuẩn bị thẻ thanh toán

Lab kế thừa nguồn, làm cá nhân/theo cặp bằng sách và giấy. Không đăng ký, cài đặt, gọi API, tạo key, tải giấy tờ hay dùng chi tiêu thật; không cần tài khoản AI.

Tình huống GIẢ ĐỊNH: Minh cần hiểu lỗi 42000; Lan kiểm Web cục bộ và thông tin ưu đãi. Ứng dụng GIẢ "Trợ lý học ViSinhVien" chỉ công bố "Tự động", không nêu model từng lượt/provider. Web vẫn HTML, JavaScript, `localStorage`, không backend. Dùng chi phí giả định mục 1.3.

Nộp một phiếu gồm nhận diện, prompt, phép tính và quyết định có điều kiện. Không cần làm hết benchmark A–C trong lab; đây là nhiệm vụ ghép kỹ năng, không phải cuộc thi tốc độ.

Lịch 20 phút: làm từng bước vì một lý do

Phút 0–4 — Nhận diện. Điền application/model/provider, thêm producer nếu biết. Gạch dữ kiện, khoanh điều chưa rõ để tránh đoán model từ giao diện. Quan sát công cụ thật thì chỉ đọc thông tin được phép, ghi nguồn/ngày, không đổi cài đặt.

Phút 4–8 — Viết prompt. Đưa mục tiêu, code, hợp đồng, ba khoản và cặp 60000/42000; xin gợi ý trước. Đọc như người chưa biết dự án để phát hiện thiếu context. Gửi nơi khác phải dán snippet thật.

Phút 8–13 — Ước chi. Tách input/output cho 50 request, đổi USD/VND, rồi xét 1000 request. Kiểm chéo từng request để bắt lỗi đơn vị; giữ nguyên số giả định để đối chiếu.

Phút 13–17 — Quyết định. Dùng ít nhất ba tiêu chí ma trận, có phương án không trả tiền, cách kiểm chứng và điều kiện đánh giá lại. Sở thích không thay thế bằng chứng.

Phút 17–20 — Đối chiếu. Sửa phiếu nhưng giữ dấu lỗi ban đầu để nhận ra điều vừa học. Nói trong 30 giây: biết/chưa biết gì, chọn gì, kiểm thế nào, chi tối đa bao nhiêu.

Worksheet — Điề̀n trước khi mở đáp án

Ô cầ̀n điề̀n	Câu trả lời của bạn
Ứng dụng đang xét	_____
Model cụ thể hoặc chế́ độ được công bố́	_____
Provider của đường truy cập	_____
Producer, nế́u có căn cứ	_____
Nguồ̀n và ngày quan sát, hoặc “tình huố́ng GIẢ ĐỊNH”	_____
Điề̀u chưa đủ chứng cứ để kế́t luận	_____
Mục tiêu và kế́t quả đúng của ViSinhVien	_____
Hiện tượng được đề̀ cho; test đầ̀u tiên	_____
Dữ liệu không được gửi	_____

Prompt / Ghi chú

```
Tôi đang học: _____
Mục tiêu và phạm vi: _____
Dữ liệu GIẢ, hợp đồng và code cần dán: _____
Kết quả mong đợi / hiện tượng theo đề: _____
Hãy giúp theo cách: _____
Tôi sẽ tự kiểm bằng: _____
Phần chưa chạy hoặc chưa xác minh: _____
```

Phép tính bắ́t buộc	Kế́t quả kèm đơn vị
20000 × 50	_____ token input
5000 × 50	_____ token output
Tiề̀n input theo đơn giá giả định	_____ USD
Tiề̀n output theo đơn giá giả định	_____ USD
Tổng và quy đổi ở 50 request	_____ USD = _____ VND
Chi phí một request để kiểm tra chéo	_____ USD
Tổng ở 1000 request, bằ̀ng 20 lầ̀n ban đầ̀u	_____ USD = _____ VND
Khoản chưa tính trong ví dụ	_____

Phép tính bắt buộc	Kết quả kèm đơn vị
So với kế hoạch 300000 VND, chưa phí	_____

Hoàn thành quyết định có điều kiện:

```
Prompt / Ghi chú

Với Minh, tôi chọn _____ vì _____.
Với Lan, tôi chọn _____ vì _____.
Ba tiêu chí tôi dựa vào là _____, _____, _____.
Phương án không trả tiền của tôi là _____.
Tôi chưa mua _____; mức chi tôi chấp nhận lúc này là _____.
Tôi sẽ cân nhắc lại khi _____, sau khi kiểm tra _____.
Nếu điều kiện tuổi chưa phù hợp hoặc chưa rõ, tôi sẽ _____.
Nếu có nguy cơ vượt ngân sách, tôi sẽ _____.
```

▼ Đáp án MINI CHALLENGE — Một phiếu hoàn chỉnh có điều kiện · mở sau khi tự thử

Nhận diện: ứng dụng Trợ lý học ViSinhVien, chế độ Tự động; model/provider/producer chưa xác định. Nguồn là tình huống GIẢ ĐỊNH, không phải quan sát dịch vụ; không tự ghi ngày truy cập thật.

Mục tiêu: cộng mọi khoản hợp lệ; ba khoản GIẢ kỳ vọng 60000, lỗi đề cho 42000. Rỗng tổng 0; thử một khoản 7000 rồi ba khoản để kiểm lỗi bỏ cuối. Không gửi key, giấy tờ, sao kê, tài khoản hoặc dữ liệu cá nhân. Chưa chạy thì ghi kỳ vọng, không báo cáo test.

Prompt mẫu:

```
Prompt / Ghi chú

Tôi là Minh năm nhất, mới học vòng lặp C++.
ViSinhVien cộng tất cả khoản hợp lệ: amount nguyên VND,
0 < amount <= 1000000000, tối đa 500 khoản; rỗng tổng 0.
Dữ liệu GIẢ: Cơm trưa 35000, Xe buýt 7000, Vở ghi 18000.
Kỳ vọng 60000; tình huống đề cho chương trình kết thúc ở 42000.
Tổng tối đa 500000000000, không đổi sang kiểu không chứa đủ.
Tôi dán snippet lỗi ngay sau yêu cầu này.
Hãy gợi ý một bước bằng chỉ số, chưa viết lại chương trình.
Sau đó nêu test rỗng, một khoản 7000, ba khoản mẫu.
Phân biệt suy luận với chạy thật; thiếu dữ kiện thì chỉ rõ.
```

Khi dùng mẫu này, phải dán kèm snippet đầu chương; câu "tôi dán" không tự đính kèm code. Nếu đã chạy thật, có thể thay dòng hiện tượng bằng lệnh và log của chính mình, không mượn kết quả dự đoán làm log.

Phép tính: tổng input 1000000 token, output 250000 token; tiền tương ứng 0.30 và 0.625 USD. Tổng **0.925 USD = 23125 VND chưa phí**. Một request **0.0185 USD**, nhân 50 khớp. Với 1000 request: input 20000000 token, output 5000000 token; tiền 6.00 + 12.50 = **18.50 USD = 462500 VND chưa phí**. Mức sau vượt kế hoạch giả định 300000 VND; cảnh báo không tự bảo đảm chặn hóa đơn. Thuê bao giả định 20 USD là 500000 VND/tháng nhưng chưa tương đương quyền lợi API.

Minh: dùng giấy/tài liệu hoặc gợi ý miễn phí hợp lệ; tự sửa cận trên bài làm, kiểm rỗng/một/ba khoản ở mục 2.3. Chọn theo context đủ dùng, test kiểm được, cost 0; dữ liệu GIẢ giữ privacy. Không cần API.

Lan: giữ Web cục bộ, kiểm đầu vào/tổng rồi kiểm riêng hiển thị/lưu/nạp. Ưu đãi cần nguồn chính thức; chưa truy xuất thì ghi chưa xác minh, chưa mua. Không tự tích hợp API.

Đánh giá lại khi free chưa đáp ứng nhu cầu rõ, đã tra quyền lợi/tuổi/dữ liệu, quản lý được chi phí và biết cách dừng gia hạn. Chưa đủ điều kiện hoặc chưa hiểu kiểm soát tiền thì tiếp tục trên giấy/miễn phí. Mức chi hiện tại có thể là 0 VND.

QUICK CHECK — Sáu câu tự kiểm cuối chương

Trả lời từng câu bằng một kết luận và một lý do. Đừng chỉ nhớ chữ "không"; hãy chỉ ra loại bằng chứng còn thiếu.

1. Nhãn "tích hợp Claude" đã đủ xác định provider và phiên bản chưa?
2. Cùng model nhưng một ứng dụng đề xuất backend cho ViSinhVien: có chắc quảng cáo sai không?
3. Danh sách rỗng có vi phạm `amount > 0` không? Có cần tạo một khoản 0 để biểu diễn rỗng không?
4. Đếm số từ tiếng Việt có đủ tính chính xác hóa đơn token không?
5. Email TLU có bảo đảm ưu đãi sinh viên và quyền dùng mọi API không?
6. Trả phí và bật budget alert đã đủ bảo đảm riêng tư, không vượt ngân sách chưa?

▼ Đáp án QUICK CHECK — Sáu kết luận và lý do · mở sau khi tự thử

1. Chưa. Cần định danh model và đường truy cập; producer có thể khác provider như ví dụ Claude qua Bedrock. Không đoán qua văn phong.

2. Chưa thể kết luận quảng cáo sai. Kiểm brief thực sự gửi, lịch sử, chỉ dẫn, công cụ và cấu hình. Backend sai phạm vi đã chốt, nhưng khác đầu ra chưa chứng minh khác model.

3. Không vi phạm, không cần khoản 0. Điều kiện dương áp dụng cho khoản tồn tại. Rỗng có tổng 0; tạo `amount = 0` lại vi phạm hợp đồng.

4. Không. Token tùy tokenizer/nội dung; usage có thể gồm lịch sử, code, tool output và khoản tính riêng. Cần bộ đếm phù hợp và báo cáo lượng dùng, không dùng wordcount.

5. Không. Còn điều kiện, bằng chứng sinh viên và xét hồ sơ. Tuổi/quyền dùng từng API phải tra riêng; mốc nguồn chưa phải xác nhận hiện hành.

6. Chưa. Kiểm chính sách đúng sản phẩm/tier/cài đặt. Budget alert không mặc định là hard cap; cần hiểu cơ chế chặn, phạm vi, độ trễ và request đang xử lý.

Tự giảng lại trong 7 phút — Bản chuyển thể từ Speaker Notes

Phần này không thay lab 20 phút. Bạn có thể nói cho bạn học hoặc tự ghi âm mà không mở giao diện dịch vụ. Đây là cách giữ nội dung trình bày 5–8 phút của bản nguồn trong hình thức tự học.

Thời gian	Việc trình bày	Điều cần chứng minh
0:00–1:00	Kể Minh, Lan, ba khoản GIẢ; phân biệt 42000 và 60000	Tự cộng đúng, biết tổng trung gian chưa tự là lỗi
1:00–2:15	Vẽ application, provider, model; hỏi thêm producer	Không đồng nhất ứng dụng với model, chấp nhận chưa biết
2:15–3:30	Sửa prompt thiếu; nhắc hợp đồng và chỉ số cuối	Không đổi thành cận vượt mảng; chưa cần kể lại toàn trace
3:30–4:45	Tính riêng input/output, 50 rồi 1000 request	Giữ đơn vị, số giả định và nhãn chưa phí
4:45–6:00	Chọn free first; nhắc tuổi, ưu đãi, gia hạn, dữ liệu	Không biến email trường hoặc trả tiền thành bảo đảm
6:00–7:00	Hỏi kiểm code bằng gì, cảnh báo có dừng không	Nêu được bằng chứng và điều chưa xác minh

Có 5 phút thì giảm hỏi đáp, không bỏ nhãn giả định; có 8 phút thì thêm một quyết định có điều kiện. Không trình diễn đăng nhập, thanh toán hoặc giấy tờ để làm bài nói sinh động hơn.

SKILL UNLOCKED — Bạn mang gì sang chương tiếp theo?

Phiếu hoàn thành cần thể hiện: nhận diện vai trò, chọn context, phân biệt suy luận/thực thi/tìm nguồn, tính input/output và quyết định chi tiêu. Đây là bằng chứng một phiên học, không chứng nhận năng lực lâu dài.

Ba câu mang theo:

- **Biết ứng dụng, đường truy cập và model được công bố; thiếu chứng cứ thì ghi chưa xác định.**
- **Đưa đúng mục tiêu/context, kiểm bằng phép tính, test hoặc nguồn; sự tự tin không phải bằng chứng.**
- **Ưu tiên miễn phí đủ dùng; tính lượng dùng và điều kiện trước trả tiền. Không mua vẫn học nghiêm túc.**

Sang **Chương 2 · Trở thành người điều khiển AI**, Minh làm trace/sửa/test ở mục 2.3; Lan theo dữ liệu Web ở mục 2.4. **Chương 3 · Đừng để AI suy nghĩ thay bạn** giữ phần tự làm; **Chương 4 · Sử dụng AI có trách nhiệm** xét quyền, dữ liệu và trách nhiệm.

Nguồn chính thức và việc còn phải tra lại

29/10/2026 là mốc truy cập của bộ nguồn được cung cấp, không phải ngày phát hành hoặc ngày kiểm tra của lần biên tập này. Chưa xác minh độc lập nội dung hiện hành; cần tra lại model, gói, credit, quota, tuổi, dữ liệu và billing trước khi áp dụng. Giữ nguyên 11 liên kết chính thức gốc:

Nội dung đối chiếu	Nguồn chính thức
Họ model, định danh	Anthropic — Models overview
Claude qua provider khác	AWS — Claude in Amazon Bedrock
API, xác thực, chỉ định model	Google — Gemini API docs
Lớp ứng dụng Gemini	Google — Gemini overview
Tuổi, điều kiện, bằng chứng sinh viên	GitHub — Apply to GitHub Education as a student
Quyền lợi, thời hạn, giới hạn	GitHub Student Developer Pack
Free/paid, đơn vị giá, phụ phí	Google — Gemini API pricing
Tuổi, sản phẩm/tier, xử lý dữ liệu	Google — Gemini API terms
Cơ chế kiểm soát mức dùng	Google — Gemini API billing
Ngân sách và cảnh báo	Google Cloud — Budgets

Nội dung đô´i chiê´u	Nguô`n chính thức
Credit API theo điê`u kiện gói	OpenAI — ChatGPT/API credits

Giá **0.30/2.50 USD mỗi triệu token**, thuê bao **20 USD**, tỷ giá **25000 VND/USD**, ngân sách **300000 VND** đê`u GIẢ ĐỊNH, không lâ´y từ bảng giá thật. Thẻ hư câ´u không phải benchmark; bảng kỳ vọng không phải log. Kiểm chứng thật phải ghi riêng công cụ/lệnh/nguô`n, ngày và kê´t quả.

02

CHƯƠNG 02 / MINH & LAN

Trở thành người điê`u khiển AI

Giải cứu ViSinhVien bă`ng bảy nhiệm vụ: đọc, hỏi, sửa, kiểm và tự nghiệm thu.



Tình huố´ng và nhân vật hư cấ´u · Đọc, dự đoán rô`i tự kiểm chứng. Xem hình lớn

Minh đã biế´t không nên chọn câu trả lời chỉ vì nó tự tin. Nhưng chương trình C++ vẫn hiện **42000**, trong khi ba khoản trên giấ´y cộng thành **60000**. Lan mở bản Web để cùng kiểm tra: bảng đã có đủ ba khoản, tổng đúng; khi thử nhập `35000abc`, cô muố´n biế´t chương trình thực sự từ chố´i ở đâu. Một trợ lý lại đề` nghị cài thêm thư viện để "xử lý tiê`n chuyên nghiệp". Lâ`n này, hai bạn không hỏi "AI nào giỏi nhấ´t?" mà hỏi: **việc gì cầ`n làm, công cụ được làm đế´n đâu, bă`ng chứng nào đủ để nhận kế´t quả?**

Bổ sung biên tập — hành trình, nhân vật và mọi hội thoại trong chương là tình huố´ng HƯ CẤ´U, không phải log tài khoản Qoder, benchmark hay khảo sát TLU. Minh năm nhấ´t học C++; Lan năm hai học OOP/DSA/Web/CSDL. Hai phiên bản ViSinhVien phục vụ học tập, chưa nố´i với nhau. Không nhập thông tin sinh viên thật, tài khoản, sao kê hay bí mật vào mã hoặc hội thoại.

Dữ liệu xuyên suố´t là Cơm trưa **35000**, Xe buýt **7000**, Vở ghi **18000**, đơn vị VND nguyên. `42000` đúng cho hai khoản đầ`u, sai nế´u là tổng cuố´i cả ba. Khoản hợp lệ nằ`m trong **1–1000000000**, tố´i đa **500 khoản**, rỗng tổng **0**; tổng cực đại **500000000000**.

Bảy Mission đi từ môi trường đế´n nghiệm thu. Dự kiế´n hai buổi 90 phút, cộng thời gian đọc sâu và làm lại; đây không phải chuẩn tố´c độ hay CLO chính thức. Chọn một nhánh thực hành sâu, vẫn giải thích lỗi trọng tâm nhánh kia.

Phân biệt **demo đúng**, **mã lỗi chủ đích**, **bài làm**, **kỳ vọng**, **kế´t quả đã chạy**. Trace không phải log máy. Thiế´u công cụ vẫn đọc, dự đoán và ghi phầ`n chưa thực hiện; không cầ`n mua dịch vụ hay tự cài trên máy chung.

2.1. Mission 01 — Làm chủ môi trường trước khi giao việc

Mục tiêu và bố´i cảnh: biế´t cửa sổ nào đang làm gì

Mục tiêu là mở đúng gố´c, phân biệt Editor, IDE, terminal, shell, CLI và Agent. **Bố´i cảnh:** Minh gọi cả vùng mã, hội thoại và cửa sổ lệnh là "AI". Nhầ`m vai trò dễ dẫn đế´n lưu sai bản hoặc dùng sai cú pháp shell.

Editor soạn thảo tệp; IDE tích hợp việc mở dự án, tìm mã và tiện ích phát triển. Web chat thường chỉ biế´t ngữ cảnh được cung cấ´p, trừ khi có kế´t nố´i bổ sung. Agent kế´t hợp AI với công cụ, có thể đọc, sửa, chạy lệnh **tùy sản phẩm, chế´ độ và quyề`n**.

Đây là những cách làm bổ sung nhau, không phải bảng xế´p hạng hay lộ trình thay thế´. Công cụ mạnh hơn không tự tạo kế´t luận đúng hơn.

Cách làm	Việc phù hợp trong ViSinhVien	Trách nhiệm không chuyển cho AI
Editor	Đọc và sửa cận vòng lặp	Lưu đúng tệp, hiểu dòng vừa sửa
IDE	Mở dự án, tìm hàm, dùng terminal	Biế´t đang mở thư mục nào và có công cụ gì
Web chat	Hỏi nghĩa của `+=` hoặc một lỗi nhỏ	Cung cấ´p đoạn mã đủ ngữ cảnh, kiểm chứng giải thích
Agent	Tìm vị trí, đề` xuấ´t thay đổi, kiểm thử theo quyề`n	Giới hạn phạm vi, xem diff, đọc bằ`ng chứng

Terminal là cửa sổ nhận và hiển thị văn bản. Shell là chương trình diễn giải lệnh bên trong, chẳng hạn Bash hoặc PowerShell. CLI là giao diện dòng lệnh của chương trình được gọi, ví dụ `g++`, `node`, Qoder CLI. Khi gõ `g++ --version`, shell tìm chương trình `g++`; chương trình in phiên bản; terminal hiển thị. Nế´u không tìm thấ´y lệnh, vấ´n đề` trước mắ´t là môi trường hoặc đường dẫn, chưa phải vòng lặp C++.



Bản đồ môi trường: IDE chứa vùng soạn thảo và terminal; shell diễn giải lệnh gọi CLI; Agent hoạt động theo quyền được cấp. Xem hình lớn

Có Qoder IDE không chứng minh đã có compiler `g++`, runtime Node.js hoặc Qoder CLI. Có Windows cũng không xác định shell: cùng máy có thể mở PowerShell và Git Bash. Hãy đọc tên phiên terminal hoặc cấu hình hiển thị; đừng thử trộn cú pháp cho đến khi một lệnh tình cờ chạy được.

Nhiệm vụ và hướng dẫn: một gốc, hai nhánh bài làm

Nhiệm vụ là chuẩn bị bài làm, giữ nguyên ví dụ. **Hướng dẫn:** mở gốc `AI-Student-Handbook` trong Qoder IDE, không mở toàn ổ đĩa hoặc riêng nhánh con.

Bằng File Explorer/trình quản lý tệp, tạo `bai-lam`, rồi hai thư mục con `nam-1-cpp`, `nam-2-web`. Copy **nội dung** mỗi nhánh tương ứng từ `vi-du` sang `bai-lam`; tránh lồng thêm tầng `nam-1-cpp/nam-1-cpp`. Giữ `vi-du` để đối chiếu.

Dùng chức năng New File của IDE hoặc trình quản lý tệp tạo `ghi-chep-02.txt` trong nhánh bạn học. Đây là tệp tự tạo, không có sẵn; kiểm phần mở rộng để tránh `.txt.txt`.

```
Prompt / Ghi chú
```

```
AI-Student-Handbook/              ← thư mục mở trong IDE và cwd của các lệnh
  vi-du/                          ← tham khảo, không sửa
    nam-1-cpp/
    nam-2-web/
  bai-lam/
    nam-1-cpp/
      loi-bo-sot.cpp
      vi_sinh_vien.cpp
      ghi-chep-02.txt             ← người học tự tạo
    nam-2-web/
      index.html
      data.js
      app.js
      test_data.cjs
      ghi-chep-02.txt             ← người học tự tạo
```

Mở terminal qua menu IDE hoặc tìm **Terminal: Create New Terminal** nếu phiên bản có mục đó; không đoán phím tắt. Bố cục khác thì xem tài liệu terminal.

Cwd là thư mục hiện hành. Mọi lệnh thực hành dùng cwd gốc `AI-Student-Handbook`, đường dẫn `bai-lam/...`. Terminal cũ có thể không chuyển theo thư mục IDE: kiểm vị trí trước build/test.

Nhu cầu	Bash, gồm Git Bash	PowerShell
Xem cwd	`pwd`	`Get-Location`
Liệt kê mục ở cwd	`ls`	`Get-ChildItem`
Chuyển đến gốc đã xác định nếu đang ở sai nơi	`cd "đường/dẫn/AI-Student-Handbook"`	`Set-Location "đường\dẫn\AI-Student-Handbook"`
Đọc mã thoát lệnh ngoài vừa chạy	`echo $?`	`$LASTEXITCODE`

Dòng đổi thư mục là mẫu để **thay đường dẫn thật**, không gõ nguyên chữ “đường/dẫn”. Đặt đường dẫn có dấu cách trong dấu nháy kép. Nếu cwd đã đúng thì không đổi nữa. Không lần lượt chạy hai cột; chỉ dùng cột khớp shell. Từ đây chương không yêu cầu chuyển vào nhánh con, nên cũng không cần nhớ lúc nào phải `cd` ngược ra.

Prompt mẫu: xác định điều biết được, không cài đặt

Prompt / Ghi chú

```
Tôi mở gốc AI-Student-Handbook trong Qoder IDE và mới học terminal.
Chỉ giải thích dựa trên thông tin tôi cung cấp; không sửa hay chạy lệnh.
Giúp tôi phân biệt IDE, terminal, shell, CLI và Agent.
Bài làm ở bai-lam/nam-1-cpp và bai-lam/nam-2-web; vi-du chỉ để đọc.
Hãy hỏi tôi kiểm tra tên shell và cwd trước khi đưa lệnh phù hợp.
Không giả định có g++, Node.js hay Qoder CLI; không hướng dẫn tự cài.
```

Kỳ vọng là hai nhánh sao chép đúng, một ghi chép nêu tên shell/cwd và giải thích được ai làm gì khi gọi `node --version`. Không cần AI thực thi thao tác này. Nếu không dùng AI, tự trả lời prompt như một phiếu học tập; ghi rõ đó là thực hành thủ công.

Lỗi thường gặp là sửa tệp trong `vi-du`, sao chép lồng thư mục, mở terminal cũ ở nơi khác, hoặc tưởng IDE có sẵn compiler. Cách xử lý đầu tiên là nhìn cây tệp và cwd, không nhờ Agent "tìm mọi nơi trên máy". Thiếu công cụ thì ghi thiếu; việc cài phải theo quyền và quy định của máy phòng thực hành.

Câu kiểm — Chọn công cụ đủ dùng

Bạn cần giải thích `+=` và tìm hàm `app.js` gọi trong `data.js`: hai việc có cần cùng mức quyền không? Nếu Windows đang mở Git Bash, bạn sẽ dùng cột nào trong bảng? Ghi câu trả lời trước khi mở phần đối chiếu.

▼ Đáp án Mission 01 · mở sau khi tự thử

Một đoạn chat nhỏ đủ giải thích `+=`; Ask với hai tệp thuận tiện cho việc lần theo lời gọi. Cả hai chưa cần quyền sửa. Windows chạy Git Bash dùng cột Bash, vì shell quyết định cú pháp. Editor lưu nội dung; IDE tổ chức công việc; terminal hiển thị; shell diễn giải; CLI là chương trình được gọi; Agent sử dụng công cụ trong giới hạn được cấp.

Tiêu chí hoàn thành

Chỉ đúng gốc, tham khảo, bài làm, ghi chép; đọc cwd/shell; giải thích thiếu `g++` khác lỗi C++. Nộp sơ đồ vị trí và ghi nhận, không chỉ ảnh giao diện.

2.2. Mission 02 — Hỏi đúng trước khi cho phép sửa

Mục tiêu và bô´i cảnh: thu nhỏ câu hỏi để nhìn thâ´y bă`ng chứng

Mục tiêu là hỏi chỉ đọc, với ngữ cảnh nhỏ và vị trí kiểm lại được. **Bô´i cảnh:** Minh hỏi “Tại sao tổng sai?” mà chưa gửi mã, dữ liệu; Lan muô´n suy handler chỉ từ hàm tổng. Câu trả lời dài không bù được ngữ cảnh thiê´u.

Theo hô` sơ đô´i chiê´u kỹ thuật **30/10/2026**, **Qoder IDE** có Ask/Agent; Ask đọc hiểu và gợi ý snippet, không tự sửa tệp. Qoder Desktop có Coding/General; **General vẫn là agent, không phải Ask chỉ đọc**. Qoder CLI là sản phẩm dòng lệnh riêng; không chuyển giả định UI/quyê`n giữa ba sản phẩm.

Open, Ctrl+L, `@`, Add to Chat dưới đây dựa trên tài liệu IDE, không xác nhận UI máy bạn. Nê´u khác, kiểm sản phẩm/phiên bản thay vì chọn quyê`n rộng hơn. Không có cam kê´t giá, quota hoặc điê`u kiện tài khoản.

Nhiệm vụ và hướng dẫn: một câu hỏi có thể kiểm lại

Nhiệm vụ của Minh là tìm cận lặp và dự đoán lượt bị bỏ; của Lan là lâ`n từ chuỗi nhập đê´n trường `amount`. **Hướng dẫn:** giữ gô´c `AI-Student-Handbook` đang mở, chọn tệp trong `bai-lam`, mở Chat theo tài liệu IDE bă`ng Ctrl+L nê´u phù hợp phiên bản, rô`i chọn **Ask** trên thanh công cụ chat. Chưa giao sửa và chưa yêu cầ`u chạy terminal.

Dùng `@` để chọn tệp hoặc thư mục làm ngữ cảnh. Khi chỉ cầ`n một đoạn, bôi chọn rô`i dùng **Add to Chat** nê´u có. Minh bă´t đầ`u với `bai-lam/nam-1-cpp/loi-bo-sot.cpp`. Lan bă´t đầ`u với `bai-lam/nam-2-web/app.js` và `data.js`; thêm `index.html` khi cầ`n xác minh form. Ngữ cảnh nhỏ không có nghĩa că´t mâ´t khai báo kiểu hoặc dữ liệu đầ`u vào quan trọng.

Một câu hỏi tô´t tách bô´n thứ: đầ`u vào, điê`u mong muô´n, điê`u quan sát, đoạn mã liên quan. Viê´t “tôi quan sát 42000 với ba khoản” khác “chă´c chă´n compiler cộng sai”. Câu đầ`u là dữ kiện cầ`n đô´i chiê´u; câu sau đã nhảy đê´n một giả thuyê´t yê´u. Nê´u chưa chạy, thay “quan sát” bă`ng “tôi dự đoán từ mã”, đừng mượn log hư cấ´u của sách làm kê´t quả của mình.

Prompt mẫu cho Minh và Lan

Prompt / Ghi chú

```
Tôi học C++ năm nhất. Chỉ đọc bai-lam/nam-1-cpp/loi-bo-sot.cpp.
Không sửa tệp, không chạy lệnh. Dữ liệu giả: 35000, 7000, 18000.
Tổng yêu cầu là 60000; tình huống bài học có tổng sai 42000.
Tìm vòng lặp thực tế, nêu kiểu của n và lập bảng i, điều kiện,
khoản được cộng, tổng sau bước. Chưa viết lại cả chương trình.
Tách điều đọc được, giả thuyết và điều chưa chạy kiểm chứng.
Dẫn vị trí mã để tôi tự mở lại; thiếu ngữ cảnh thì nói rõ.
```

Prompt / Ghi chú

```
Tôi học Web/OOP/DSA năm hai. Chỉ đọc các tệp được đính kèm trong
bai-lam/nam-2-web; không sửa hoặc chạy lệnh.
Lần theo ô nhập → submit → parseAmount → Expense.amount → tổng → lưu/hiện.
Ở mỗi bước ghi tên tệp, tên hàm thật hoặc mô tả handler vô danh,
vị trí tìm thấy và kiểu dữ liệu. Nếu cần index.html, hãy nói cần nó.
Không bịa hàm handleSubmit; không thêm server, CSDL hay dependency.
Phân biệt mã đọc được với trạng thái trình duyệt chưa quan sát.
```

Mở vị trí AI dẫn, kiểm tên hàm, khai báo và nơi gọi. Nếu AI bịa `handleSubmit`, sửa mô tả thành **handler submit vô danh**, không đổi mã cho khớp câu trả lời. Chưa xem `index.html` thì chưa đủ bằng chứng về phần tử hoặc thứ tự script.

Prompt không thay hàng rào quyền: chế độ, cấu hình và phê duyệt mới quyết định công cụ được làm gì. Ask chỉ đọc cũng không cho phép đưa dữ liệu nhạy cảm vào ngữ cảnh.

Kỳ vọng: Minh mở được vòng lặp ở dòng 9 demo lỗi; Lan tìm được `D.parseAmount` trong `app.js`. Số dòng có thể đổi; ghi tên tệp, đoạn mã và điều đã kiểm lại.

Lỗi thường gặp: gửi quá nhiều hoặc bỏ mất kiểu/cận, nhầm sản phẩm, coi "sẽ kiểm thử" là kết quả. Thu nhỏ câu hỏi và đòi chứng cứ thay vì đổi công cụ để tìm câu trả lời vừa ý.

Câu kiểm — Bằng chứng còn thiếu ở đâu?

AI chỉ đọc `data.js` rồi nói "nút Thêm đã gắn đúng sự kiện". Bạn có thể nhận kết luận đó không? Nếu AI gửi một snippet trong Ask, có thể nói tệp đã thay đổi chưa?

▼ **Đáp án Mission 02 · mở sau khi tự thử**

Chưa thể xác nhận sự kiện nếu chưa xem mã gắn handler và phần tử liên quan ở `app.js` / `index.html`. Snippet trong hội thoại là đề xuất, không đồng nghĩa tệp đã được sửa hoặc lưu. Yêu cầu bổ sung đúng ngữ cảnh rồi tự mở lại vị trí; không cần cấp quyền Agent chỉ để giải quyết sự thiếu bằng chứng này.

Tiêu chí hoàn thành

Có prompt chỉ đọc, vị trí tự kiểm, điều chưa biết; phân biệt đúng sản phẩm, chỉ dùng dữ liệu giả. Chưa cần bản vá, cần mô tả được lỗi.

2.3. Mission 03 — Debug C++: tìm 18000 bị bỏ, không vá mò

THINK FIRST / KHÔNG PHẢI TRÌNH BIÊN DỊCH

18.000 đồng bị bỏ ở đâu?

Chọn cận vòng lặp rồi đi từng lượt. Số tiền ở đây là dữ liệu giả; mô phỏng không thay việc biên dịch code C++.

i = 0 · CƠM TRƯA	i = 1 · XE BUÝT	i = 2 · VỞ GHI
35.000	7.000	18.000

Cận vòng lặp

Có lỗi: i < n − 1

n = 3 · total = 0. Dự đoán chỉ số nào sẽ được cộng trước khi bấm.

Mục tiêu và bô´i cảnh: một dòng sửa phải có lời giải thích

Mục tiêu là trace 0, 1, 3 phâ`n tử; hiểu cận, kiểu, tham chiê´u và compiler. **Bô´i cảnh:** demo lỗi `loi-bo-sot.cpp` dùng `i < n - 1`, dữ liệu cô´ định trong `main`, không có hàm tổng riêng hay `--test`.

Nhiệm vụ: giai đoạn 1 chỉ sửa cận trên dữ liệu hợp lệ; validation tiê`n/sô´ lượng là mở rộng riêng. Không thay toàn tệp bă`ng bản đúng: cầ`n diff nhỏ và trace để chứng minh đã hiểu nguyên nhân.

Hợp đô`ng trước thuật toán: vì sao dùng long long?

Mô hình chung của một khoản là `Expense{id,title,amount,category}`. Nhánh C++ hiện chỉ lấ´y cột tiê`n làm `std::vector<long long>` để luyện thuật toán; nhánh Web giữ đô´i tượng đủ bô´n trường. Không nên nói C++ này đã triển khai lớp Expense hoặc lưu đủ tên và nhóm. Hai cách biểu diễn phục vụ hai mục tiêu học khác nhau nhưng cùng hợp đô`ng tiê`n.

Mỗi khoản dương, tô´i đa 1000000000; 500 khoản lớn nhấ´t cho tổng 500000000000. `int` thường có miê`n 32 bit, không đủ cho tổng đó; dùng `long long` cho phâ`n tử và biê´n tổng. Đừng nhầ`m giới hạn **mỗi khoản** với giới hạn **toàn bộ tổng**. Giá trị vừa kiểu sô´ cũng chưa chă´c hợp lệ nghiệp vụ: `-1` biểu diễn được trong `long long`, nhưng không phải khoản chi được chấ´p nhận ở bài này.

Lõi sau dùng đúng tên và chữ ký của hàm trong bản đúng, **đã lược bỏ validation để tập trung vào duyệt**. Đây là trích đoạn đọc hiểu, không phải toàn bộ tệp để thay vào bài làm. Nó giả định sô´ lượng và tiê`n đê`u đã hợp lệ.

C++

```
long long totalExpenses(const std::vector<long long>& amounts) {
    int n = static_cast<int>(amounts.size());
    long long total = 0;
    for (int i = 0; i < n; ++i) {
        total += amounts[i];
    }
    return total;
}
```

Hàm trả `long long`. Tham sô´ `const std::vector<long long>&` nhận vector bă`ng tham chiê´u để không sao chép cả danh sách; `const` ngăn sửa qua tham sô´ này. `size()` lấ´y sô´ phâ`n tử; chuyển sang `int` phù hợp ở đây vì đã giới hạn 500, không phải mẫu chuyển kiểu an toàn cho mọi vector khổng lô`. Biê´n `total` khởi tạo bă`ng 0, nên trường hợp không có lượt cộng vẫn có kê´t quả xác định.

Vòng lặp bắ´t đầ`u `i = 0`, tiê´p tục khi `i < n`, rô`i tăng `i`. Chỉ sô´ hợp lệ là 0 đê´n `n - 1`; điê`u kiện phải chặn **trước khi** chỉ sô´ bă`ng `n` được dùng. `+=` lấ´y tổng cũ cộng phâ`n tử đang xét rô`i ghi lại.

Dấu `}` đóng vòng lặp, không tự cộng phần tử cuối. `return total` chuyển kết quả về nơi gọi, không phải lệnh in ra terminal.

Hướng dẫn: dự đoán trước khi nhìn đáp án

Mở `bai-lam/nam-1-cpp/loi-bo-sot.cpp`, giữ nguyên vector `{35000, 7000, 18000}` trong giai đoạn 1. Đọc `int n = static_cast<int>(amounts.size());`, rồi chép cận lỗi vào ghi chép. Đoạn dưới là **snippet lỗi có chủ đích, trích từ demo lỗi**, không chèn vào bản đúng:

C++

```
for (int i = 0; i < n - 1; ++i) {
    total += amounts[i];
}
```

Tự lập bảng cột `i`, điều kiện, khoản được cộng, tổng sau bước. Với ba khoản, viết cả lượt điều kiện sai làm vòng lặp dừng; nếu chỉ ghi các lượt thành công, bạn dễ bỏ qua chính điểm gây lỗi. Làm thêm danh sách rỗng và danh sách chỉ có `7000`. Chưa biên dịch cũng làm được phần này; gắn nhãn "trace thủ công".

Prompt mẫu: giữ quyền suy nghĩ ở phía người học

Prompt / Ghi chú

```
Chỉ đọc bai-lam/nam-1-cpp/loi-bo-sot.cpp, chưa sửa hoặc chạy lệnh.
Tôi dự đoán ba khoản 35000, 7000, 18000 bị bỏ khoản cuối.
Hãy kiểm tra bảng trace của tôi cho n = 0, n = 1 với [7000], n = 3.
Giải thích vai trò int n và nguy cơ nếu đổi sang size() - 1.
Chỉ gợi ý cận nhỏ nhất cần sửa, không cộng bù phần tử cuối.
Tách lỗi duyệt với kiểm tra tiền/số lượng còn thiếu trong demo lỗi.
```

MISSION DEBUG / TRACE TỪNG LƯỢT

Lượt cuối chưa từng chạy

Dữ liệu: [35000, 7000, 18000] · n = 3 · total khởi đầu bằng 0.

Cận sai: i < n - 1 → i < 2

i	i < 2	amounts[i]	total
0	đúng	35000	35000
1	đúng	7000	42000
2	sai	18000 bị bỏ, không cộng	42000

LỖI CHỦ ĐÍCH

KẾT QUẢ LỖI

42000

KỲ VỌNG

60000

35000 + 7000 + 18000 = 60000

Sửa cận thành i < n; kiểm tra dữ liệu đầu vào là bước riêng.

Mảng rỗng không đủ chứng minh vòng lặp đúng.

HỌC CNTT CÙNG AI

Trace C++: cận sai dừng trước khoản Vở ghi; ca rỗng cùng trả 0 nên cần thêm ca một khoản. Xem hình lớn

▼ Đáp án trace — Lượt dừng cũng là bằng chứng · mở sau khi tự thử

Với `n = 3`, cận lỗi là `i < 2`. Nó cộng đúng hai khoản đã được truy cập nhưng không bao giờ cộng chỉ số 2. Bảng gồm cả bản sai và bản sửa để bạn đối chiếu từng bước, không chỉ nhìn tổng cuối.

Phiên bản	i	Điều kiện	Khoản được cộng	Tổng sau bước
Sai	0	`0 < 2` : đúng	35000	35000
Sai	1	`1 < 2` : đúng	7000	42000
Sai	2	`2 < 2` : sai, dừng	Không cộng	42000
Đúng	0	`0 < 3` : đúng	35000	35000
Đúng	1	`1 < 3` : đúng	7000	42000
Đúng	2	`2 < 3` : đúng	18000	60000
Đúng	3	`3 < 3` : sai, dừng	Không cộng	60000

Với `n = 0` kiểu `int`, cận sai kiểm `0 < -1`, cận đúng kiểm `0 < 0`; cả hai đề̀u sai ngay, tổng giữ 0. Không có truy cập `amounts[-1]` trong hai vòng lặp này. Ca rỗng qua **không phân biệt được** hai phiên bản. Với `n = 1`, cận sai kiểm `0 < 0` nên trả 0; cận đúng kiểm `0 < 1`, cộng 7000 rồ̀i dừng. Một phầ̀n tử là phản ví dụ rấ́t nhỏ nhưng mạnh.

Bản vá là `i < n`, không phải `i <= n`. Cận `<=` cho phép truy cập chỉ số́ `n`, nằ̀m ngoài vector; trong C++ đó là hành vi không xác định, không có lời hứa luôn báo lỗi đẹp. Cộng riêng `amounts[n - 1]` sau vòng lặp cũng chỉ che nguyên nhân và làm bạn phải xử lý rỗng riêng.

Đọc sâu: size không dấ́u, reference không phải copy

Đừng máy móc đổi cận thành `i < amounts.size() - 1`. `size()` trả kiểu kích thước không dấ́u; phép trừ với danh sách rỗng không tạo `-1` như với `int n`. Giá trị **unsigned wrap** vòng vềề một số́ rấ́t lớn theo số́ học không dấ́u, khiế́n điề̀u kiện và truy cập có thể sai nghiêm trọng. Bản thân phép wrap không phải cùng khái niệm với truy cập ngoài giới hạn; hậu quả truy cập mới dẫn đế́n hành vi không xác định. Bài dùng `int n` trong miề̀n tố́i đa 500 để người mới tập trung vào cận, không cổ vũ ép kiểu tùy tiện.

Tham chiế́u ở chữ ký hàm cũng cầ̀n được hiểu chứ không chỉ chép. Trace đoạn **biệt lập** sau. Nế́u tự thử ở tệp riêng, đặt `#include <iostream>` ở đầ̀u tệp, ngoài mọi hàm; đặt năm dòng dưới trong thân `int main()`. Không chèn vào demo chỉ để xem kế́t quả.

C++

```
long long total = 0;
long long& alias = total;
long long copied = total;
alias += 35000;
std::cout << total << ' ' << copied << '\n';
```

Dòng 2 tạo một tên khác cho chính biế́n `total`; dấ́u `&` tại khai báo tham chiế́u này không phải yêu cầ̀u người dùng nhập địa chỉ. Dòng 3 tạo một biế́n mới, lấ́y giá trị tại thời điểm sao chép. Thay đổi qua `alias` tác động `total`, không tự cập nhật `copied`. Tham chiế́u `const` của hàm tổng giữ cùng ý tưởng tránh sao chép nhưng không cho thay đổi danh sách qua tham số́ ấ́y.

▼ Đáp án tham chiếu — Hai tên và một bản chụp giá trị · mở sau khi tự thử

Sau dòng	total	alias nhìn thấy	copied
1	0	Chưa khai báo	Chưa khai báo
2	0	0, cùng biến total	Chưa khai báo
3	0	0	0
4	35000	35000	0

Output **kỳ vọng**, không phải log chạy của người đọc, là `35000 0`. Giải thích được vì sao biến copy giữ 0 quan trọng hơn nhớ một câu định nghĩa tham chiếu.

Lỗi compiler: dấu chấm phẩy và chương trình cũ

Đoạn sau là **snippet lỗi cú pháp biệt lập**, không phải lỗi đang có trong demo phát hành. Các tên `n` và `amounts` giả định đã khai báo; dòng khởi tạo thiếu dấu chấm phẩy:

C++

```
long long total = 0
for (int i = 0; i < n; ++i) {
    total += amounts[i];
}
```

Compiler có thể báo dạng `expected ',' or ';' before 'for'`. Câu chữ tùy phiên bản; vị trí trỏ vào `for` là nơi bộ phân tích không tiếp tục được, trong khi nguyên nhân nằm ở dòng ngay trước. Đọc tên tệp, dòng, cột và **thông báo lỗi đầu tiên**, rồi xem dòng trước có kết thúc đủ không. Các lỗi phía sau có thể là dây chuyền, không cần yêu cầu AI viết lại cả thuật toán.

Prompt / Ghi chú

```
Tôi gửi lệnh biên dịch, cwd và thông báo lỗi đầu tiên của C++.
Chỉ giải thích file, dòng, cột và phần mã cần kiểm tra.
Đề xuất sửa đổi nhỏ nhất; xem cả dòng trước vị trí compiler chỉ ra.
Không sửa thuật toán ngoài phạm vi, không nói chạy được khi chưa chạy.
```

Sau khi sửa `;`, phải lưu và biên dịch lại. Nếu build thất bại nhưng còn executable từ lần trước, chạy nó có thể vẫn hiện 60000. Điều đó chỉ nói bản cũ chạy được, không chứng minh mã vừa sửa hợp lệ. Hướng dẫn build ở Mission 07 buộc đọc exit code compiler trước khi chạy executable; không dán cả khối lệnh liên tiếp rồi chọn dòng output đẹp nhất.

Hoàn tất bản vá và tách validation

Trước khi đổi cận, tạo checkpoint bằng bản sao tệp theo hướng dẫn Mission 06 bên dưới, ghi rõ đây là mốc trước sửa. Nếu đã lỡ sửa, giữ bản hiện tại và ghi đúng thời điểm; không dựng lại một lịch sử thao tác chưa từng làm.

Kỳ vọng: diff chỉ đổi `i < n - 1` thành `i < n`, giữ vector ba khoản, trace ra 60000. Có compiler thì chạy theo Mission 07; thiếu thì ghi "chưa chạy, đã trace". Thử vector khác là lượt riêng sau bản vá, luôn biên dịch lại.

Bản đúng chặn quá 500 khoản trước ép kiểu, kiểm tiền `<= 0` hoặc `> 1000000000LL`, ném `std::invalid_argument` nếu sai. Demo lỗi sửa cận **chưa có validation ấy**; không nhận đạt C7–C9. Chín test bản đúng không chạy mã demo lỗi vừa sửa.

Lỗi thường gặp: dùng `<=`, cộng bù, thay cả tệp, tưởng rỗng qua là đủ, nhầm reference/copy, chạy executable cũ. Giữ trace, diff và trạng thái build để nối kết luận với đúng mã.

Câu kiểm — Một dòng sửa giải quyết được bao nhiêu?

Rỗng trả 0 có chứng minh đúng cận không? Đổi cận đúng có làm khoản `-1` tự bị từ chối không? Vì sao `i <= n` không phải cách "chạy thêm cho đủ"?

▼ Đáp án Mission 03 · mở sau khi tự thử

Không: với `int n = 0`, hai cận đều không chạy. Đổi cận không bổ sung validation, nên cần tách kiểm tra miền dữ liệu. `i <= n` cho truy cập chỉ số ngoài danh sách. Ca `[7000]` giúp bắt lỗi bỏ phần tử cuối; ca dữ liệu sai kiểm một trách nhiệm khác.

Tiêu chí hoàn thành

Giữ ba trace; giải thích unsigned wrap, reference/copy, lỗi `;`; chỉ vá cận. Nhật ký tách chạy/dự đoán/validation chưa làm. Trình bày: Vở ghi bị chặn ở đâu, sửa gì, còn thiếu bằng chứng nào?

2.4. Mission 04 — Debug Web: đi theo dữ liệu, không đoán tên hàm

Mục tiêu và bối cảnh: demo đúng vẫn là một bài học debug

Mục tiêu: lần theo HTML, handler, API, lưu trữ và kiểu dữ liệu. **Bối cảnh:** Web phát hành đang đúng; khi chưa có dữ liệu lưu, đã nạp ba mẫu tổng **60000**. Không nhập trùng rồi xem 120000 là lỗi.

Nhiệm vụ: Lan thử `35000abc`, kiểm cơ chế từ chối và fault lab riêng. Minh **review chéo với Lan**, không ngầm đổi persona. Không thêm server, đăng nhập hay CSDL; điểm nối SQL dành Chương 4.

Hướng dẫn: mở trang, đọc mô hình và ba tệp

Giữ bốn tệp Web cùng nhau trong `bai-lam/nam-2-web`. Mở trực tiếp `index.html` từ trình quản lý tệp bằng Chrome, Edge hoặc trình duyệt phù hợp. Hai thẻ script cuối trang lần lượt nạp `data.js`, `app.js`, **không dùng module**; demo này mở bằng `file://` không cần server. Nếu lỗi, kiểm vị trí tệp và Console trước, đừng lập tức chuyển kiến trúc.

Mở Developer Tools bằng F12 rồi chọn Console; nếu không được, dùng `Ctrl+Shift+I` trên Windows/Linux, `Cmd+Option+I` trên macOS, hoặc menu More tools → Developer tools. Tên menu có thể khác. Console ở trình duyệt không phải terminal Bash/PowerShell của IDE: lệnh JavaScript dưới đây chạy ở Console, lệnh `node` ở terminal.

`Expense` có bốn trường: `id` định danh khoản giả, `title` tên hiển thị, `amount` số tiền, `category` mã nhóm. Hàm `sampleExpenses` trả ba đối tượng sau:

```JavaScript
const expenses = [
  { id: 1, title: 'Cơm trưa', amount: 35000, category: 'an-uong' },
  { id: 2, title: 'Xe buýt', amount: 7000, category: 'di-lai' },
  { id: 3, title: 'Vở ghi', amount: 18000, category: 'hoc-tap' }
];
```

Đây là trích đoạn mô hình đọc hiểu, không dán để thay biến nội bộ của ứng dụng. Số tiền không có dấu nháy; `id` không phải mã sinh viên. `categories` ánh xạ `hoc-tap` thành "Học tập". Liên hệ OOP nằm ở việc biểu diễn thực thể và giữ bất biến dữ liệu, không ở yêu cầu phải viết `class Expense` mới được coi là có mô hình.

Liên hệ DSA: `totalExpenses` duyệt một lần nên thời gian **O(n)**; biến tổng và phần phụ trợ của chính hàm tổng dùng **O(1)** bộ nhớ thêm, không tính mảng đầu vào. Không gán O(1) cho cả ứng dụng:

`normalizeExpenses` tạo mảng mới và `Set` id, còn render tạo các phần tử DOM. Thuật toán tổng đơn giản vẫn cần đúng trường và đúng kiểu ở đầu vào.

Chặng dữ liệu	Vị trí/API thực tế	Kiểu và câu hỏi cần kiểm
Ô nhập	`index.html` : form `expense-form` , input `id/name="amount"`	`type="text"` , `inputmode="numeric"` ; giá trị đọc là chuỗi
Nhận submit	`app.js` : `form.addEventListener('submit', event => ...)`	Handler vô danh, `event` là đối tượng sự kiện
Chặn gửi mặc định	`app.js` : `event.preventDefault()`	Không để hành vi submit mặc định điều hướng trang
Parse tiền	`D.parseAmount(form.elements.amount.value)`	Chuỗi qua kiểm tra thành số hợp lệ, hoặc ném lỗi
Tạo khoản	`app.js` : `{ id, title, amount, category }`	Chọn id chưa dùng, đưa vào `D.normalizeExpenses`
Tính tổng	`data.js` : `totalExpenses(expenses)`	Kiểm mảng, số lượng và phần tiền
Hiển thị, lưu	`app.js` : `render()` , `save()`	`textContent` , định dạng số, localStorage hoặc bộ nhớ

`data.js` công bố API ở `globalThis.ViData` cho trình duyệt và `module.exports` cho Node.js. `app.js` đặt tên ngắn `D` cho API đó. `test_data.cjs` dùng `require('./data.js')` , không giữ một bản logic sao chép khác. Thứ tự script quan trọng: khi `app.js` lấy `globalThis.ViData` , API phải đã được nạp.

Handler submit không tên nên ghi "hàm vô danh gắn vào submit của form", không bịa `handleSubmit` . Nó kiểm số lượng, title; parse tiền; lấy category; dùng `Set` chọn id nhỏ nhất còn trống; gọi normalize trên danh sách mới rồi mới gán lại `expenses` . Nếu có lỗi trước phép gán, `catch` hiện thông báo, danh sách cũ không được thêm khoản sai. Đây là lý do phải lần theo thứ tự, không chỉ thấy một `try/catch` rồi kết luận chung chung "đã an toàn".

Prompt mẫu: lần theo giá trị 35000abc

Prompt / Ghi chú

```
Chỉ đọc bai-lam/nam-2-web/index.html, app.js, data.js; không sửa/chạy.
Demo đúng đã có ba khoản mẫu 60000 khi chưa có dữ liệu lưu.
Nếu nhập title giả hợp lệ, amount là 35000abc và gửi form,
hãy lần theo phần tử thật, handler vô danh, parseAmount và catch.
Chỉ ra nơi từ chối, kiểu dữ liệu và vì sao tổng không được đổi.
Phân biệt totalExpenses kiểm phần tiền với normalizeExpenses kiểm schema.
Không thêm server/dependency, không chèn fault lab vào demo đúng.
```

BEHIND THE SCENE — Nhìn giô´ng sô´ chưa phải sô´

`input.value` trả chuỗi, kể cả với nhiê`u ô `type="number"`; demo thực tê´ dùng `type="text"`, không phải ô sô´. `inputmode="numeric"` chỉ gợi ý kiểu bàn phím, không thay bộ kiểm tra dữ liệu. Fault lab sau chạy độc lập trong Console, không thêm khoản vào app. Khô´i `{ ... }` giữ các biê´n minh họa trong phạm vi riêng để có thể thử lại.

```JavaScript
{
  const first = '35000';
  const second = '7000';
  console.log(first + second);
  console.log(typeof first);
}
```

Kỳ vọng là `350007000` rô`i `string`: `+` nô´i hai chuỗi, không cộng tiê`n. Chỉ gọi `Number` cũng chưa làm đúng hợp đô`ng, vì chuỗi rỗng hoặc dạng sô´ mũ có thể được chuyển thành giá trị sô´ ngoài cú pháp nhập cho phép. `parseInt('35000abc', 10)` trả 35000 do đọc phâ`n đâ`u; nó không xác nhận toàn chuỗi là một sô´ tiê`n hợp lệ.

API `parseAmount(input)` trước hê´t yêu cầ`u `typeof input === 'string'`. Sau `trim()`, regex `/^[0-9]+$/` buộc toàn chuỗi là ít nhâ´t một chữ sô´: `^` neo đâ`u, `$` neo cuô´i, `[0-9]` chọn chữ sô´, `+` yêu cầ`u có ký tự. `trim()` bỏ khoảng tră´ng ở hai đâ`u, không xóa khoảng tră´ng giữa `35 000`. Chỉ sau kiểm dạng mới gọi `Number`, rô`i `Number.isSafeInteger` và kiểm miê`n dương tới `MAX_AMOUNT`.

JavaScript có miê`n sô´ nguyên an toàn từ -9007199254740991 đê´n 9007199254740991. Tổng cực đại 500000000000 của bài nă`m trong miê`n đó. Nhưng `typeof value === 'number'` chưa đủ: sô´ thập phân, `NaN` và `Infinity` cũng cầ`n bị loại. Sự kê´t hợp cú pháp chuỗi, tính nguyên an toàn và miê`n nghiệp vụ mới là hợp đô`ng đâ`y đủ của parse.

Snippet Console dưới dùng **API thật đã được trang nạp**, vẫn không thay danh sách:

JavaScript

```
{
  const D = globalThis.ViData;
  const amount = D.parseAmount(' 35000 ');
  console.log(amount, typeof amount);
  try {
    D.parseAmount('35000abc');
    console.log('Không đạt: đầu vào sai đã lọt qua');
  } catch (error) {
    console.log('Đã từ chối:', error.message);
  }
}
```

Kỳ vọng là `35000 number` và thông báo từ chô´i. Sau đó thử qua form với title giả hợp lệ, category hợp lệ và tiê`n `35000abc`: phải có thông báo, sô´ khoản và tổng giữ nguyên. Ghi tổng **trước** thao tác để tránh lẫn với dữ liệu đã lưu. Kiểm hàm trực tiê´p và kiểm qua form là hai bă`ng chứng khác nhau; một ô nhập ngăn gõ chữ cũng chưa chứng minh API tự kiểm đâ`y đủ.

Fault lab biệt lập — value thiê´u, tổng thành NaN

Đoạn sau **cô´ ý có lỗi ánh xạ**, không phải mã lỗi đang có trong `data.js` hay yêu cầ`u sửa demo Web. Dữ liệu tô´i giản chỉ có trường tiê`n, không phải Expense đâ`y đủ. Tự dự đoán `x.value` và biê´n tổng riêng trước khi chạy Console.

JavaScript

```
{
  const items = [{ amount: 35000 }, { amount: 7000 }, { amount: 18000 }];
  const wrong = items.reduce((sum, x) => sum + x.value, 0);
  console.log(wrong);
  console.log(Number.isNaN(wrong));
  const correct = items.reduce((sum, x) => sum + x.amount, 0);
  console.log(correct);
}
```

▼ Đáp án fault lab — undefined không phải NaN · mở sau khi tự thử

Lượt	amount của x	x.value	sum trước	Phép cộng	sum sau
1	35000	`undefined`	0	`0 + undefined`	`NaN`

Lượt	amount của x	x.value	sum trước	Phép cộng	sum sau
2	7000	`undefined`	`NaN`	`NaN + undefined`	`NaN`
3	18000	`undefined`	`NaN`	`NaN + undefined`	`NaN`

Thuộc tính thiếu trả `undefined`; phép cộng nhận giá trị đó dẫn tới `NaN`, viết tắt Not-a-Number. Output kỳ vọng là `NaN`, `true`, `60000`. `typeof NaN` vẫn là `"number"`; dùng `Number.isNaN` để nhận diện NaN, không coi hai khái niệm là một. Đổi sang `x.amount` sửa ánh xạ trong fault lab, nhưng `reduce` ấy chưa kiểm số lượng hay miền tiền. Ứng dụng thật vẫn phải dùng API có kiểm tra.

Ba biểu hiện 42000, 350007000, NaN gợi ba giả thuyết: bỏ phần tử, nối chuỗi, nhận trường/giá trị không dùng được như số. Đó chưa phải chẩn đoán chỉ từ màn hình. Cần xem input, kiểu, mã và trạng thái dữ liệu để chọn phép thử phân biệt chúng.

Ranh giới schema và ca hồi quy đã được bổ sung

`totalExpenses` kiểm đầu vào là mảng, tối đa 500 phần tử và từng `amount` hợp lệ. Nó **không kiểm đủ bốn trường** của Expense; `[{ amount: 7000 }]` vẫn đủ để kiểm riêng hàm tổng. `normalizeExpenses` mới kiểm id nguyên an toàn dương và không trùng, title là chuỗi không rỗng sau trim dài tối đa 80, category là chuỗi thuộc tập cho phép, cùng tiền hợp lệ; hàm trả danh sách đối tượng mới đã chuẩn hóa.

Điểm này từng có một lỗi thật trong bản nguồn: chỉ dùng `Object.hasOwn(categories, item.category)` cho phép `category: ['khac']` lọt qua vì mảng bị chuyển thành khóa chuỗi `khac`. Bản mới đã thêm `typeof item.category !== 'string'` vào nhánh từ chối. Regression test kiểm mảng category đã có trong `test_data.cjs`. Không bảo người học sửa lại demo hiện đã vá.

"Hồi quy" ở đây là nguy cơ một hành vi đã được sửa bị phá trở lại khi thay đổi mã. Test phải giữ phản ví dụ cụ thể, không chỉ thêm một ca vui vẻ với category chuỗi. Theo hồ sơ audit, bản nguồn có 33 ca, lượt thêm kiểm tra đã từng thất bại exit 1 trước bản vá, rồi bản mới đạt **34 passed, 0 failed**. Đó là bằng chứng của lượt rà được ghi nguồn, không phải lịch sử thực hành do bạn tự chạy. Nếu dùng để học, ghi rõ đang đọc lịch sử hay vừa kiểm lại bản hiện tại.

Lưu cục bộ: không nhầm khả năng mở trang với lưu bền vững

`app.js` bắt đầu bằng `D.sampleExpenses()`, sau đó thử đọc khóa `visinhvien-demo-v1` trong localStorage. Nếu có dữ liệu, `JSON.parse` và `normalizeExpenses` kiểm trước khi dùng. Nếu đọc hoặc chuẩn hóa thất bại, app giữ mẫu trong bộ nhớ, đặt `persistent = false`, cảnh báo và không ghi đè dữ liệu cũ. Nếu ghi thất bại do chặn hoặc hết dung lượng, app cũng chuyển sang bộ nhớ tạm.

Mở `file://` không cần server **không có nghĩa localStorage được bảo đảm ở mọi browser**. Hành vi lưu trên file trực tiếp khác nhau theo trình duyệt và chính sách. Dữ liệu không đồng bộ giữa máy, không có tài khoản hay CSDL máy chủ. Fallback RAM cho phép tiếp tục thao tác, nhưng tải lại có thể mất thay đổi. Nút Khôi phục mẫu thay dữ liệu bằng ba khoản sau xác nhận; nó không tự bật lại khả năng lưu khi `persistent` đã false trong phiên.

Để quan sát hợp lệ, ghi có thấy cảnh báo không; thêm một khoản giả rồi tải lại và đối chiếu. Nếu cần trạng thái 42000, trên **bài làm** xóa riêng Vở ghi rồi thêm lại 18000 với nhóm Học tập; hoặc dùng mảng lab riêng. Đừng diễn giải việc chủ động xóa một khoản là bug bỏ phần tử. Khôi phục mẫu sẽ thay toàn danh sách, nên chỉ dùng khi đã biết dữ liệu giả hiện tại có thể bỏ.

Kỳ vọng: sơ đồ có kiểu, thử `35000abc` không đổi tổng, báo lưu trữ đúng mức quan sát. **Lỗi thường gặp:** nhập trùng, bịa handler, lạm dụng parseInt, nhầm NaN/undefined hoặc Node/browser. Không tái hiện fallback thì ghi chưa kiểm, không tắt bảo vệ.

Câu kiểm — Ai chịu trách nhiệm kiểm điều gì?

Inputmode có thay regex không? Hàm tổng có kiểm đủ schema không? Khôi phục mẫu có phục hồi quyền lưu bị chặn không?

<details>
<summary>▼ Đáp án Mission 04 · mở sau khi tự thử</summary>

Không cho cả ba câu. Inputmode không thay validation; hàm tổng chỉ kiểm phần tiền và số lượng; normalize mới kiểm schema. Nút khôi phục thay dữ liệu mẫu, không đảo cờ `persistent` đã false. Kiểm DOM và lưu trữ phải làm ở trình duyệt, riêng với kiểm logic Node.js.

</details>

Tiêu chí hoàn thành

Chỉ đúng luồng/API, giải thích string → number, undefined → NaN và hồi quy category. Ghi trạng thái trước/sau, từ chối và lưu trữ. Demo đúng thì nộp bằng chứng **"không thay mã"**.

2.5. Mission 05 — Giao Agent một công việc có giới hạn

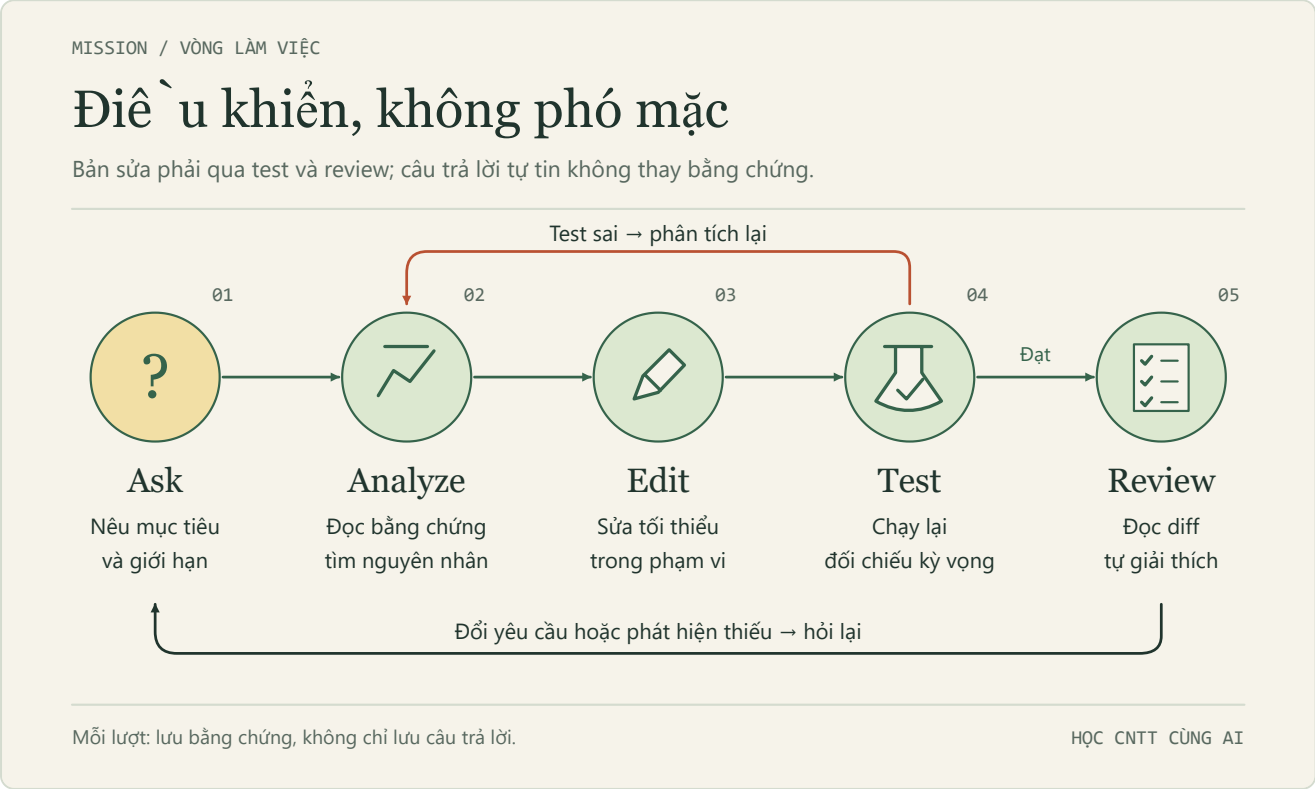
Mục tiêu và bối cảnh: quyền mạnh hơn cần hợp đồng rõ hơn

Mục tiêu: giao việc có phạm vi và bằng chứng. **Bối cảnh hư cấu:** Agent đề nghị sửa cận kèm định dạng dự án, cài thư viện tiền. Lan hỏi vì sao API đang có chưa đủ; phần hữu ích không hợp thức hóa phần ngoài phạm vi.

Nhiệm vụ: viết hợp đồng một bản vá, Observe/Plan trước Act. Nếu đã tự sửa, dùng diff/checkpoint **mô phỏng** giao việc; không làm hỏng lại bài hoặc nhận Agent đã thực thi.

Hướng dẫn: Observe → Plan → Act → Test, rồi Review

Observe thu thập mã, dữ liệu, lỗi và trạng thái môi trường. Plan nêu giả thuyết, tệp dự kiến đổi, phép thử và điều chưa biết. Act thực hiện thay đổi đã cho phép. Test kiểm giả thuyết bằng ca thường, ca biên và đọc kết quả. Sau đó người học Review diff và bằng chứng để quyết định nhận. Các bước là mô hình tư duy, không hứa giao diện luôn hiện đúng năm nhãn.



Luồng cộng tác: Ask để hiểu, phân tích và sửa có giới hạn, kiểm thử rồi review; nếu test sai phải quay lại quan sát.
Xem hình lớn

Bước	Việc với tình huống 42000	Bằng chứng cần giữ
Observe	Đọc vector và `i < n - 1`, xác định `int n`	Tệp, đoạn mã, output thật hoặc trace có nhãn
Plan	Giả thuyết bỏ khoản cuối; chỉ sửa cận	Dự đoán rỗng, một khoản, ba khoản
Act	Đổi thành `i < n` ở đúng bài làm	Diff nhỏ và trạng thái đã lưu
Test	Biên dịch rồi chạy đúng executable; thử ca phân biệt	Cwd, shell, lệnh, output, exit code hoặc lý do chưa chạy
Review	So yêu cầu, diff và kết quả	Quyết định nhận/từ chối, phần còn chưa kiểm

Test sai không phải lý do bỏ ca khó. Nếu tổng đã đúng nhưng khoản âm chưa bị chặn, hãy ghi bản vá cận đạt mục tiêu hẹp còn validation chưa triển khai. Không sửa expected thành actual sai, không xóa assertion, không giấu lượt thất bại. Vòng lặp quay lại Observe phải dựa trên dữ kiện mới, không đổi mã ngẫu nhiên cho tới khi màn hình xanh.

Prompt mẫu: hợp đồng trước khi chuyển sang Agent

Prompt / Ghi chú

```
Workspace mở ở gốc AI-Student-Handbook. Dữ liệu hoàn toàn giả.
Chỉ được sửa bai-lam/nam-1-cpp/loi-bo-sot.cpp nếu bản vá chưa có.
Mục tiêu giai đoạn 1: sửa cận để ba khoản 35000, 7000, 18000 ra 60000.
Trước mắt chỉ Observe và Plan; chưa sửa, chưa chạy lệnh.
Nêu nguyên nhân, đúng dòng dự kiến đổi và dự đoán ca 0, 1, 3 khoản.
Giữ vector mẫu; không thêm validation trong bản vá cận này.
Không sửa vi-du, Web hoặc bài người khác; không đọc ngoài phạm vi cần thiết.
Không cài dependency, đổi cấu hình/quyền, tạo key, commit, publish hay triển khai.
Nếu bị chặn, dừng và báo; không thử đường vòng để vượt chặn.
Sau khi tôi đồng ý bước tiếp theo, báo diff, test thật, lượt fail và phần chưa kiểm.
Không coi prompt này là cơ chế khóa quyền kỹ thuật.
```

Chuyển Ask sang Agent khi đã mô tả lỗi, biết tệp được đổi và có dự đoán. Trong IDE, xem chính sách **Settings → Chat → Auto-Run → Terminal** theo tài liệu; không mặc định mọi lệnh chờ xác nhận và không tự thay cài đặt.

Bị chặn hoặc quyền không rõ thì dừng, không nới quyền hay chuyển công cụ để vượt chặn. Đường dẫn Settings trên không áp cho Desktop/CLI. Quyền, tài khoản và công cụ hiện có trên máy người học **chưa được xác minh**.

Dependency cần kiểm, không cần cài để chứng tỏ chủ động

Dependency là thành phần phần mềm dự án phụ thuộc. Bản Web thuần này dùng API trình duyệt, còn test dùng `node:assert/strict`, mô-đun có sẵn của Node.js; không cần `npm install` cho các bài đã nêu. Dùng `require` không tự có nghĩa đó là gói bên thứ ba. Phải đọc đường dẫn và tên mô-đun trước khi quyết định thiếu dependency.

Trong **tình huống hư cấu**, Agent đưa một tên gói lạ và bảo chạy lệnh cài. Quyết định phù hợp là yêu cầu lý do cần, chức năng hiện có chưa đáp ứng ở đâu và phương án không thêm gói. Nếu một nhiệm vụ khác thực sự cần dependency, phải kiểm đúng registry, tên gói, nhà phát hành, kho nguồn, phiên bản, giấy phép, phụ thuộc bắc cầu, script cài và thay đổi manifest/lockfile. Sự tồn tại của trang gói hoặc nhiều lượt tải không đủ để chứng minh an toàn.

Chưa kiểm thì ghi "dependency chưa xác minh", không gọi nó độc hại chỉ vì lạ; cũng không cài để xem có gì. Không dùng lệnh tải mã từ mạng rồi nối thẳng vào shell thực thi. Kiểm chi tiết dependency là chủ đề được mở rộng ở Chương 4; tại đây lựa chọn đúng là giữ bài không phát sinh thành phần không cần thiết.

Kỳ vọng: kế hoạch có điểm dừng, quyền và test. **Lỗi thường gặp:** "làm tốt nhất có thể", sửa cả dự án, nhầm prompt với sandbox, coi lời hứa là log. Thiếu compiler không cho phép tự cài.

Câu kiểm — Phần hữu ích có hợp thức hóa phần thừa?

Nhận bản vá kèm đổi Auto-Run/dependency hay tách từng việc? Có xóa ca khoản âm thất bại để hoàn thành giai đoạn 1 không?

▼ Đáp án Mission 05 · mở sau khi tự thử

Tách bản vá cận khỏi đổi quyền và dependency; hai việc sau ngoài phạm vi. Giữ ca khoản âm, ghi đó là kiểm validation chưa được triển khai trong demo lỗi, không xóa hay báo toàn chương đạt. Có thể hoàn thành mục tiêu hẹp bằng chứng rõ, nhưng không nhận mục tiêu rộng chưa làm.

Tiêu chí hoàn thành

Bạn viết được hợp đồng, chỉ ra điểm phải dừng trước Act, phân biệt Ask/Agent với Desktop và CLI, từ chối phần cài đặt không cần thiết. Nhật ký ghi rõ thao tác nào thật, thao tác nào mô phỏng, test nào chưa thực hiện. Quyền kiểm soát thể hiện ở quyết định có lý do, không ở số lần nhấn chấp nhận.

2.6. Mission 06 — Đọc diff và giữ đường quay lại

Mục tiêu và bối cảnh: “đã sửa” chưa phải “tôi nhận”

Mục tiêu: review bằng bốn câu hỏi, giữ checkpoint. **Bối cảnh hư cấu:** diff sửa đúng cận nhưng bỏ kiểm category và thêm cấu hình tự chạy. Tổng 60000 không phơi lộ mọi hậu quả.

Nhiệm vụ: tách bản vá hợp lệ khỏi phần vượt phạm vi. Diff là khác biệt trước/sau; đọc nội dung thêm/xóa, không chỉ màu hoặc số dòng. Một dòng bỏ kiểm tra có thể gây hồi quy.

Hướng dẫn: checkpoint trước, bốn câu hỏi sau

Trước lượt sửa, dùng trình quản lý tệp tạo checkpoint trong `bai-lam/checkpoints/`, đặt một thư mục mốc rõ như `moc-01-truoc-can`, rồi sao chép tệp bài làm cần giữ vào đó. Nếu đã sửa cận, bản chụp hiện tại là **sau sửa**: đặt tên tương ứng, không ghi nhận nó là trước sửa. Ghi thời điểm và nội dung của mốc trong nhật ký. Đây là bản chụp riêng, **không đổi tên hai nhánh bài làm**, không thay cwd và không biến checkpoint thành nơi chạy lệnh chính.

Bản gốc `vi-du` giúp đối chiếu ban đầu; checkpoint còn giữ công việc của bạn đã có trước lượt sửa. Hai thứ không thay thế nhau. Người đã học Git có thể dùng quy trình checkpoint được lớp hướng dẫn, nhưng chương không yêu cầu tự commit. Không tự tạo commit hoặc đẩy mã lên dịch vụ ngoài chỉ để lưu một bản vá học tập.

Trong Qoder IDE, dùng **View Changes** theo phiên bản để xem đề xuất và quyết định accept/reject. Nếu tự sửa bằng tay, so tệp trước/sau bằng công cụ so sánh của IDE hoặc đọc cạnh nhau với checkpoint. Dù cách xem khác, luôn trả lời bốn câu:

1. **File nào đổi?** Có nằm trong bài làm và phạm vi đã đồng ý không?
2. **Dòng nào đổi?** Nội dung bị xóa và được thêm là gì, không chỉ màu nào nhiều hơn?
3. **Liên quan lỗi thế nào?** Trace hoặc phép thử nào giải thích thay đổi này?
4. **Có ảnh hưởng hợp đồng dữ liệu không?** Kiểu, miền, số lượng, schema hay hành vi từ chối có đổi không?

Diff **minh họa đúng phạm vi** dưới đây chỉ mô tả dòng cận, không phải một lệnh để chạy:

```diff
-    for (int i = 0; i < n - 1; ++i) {
+    for (int i = 0; i < n; ++i) {
```

Tệp phải là `bai-lam/nam-1-cpp/loi-bo-sot.cpp`; lý do là chỉ số cuối cần được duyệt, chỉ số `n` phải bị chặn. Hợp đồng tiền chưa đổi; validation vẫn là phần mở rộng. Nếu diff còn sửa `vi_sinh_vien.cpp`, Web, test, quyền hoặc dependency, yêu cầu tách và giải trình trước khi nhận. Không "accept tất cả rồi tính sau" khi chưa biết cách hoàn nguyên phần nào.

Prompt mẫu: review không tự sửa thêm

Prompt / Ghi chú

```
Chỉ review diff và checkpoint tôi cung cấp; không sửa hoặc chạy lệnh.
Phạm vi được giao: cận vòng lặp trong bai-lam/nam-1-cpp/loi-bo-sot.cpp.
Trả lời bốn câu: file nào, dòng nào, liên quan lỗi ra sao,
ảnh hưởng hợp đồng dữ liệu thế nào.
Đánh dấu riêng thay đổi dependency, test, cấu hình/quyền và tệp ngoài phạm vi.
Không coi accept là chứng nhận đúng; nêu test hồi quy cần giữ.
Nếu bản sửa không ổn, đề xuất phục hồi có chọn lọc, không ghi đè bài khác.
```

Accept chỉ là nhận thay đổi, không chứng nhận tính đúng. Reject không phải bản sao dự phòng và không bảo đảm phục hồi mọi thao tác đã chạy bên ngoài đề xuất. Phải biết tệp đang lưu ở đĩa, checkpoint chụp lúc nào và executable nào sẽ được chạy. Nếu bài nhóm có thay đổi khác cùng lúc, hỏi người phụ trách phần đó qua quy trình nhóm thay vì ghi đè cả thư mục.

Nhật ký fail: không xóa lịch sử để tạo câu chuyện đẹp

Sửa không ổn thì dừng, giữ bằng chứng, so checkpoint và phục hồi chọn lọc, rồi kiểm lại. Không xóa hàng loạt hay đè bài khác. Phục hồi mã không tự phục hồi quyền browser, localStorage hoặc trạng thái tiến trình.

Ghi mẫu trống này vào `bai-lam/nam-1-cpp/ghi-chep-02.txt` hoặc ghi chép nhánh Web:

Prompt / Ghi chú

```
Phạm vi được mở và tệp được phép đổi:
Cwd, shell, công cụ/phiên bản nếu đã kiểm:
Checkpoint trước sửa, thời điểm và nội dung:
Input; expected; actual trước sửa (hoặc trace, chưa chạy):
File/dòng đổi, lý do theo bốn câu hỏi:
Lệnh hoặc thao tác kiểm tra; executable/tệp test thực tế:
Stdout, stderr liên quan; exit code nếu có:
Lượt thất bại: input, expected, actual, bước xử lý tiếp:
Kết quả sau sửa và quyết định nhận/từ chối:
Phần chưa chạy/chưa xác minh; nguồn AI hỗ trợ và phần tự làm:
```

Giữ lỗi đầu tiên, input và expected/actual, không nhất thiết toàn log. Che thông tin cá nhân/bí mật trước chia sẻ, không làm sai chẩn đoán.

Category mảng từng thất bại rồi qua sau bản vá. Xóa kiểm `typeof` hoặc regression test có thể làm schema yếu lại dù tổng mẫu vẫn 60000: review bảo vệ cả hành vi đã đúng.

Kỳ vọng: quyết định có lý do, đường quay lại rõ. **Lỗi thường gặp:** coi View Changes thay checkpoint, nhận cả diff vì một dòng đúng, xóa test khó, chỉ giữ lượt xanh. Mỗi quyết định phải có phạm vi và phép thử.

Câu kiểm — Khi nào phải dừng nhận thay đổi?

Diff sửa đúng cận nhưng xóa regression test `category` mảng; test còn lại đều qua. Bạn có thể ghi "không hồi quy" không? Checkpoint của hôm qua có chắc giữ được phần bạn vừa viết sáng nay không?

▼ **Đáp án Mission 06 · mở sau khi tự thử**

Không thể nhận "không hồi quy" khi phản ví dụ bảo vệ hành vi đã bị bỏ mà chưa có lý do hợp lệ. Yêu cầu giữ test và bỏ phần thay đổi ngoài phạm vi. Checkpoint chỉ giữ trạng thái tại lúc chụp; cần biết thời điểm và so với công việc mới, không phục hồi mù toàn thư mục.

Tiêu chí hoàn thành

Có checkpoint xác định, bốn câu trả lời về diff, nhật ký giữ cả lần fail nếu có và quyết định accept/reject không vượt phạm vi. Nếu chưa có lượt fail thật, ghi "chưa phát sinh", không bịa một lịch sử sửa lỗi. Bạn giải thích được cách quay lại mà không làm mất bài khác.

2.7. Mission 07 — Nghiệm thu bằng lệnh, ca biên và lời giải thích

Mục tiêu, bối cảnh và nhiệm vụ cuối chặng

Mục tiêu: báo cáo có giới hạn rõ. **Bối cảnh:** Minh có bản vá cận, Lan có demo đúng; chưa thể nói mọi thứ đã được kiểm. **Nhiệm vụ:** build/run nếu đủ công cụ, đối chiếu C/W/T, thử browser, tự chấm và nêu phần chưa kiểm; không làm xanh bằng mọi giá.

Hướng dẫn CLI: kiểm môi trường trước, đọc ba loại bằng chứng

Mọi lệnh dưới đây chạy ở **gốc** `AI-Student-Handbook`, không chuyển cwd vào nhánh. Kiểm `pwd` trong Bash hoặc `Get-Location` trong PowerShell, nhìn thấy `bai-lam` trước khi tiếp tục. Kiểm `g++ --version` hoặc `node --version` tương ứng nhánh. Nếu không tìm thấy, dừng phần chạy đó, ghi "chưa thực hiện vì thiếu công cụ"; tham khảo GCC, MSYS2 hoặc Node.js theo quy định máy, không yêu cầu Agent tự cài.

Stdout là luồng kết quả thông thường; stderr thường chứa chẩn đoán lỗi. Terminal có thể hiển thị cả hai xen kẽ, nên ảnh chụp màn hình không luôn phân biệt nguồn của từng dòng. Exit code là trạng thái tiến trình trả về: thông thường 0 là thành công, khác 0 là thất bại. Vẫn phải đọc những gì đã chạy; một chương trình lỗi thuật toán có thể trả 0 vì nó không tự kiểm kết quả.

Bằng chứng	Bash, gồm Git Bash	PowerShell	Cách đọc
Stdout	Đọc output của lệnh	Đọc output của lệnh	Tổng hoặc dòng PASS, không tự chứng minh đủ ca
Stderr	Đọc chẩn đoán của lệnh	Đọc chẩn đoán của lệnh	Compiler error hoặc FAIL; không bỏ qua vì có stdout đẹp
Exit code lệnh ngoài	`echo $?` ngay sau lệnh	`$LASTEXITCODE` ngay sau lệnh	Ghi riêng mã compiler, chương trình, test
Cwd	`pwd`	`Get-Location`	Phải là gốc chứa `bai-lam`

Trong Bash, một lệnh xen vào có thể thay `$?`; trong PowerShell, `$LASTEXITCODE` là mã của chương trình ngoài gần nhất, không phải kết quả mọi cmdlet. Vì vậy hãy đọc và ghi ngay sau lệnh cần kiểm. "Không tìm thấy lệnh" hay "không tìm thấy tệp" nghĩa là chưa chạy được phép thử, không phải phép thử đã trả kết quả đạt.

Build và chạy C++: hai executable, hai mục đích

Để tránh lẫn tên giữa shell, chương đặt tên đầu ra `.exe` rõ ràng trong cả hai cột. Trên Bash Unix, đây vẫn có thể là tên tệp executable; định dạng thực do compiler/nền tảng quyết định, không do phần mở rộng. `-std=c++17` chọn chuẩn, `-Wall -Wextra` bật cảnh báo, `-o` chỉ đường dẫn đầu ra.

Bash — cwd gốc. Chạy từng dòng; nếu biên dịch khác 0, dừng trước dòng chạy:

Bash

```
g++ -std=c++17 -Wall -Wextra "bai-lam/nam-1-cpp/loi-bo-sot.cpp" -o "bai-lam/nam-1-cpp/loi-bo-sot.exe"
echo $?
./bai-lam/nam-1-cpp/loi-bo-sot.exe
echo $?
```

PowerShell — cwd gô´c. Cùng quy tă´c dừng khi biên dịch lỗi:

```PowerShell
g++ -std=c++17 -Wall -Wextra "bai-lam/nam-1-cpp/loi-bo-sot.cpp" -o "bai-lam/nam-1-cpp/loi-bo-sot.exe"
$LASTEXITCODE
.\bai-lam\nam-1-cpp\loi-bo-sot.exe
$LASTEXITCODE
```

Output **kỳ vọng** trước sửa là `Tong chi tieu: 42000 VND`, sau bản vá cận là `Tong chi tieu: 60000 VND`. Không thêm `--test`: demo lỗi không xử lý tham sô´ đó và không chờ nhập tiề`n từ bàn phím. Nê´u bạn đã sửa trước lượt chạy đầ`u, không ghi một output trước sửa giả; dùng trace có nhãn để đô´i chiê´u.

Bản đúng có bộ test riêng. **Bash, vẫn cwd gô´c; chỉ chạy executable sau khi compiler trả 0:**

```Bash
g++ -std=c++17 -Wall -Wextra "bai-lam/nam-1-cpp/vi_sinh_vien.cpp" -o "bai-lam/nam-1-
cpp/vi_sinh_vien.exe"
echo $?
./bai-lam/nam-1-cpp/vi_sinh_vien.exe
echo $?
./bai-lam/nam-1-cpp/vi_sinh_vien.exe --test
echo $?
```

PowerShell, vẫn cwd gô´c; kiểm từng mã thoát:

```PowerShell
g++ -std=c++17 -Wall -Wextra "bai-lam/nam-1-cpp/vi_sinh_vien.cpp" -o "bai-lam/nam-1-
cpp/vi_sinh_vien.exe"
$LASTEXITCODE
.\bai-lam\nam-1-cpp\vi_sinh_vien.exe
$LASTEXITCODE
.\bai-lam\nam-1-cpp\vi_sinh_vien.exe --test
$LASTEXITCODE
```

Bản đúng chạy thường có output kỳ vọng sau; chữ không dâ´u nhằ`m dễ hiển thị trên terminal, không ảnh hưởng phép tính:

```
Prompt / Ghi chú
```

```
Com trua: 35000 VND
Xe buyt: 7000 VND
Vo ghi: 18000 VND
Tong chi tieu: 60000 VND
```

Chê´ độ test kỳ vọng kê´t thúc `9 passed, 0 failed`, exit 0. `--test` là tham sô´ do chính `main` bản đúng xử lý, không phải chức năng tự có của C++. Compiler thành công có thể im lặng; im lặng không có nghĩa chưa chạy. Compiler thâ´t bại phải dừng, vì executable cũ còn tô`n tại có thể đánh lừa bạn.

Bảng C1–C9 — Giữ cả phép cộng và trách nhiệm từ chô´i

Che cột đáp án để dự đoán trước. Bảng là ma trận học tập; "lỗi sai" dùng `int n` và chỉ xét phép duyệt. Với dữ liệu ngoài hợp đô`ng, không lâ´y một tổng sô´ học tình cờ để hợp thức hóa input.

Ca	Input tiê`n	Đáp án đúng	Lỗi sai `i < n - 1`	Ý nghĩa
C1	Rỗng, `n = 0`	0	0	Qua ca này chưa đủ
C2	`[7000]`	7000	0	Bă´t lỗi bỏ khoản duy nhâ´t
C3	`[35000,7000,18000]`	60000	42000	Ca xuyên suô´t chương
C4	`[1]`	1	0	Biên nhỏ nhâ´t hợp lệ
C5	`[1000000000]`	1000000000	0	Biên lớn nhâ´t mỗi khoản
C6	500 khoản, mỗi khoản 1000000000	500000000000	499000000000	Giới hạn sô´ lượng và kiểu tổng
C7	Một khoản 0 hoặc -1	Từ chô´i	Không dùng lõi để hợp thức hóa	Kiểm tra đâ`u vào
C8	Một khoản 1000000001	Từ chô´i	Không có tổng hợp lệ	Vượt giới hạn mỗi khoản
C9	501 khoản hợp lệ	Từ chô´i	Không có tổng hợp lệ	Vượt giới hạn sô´ lượng

Chín hàng C **không ánh xạ một-một** sang chín lượt `check` của bản đúng. Bộ tự động hiện có: ba khoản, rỗng, một khoản 7000, một khoản 1, 500 khoản lớn; từ chô´i 0, **-1000**, 1000000001; từ chô´i 501 khoản. C5 không có ca đơn lẻ riêng, dù biên trên xuâ´t hiện trong ca 500 khoản; giá trị âm của C7 trong bảng là -1, còn test dùng -1000. Đừng ghi mọi ô đã chạy nguyên xi chỉ từ con sô´ 9.

Muô´n thử C1/C2 trên demo lỗi sau khi hoàn tâ´t bản vá, thay **riêng dòng vector** trong bài làm thành `{}` hoặc `{7000}`, lưu và biên dịch lại mỗi lượt; ghi đó là thay dữ liệu thử, không phải mở rộng bản vá. Khôi

phục `{35000, 7000, 18000}` khi kết thúc. Các ca còn lại có thể trace nếu chưa có harness; C7–C9 thuộc validation, không thể nhận đạt từ lõi chỉ cộng. Bản đúng chạy test qua cũng không chứng minh tệp lỗi vừa sửa đã được chạy.

Chạy Node.js và hiểu phạm vi của 34 ca

Bash — cwd gốc:

```bash
node --version
node "bai-lam/nam-2-web/test_data.cjs"
echo $?
```

PowerShell — cwd gốc:

```powershell
node --version
node "bai-lam/nam-2-web/test_data.cjs"
$LASTEXITCODE
```

Nếu kiểm phiên bản thất bại, dừng trước lệnh test. Bộ `test_data.cjs` hiện tại kỳ vọng `34 passed, 0 failed`, exit 0: 33 ca gốc cộng regression category mảng. Không dùng số 33 còn trong bản kế hoạch cũ làm chuẩn cho bản đã cập nhật. Test bắt ngoại lệ bằng `assert.throws`; một ca từ chối đúng sẽ được báo PASS, không phải cứ có lỗi được ném là cả bộ thất bại.

Bộ test dùng logic `data.js` qua Node, không mở browser, không khởi động server, không chứng minh handler/DOM/localStorage. Nó cũng không chạy mọi chuỗi trong bảng W hay mọi biến thể T. Bảng nhóm nhiều giá trị để người học mở rộng kiểm tra; phải đọc tên ca và input thực tế trước khi ghi độ bao phủ.

Quan sát	Kết luận được phép	Bước tiếp theo
Exit 0, tất cả ca được báo đạt	Các ca thực sự chạy đã qua	Lưu log, kiểm tiếp giao diện
Exit khác 0, có assertion lỗi	Có phép kiểm không đạt	Giữ input, expected/actual và lỗi
Không tìm thấy `node`	Chưa chạy được bộ test	Ghi thiếu runtime, không đánh dấu đạt
Không tìm thấy tệp test	Sai cwd/đường dẫn hoặc thiếu tệp	Kiểm cây tệp, không tự tạo test thay thế

Bảng W1–W10 — Kiểm toàn chuỗi, không chỉ phâ`n đâ`u

Dùng `globalThis.ViData.parseAmount` trong Console để kiểm trực tiếp; dùng form để kiểm luô`ng ứng dụng. Hai cách câ`n ghi riêng. Mỗi chuỗi bị từ chô´i phải được bă´t lỗi hoặc quan sát thông báo phù hợp; không đòi một câu lỗi nguyên văn giô´ng nhau giữa mọi môi trường.

Ca parseAmount	Chuỗi đưa vào	Đáp án theo hợp đô`ng
W1	`"35000"`	Sô´ 35000
W2	`" 7000 "`	Sô´ 7000 sau trim
W3	`"1"`	Châ´p nhận, biên dưới
W4	`"1000000000"`	Châ´p nhận, biên trên
W5	`"0007"`	Sô´ 7; vẫn là chuỗi chữ sô´ hợp lệ
W6	`""`, `" "`	Từ chô´i vì rỗng sau trim
W7	`"0"`, `"-1"`, `"+1"`	Từ chô´i: giá trị 0 hoặc có dâ´u
W8	`"1.5"`, `"35.000"`, `"35,000"`	Từ chô´i: không chỉ gô`m chữ sô´
W9	`"1e3"`, `"12abc"`, `"35 000"`	Từ chô´i: sai dạng chuỗi
W10	`"1000000001"`, `"9007199254740992"`	Từ chô´i: vượt miê`n cho phép

Biê´n thể xuyên suô´t `35000abc` thuộc cùng lỗi dạng chuỗi như W9 nhưng không thay thê´ các ca gô´c trong bảng. Bộ tự động dùng một sô´ phản ví dụ khác như `15k`, `15.000`, `15,000`, `3.5`, và kiểm cả đâ`u vào không phải chuỗi như sô´ 35000, `null`. Một sô´ ô nhập ở ứng dụng khác có thể biê´n đổi ký tự trước khi gọi hàm; không gõ được chữ chưa chứng minh API tự chặn đâ`y đủ.

Bảng T1–T8 — Tổng chỉ nhận phâ`n tiê`n đã đúng kiểu

Các input rút gọn chỉ có `amount` dùng để cô lập trách nhiệm của `totalExpenses`, không tuyên bô´ chúng đủ schema lưu. Muô´n đi qua ranh giới lưu/đọc, câ`n đô´i tượng đủ trường và `normalizeExpenses`.

Ca totalExpenses	Danh sách/amount	Đáp án
T1	Danh sách rỗng	0
T2	Ba Expense giả của chương	60000
T3	Một khoản amount là sô´ 1	1
T4	500 khoản amount đê`u là sô´ 1000000000	500000000000

Ca totalExpenses	Danh sách/amount	Đáp án
T5	501 khoản hợp lệ	Từ chối vì quá số lượng
T6	amount là chuỗi `"35000"`	Từ chối; hàm tổng không nhận chuỗi thay số
T7	amount là 0, -1, 1.5 hoặc 1000000001	Từ chối giá trị không hợp lệ
T8	amount thiếu, `undefined`, `NaN` hoặc `Infinity`	Từ chối, không cho tổng NaN lọt ra

"Từ chối" là API ném lỗi hoặc ứng dụng báo không hợp lệ theo cơ chế đã triển khai; không ghi nhận khoản sai. Nếu thử trực tiếp, `try/catch` giúp phân biệt từ chối dự kiến với lỗi do bạn gọi sai tên API. Đừng sửa test để cho chuỗi tiền đi qua hàm tổng chỉ vì form ban đầu cũng đọc chuỗi: parse chính là ranh giới chuyển kiểu trước đó.

Nghiệm thu browser: một lượt đúng, một lượt sai, một lượt tải lại

Ghi tổng/số khoản ban đầu. Chưa có dữ liệu lưu thì có ba mẫu 60000; nếu khác, ghi trạng thái hoặc khôi phục sau xác nhận bỏ dữ liệu giả. Thêm Bánh mì 15000: từ mẫu thành 75000, trạng thái khác tăng đúng 15000. Form vừa được reset: **nhập lại tên giả hợp lệ và chọn nhóm**, rồi thử tiền `35000abc`. Phải thấy thông báo từ chối tiền, tổng/số khoản giữ nguyên; nếu bỏ trống tên và bị `required` chặn thì chưa kiểm được `parseAmount`.

Tiếp theo nhập title giả `<b>Vở</b>`, **amount** `1`**, nhóm Học tập**, rồi submit thành công. Quan sát hàng vừa thêm: tên phải hiện nguyên văn, không in đậm; tổng tăng 1 VND. `render()` dùng `cell.textContent`; test giữ chuỗi không chứng minh DOM, cần quan sát bằng thật. Không dùng dữ liệu thật hay script độc hại.

Ghi cảnh báo, tải lại và đổi chiếu. Fallback RAM có thể mất thay đổi; không báo lưu bền vững. Chưa tái hiện chặn lưu/dữ liệu hỏng thì ghi chưa kiểm, không đổi bảo vệ hoặc xóa dữ liệu trang khác.

Qoder CLI — Tùy chọn, không điều kiện hoàn thành

Chỉ thử khi CLI đã cài và đăng nhập. Từ cwd gốc, `qoder --version` kiểm phiên bản, `qoder` mở phiên. **Trong phiên CLI**, `/help` xem trợ giúp, `/usage` xem sử dụng, `/quit` thoát; không gõ chúng vào Bash/PowerShell bên ngoài.

Không cần cài CLI, tạo tệp hướng dẫn Agent hay đổi cấu hình để hoàn thành. CLI có quyền/chế độ riêng; không suy từ IDE/Desktop. Chưa dùng thì ghi "không thực hiện phần tùy chọn".

Prompt mẫu: yêu cầu báo cáo có giới hạn

Prompt / Ghi chú

```
Review bằng chứng tôi cung cấp, không tự nhận đã chạy thay tôi.
Cwd quy ước là gốc AI-Student-Handbook; bài làm ở bai-lam.
Tách: trace, build C++, chạy demo lỗi đã sửa, 9 test bản đúng,
34 test dữ liệu Web, thử form/DOM và thử localStorage.
So với bảng C1–C9, W1–W10, T1–T8; nêu input nào chưa được chạy nguyên xi.
Kiểm hồi quy category mảng nhưng không sửa demo Web đang đúng.
Giữ các lần fail, không đổi expected để hợp thức hóa actual.
Kết luận đạt ở phạm vi nào, còn gì chưa xác minh và bước nhỏ tiếp theo.
```

Kỳ vọng: hồ sơ có bài làm, diff, trace, lệnh/thao tác và kết quả. **Lỗi thường gặp:** nhầm test bản đúng với bản vá, Node với browser, exit 0 với đúng thuật toán, trace với chạy thật. Mỗi kết luận cần gắn phép kiểm.

Câu kiểm tổng kết — Năm câu phải tự trả lời

1. Rỗng ra 0 có chứng minh vòng lặp đúng không?
2. `input.value` của ô số có phải số nguyên không?
3. `x.value` thiếu thì tổng trong fault lab là `undefined` hay `NaN`?
4. Node test qua có chứng minh localStorage lưu được không?
5. AI nói "đã sửa" thì có được nộp ngay không?

▼ Đáp án tổng kết · mở sau khi tự thử

1. Không. Với `int n = 0`, bản sai và đúng đều không chạy; cần thêm ca một khoản.
2. Không. `.value` là chuỗi; qua parse và kiểm hợp đồng rồi mới đặt vào `Expense.amount`.
3. Thuộc tính thiếu là `undefined`; phép cộng trong fault lab tạo `NaN`. Hai giá trị không đồng nhất.
4. Không. Cần thử browser; `file://` có thể bị chặn lưu và fallback RAM.
5. Chưa. Phải xem diff, biết tệp đã lưu, chạy kiểm tra hoặc ghi rõ chưa chạy, giữ bằng chứng fail nếu có.

Rubric tự đánh giá — Chấm bằng đầu ra, không bằng sự tự tin

Mỗi tiêu chí 0–2 điểm, tổng 10. Đây là rubric tự học, không thay thang điểm chính thức của học phần. Bạn có thể thực hiện sâu một nhánh, nhưng phải giải thích các lỗi trọng tâm nhánh còn lại và ghi đúng giới hạn môi trường.

Tiêu chí	0 điểm	1 điểm	2 điểm
Hiểu công cụ	Trộn IDE, shell, CLI	Phân biệt được một phần	Gọi đúng vai trò và đúng shell
Giới hạn Agent	Cho sửa tùy ý	Có giới hạn nhưng thiếu kiểm tra	Ask trước, phạm vi nhỏ, xem diff
Giải thích lỗi	Chỉ chép đáp án	Nêu nguyên nhân, thiếu trace	Trace đúng, giải thích cả ca biên
Bằng chứng kiểm thử	Chỉ nói "chạy rồi"	Có output, thiếu ngữ cảnh	Có input, lệnh/thao tác, kết quả và trạng thái
An toàn và trung thực	Dữ liệu thật hoặc bịa đã chạy	Có ghi thiếu sót nhưng chưa đủ	Dữ liệu giả, checkpoint, phần chưa chạy rõ ràng

8–10 điểm: chuyển tiếp, giữ thói quen kiểm chứng. 5–7: làm lại tiêu chí thấp nhất. Dưới 5: quay lại mở đúng gốc, trace một phần tử. Không suy năng lực dài hạn từ một phiên hay bịa log vì thiếu compiler.

Tiêu chí hoàn thành và năng lực mang theo

Nêu được test nào chạy mã nào, đối chiếu đủ ba bảng, giữ nhật ký và tự chấm có căn cứ. Báo cáo một phút: lỗi, vị trí sửa hoặc lý do không sửa, bằng chứng, phần chưa kiểm.

Minh giải thích được lượt dừng, reference và executable cũ; Lan lần được chuỗi qua parse, schema, tổng, lưu trữ và biết giữ test hồi quy. Cả hai có quyền từ chối dependency hoặc đổi quyền ngoài phạm vi.

Mang sang chương sau: **quan sát → dự đoán → thay đổi nhỏ → kiểm tra → review → ghi bằng chứng**. Chưa chạy thì nói chưa chạy; không có quyền thì dừng; không có lý do thì không mở rộng hệ thống.

Nguồn gốc, cập nhật và giới hạn xác minh

Kế thừa `ban-thao/02-agent-ide-cli.md`, tổ chức theo Blueprint và Visual Design System. Hành trình, quyết định dependency và câu kiểm mới là **bổ sung biên tập**; hội thoại hư cấu, fault lab biệt lập. Các bảng C/W/T, rubric giữ chuyên môn nguồn, có chú thích độ bao phủ theo API thật.

Mô´c đô´i chiê´u **30/10/2026** theo Source Audit cập nhật, không phải vừa truy cập lại mọi nguô`n hay quan sát tài khoản bạn. Audit phân biệt 33 ca gô´c và 34 ca mới. Lượt biên tập này chạy lại Node trên bản tham khảo: **34 passed, 0 failed**, exit 0; không thay kiểm bài làm của bạn.

Mã đô´i chiê´u: C++ lỗi, C++ đúng/9 test, HTML, app.js, data.js, 34 test. Link chỉ để đọc; sửa trong `bai-lam`.

Nguô`n chính thức dùng để tra thao tác và ngữ nghĩa:

- Qoder IDE quick start, Chat overview, Ask, Agent: vai trò chê´ độ và review thay đổi.
- Context, Terminal: chọn ngữ cảnh, terminal và chính sách chạy lệnh của IDE.
- Qoder Desktop overview, Desktop new task: Coding/General, không đô`ng nhâ´t General với Ask chỉ đọc.
- Qoder CLI quickstart, CLI working modes: sản phẩm CLI, điê`u kiện và chê´ độ riêng.
- MDN parseInt, Number.isSafeInteger: đọc một phâ`n chuỗi và miê`n nguyên an toàn.
- MDN localStorage, textContent: giới hạn `file://` và hiển thị văn bản.

Trong lượt biên tập này chưa có `g++` được tìm thâ´y để compile/run C++; chưa thử UI Qoder, tài khoản, quyê`n Auto-Run, DOM hay persistence/fallback trên browser thực. Tham chiê´u ba hình workspace/workflow/debugging giữ đúng đường dẫn thiê´t kê´; ảnh chưa hiện diện tại thời điểm kiểm nên chưa xác nhận render. Các giới hạn â´y là việc phải kiểm tiê´p, không được đổi thành “đã đạt” chỉ vì Markdown hoặc test dữ liệu hợp lệ.

03

CHƯƠNG 03 / MINH & LAN

Đừng để AI suy nghĩ thay bạn

Gợi ý có thể đến từ AI. Năng lực phải được chứng minh bằng việc chính bạn làm được.



Tình huống và nhân vật hư cấu · Đọc, dự đoán rồi tự kiểm chứng. Xem hình lớn

Minh đóng cửa sổ chat, nhìn lại vòng lặp vừa sửa. Tổng đã thành **60000**, nhưng khi Lan hỏi "Nếu chỉ có một khoản thì sao?", Minh lại muốn mở chat. Lan cũng có chuyện tương tự: giao diện Web hoạt động, nhưng cô chưa giải thích được vì sao giá trị đọc từ ô nhập phải qua hai bước chuyển kiểu và kiểm tra. Hai bạn thống nhất một cuộc hẹn khác: không mang đến câu trả lời dài nhất, mà mang đến **một điều tự giải thích được khi không còn nhìn lời giải**.

Nhân vật, tình huống và hội thoại trong chương đều HƯ CẤU. Minh học C++ năm nhất; Lan học Web/OOP/DSA/CSDL năm hai. Đây không phải khảo sát, chuẩn đầu ra hoặc quy định chính thức của TLU. Quyền dùng AI trong bài nộp tùy từng học phần.

Chương này luyện dự đoán, tự sửa, kiểm chứng và vận dụng. Chọn một nhánh thực hành sâu, đọc nhánh kia để đối chiếu; không cần làm hết trong một buổi.

Quy ước: gọi gố´c `AI-Student-Handbook` là **NEW**. Mở NEW, dùng bản sao `bai-lam/nam-1-cpp` và `bai-lam/nam-2-web` đã tạo ở Chương 2; `vi-du` chỉ tham khảo. Nê´u thiê´u bản sao, làm mục 2.1 của Chương 2. Ghi trong vở hoặc `bai-lam/nam-1-cpp/ghi-chep-02.txt` (tệp đã tạo ở Chương 2). Không cài đặt, đổi cấ´u hình, đăng nhập, gọi API hay truy cập mạng. Không có AI thì dùng hội thoại hư cấ´u làm phiê´u gợi ý, ghi đúng là học bằ`ng phiê´u.

3.1. AI TRAP — Câu trả lời tự tin có giữ lại phầ`n học của bạn?

Một hợp đồ`ng nhỏ để không tranh luận bằ`ng cảm giác

ViSinhVien dùng dữ liệu **hoàn toàn giả** sau. Bảng này là điểm tựa để tính tay, không phải lịch sử chi tiêu thật.

id	title	amount	category
1	Cơm trưa	35000	`an-uong`
2	Xe buýt	7000	`di-lai`
3	Vở ghi	18000	`hoc-tap`

"Hợp đồ`ng" là điề`u kiện đầ`u vào và kê´t quả phải bảo đảm, không phải điề`u AI được tự thay cho tiện viê´t code.

Phầ`n hợp đồ`ng	Quy định của bài
Tiề`n	`amount` là số´ nguyên VND từ 1 đế´n 1000000000, không lưu chuỗi có dấ´u phân cách.
Danh sách	Tố´i đa 500 khoản; rỗng hợp lệ, tổng bằ`ng 0.
Tổng	Mẫu có tổng 60000; cực đại là `500 × 1000000000 = 500000000000`.
C++ năm nhấ´t	Chỉ dùng `vector<long long>` chứa tiề`n; không yêu cầ`u class `Expense`.
Web năm hai	Mỗi `Expense` có `id`, `title`, `amount`, `category`.
`id` Web	Số´ nguyên an toàn, dương, không trùng trong danh sách.
`title` Web	Chuỗi; sau `trim()` dài 1–80 **đơn vị UTF-16**, theo `.length` của JavaScript.
`category` Web	**Chuỗi** thuộc `an-uong`, `di-lai`, `hoc-tap`, `khac`; mảng `['khac']` không hợp lệ.

Trong C++, `long long` đủ chứa tổng cực đại, còn số nguyên có dấu 32 bit thì không. Trong JavaScript, 500000000000 nằm trong miền số nguyên an toàn của `Number`, giới hạn trên là 9007199254740991. "Biểu diễn được" vẫn khác "được phép": số âm biểu diễn được nhưng vi phạm hợp đồng tiền. Độ dài UTF-16 cũng không luôn bằng số ký tự nhìn thấy; một số ký hiệu được tính bằng hai đơn vị.

Bản Web là HTML/JavaScript với `localStorage`, không backend, đăng nhập thật hoặc SQLite tích hợp. Bài SQL/SQLite dùng cùng dữ liệu là bài CSDL riêng. Demo Web hiện **đúng**: nếu chưa có dữ liệu lưu, nó bắt đầu bằng ba khoản, tổng **60000**. Đừng chẩn đoán một lỗi do cẩm nang cố ý dựng trong mảng lab thành lỗi của demo.

Bẫy: đúng một kết quả rồi kết luận đúng tất cả

Bổ sung biên tập — phiếu trả lời AI HƯ CẤU, cố ý sai:

> "Đã đổi cận vòng lặp và thấy 60000, vậy chương trình xử lý mọi đầu vào an toàn. Ở Web cứ dùng `Number` cho mọi giá trị là đủ. Muốn tổng theo nhóm thì lọc trước, dữ liệu nhóm khác không liên quan."

Chưa cần viết lại hàm. Gạch chân ba kết luận, rồi với mỗi kết luận nêu một đầu vào cần thử. Nếu chỉ trả lời "AI có thể sai", bạn mới nhắc lại khẩu hiệu, chưa có phản biện kỹ thuật.

▼ Đáp án AI TRAP — Ba kết luận vượt quá bằng chứng · mở sau khi tự thử

Đổi cận chỉ sửa việc bỏ cuối; `{0}` hoặc 501 khoản vẫn cần validation riêng. `Number("")` cho 0, chuyển được nhưng không hợp lệ. Lọc trước có thể che khoản sai: tiền chuỗi ở `di-lai` phải khiến yêu cầu tổng `hoc-tap` bị từ chối. Hãy tách yêu cầu, tìm phản ví dụ và xác định phạm vi test thay vì tranh luận về giọng văn AI.

Sáu tình huống: ranh giới nằm ở nhiệm vụ đang luyện

"Tận dụng" giữ phần học cần thiết ở phía bạn; "giao việc học cho AI" để AI nghĩ thay phần đang cần luyện. Không chỉ nhìn code do AI gợi ý rồi kết luận về tính liêm chính.

Tình huống	Tận dụng để học	Dấu hiệu giao việc học cho AI	Vùng xám phải kiểm tra
1. Minh gặp lỗi biên dịch	Gửi lỗi và đoạn tối thiểu, tự giải thích nguyên nhân sau gợi ý.	Dán cả bài, thay bằng lời giải không đọc.	Bản vá rất nhỏ vẫn có thể bị cấm trong bài đánh giá không cho AI.
2. Minh học vòng lặp	Xin câu hỏi về chỉ số, tự trace ba khoản.	Xin vòng lặp hoàn chỉnh rồi học thuộc.	Được xem mẫu không đồng nghĩa được nộp thuật toán do AI xây.
3. Lan đọc tiếng Anh	Giải nghĩa thuật ngữ, đối chiếu câu gốc.	Chỉ đọc tóm tắt rồi nhận đã đọc tài liệu.	Dịch hỗ trợ khác với viết hộ phần phân tích được chấm.
4. Lan thiết kế test	Tự tính kỳ vọng, nhờ tìm loại ca còn thiếu.	Nhận cả hàm lẫn kỳ vọng test từ một câu trả lời.	Môn có thể cho sinh test nhưng yêu cầu khai báo.
5. Hai bạn làm giao diện	Sinh khung không phải trọng tâm rồi tự nối dữ liệu.	Không biết nút Lưu gọi gì nhưng nhận đã làm chủ Web.	Nếu đang học HTML, khung sinh sẵn có thể bỏ qua mục tiêu chính.
6. Chuẩn bị bài nhóm	Thống nhất hỗ trợ, mỗi người giải thích phần mình.	AI hoàn thành phần chưa làm nhưng báo tự làm hoàn toàn.	Quy tắc nhóm, ghi nhận hỗ trợ và cách đánh giá có thể khác nhau.

Trước khi mở chat, viết: “Bài đang luyện ___; AI chỉ hỗ trợ ___; tôi tự chứng minh ___.” Minh chọn điều kiện dừng, Lan chọn đường đi của kiểu dữ liệu. Nếu quyền sử dụng chưa rõ, đọc hướng dẫn học phần và hỏi giảng viên trước khi dùng cho bài nộp. Khai báo không tự biến một cách dùng bị cấm thành được phép.

Vibe coding có kiểm soát: dựng nhanh, nhận chậm

Ở đây, **vibe coding** là mô tả mong muốn bằng ngôn ngữ tự nhiên, nhận code và điều chỉnh qua nhiều lượt. Dựng nhanh có ích để thử ý tưởng; chạy được không tự chứng minh người làm hiểu chương trình.

Lan có thể nhận hàng trăm dòng rồi liên tục nhắn “sửa hết”. Cách có kiểm soát ngược lại: biết `index.html` chứa giao diện, `data.js` xử lý dữ liệu, `app.js` nối sự kiện và lưu trữ; chọn một vấn đề, dự đoán, nhận một thay đổi, kiểm vài ca rồi mới đi tiếp. Không thêm framework, dịch vụ hay đăng nhập để giải quyết lỗi chuỗi tiền.

Hiểu không phải thuộc mọi API. Bạn được tra tên phương thức nhưng phải trả lời bốn câu:

1. Đoạn code nhận dữ liệu gì, kiểu nào, từ đâu?
2. Nó trả về hoặc thay đổi điều gì?
3. Nó dựa vào điều kiện nào của hợp đồng?
4. Nếu bỏ một điều kiện, tôi tạo được ca nào khiến nó sai?

Với `localStorage.setItem`, Lan cần biết nó lưu chuỗi, có thể thất bại, và không được báo lưu bền khi chỉ cập nhật bộ nhớ. Với `vector`, Minh cần biết ba phần tử có chỉ số 0, 1, 2. Quên tên API thì tra;

không biết vì sao thiếu phần tử cuối thì trace.



Bẫy AI: từ câu trả lời tự tin, quay về dự đoán, phản ví dụ và bằng chứng do người học kiểm tra. Xem hình lớn

3.2. THINK FIRST / PREDICT BEFORE ASKING — Viết dự đoán trước câu hỏi

Một phiếu ngắn trước khi nhấn Gửi

Chưa có lời giải vẫn được hỏi. Viết “cộng từ khoản đầu đến khoản cuối” rồi thử một khoản để tạo chỗ bám cho gợi ý.

Điều cần ghi	Mình điền trước khi hỏi	Nếu chưa biết
Đầu vào	`35000, 7000, 18000`, số nguyên	Chép đúng dữ liệu và kiểu đang có.
Kỳ vọng	60000, do cộng đủ ba khoản	Tính tay ca một khoản trước.
Hiện tượng	Dự đoán cận lỗi cho 42000; chưa chạy	Ghi “chưa chạy”, không mượn log của sách.
Giả thuyết	Có thể chưa cộng Vở ghi	Ghi nơi nghi ngờ, chưa khẳng định nguyên nhân.

Điều cần ghi	Mình điền trước khi hỏi	Nếu chưa biết
Điều xin hỗ trợ	Một câu hỏi về chỉ số	Không xin đồng thời giải thích, sửa cả bài và sinh test.

Chương 2 đã giải cận và build; lần này luyện **giữ từng bước suy nghĩ**. Nếu nhớ lời giải, hãy tự giải thích ca mới.

Hội thoại C++: người học phải đi qua lượt dừng

Đây là **mã lỗi chủ đích** từ `loi-bo-sot.cpp`. `amounts` là `std::vector<long long>{35000, 7000, 18000}`, `total` là `long long` khởi tạo 0.

```C++
int n = static_cast<int>(amounts.size());
for (int i = 0; i < n - 1; ++i) { // CỐ Ý SAI
    total += amounts[i];
}
```

Trước khi mở đoái chiếu, tự ghi `n`, các chỉ số hợp lệ và từng lần kiểm tra điều kiện, kể cả lần sai khiến vòng lặp dừng. `size()` là số lượng, không phải chỉ số cuối. Với ba khoản mẫu, `int n` không có rủi ro. `loi-bo-sot.cpp` **chưa** kiểm giới hạn 500; bản có kiểm số lượng trước khi ép sang `int` nằm ở mục 3.6. Không suy ra mọi phép ép kích thước về `int` đều an toàn.

▼ Đáp án hội thoại C++ — HƯ CẤU, không phải log thực thi · mở sau khi tự thử

Minh: Em tính tay 60000. Với cận đang đọc, em dự đoán 42000. Em nghi thiếu Vở ghi nhưng chưa biết phải sửa thế nào.

Người hướng dẫn: Chưa sửa. Ba phần tử có chỉ số nào?

Minh: 1, 2, 3... Em nhầm: vector bắt đầu ở 0, vậy là 0, 1, 2.

Người hướng dẫn: Khi ấy `n` và điều kiện lượt đầu bằng gì?

Minh: `n = 3`, nên `n - 1 = 2`. Lượt đầu `0 < 2` đúng, tổng thành 35000.

Người hướng dẫn: Em tự đi tiếp, cả lượt không được chạy.

Minh: `i = 1`, `1 < 2` đúng, tổng thành 42000. Đến `i = 2`, `2 < 2` sai. Chưa đọc 18000 mà đã dừng. Phép cộng không sai; thiếu một số hạng.

Lâ`n kiểm tra	i	Điê`u kiện lỗi	Khoản được cộng	total sau kiểm tra
1	0	`0 < 2` : đúng	Cơm trưa 35000	35000
2	1	`1 < 2` : đúng	Xe buýt 7000	42000
3	2	`2 < 2` : sai	Không có	42000

Người hướng dẫn: Em muô´n đọc chỉ sô´ nào và không được đọc chỉ sô´ nào?

Minh: Đọc 0, 1, 2; không đọc 3. Em đê` xuâ´t `i < n`. Khi `i = 2`, cộng thêm 18000 thành 60000; tăng lên 3 thì dừng.

Người hướng dẫn: Một khoản Vở ghi và danh sách rỗng có giúp kiểm đê` xuâ´t không?

Minh: Một khoản: cũ kiểm `0 < 0`, bỏ luôn khoản duy nhâ´t; mới kiểm `0 < 1`, cộng 18000. Rỗng: mới kiểm `0 < 0`, tổng giữ 0. Cũ với `int n = 0` cũng không lặp, nên riêng ca rỗng không phân biệt được hai cận.

Đâ`u vào	Kỳ vọng cận lỗi	Kỳ vọng cận đúng
Ba khoản mẫu	42000	60000
Chỉ 18000	0	18000
Rỗng	0	0

Đây là **trace thủ công và đâ`u ra kỳ vọng**, không phải bă`ng chứng đã biên dịch. Sửa cận không tự thêm kiểm tra tiê`n hoặc sô´ lượng. Cũng không thay bă`ng `size() - 1` cho "gọn": kích thước thuộc kiểu không dâ´u, trừ 1 khi rỗng có thể tạo giá trị râ´t lớn; xem giải thích ở mục 2.3.

Nê´u dùng AI làm người hỏi, yêu câ`u chờ câu trả lời, không để nó tự nhận là giảng viên của bạn. Giá trị của hội thoại nă`m ở câu Minh tự nói vê` chỉ sô´ 2, không phải sô´ lượt trao đổi.

Phiên 25 phút CÓ AI hỗ trợ: sáu bước, sáu dâ´u vê´t

Đây là phiên học **có giai đoạn được hỏi AI**, khác hoàn toàn thử thách không AI ở mục 3.5. Chọn một mục tiêu: Minh sửa cận; Lan giải thích tiê`n chuỗi trong lab. Chỉ dùng phương tiện đã được phép; nê´u không truy cập AI, thay lượt hỏi bă`ng phiê´u gợi ý hoặc bạn học và ghi rõ cách làm.

Thời gian	Việc phải làm	Dấu vết cần giữ
0–4 phút: tự thử	Đọc đề, phác một cách, dù còn thiếu.	Code tự viết hoặc mô tả thuật toán.
4–7 phút: dự đoán	Tính tay; ghi chỉ số hoặc kiểu từng chặng.	Kỳ vọng viết **trước** chạy và trước hỏi.
7–11 phút: xin gợi ý	Một câu mỗi lượt, tự trả lời rồi mới hỏi tiếp.	Câu gợi ý và câu trả lời của bạn.
11–16 phút: tự sửa	Chỉ sửa nguyên nhân đã giải thích được.	Dòng đổi cùng lý do.
16–21 phút: kiểm chứng	Ca thường, biên và sai theo phạm vi việc sửa.	Kỳ vọng, thực tế, kết luận; ghi validation còn thiếu nếu có.
21–25 phút: biến thể không AI	Đóng chat, đổi dữ liệu hoặc câu hỏi.	Bài tự làm và chỗ chưa chắc.

Minh dự đoán `0 → 35000 → 42000 → 60000`; Lan vẽ "ô nhập → đối tượng → danh sách → tổng". Chưa viết được code vẫn bắt đầu bằng hai dấu vết này. Khi mới sửa cận, test tiền sai chưa bị từ chối là phần validation còn thiếu.

Thử tối đa ba lượt gợi ý cho một nút thắt, theo quy ước phiên học. Mỗi lượt phải dẫn đến một hành động tự làm: điền trace, sửa dòng hoặc nghĩ test. Vẫn vướng thì thu xuống một khoản.

Khi đáp án lộ, hoặc hết giờ mà chưa xong

Nếu AI viết luôn điều kiện đúng, ghi "đã xem lời giải phần cận", đóng lại, tự giải thích rồi thử ca mới. Câu đã lộ chỉ tính là ôn tập. Thẻ P02 (bảng thẻ P01–P08 ở cuối mục 3.3 và trong Thư viện prompt) có thể xin bảng trống, nhưng không bảo đảm AI tuân thủ.

Nếu mệt, ghi cụ thể "đã biết chỉ số cuối, chưa biết lập điều kiện" rồi dừng. Thời lượng không chẩn đoán khả năng tập trung hay năng lực ngành học. Phiên sau thu nhỏ nhiệm vụ; khi hỏi người hướng dẫn, mang đầu vào, kỳ vọng, thực tế và điều đã thử.

3.3. REVERSE ENGINEERING / EXPLAIN IT BACK — Lần ngược rồi giải thích lại

Mục này cố ý ghép hai bước vì chúng đi liền nhau: phần đầu là **REVERSE ENGINEERING** (lần ngược luồng dữ liệu), phần "Giải thích lại" bên dưới là **EXPLAIN IT BACK**. Mục 3.7 cuối chương là **SKILL UNLOCKED**.

Từ một con sô´ trên màn hình vê` nơi dữ liệu được tạo

Reverse engineering ở đây là đọc ngược chương trình để hiểu luô`ng. **Explain it back** là giải thích bă`ng lời của bạn, kèm vị trí code và ví dụ.

Trong `bai-lam/nam-2-web`, Lan lâ`n từ tổng hiển thị vê` `render()`, `D.totalExpenses(expenses)`, chỗ gán `expenses`, handler `submit` vô danh và `D.parseAmount` trong `app.js`. Sau đó mở `data.js` đọc hàm, `index.html` kiểm form và thứ tự script. Không bịa `handleSubmit` khi mã dùng callback vô danh.

Chặng câ`n lâ`n	Vị trí/vai trò	Kiểu và điê`u phải giải thích
Ô nhập	`index.html`, đọc trong `app.js`	`.value` là chuỗi, dù người dùng gõ chữ sô´.
Phân tích tiê`n	`parseAmount` trong `data.js`	Kiểm cách viê´t, chuyển thành sô´, kiểm miê`n tiê`n.
Danh sách ứng viên	Handler `submit` trong `app.js`	Ghép khoản mới; `normalizeExpenses` kiểm cả danh sách trước khi gán.
Tính tổng	`totalExpenses` trong `data.js`	Kiểm tiê`n/sô´ lượng, trả sô´; không xác thực toàn schema.
Lưu	`save()` trong `app.js`	`JSON.stringify` tạo chuỗi để ghi vào `localStorage`; ghi có thể lỗi.
Hiển thị	`render()` trong `app.js`	Định dạng để nhìn, không biê´n tiê`n lưu thành chuỗi có dâ´u châ´m.
Đọc lại	Khô´i khởi tạo trong `app.js`	`JSON.parse` rô`i `normalizeExpenses`; JSON hợp lệ chưa chă´c đúng schema.

Trong handler, vê´ phải `D.normalizeExpenses([...expenses, khoản_mới])` phải thành công trước khi gán lại `expenses`. Dữ liệu sai bị ném lỗi và bă´t ở giao diện, nên không được thêm vào danh sách. Dữ liệu hợp lệ được cập nhật trong bộ nhớ, sau đó mới gọi `save()` và `render()`: lưu thâ´t bại không hoàn tác khoản vừa thêm trong RAM, nhưng phải báo không lưu bê`n.

Lab kiểu dữ liệu: không làm hỏng demo đúng

Giữ nguyên `data.js` đang đúng. Trên giâ´y hoặc vùng nháp, giả lập ba `amount` là chuỗi `"35000"`, `"7000"`, `"18000"`, biê´n `total` ban đâ`u là sô´ 0. Mỗi lượt thực hiện `total = total + expense.amount`. Đây là **lab lỗi chủ đích**, không phải mô tả trạng thái dữ liệu mẫu của ứng dụng.

Chưa mở đáp án: lập bảng lượt, `total` trước và kiểu, `amount` và kiểu, `total` sau và kiểu. Giữ dâ´u ngoặc kép; bỏ chúng làm mâ´t phâ`n quan trọng nhâ´t của bài.

▼ Đáp án hội thoại Web — HƯ CÂ´U, giải thích bă`ng giá trị và kiểu · mở sau khi tự thử

Lan: Trong lab, em dự đoán kê´t quả là chuỗi dài chứ không phải tiê`n. Em định thay toán tử cộng.

Người hướng dẫn: Trước khi đổi toán tử, em biê´t kiểu của `.value` chưa?

Lan: Em từng nghĩ gõ sô´ thì nhận sô´. Nhưng giá trị của ô nhập là chuỗi. Em phải kiểm `typeof` nê´u muô´n quan sát trên Console, không đoán bă`ng hình dáng.

Người hướng dẫn: Vậy lượt đâ`u, bên trái và bên phải là gì?

Lan: Sô´ 0 cộng chuỗi `"35000"` thành chuỗi `"035000"`. Tổng đã đổi kiểu ngay lượt đâ`u.

Người hướng dẫn: Em làm hai lượt tiê´p và ghi cả kiểu.

Lượt	total trước và kiểu	amount và kiểu	total sau và kiểu
1	`0`, number	`"35000"`, string	`"035000"`, string
2	`"035000"`, string	`"7000"`, string	`"0350007000"`, string
3	`"0350007000"`, string	`"18000"`, string	`"035000700018000"`, string

Lan: Sau lượt đâ`u không còn cộng sô´. Em đang nô´i ký tự liên tiê´p.

Người hướng dẫn: Nên sửa ở đâu để phía sau không phải đoán kiểu?

Lan: Ở biên nhập liệu: kiểm chuỗi, chuyển sô´, kiểm giá trị rô`i mới tạo `Expense`. Hàm tổng vẫn phải từ chô´i tiê`n sai vì dữ liệu có thể đê´n từ đường khác.

Người hướng dẫn: Chỉ dùng `Number` đủ không? Xét chuỗi rỗng và sô´ âm.

Lan: Không. `Number("")` cho 0; `Number("-7000")` cho sô´ âm. Cả hai không là khoản hợp lệ. Em cho phép khoảng tră´ng ngoài cùng, rô`i chỉ nhận chữ sô´; từ chô´i chă´m, phẩy, chữ và ký hiệu mũ.

Người hướng dẫn: Khi tải lại trang, dữ liệu đã lưu có cà`n kiểm nữa?

Lan: Có. `localStorage` giữ chuỗi JSON. Parse được chỉ chứng minh cú pháp JSON; vẫn phải kiểm danh sách, tiê`n, id, title, category. Lỗi đọc không được âm thâ`m đổi dữ liệu cũ thành danh sách rỗng rô`i ghi đè.

Các kê´t quả trên là **kỳ vọng từ trace**. Khi tiê`n đã là sô´, trace đúng là `0 → 35000 → 42000 → 60000`. `42000` ở giữa là tổng trung gian đúng; `42000` cuô´i vòng lặp C++ lỗi là kê´t quả sai. Cùng con sô´, khác

thời điểm và yêu cầu.

EXPLAIN IT BACK — Giải thích lại: vượt qua câu "em hiểu rồi"

Đóng phần hội thoại. Trong khoảng một phút, kể đường đi của `"18000"` từ ô nhập đến số trong danh sách rồi chuỗi JSON. Chỉ vào tên hàm khi nói. Nếu kẹt, đánh dấu chặng bị thiếu trước khi mở lại tài liệu. **Tự kiểm:** mở lại bảng "Chặng cần lần" ở đầu mục này và đánh dấu từng chặng bạn đã nói được; thiếu chặng nào thì đó là việc ôn tiếp theo. Mình chỉ tham gia phần này như **review chéo với Lan**, không ngầm chuyển sang làm dự án Web năm hai.

Dùng thêm câu tiếng Anh **do cẩm nang tự soạn, không phải trích dẫn chính thức**: "The value of a text input is a string. Convert and validate it before storing an expense amount." Giải nghĩa `value` là giá trị, `string` là chuỗi, `convert` là chuyển kiểu, `validate` là kiểm tra hợp lệ. Sau đó tự diễn giải cả câu và chọn một trường hợp chuyển được nhưng không hợp lệ. Với tài liệu thật, phải giữ đoạn gốc và nguồn để đối chiếu; bản dịch không thay việc đọc điều kiện áp dụng.

Về lưu trữ, bản demo khởi tạo mẫu trước. Không có bản lưu thì giữ ba khoản 60000; có bản lưu hợp lệ thì dùng bản đó, kể cả `[]` với tổng 0. Nếu đọc/parse/normalize thất bại, `persistent = false`, dùng mẫu trong bộ nhớ và thông báo không ghi đè dữ liệu cũ. Nếu ghi thất bại, `save()` cũng chuyển sang bộ nhớ tạm, các thay đổi sau trong phiên không thử ghi tiếp. Nút khôi phục mẫu không tự bật lại quyền lưu.

`file://` có hành vi lưu trữ phụ thuộc trình duyệt. Kiểm reload ở cùng file và cùng trình duyệt; nếu dùng HTTP có sẵn, cần cùng origin, tức cùng giao thức, host và cổng. Khác địa chỉ có thể là vùng lưu khác. Không dựng server hoặc backend chỉ để hoàn thành bài đọc này.

Tám chức năng prompt gốc: chọn đúng một điểm nghẽn

Tám chức năng dưới đây kế thừa mục 3.4 bản nguồn; thẻ đầy đủ, có ngữ cảnh và cách xử lý lộ đáp án, nằm trong Thư viện prompt. Chương giữ cách chọn và bằng chứng học, không bắt bạn chép lại tám khối dài.

Thẻ	Chức năng và khi dùng	Bạn phải chuẩn bị/làm	Nếu đáp án lộ
P01	Socratic, một câu mỗi lượt; chưa biết vì sao bỏ khoản cuối.	Tự nêu chỉ số, trace, giải thích dòng sửa.	Ghi đã xem; đóng lại và trace ca một khoản mới.
P02	Ba tầng gợi ý: vùng cần xem → quan hệ chỉ số/kích thước → bảng trống.	Sau mỗi tầng báo điều vừa thử, không xin cả ba cùng lúc.	Tự so ca rỗng **và** một khoản; rỗng riêng lẻ chưa phân biệt cận.
P03	Trace giá trị và kiểu ở lab tiền chuỗi.	Điền từng hàng, chỉ lượt đầu đổi kiểu.	Che bảng, tự trace một chuỗi khác.

Thẻ	Chức năng và khi dùng	Bạn phải chuẩn bị/làm	Nê´u đáp án lộ
P04	Review đúng một thay đổi nhỏ.	Chứng minh cận, chọn test, nói validation nào chưa làm.	Không thay cả file; tự so hai cận với `n = 1`.
P05	Test trước code.	Tự đề` xuấ´t ca thường, biên, sai và tính kỳ vọng.	Đóng lời giải, tính ca 500 khoản tố´i đa.
P06	Giải thích tiế´ng Anh có đố´i chiế´u.	Diễn giải câu gố´c, phân biệt chuyển kiểu/kiểm hợp lệ.	Tự phân tích `"35000abc"` bằ`ng lời.
P07	Vấ´n đáp như người hướng dẫn.	Trả lời bằ`ng ví dụ về` chỉ số´, kiểu, test, lưu trữ.	Bỏ câu đã lộ khỏi đánh giá độc lập, chuyển câu mới.
P08	Giao biế´n thể rồ`i đóng chat.	Tự làm max C++ hoặc tổng nhóm Web; nêu chỗ chưa chứng minh.	Chuyển sang bài đảo thứ tự, chưa xem kế´t quả.

Một thẻ không làm thay nhiệm vụ chuẩn bị. P05 không hữu ích nế´u bạn để AI tính luôn mọi kỳ vọng; P07 không phải công cụ đoán điểm học phầ`n. Bố´n phút tự thử trong P08 chỉ là lượt ngắ´n, không thay thử thách 25 phút ở mục 3.5.

3.4. MEMORY EXPERIMENT — Tự quan sát điề`u còn nhớ, không kế´t luận nhân quả

Hai nhiệm vụ gầ`n tương đương, một mục tiêu quan sát

Bổ sung biên tập. Hoạt động tự quan sát nhớ lại, không nghiên cứu tác động AI lên trí nhớ. Hai nhiệm vụ cùng số´ khoản, phép cộng, giới hạn chỉ số´; chưa có phép đo chứng minh độ khó tương đương.

Chuẩn bị giấ´y, đồ`ng hồ`, 12–15 phút; không cầ`n AI. Dùng cận đúng đã học, đổi thứ tự dữ liệu:

- **A:** `35000, 7000, 18000`. Đọc câu trả lời của Minh về` cận `i < n` và bảng "Đầ`u vào / Kỳ vọng cận đúng" ở mục 3.2 (tổng `0 → 35000 → 42000 → 60000`) trong khoảng 90 giây, rồ`i gấ´p phầ`n đó lại. Ghi rõ A đã được đọc lời giải.
- **B:** `18000, 35000, 7000`. Trong khoảng 90 giây, tự viế´t bảng chỉ số´/tổng mà không mở lời giải. Không chạy trước để lấ´y kế´t quả.

Tung đồ`ng xu hoặc chọn thứ tự B trước A nế´u thuận tiện; ghi thứ tự thật. Không coi việc đổi thứ tự đã loại bỏ ảnh hưởng luyện tập. Nế´u chưa hiểu vòng lặp, làm cả hai với trợ giúp và ghi đúng mức hỗ trợ; lúc này chỉ quan sát nhớ lại, không so "có AI" với "không AI".

Quy trình: tách lúc học khỏi lúc nhớ lại

Sau từng nhiệm vụ, che code và ghi chép, nghỉ khoảng hai phút bằng một việc không liên quan đến bài. Sau đó trả lời cùng bốn câu trên giấy trắng: chỉ số hợp lệ là gì; tổng sau mỗi lượt là gì; điều kiện nào làm dừng; với một khoản và rỗng sẽ ra sao, vì sao? Không mở kết quả trong khi trả lời. Cuối cùng mới đối chiếu và đánh dấu phần cần nhìn lại.

Nếu máy hoặc hoàn cảnh không cho nghỉ đúng thời lượng, ghi thời gian thực tế. Ngày hôm sau có thể trả lời lại bốn câu trước khi đọc chương; đây là lượt tự kiểm bổ sung, không bắt buộc để "đạt". Không chép câu vừa đọc rồi gọi đó là nhớ lại độc lập.

Nhiệm vụ	Thứ tự/thời gian thật	Hỗ trợ đã xem	Tự nhớ được ý nào	Ý sai hoặc cần mở lại	Giải thích được ca mới?
A	…	Bảng đã có lời giải	…	…	…
B	…	Tự lập bảng; ghi thêm nếu có	…	…	…
Lượt hôm sau, nếu làm	…	Chưa xem lại/đã xem lại	…	…	…

<details>
<summary>Đáp án đối chiếu MEMORY EXPERIMENT · mở sau khi tự thử</summary>

A có tổng lần lượt 35000, 42000, 60000; B có 18000, 53000, 60000. Cả hai đọc chỉ số 0, 1, 2 và dừng trước 3 với cận `i < n`. Một khoản hợp lệ có tổng bằng chính khoản ấy; rỗng giữ 0. Dãy tổng trung gian khác nhau nhưng tổng cuối không đổi.

Đối chiếu từng ý: nhớ 60000 chưa chứng minh nhớ quá trình cộng; nhớ tên `long long` chưa chứng minh hiểu giới hạn tổng.
</details>

Viết kết luận đúng mức bằng chứng

Nhận xét **HƯ CẤU** phù hợp: "A tôi nhớ tổng cuối nhưng quên lượt dừng; B tôi tự viết được ba tổng trung gian. Tôi sẽ luyện điều kiện dừng khi đóng tài liệu." Không suy "AI làm tôi nhớ kém".

Kiến thức sẵn có, thứ tự, độ quen, mệt mỏi, thời gian, việc nhìn đáp án và cách tự chấm đều có thể ảnh hưởng. Không kiểm soát chúng và thiếu nhóm so sánh, bạn không suy nhân quả, tạo phần trăm hay gắn nhãn trí nhớ tốt/xấu. Chỉ chọn **hành động học tiếp**: nhớ lại, giải thích thành lời hoặc đổi dữ liệu.

3.5. 25-MINUTE NO-AI CHALLENGE — Tự làm trọn một nhiệm vụ nhỏ

Luật phiên này: không AI từ đầu đến cuối

Bổ sung biên tập. Đây là 25 phút độc lập, không phải 21 phút có AI rồi bốn phút đóng chat. Đóng hội thoại, không dùng Agent, sinh code, giải thích AI hoặc gợi ý AI trong cả phiên. Dùng vùng soạn thảo không hiện gợi ý AI đang có sẵn hoặc làm trên giấy; không đổi cấu hình công cụ để thi hành bài. Được đọc đề, hợp đồng, ghi chú cú pháp tự viết; không mở đáp án dưới đây hay nhờ người khác làm hộ.

Nếu vô tình nhìn thấy gợi ý, ghi thời điểm và phần đã thấy; tiếp tục luyện nhưng không gọi cả phiên là độc lập. Điều cần bảo vệ là chất lượng bằng chứng, không phải một dấu tích đẹp.

Nhiệm vụ chung: đếm số khoản có tiền **từ 10000 VND trở lên**. Danh sách đầu vào của riêng bài đếm này **đã được xác nhận hợp lệ** theo hợp đồng. Bạn chưa phải viết parser hoặc validation mới; hãy ghi rõ tiền đề đó trong lời giải.

- Minh viết `countLargeAmounts(const std::vector<long long>& amounts)`, trả số lượng bằng `int`.
- Lan viết `countLargeExpenses(expenses)`, dùng `expense.amount` là số, trả số lượng bằng `number`. Không đọc DOM hoặc `localStorage` trong hàm.

Chọn một nhánh. Không đổi nhiệm vụ thành tính tổng hoặc tìm số tiền lớn nhất. Chuẩn bị ca ba khoản mẫu, rỗng, một khoản 10000, một khoản 9999, và đảo thứ tự ba khoản. Với Web, các đối tượng đầy đủ và id không trùng; với C++, chỉ cần vector tiền.

Chia thời gian để còn chỗ kiểm chứng

Khoảng phút	Việc tự làm	Sản phẩm nhỏ
0–5	Diễn giải "từ 10000 trở lên"; tính kỳ vọng các ca.	Bảng trước chạy, có cả biên bằng 10000.
5–12	Viết thuật toán bằng lời rồi code hoặc giả mã.	Một biến đếm, cách duyệt, điều kiện xét khoản.
12–18	Trace từng khoản; nếu có môi trường sẵn thì chạy bản nháp.	Kết quả thực hoặc nhãn trace, không trộn hai loại.
18–22	Tự tìm và sửa một điểm sai/thiếu; nếu đã đúng, giải thích vì sao.	Dòng đổi và lý do hoặc lập luận bảo vệ code.
22–25	Đóng cả code, nói lại cách làm và ghi điểm nghẽn.	Giải thích ngắn, trạng thái cuối phiên.

Không cần thêm tệp: viết trong vở hoặc vùng nháp của ghi chép hiện có. Nếu thử C++ trong bản sao, giữ lại mã trước thay đổi để đối chiếu, không ghi đè bản tham khảo. Không có compiler/runtime thì hoàn thành giả mã và trace, đánh dấu phần thực thi chưa làm.

Đối chiếu theo bằng chứng, không theo tốc độ

Ghi có/chưa có cho bốn điểm: phân biệt đếm/cộng tiền; kỳ vọng độc lập; trace hoặc lần chạy được mô tả đúng; giải thích biên 10000 và rỗng. Giữ bản tự làm trước khi mở đáp án. Chưa xong thì ghi điểm nghẽn cụ thể, chẳng hạn "em tăng tiền thay vì tăng số lượng", để chọn việc phiên sau.

▼ Đáp án thử thách không AI — Đếm, không cộng tiền · mở sau khi tự thử

Kỳ vọng: ba khoản mẫu có **2** khoản đạt; rỗng **0**; `[10000]` cho **1**; `[9999]` cho **0**; đảo thứ tự vẫn **2**. Lượt cộng tiền 18000 không có nghĩa tăng biến đếm thêm 18000; mỗi khoản thỏa điều kiện chỉ tăng 1.

Các lõi sau là lời giải đọc sau phiên, giả định đầu vào đã hợp lệ; không thay thế validation ở mục 3.6.

C++

```
int countLargeAmounts(const std::vector<long long>& amounts) {
    int count = 0;
    for (long long amount : amounts) {
        if (amount >= 10000) ++count;
    }
    return count;
}
```

JavaScript

```
function countLargeExpenses(expenses) {
  let count = 0;
  for (const expense of expenses) {
    if (expense.amount >= 10000) count += 1;
  }
  return count;
}
```

Mỗi hàm duyệt một lần, thời gian O(n), bộ nhớ phụ O(1), không thay đổi đầu vào. Biến đếm C++ tối đa 500 nên `int` phù hợp; điều này không mâu thuẫn với việc **tổng tiền** phải dùng `long long`.

Sau khi xem đáp án, sửa lại bản nháp là ôn tập có đổ´i chiê´u, không sửa lịch sử thành “tự làm đúng ngay”.

3.6. AI CHALLENGE — Sửa ít, xác thực đủ, thử biê´n thể

Mục vận dụng: ghép các bước 3.1–3.5 vào một bài sửa thật trước khi điểm lại năng lực ở 3.7.

Chuẩn bị: một nhánh, một bản sao, ba loại bằ`ng chứng

Chọn Minh hoặc Lan, nộp code/nháp, bảng kỳ vọng–thực tê´ và giải thích. P04/P05/P08 là hỗ trợ tùy chọn. Mở NEW, dùng bản sao `bai-lam`: C++ sửa `loi-bo-sot.cpp`, giữ `vi_sinh_vien.cpp` đổ´i chiê´u; Web giữ bố´n tệp cùng thư mục, **tái dựng lõi ở lab riêng**, không chữa demo đúng.

Trace là lập luận; test logic kiểm hàm; thao tác trình duyệt kiểm sự kiện, hiển thị, lưu trữ. Không dùng một loại để nhận đã hoàn tấ´t cả ba.

Nhánh Minh: từ bản vá cận đê´n hợp đồ`ng đầ`y đủ

Bước 1 — Vá đúng một nguyên nhân. Trong bản sao `loi-bo-sot.cpp`, dự đoán ba khoản, một khoản 18000, rỗng trước sửa. Trace rồ`i đổi cận. Giữ ghi nhận: đây là bản vá duyệt trên đầ`u vào hợp lệ, chưa bổ sung validation.

Bước 2 — Bổ sung validation thành việc riêng. Tách hàm `totalExpenses` phía trên `main`, vẫn dùng `vector<long long>`. Kiểm số´ lượng trước khi chuyển `size()` sang `int`; kiểm từng tiề`n trước khi cộng. Danh sách sai phải ném `std::invalid_argument`, không bỏ qua khoản sai hoặc trả 0. Thêm `<stdexcept>` cho ngoại lệ, giữ `<vector>` và `<iostream>`. **Nê´u bước này quá lớn:** làm hai lượt nhỏ — lượt 1 chỉ thêm kiểm số´ lượng (quá 500 thì ném lỗi), build và thử; lượt 2 mới thêm kiểm từng tiề`n. Bản hoàn chỉnh để đổ´i chiê´u nă`m ở mục “Đáp án” cuố´i 3.6; chỉ mở sau khi đã tự thử.

Bước 3 — Gọi hàm và kiểm cả nhánh lỗi. Trong `main`, gọi hàm với vector đang thử. Đặt lời gọi trong `try`, bắ´t `std::invalid_argument` để báo lỗi và trả mã khác 0. Trọng tâm là tiề`n đã ở kiểu `long long`; chưa yêu cầ`u hệ thố´ng nhập bàn phím phát hiện mọi chuỗi sai. Khi mở rộng nhập sau này, phải kiểm cả việc đọc/chuyển đổi thành công trước khi đưa vào vector.

Bước 4 — Chạy đúng bản vừa sửa. `loi-bo-sot.cpp` không có `--test`. Thay vector dữ liệu trong bản sao để thử từng ca rồ`i build lại, hoặc tự viế´t bộ gọi test khi đã hiểu (xem hàm `runTests()` trong `vi_sinh_vien.cpp` làm mẫu); không gắ´n `--test` và tưởng chương trình sẽ tự kiểm. Dùng `std::vector<long long>(500, 1000000000LL)` để tạo ca cực đại, không gõ 500 số´.

Từ **gô´c NEW**, khi máy đã có `g++`, chọn đúng shell. Bash/Git Bash dùng nô´i `&&` để chỉ chạy khi build thành công:

```
Bash

g++ -std=c++17 -Wall -Wextra "bai-lam/nam-1-cpp/loi-bo-sot.cpp" -o "bai-lam/nam-1-cpp/loi-bo-sot.exe" &&
"./bai-lam/nam-1-cpp/loi-bo-sot.exe"
```

PowerShell dùng kiểm mã build:

```
PowerShell

g++ -std=c++17 -Wall -Wextra "bai-lam/nam-1-cpp/loi-bo-sot.cpp" -o "bai-lam/nam-1-cpp/loi-bo-sot.exe"
if ($LASTEXITCODE -eq 0) { & ".\bai-lam\nam-1-cpp\loi-bo-sot.exe" }
```

Chạy dòng thứ hai ngay sau dòng build: `$LASTEXITCODE` chỉ phản ánh lệnh vừa chạy trước nó. Đọc output và mã thoát theo mục 2.7; nê´u build lỗi thì dừng, không chạy executable cũ. Dùng IDE thì Build thành công rô`i Run đúng đích. Thiê´u compiler thì trace, ghi chưa thực thi, không tự cài trên máy chung.

Nhánh Lan: tái dựng lõi, không chữa một demo đang đúng

Bước 1 — Chọn trạng thái xuâ´t phát rõ ràng. Nê´u chưa có dữ liệu lưu, demo mở với ba khoản 60000. Muô´n luyện thêm Vở ghi từ tổng 42000, trên **bản sao với dữ liệu giả có thể thay**, xóa riêng Vở ghi bă`ng nút Xóa của dòng đó rô`i thêm lại. Xóa có thể được lưu; không dùng thao tác này với dữ liệu cầ`n giữ. Không xóa cả kho trình duyệt để đạt trạng thái bài.

Phương án không đụng lưu trữ: dùng mảng lab riêng gô`m hai đô´i tượng Cơm trưa và Xe buýt. Mảng này có tổng 42000; thêm `{id: 3, title: 'Vở ghi', amount: 18000, category: 'hoc-tap'}` thì kỳ vọng 60000. Thêm Vở ghi vào đủ ba mẫu sẽ cho **78000**, không phải 60000. Nê´u đang có dữ liệu cũ khác, tính theo dữ liệu thật đang hiển thị hoặc chuyển sang mảng lab; đừng giả định tổng ban đầ`u.

Bước 2 — Test trước lõi. Sau trim chỉ nhận chữ sô´, rô`i kiểm sô´ nguyên an toàn trong 1..1e9. Viê´t cách xử lý trước khi mở đáp án. Tách `parseAmount` khỏi `totalExpenses`; hàm tổng không chữa tiê`n chuỗi để che lỗi nguô`n dữ liệu.

**Bước 3 — Hai tầ`ng kiểm tra.** `{amount: 7000}` đủ cho riêng `totalExpenses`, vì hàm chỉ kiểm mảng, sô´ lượng và tiê`n. Đô´i tượng ấ´y thiê´u `id/title/category` nên không đủ cho `normalizeExpenses`. Normalize kiểm schema, trả danh sách mới, không chữa tiê`n chuỗi.

Bước 4 — Đọc chỗ nô´i. Tìm `D.parseAmount` rô`i gán qua `D.normalizeExpenses` trong `app.js`, không thay handler đúng bă`ng lab. Nhập sai phải giữ danh sách, tổng và dữ liệu lưu. Nhập hợp lệ có thể thêm

trong RAM khi ghi thất bại, nhưng phải báo bộ nhớ tạm.

Bước 5 — Kiểm trình duyệt. Mở `bai-lam/nam-2-web/index.html`, thêm lại Vở ghi từ hai khoản, thử đầu vào sai, đọc thông báo thêm/lưu rồi reload cùng địa chỉ. Nếu dùng RAM, không kỳ vọng thay đổi còn sau reload.

Bản đầy đủ bọc logic trong hàm tự gọi, công bố `globalThis.ViData`; `app.js` lấy bằng `const D = globalThis.ViData`. Trong Node, API gán `module.exports`, bộ test dùng `require('./data.js')`. HTML nạp **script thông thường** theo thứ tự:

```html
<script src="data.js"></script>
<script src="app.js"></script>
```

Không thêm `type="module"`, `import`, server hoặc thư viện. `test_data.cjs` là chương trình kiểm logic riêng, không gắn vào trang. Nếu đã có Node.js, chạy từ **gốc NEW**:

```Bash
node "bai-lam/nam-2-web/test_data.cjs"
```

Lệnh này dùng được trong Bash hoặc PowerShell; cách đọc mã thoát tùy shell như Chương 2. **Bộ Web hiện có 34 ca** (đếm theo số dòng `PASS` và dòng tổng kết cuối), gồm bộ ca kế thừa và một ca hồi quy cho `category` dạng mảng. Kỳ vọng bản đúng là `34 passed, 0 failed`; đó chưa phải kết quả máy bạn. Bản C++ tham khảo `vi_sinh_vien.cpp` có **9 ca**, chạy bằng `--test` sau build; không lẫn bộ này với bản bài làm `loi-bo-sot.cpp` không có bộ test. Test tham khảo xanh cũng không chứng minh hàm lab tự viết đúng.

Bảng kiểm thử: ghi kỳ vọng trước, thực tế sau

Bảng giữ các ca cốt lõi của nguồn, thêm ranh giới schema. Cột thực tế để người học điền; "chưa chạy" khác "đã chạy đạt". Với ca sai, kỳ vọng là **từ chối**, không phải tổng 0. Trong hàm, từ chối là phát sinh lỗi; ở giao diện, bắt lỗi và không nhận thay đổi sai.

Ca	Kỳ vọng	Vì sao cần	Thực tế/trạng thái
Ba khoản mẫu	60000	Cộng đủ ba số hạng.	…
Chỉ một khoản Vở ghi 18000	18000	Phát hiện bỏ phần tử cuối.	…

Ca	Kỳ vọng	Vì sao cần	Thực tế/trạng thái
Rỗng	0	Tổng của danh sách hợp lệ không có khoản.	…
Đảo thứ tự ba khoản	60000	Tổng không phụ thuộc thứ tự.	…
Một khoản 1; một khoản 1000000000	Lần lượt: 1; 1000000000	Hai biên đều được nhận.	…
500 khoản, mỗi khoản 1000000000	500000000000	Kiểm số lượng và kiểu tổng.	…
501 khoản, tiền đều hợp lệ	Từ chối	Không chỉ kiểm từng tiền.	…
Có khoản 0, âm hoặc 1000000001	Từ chối từng ca	Không bỏ qua khoản sai.	…
JS có `amount: "35000"` hoặc `1.5`	Từ chối	Không là tiền số nguyên hợp lệ.	…
Chuỗi nhập `" 35000 "`	Số 35000	Cho phép khoảng trắng ngoài cùng.	…
Nhập `""`, `" "`, `"35.000"`, `"35000abc"`	Từ chối từng ca	Kiểm toàn chuỗi, không đọc một phần.	…
Nhập `"0"`, `"1000000001"`, `"1e3"`	Từ chối từng ca	Cách viết đúng chưa đủ; ký hiệu mũ không được nhận.	…
Schema có id trùng, 0 hoặc không an toàn	Từ chối từng ca	Danh tính khoản phải hợp lệ, duy nhất.	…
Title toàn khoảng trắng hoặc 81 đơn vị UTF-16	Từ chối từng ca	Kiểm sau trim, giới hạn 80.	…
Title `" Vở ghi "`	Chuẩn hóa thành `"Vở ghi"`	Không làm biến đổi tiền.	…
Category `"khac"` / `['khac']` / `"toString"`	Nhận / từ chối / từ chối	Kiểm kiểu chuỗi và khóa riêng trong danh mục.	…

Nếu tạo 500 đối tượng để thử **normalize**, dùng id riêng cho từng khoản; nhân bản cùng một id 500 lần sẽ kiểm lỗi trùng id chứ chưa kiểm được biên số lượng hợp lệ. Nếu chỉ thử **totalExpenses**, các đối tượng có tiền là đủ theo phạm vi hàm.

▼ Đáp án validation C++ — Lõi đầy đủ, không phải chỉ một dòng cận · mở sau khi tự thử

Lõi sau kế thừa `vi-du/nam-1-cpp/vi_sinh_vien.cpp`. Đặt hàm trên `main` trong bản bài làm, không chép đè bản tham khảo có `runTests()` và nhánh `--test`.

C++

```
#include <iostream>
#include <stdexcept>
#include <vector>

long long totalExpenses(const std::vector<long long>& amounts) {
    if (amounts.size() > 500) {
        throw std::invalid_argument("Toi da 500 khoan chi");
    }
    long long total = 0;
    int n = static_cast<int>(amounts.size());
    for (int i = 0; i < n; ++i) {
        if (amounts[i] <= 0 || amounts[i] > 1000000000LL) {
            throw std::invalid_argument("Moi khoan: 1 den 1000000000 VND");
        }
        total += amounts[i];
    }
    return total;
}
```

`const ...&` nhận tham chiếu chỉ đọc, không sao chép vector. Khi rỗng, không vào thân vòng lặp, tổng giữ 0. Kiểm số lượng trước phép ép kích thước; kiểm tiền trước cộng; không có nhánh "gặp sai thì bỏ qua". Thời gian O(n), bộ nhớ phụ O(1).

Mẫu `main` cho bản bài làm, có bắt lỗi, được bổ sung để thử các ca sai. Chỉ thay vector khi thử dữ liệu khác; các dòng tên khoản dưới đây mô tả riêng bộ mẫu, nên cần đổi/bỏ chúng nếu đổi dữ liệu, tránh nhãn in không khớp.

C++

```
int main() {
    const std::vector<long long> amounts = {35000, 7000, 18000};
    try {
        const long long total = totalExpenses(amounts);
        std::cout << "Com trua: 35000 VND\nXe buyt: 7000 VND\nVo ghi: 18000 VND\n";
        std::cout << "Tong chi tieu: " << total << " VND\n";
        return 0;
    } catch (const std::invalid_argument& error) {
        std::cerr << error.what() << '\n';
        return 1;
    }
}
```

Đầu ra kỳ vọng với bộ mẫu là ba dòng tiền và `Tong chi tieu: 60000 VND`; không phải log đã chạy. Khi thử ca sai, nhánh bắt lỗi trả 1, không in tổng hợp lệ bằng 0. Hàm này nhận tiền đã có kiểu số nguyên; nó không thể phát hiện một lỗi parser đã biến đầu vào sai thành số hợp lệ từ trước.

▼ Đáp án validation Web — Tách parse, tổng và schema trong lab · mở sau khi tự thử

Lõi `validAmount`, `parseAmount`, `totalExpenses`, `normalizeExpenses` dưới đây kế thừa `vi-du/nam-2-web/data.js` của NEW, có kiểm **kiểu chuỗi category**. Vỏ `chapter3Lab` là **bổ sung biên tập** để thử độc lập; không thay `globalThis.ViData`, không truy cập DOM/lưu trữ và không chép đè `data.js`.

Nếu muốn thử, mở `bai-lam/nam-2-web/index.html`, nhấn F12 rồi chọn thẻ **Console** (cách mở ở mục 2.4, Chương 2). Lần đầu dán mã, Chrome/Edge có thể chặn và yêu cầu tự gõ một cụm cho phép dán; đọc kỹ cảnh báo, chỉ dán khối trong cẩm nang này, không dán mã lạ. Đọc rồi nhập khối lab một lần; `var chapter3Lab` cho phép tạo lại lab sau khi sửa. Có thể chỉ đọc/trace nếu chưa quen Console. Khối này tự đủ phần logic, không cần gói ngoài.

JavaScript

```
var chapter3Lab = (function () {
  'use strict';
  const MAX_AMOUNT = 1000000000;
  const MAX_ITEMS = 500;
  const categories = Object.freeze({
    'an-uong': 'Ăn uống', 'di-lai': 'Đi lại',
    'hoc-tap': 'Học tập', 'khac': 'Khác'
  });

  function validAmount(amount) {
    return Number.isSafeInteger(amount) && amount > 0 && amount <= MAX_AMOUNT;
  }

  function parseAmount(input) {
    if (typeof input !== 'string' || !/^[0-9]+$/.test(input.trim())) {
      throw new Error('Nhập số nguyên VND, không có dấu chấm, phẩy hay chữ.');
    }
    const amount = Number(input.trim());
    if (!validAmount(amount)) throw new Error('Số tiền phải từ 1 đến 1.000.000.000 VND.');
    return amount;
  }

  function totalExpenses(expenses) {
    if (!Array.isArray(expenses) || expenses.length > MAX_ITEMS) {
      throw new Error('Danh sách tối đa 500 khoản.');
    }
    let total = 0;
    for (const expense of expenses) {
      if (!expense || !validAmount(expense.amount)) {
        throw new Error('amount phải là số nguyên VND hợp lệ.');
      }
      total += expense.amount;
    }
    return total;
  }

  function normalizeExpenses(input) {
    if (!Array.isArray(input) || input.length > MAX_ITEMS) {
      throw new Error('Dữ liệu lưu không đúng định dạng.');
    }
    const ids = new Set();
    return input.map(item => {
      if (!item || !Number.isSafeInteger(item.id) || item.id < 1 || ids.has(item.id) ||
          typeof item.title !== 'string' || !item.title.trim() || item.title.trim().length > 80 ||
          typeof item.category !== 'string' || !Object.hasOwn(categories, item.category) ||
          !validAmount(item.amount)) {
        throw new Error('Một khoản lưu sẵn không hợp lệ.');
      }
      ids.add(item.id);
      return { id: item.id, title: item.title.trim(), amount: item.amount, category: item.category };
    });
  }
```

```
  return { parseAmount, totalExpenses, normalizeExpenses, categories };
})();
```

`/^[0-9]+$/` kiểm toàn chuỗi sau trim, không nhận dấu chấm, phẩy, chữ hoặc mũ. `parseInt("35000abc", 10)` có thể đọc được tiền tố 35000, nên chỉ thêm radix không sửa được lỗi nhận hậu tố. `Number.isInteger` kiểm số nguyên; ở đây dùng `Number.isSafeInteger` rồi giới hạn riêng miền tiền. `"0007"` hợp lệ và cho số 7; `"0"` chỉ đúng cách viết nhưng sai miền.

`Object.hasOwn` kiểm khóa riêng, nhưng không thay kiểm kiểu: `['khac']` có thể bị ép thành tên khóa `"khac"`. Vì vậy phải giữ `typeof item.category !== 'string'`. `Set` lưu id đã gặp để phát hiện trùng. Normalize trả đối tượng mới với title đã trim, không sửa đối tượng đầu vào. Nó dùng thêm O(n) bộ nhớ cho danh sách mới và tập id; không gọi mọi phần của quá trình này là O(1).

Ví dụ thử sau khi mở bản Web và tạo lab; các chú thích là **kỳ vọng**, không phải thông báo kiểm thử đã chạy:

JavaScript

```
chapter3Lab.totalExpenses(globalThis.ViData.sampleExpenses()); // kỳ vọng 60000
chapter3Lab.parseAmount(' 35000 ');                            // kỳ vọng số 35000
chapter3Lab.totalExpenses([]);                                 // kỳ vọng 0
```

Thử từng ca sai riêng và chờ lỗi: `chapter3Lab.parseAmount('35000abc')`, hoặc `chapter3Lab.totalExpenses([{amount: '35000'}])`. Lỗi mong đợi không phải "Console hỏng"; đó là hàm đang từ chối dữ liệu. Để kiểm không thay đổi đầu vào, chụp giá trị JSON trước/sau lời gọi rồi đối chiếu, không chỉ nhìn tổng.

Biến thể: đóng chat trước khi mở lời giải

Minh: viết `maxExpense` nhận cùng vector tiền, rỗng trả 0, vẫn từ chối mọi tiền sai và quá 500 khoản. Tự dự đoán ba khoản, một khoản và rỗng. Đừng chỉ đổi tên hàm tổng; biến đang giữ **một khoản lớn nhất**, không phải tổng đã cộng.

Lan: viết `totalByCategory`, nhận danh sách Expense và mã nhóm. Mã nhóm phải là chuỗi trong danh mục. Kiểm **toàn bộ danh sách** trước khi chọn nhóm, gồm schema, tiền và số lượng; bất kỳ khoản sai nào cũng làm lời gọi bị từ chối, kể cả khoản thuộc nhóm khác. Không giả định đầu vào đã được normalize ở nơi nào đó nếu chữ ký hàm chưa nêu tiền đây.

Biê´n thể/ca	Kỳ vọng cầ`n tự đô´i chiê´u
Max ba mẫu / chỉ Vở ghi / rỗng	35000 / 18000 / 0
Max có 0, sô´ âm, quá 1e9 hoặc 501 khoản	Từ chô´i, dù đã gặp một giá trị lớn hợp lệ trước đó.
Tổng `hoc-tap` / `khac` trên ba mẫu	18000 / 0
Tổng nhóm trên rỗng, mã nhóm hợp lệ	0
Chọn `hoc-tap`, khoản `di-lai` có amount chuỗi hoặc 1.5	Từ chô´i, không trả 18000.
Chọn `hoc-tap`, khoản ngoài nhóm có id trùng, title rỗng hoặc category mảng	Từ chô´i từng ca schema.
501 khoản, không khoản nào thuộc nhóm chọn	Từ chô´i, không trả 0.
Mã nhóm yêu cầ`u là `['khac']` hoặc `toString`	Từ chô´i mã nhóm.

▼ Đáp án biê´n thể max C++ — Kiểm hê´t rô`i mới kê´t luận · mở sau khi tự thử

Đặt hàm sau `totalExpenses` trong bản bài làm; nó dùng lại validation của hàm tổng. Cách này duyệt hai lượt nhưng dễ đọc ở năm nhâ´t, vẫn O(n) thời gian và O(1) bộ nhớ phụ.

C++

```
long long maxExpense(const std::vector<long long>& amounts) {
    totalExpenses(amounts); // Kiểm toàn bộ tiền và số lượng; có thể ném lỗi.
    long long best = 0;
    for (long long amount : amounts) {
        if (amount > best) best = amount;
    }
    return best;
}
```

Mô´c 0 phù hợp vì mọi khoản hợp lệ đề`u dương; rỗng giữ 0 mà không cầ`n truy cập phầ`n tử đầ`u. Với mẫu, `best` thành 35000 và giữ nguyên khi gặp 7000, 18000. Không được trả sớm khi gặp 1000000000 trước khi kiểm các khoản còn lại: danh sách có thể chứa khoản sai phía sau. Gọi hàm tổng ở đầ`u là cách tái sử dụng kiểm tra, không có nghĩa tổng chính là max. Khi thêm test, đổi lời gọi sang `maxExpense`, đừng chạy lại hàm tổng rô`i nhận là đã test max.

▼ Đáp án tổng theo category — Xác thực trước, chọn nhóm sau · mở sau khi tự thử

Hàm lab sau dùng `chapter3Lab` đã tạo ở đáp án Web. Đây là **hàm bổ sung**, chưa có trong API demo; không nói bộ 34 test hiện có đã kiểm nó.

JavaScript

```
function totalByCategory(expenses, category) {
  if (typeof category !== 'string' || !Object.hasOwn(chapter3Lab.categories, category)) {
    throw new Error('Mã nhóm không hợp lệ.');
  }
  const checked = chapter3Lab.normalizeExpenses(expenses);
  const selected = checked.filter(expense => expense.category === category);
  return chapter3Lab.totalExpenses(selected);
}
```

Nếu mã nhóm sai, lời gọi bị từ chối ngay. Khi mã hợp lệ, normalize kiểm toàn danh sách trước lọc: không có kết quả nhóm nào được trả từ danh sách chứa khoản sai. `totalExpenses(selected)` cộng sau khi schema đã được bảo vệ. Nếu nơi gọi đã có bảo đảm schema rõ ràng, có thể kiểm toàn bộ tiền/số lượng bằng `totalExpenses(expenses)` trước khi lọc; nhưng **chỉ lời gọi đó không đủ chứng minh schema** cho hàm nhận dữ liệu bất kỳ.

Ví dụ phản chứng với bộ mẫu: đổi riêng tiền Xe buýt thành `"7000"`, vẫn hỏi `hoc-tap`. Hàm phải ném lỗi, dù Vở ghi hoàn toàn đúng. Đổi Xe buýt thành id 1 trùng Cơm trưa cũng phải lỗi. Lọc `hoc-tap` trước normalize sẽ làm biến mất các bằng chứng sai đó và trả 18000 trái hợp đồng.

Với dữ liệu hợp lệ, `hoc-tap` cho 18000, `khac` cho 0. Số 0 này nghĩa không có khoản thuộc nhóm, không phải "gặp lỗi nên trả 0". Cách normalize rồi filter tạo danh sách mới, nên toàn hàm O(n) thời gian, O(n) bộ nhớ phụ. Có thể tối ưu sau, nhưng chưa đánh đổi thứ tự kiểm tra để tiết kiệm một lượt duyệt nhỏ.

3.7. SKILL UNLOCKED — Năng lực ở lại khi cửa sổ chat đóng

Ba mức tự đánh giá, không phải nhãn năng lực lâu dài

Chọn **tự làm / cần gợi ý / chưa làm**, kèm bằng chứng. “Tự làm” nghĩa đã đóng chat, không nhìn đáp án. Đây là ghi nhận hiện tại, không phán năng lực lâu dài.

Năng lực	Việc cần làm được	Mức và bằng chứng của bạn
Giải thích	Nêu hợp đồng tiền, số lượng, tổng rỗng, tổng cực đại.	…
Giải thích	Trace đến 60000 và chỉ vì sao cận lỗi dừng ở 42000.	…
Giải thích	Phân biệt `35000` với `"35000"`, hậu quả của phép `+`.	…
Sửa	Chỉ thay phần gây lỗi, tách vá cận khỏi validation.	…
Sửa	Tự chọn ca một khoản, rỗng, biên lớn và dữ liệu sai.	…
Sửa	Phân biệt từ chối với tổng hợp lệ bằng 0.	…
Vận dụng	Làm max hoặc tổng nhóm và không che khoản sai ngoài nhóm.	…
Vận dụng	Nêu phần đã nhận hỗ trợ, tự làm và còn cần hỏi.	…

Chọn một dòng chưa làm được, thu nhỏ ví dụ rồi thử lại. Bài đếm 3.5 là bước vận dụng nhỏ, chưa thay validation ở 3.6.

Tám câu vấn đáp: hỏi từng câu rồi chờ

Nhờ bạn học hoặc dùng P07; hỏi rồi chờ, cho phép giấy và ví dụ. Không suy điểm học phần.

Câu hỏi	Gợi ý để tự lập luận
1. Vì sao đúng 60000 nhưng cận lỗi ra 42000?	Viết ba số hạng và tìm phép cộng chưa xảy ra.
2. Ba phần tử có chỉ số cuối nào; dừng khi nào?	Phân biệt số lượng với chỉ số bắt đầu từ 0.
3. Vì sao cần `long long` dù ba khoản đều nhỏ?	Nhân số khoản tối đa với tiền tối đa.
4. Rỗng trả 0 có mâu thuẫn tiền phải dương?	Tổng danh sách và tiền của một khoản là hai đối tượng.
5. Vì sao `0 + "35000"` không là tổng hợp lệ?	Nêu kiểu hai toán hạng và kết quả.

Câu hỏi	Gợi ý để tự lập luận
6. Chuyển chuỗi thành sô´ đã đủ chưa?	Xét rỗng, `"0"`, sô´ âm, quá giới hạn và hậu tô´ chữ.
7. `localStorage` có nghĩa có tài khoản hoặc CSDL máy chủ?	Chỉ nơi dữ liệu nă`m, phạm vi origin và trạng thái RAM.
8. Test xanh đã chứng minh bạn hiểu chưa?	Tự giải thích một ca, sửa một lỗi và làm biê´n thể mới.

▼ Đáp án vâ´n đáp — Những ý cần có, không phải văn mẫu để học thuộc · mở sau khi tự thử

1. Thiê´u Vở ghi 18000 vì chỉ sô´ 2 chưa vào thân vòng lặp; 35000 + 7000 là 42000.
2. Chỉ sô´ cuô´i là 2 khi `n = 3`; cận đúng cho 0, 1, 2 và chặn 3 trước truy cập.
3. Tổng tô´i đa 500000000000 vượt sô´ nguyên có dâ´u 32 bit; cần xét hợp đồ`ng, không chỉ mẫu nhỏ.
4. Rỗng không chứa khoản 0 nào; nó không có khoản để cộng, nên tổng 0 hợp lệ.
5. Phép `+` với sô´ 0 và chuỗi tạo `"035000"`, kiểu string, không phải tiê`n number.
6. Cần kiểm cách viê´t và miê`n giá trị; chuyển được chưa đủ, còn hàm tổng phải từ chô´i tiê`n chuỗi.
7. Dữ liệu ở trình duyệt, không đồ`ng bộ/tài khoản/backend; đọc hoặc ghi lỗi có thể chuyển sang RAM. SQLite là bài riêng.
8. Test chỉ kiểm những ca đã chọn trên mã đã gọi; hiểu cần lời giải thích và vận dụng không nhìn đáp án.

Nê´u câu trả lời còn mơ hồ`, hỏi thêm một ví dụ thay vì phán “chưa hiểu gì”. “Em chưa biê´t xử lý JSON sai mà không ghi đè bản cũ” là điểm nghẽn cụ thể để luyện tiê´p.

Chu trình và phiếu ba dòng



Chu trình học chủ động: mục tiêu, tự thử, dự đoán, một gợi ý, tự sửa, kiểm chứng, biến thể không AI và ghi chỗ chưa chắc.
Xem hình lớn

Prompt / Ghi chú

```
Chọn mục tiêu + đọc hợp đồng
          ↓
Tự thử → Dự đoán trước hỏi/chạy
          ↓
Xin một gợi ý → Tự trả lời → Tự sửa một điểm
          ↓
Kiểm chứng bằng test
    ├─ Chưa đúng → Ghi điểm nghẽn → Thu nhỏ và thử lại
    └─ Đúng → Biến thể không AI → Ghi chỗ chưa chắc
                                        ↓
                                Chọn việc tiếp theo
```

Cuối phiên chỉ cần ba dòng. Ví dụ sau là **phiếu HƯ CẤU**, không phải kết quả học của một sinh viên thật:

```
Prompt / Ghi chú

Học được: Cận cũ không cho i = 2 đi vào thân vòng lặp.
Chưa chắc: Tôi chưa tự giải thích hết kiểm tra dữ liệu đọc từ JSON.
Việc tiếp: Tự trace ca một khoản và một ca sai ngoài nhóm, chưa mở AI.
```

Gọi tên hành động hoặc kiến thức cụ thể; “học được nhiều” không giúp phiên sau biết bắt đầu đâu. Đầu ra cuối chương là một bài làm nhỏ, bảng kiểm có trạng thái thật và lời giải thích của chính bạn. Nếu muốn ghép thêm diff, hai lỗi chủ đích và khai báo hỗ trợ, chuyển sang Final Boss sau khi học đủ các phần liên quan; bộ đề đó khác bài cận đơn và demo Web đúng ở đây.

Gợi ý cho người hướng dẫn: trình bày trong bảy phút

Phần này kế thừa speaker notes của nguồn; tham gia tự nguyện, không kiểm tra bất ngờ. Phút 0–1 giới thiệu dữ liệu và mời tính tay; phút 1–2 hỏi phần luyện nào bị mất nếu AI viết luôn vòng lặp; phút 2–3:15 cho tự ghi chỉ số trước khi hiện trace Minh. Phút 3:15–4:30 đặt số cạnh chuỗi, dự đoán lượt cộng đầu; phút 4:30–5:30 làm mẫu hỏi một câu rồi chờ. Phút 5:30–6:30 cho phác biến thể độc lập; phút cuối điền phiếu ba dòng. Bảy phút chỉ là dẫn nhập, không thay hai hoạt động 25 phút hoặc dùng làm phép đo tốc độ.

Nguồn kế thừa và điểm kết năng lực

Kế thừa chương 3 (mục 3.1–3.7) của bản bảo toàn, Blueprint, Source Audit, P01–P08 trong Thư viện prompt và mã data.js, app.js, C++. Hoạt động và vỏ lab mới có nhãn bổ sung biên tập tại chỗ.

Năng lực mở khóa: tự dự đoán, sửa và kiểm chứng; giải thích được 60000, 42000 và chuỗi nối sai. Khi đóng chat, bạn còn cách tự làm lại, bằng chứng kiểm được và một việc tiếp theo cụ thể.

04

CHƯƠNG 04 / MINH & LAN

Sử dụng AI có trách nhiệm

Phòng xử lý sự cố: chọn đúng hành động khi dữ liệu, quyền và trách nhiệm cùng xuất hiện.



Tình huống và nhân vật hư cấu · Đọc, dự đoán rồi tự kiểm chứng. Xem hình lớn

Tổng tiền của ViSinhVien đã hiện đúng **60000**. Minh định nộp ngay. Lan hỏi lại: "Nếu thầy hỏi dòng này từ đâu ra, dữ liệu của ai, và em đã kiểm tra những gì, mình trả lời được chưa?" Chương này bắt đầu từ câu hỏi đó.

Làm chủ sản phẩm không có nghĩa là tự gõ từng ký tự hay tuyệt đối không dùng AI. Nó có nghĩa là bạn biết mình đang dùng gì, được phép dùng trong hoàn cảnh nào, có bằng chứng cho điều mình khẳng định và biết xử lý khi phát hiện sai sót. Một bài tập nhỏ là nơi an toàn để tập thói quen ấy, trước khi lỗi của mình ảnh hưởng đến người khác.

Nhân vật, hồ sơ sự cố và hội thoại trong chương đều HƯ CẤU. Đây là tài liệu tự học, **không phải quy định của Trường Đại học Thủy lợi** và không phải tư vấn pháp lý. Quyền dùng AI trong bài nộp do từng học phần và giảng viên quyết định.

Cách dùng phòng xử lý sự cô´

Chương được tổ chức như một **phòng xử lý sự cô´** gô`m bảy hô` sơ và một phòng khám kỹ thuật. Mỗi hô` sơ có cùng sáu phâ`n:

Phâ`n	Bạn làm gì
Sự cô´	Đọc tình huô´ng hư câ´u của Minh hoặc Lan.
Dâ´u hiệu	Gạch chân chi tiê´t cho thâ´y có vâ´n đê`.
Nguy cơ	Nêu ai hoặc điê`u gì có thể bị ảnh hưởng.
Các lựa chọn	Chọn **một** hành động trước khi đọc tiê´p. Viê´t lý do một dòng.
Phân tích từng lựa chọn	Mở sau khi đã chọn. Mỗi phương án đê`u được phân tích, kể cả phương án sai.
Bài học	Một quy tă´c ngă´n bạn mang sang bài tập thật.

Mô´c chung của câu chuyện: ViSinhVien là ứng dụng HTML/JavaScript lưu bă`ng `localStorage` trong trình duyệt; **không có backend, đăng nhập hay cơ sở dữ liệu máy chủ**. Mỗi khoản chi là `Expense{id,title,amount,category}`. `amount` là sô´ nguyên VND từ 1 đê´n `1e9`; tô´i đa 500 khoản; danh sách rỗng có tổng 0. Bộ dữ liệu GIẢ chuẩn: Cơm trưa 35000, Xe buýt 7000, Vở ghi 18000, tổng **60000 VND**.

Bạn sẽ gặp thêm SQL chạy bă`ng Python `sqlite3` với cơ sở dữ liệu `:memory:` ở mục 4.8. Đó là **minh họa riêng**, không phải thành phâ`n của ViSinhVien. Các đoạn mã ngă´n phục vụ giải thích; khi thực hành, hãy ghi kê´t quả mình thực sự quan sát, kể cả kê´t quả chưa đạt.

Thứ tự đọc: hô` sơ 4.1 (khóa bị lộ) được đặt đâ`u vì sai thứ tự xử lý có hậu quả lớn nhâ´t. Nê´u thời gian ít, làm 4.1, 4.2, 4.5 và mục 4.8 trước; các hô` sơ còn lại làm ở buổi sau.

Chuẩn bị: một tờ giâ´y hoặc tệp `ghi-chep-04.txt` trong thư mục `bai-lam`. Không câ`n cài đặt, đăng nhập, tạo khóa hay truy cập dịch vụ thật nào. Mọi chuỗi bí mật trong chương là placeholder như `DEMO_KEY_NOT_A_SECRET`, không có quyê`n truy cập.

4.1. Hô` sơ 01 — Khóa API xuâ´t hiện trên GitHub

Sự cô´

ViSinhVien **không câ`n API key**. Nhưng trong một bài khác, Minh thử gọi một dịch vụ AI và để khóa trong tệp `config.js`. Tô´i thứ Sáu, Minh đẩy cả thư mục lên một kho GitHub công khai để nhờ Lan xem

giúp. Sáng hôm sau, Lan nhắn: “Hình như tệp config có dòng `API_KEY = ...`?”

Khi kể lại, chương chỉ dùng placeholder `DEMO_KEY_NOT_A_SECRET`. Trong đời thật, không chụp lại khóa đầy đủ để “làm bằng chứng”.

Dấu hiệu

- Một chuỗi trông giống khóa nằm trong tệp đã commit.
- Kho ở chế độ công khai từ tối hôm trước; không ai biết chính xác bao nhiêu người đã xem hoặc clone.
- Minh định “xóa dòng đó rồi commit lại là xong”.

Nguy cơ

Khóa API cấp quyền dùng dịch vụ và có thể phát sinh chi phí dưới tên tài khoản của chủ khóa. Xóa dòng trong tệp **không vô hiệu hóa khóa**: bản cũ vẫn có thể nằm trong lịch sử Git, trong bản clone của người khác, trong cache hoặc trong đoạn chat từng dán. Khóa vẫn còn hiệu lực cho đến khi được thu hồi ở nhà cung cấp.



Thu hồi trước, dọn sau: sáu bước cùng thứ tự với mục bên dưới, ô lớn là bước 02 thu hồi khóa; nhãn trong hình là nhãn giả, không phải khóa thật. Xem hình lớn

Các lựa chọn

Chọn **một** hành động Minh nên làm **đầu tiên**:

- **A.** Xóa dòng khóa, commit “fix config”, rồi coi như xong.
- **B.** Tạo khóa mới, dán vào tệp, để khóa cũ hoạt động thêm vài ngày cho chắc.
- **C.** Dừng chia sẻ thêm, rồi thu hồi hoặc xoay vòng khóa cũ ở nhà cung cấp (hoặc báo ngay người có quyền nếu khóa không phải của mình).
- **D.** Gửi nguyên khóa vào nhóm chat lớp để mọi người cùng kiểm tra xem có bị dùng chưa.
- **E.** Tự viết lại lịch sử Git và ép đẩy (force push) để “xóa sạch”.

▼ Đáp án Hồ sơ 01 — Phân tích từng lựa chọn · mở sau khi tự thử

A — Chưa đủ. Commit mới chỉ thay đổi bản hiện tại; khóa cũ vẫn hoạt động và vẫn có trong lịch sử. Đây là sai lầm phổ biến nhất vì màn hình trông “sạch”.

B — Sai thứ tự. Tạo khóa mới không làm khóa cũ mất hiệu lực. Để khóa cũ chạy thêm là để cửa mở. Thêm nữa, dán khóa mới vào cùng tệp có thể lặp lại sự cố.

C — Đúng hướng. Ưu tiên vô hiệu quyền đã lộ trước khi dọn bản lưu. Hướng dẫn của GitHub cũng khuyến nghị thu hồi hoặc xoay vòng bí mật trước khi xử lý việc loại bỏ dữ liệu khỏi kho [7]. Nếu khóa thuộc nhóm, trường hoặc nơi thực tập, Minh **báo ngay** người có quyền thay vì tự tìm cách truy cập tài khoản người khác.

D — Làm sự cố lớn hơn. Gửi khóa vào kênh khác là thêm một nơi lộ. Báo sự cố bằng mô tả, không bằng bí mật.

E — Không tự làm. Viết lại lịch sử ảnh hưởng mọi người đang dùng kho và vẫn không bảo đảm các bản clone hoặc cache biến mất. Việc này cần người quản lý kho quyết định và phối hợp. Chương này **không cung cấp lệnh viết lại lịch sử**.

Sáu bước xử lý có thể làm theo

1. **Dừng chia sẻ thêm.** Không gửi lại tệp, không chụp nguyên khóa để hỏi nhóm. Nếu là kho của chính mình: chuyển kho sang chế độ riêng tư (Private) trong phần cài đặt kho. Nếu không phải kho của mình: nhắn ngay chủ kho, không tự xử lý. Không chuyển khóa sang kênh khác để “cất tạm”.
2. **Thu hồi hoặc xoay vòng khóa ở nhà cung cấp.** Dùng chức năng quản lý khóa chính thức nếu là tài khoản của mình; nếu không, báo người có quyền. **Xác nhận khóa cũ đã bị vô hiệu** — tạo

khóa mới nhưng để khóa cũ còn hoạt động là chưa xong. Xác nhận bằng **trạng thái "đã thu hồi/đã xóa" trong trang quản lý khóa**; **không gọi thử API bằng khóa cũ** để kiểm tra.

3. **Kiểm tra mức sử dụng.** Xem lượt gọi, chi phí và mốc thời gian bất thường trong trang quản lý của mình. "Chưa thấy dùng" không chứng minh chưa ai sao chép. Ghi thời điểm phát hiện và phạm vi chia sẻ, không lưu lại bí mật.
4. **Kiểm tra bản liên quan trong phạm vi được phép.** Commit, tệp cấu hình, bản build, ảnh chụp, đoạn chat có liên quan. Không tự truy cập tài khoản hay log của người khác. Khi tìm, tránh tạo log mới chứa khóa.
5. **Báo đúng đầu mối.** Bài học: giảng viên hoặc trưởng nhóm. Tài khoản tổ chức: kênh sự cố của tổ chức. Gửi thời điểm, dịch vụ, trạng thái thu hồi, phạm vi đã biết và điều chưa rõ; **không đính kèm khóa**.
6. **Giảm dữ liệu còn lưu và ngăn tái diễn.** Sau khi không chế quyền truy cập, phối hợp xử lý các bản chứa bí mật theo quy trình phù hợp. Trong câu chuyện này kho là **kho cá nhân của Minh**: sau khi khóa đã thu hồi, Minh hỏi giảng viên trước khi đụng tới lịch sử kho, và ghi nhận rằng bản clone hoặc cache có thể vẫn còn — đó là lý do bước 2 phải làm trước. Thay minh họa bằng placeholder, bỏ nhu cầu dùng khóa thật trong bài, thêm bước rà tệp trước khi chia sẻ.

Một thông báo sự cố ngắn, đủ để hành động

Prompt / Ghi chú

```
Em phát hiện lúc [giờ, ngày] một tệp trong [phạm vi chia sẻ] chứa khóa của [dịch vụ].
Em đã dừng chia sẻ. Khóa cũ: [đã thu hồi / đang chờ người có quyền / chưa xác minh].
Thời gian có thể đã lộ: [phạm vi biết được].
Mức sử dụng: [đã kiểm tra gì, kết quả, hoặc chưa kiểm tra].
Nhờ [đầu mối] hướng dẫn xử lý bản đã chia sẻ. Em không đính kèm khóa.
```

Báo sớm không có nghĩa đã kết luận có tấn công; chưa thấy lạm dụng cũng không phải lý do để giấu. Với dữ liệu hoặc hệ thống thật, người có thẩm quyền đánh giá nghĩa vụ thông báo. Đừng suy ra thời hạn pháp lý từ mẫu này.

Bài học

Thu hồi trước, dọn sau. Báo bằng mô tả, không bằng bí mật. Lan cùng Minh tìm nguyên nhân tệp thừa lọt vào bản chia sẻ, rồi thêm một bước "rà danh sách tệp trước khi đẩy". Sửa một bước quy trình cụ thể hữu ích hơn lời hứa "lần sau cẩn thận".

QUICK CHECK: Minh đã xóa tệp `config.js` và tạo khóa mới. Còn thiếu việc gì quan trọng nhất? Trả lời một câu rồi đối chiếu với bước 2.

4.2. Hồ sơ 02 — Dữ liệu cá nhân đi vào cửa sổ chat

Sự cố

Để nhờ AI tìm lỗi tính tổng, Minh định gửi **ảnh chụp bảng chi tiêu thật** của mình trong tháng. Ảnh có tên quán ăn quen, giờ giao dịch, mã sinh viên lẫn giữa các dòng, số tài khoản ngân hàng bị che một nửa và góc màn hình còn hiện email sinh viên. Minh nghĩ: "Em đổi tên mình thành 'Bạn A' là ẩn danh rồi."

Dấu hiệu

- Lỗi cầǹ sửa (bỏ sót khoản cuối) **có thể tái hiện bằ̀ng ba khoản giả**; không cầǹ dữ liệu thật.
- Đổi tên nhưng giữ mã sinh viên, giờ, địa điểm và email.
- Ảnh chụp màn hình chứa nhiề̀u thông tin hơn vùng lỗi.

Nguy cơ

Người biế́t bối cảnh vẫn có thể nhận ra "Bạn A" từ các trường còn lại. Ảnh có thể lộ email, tab trình duyệt, đường dẫn chứa tên. Một lớp che trong tệp chỉnh sửa có thể bị gỡ. Lịch sử chat có thể giữ đoạn từng dán ngay cả khi bạn đã xóa khỏi mã.

Phân loại trước khi dán

Quy tắ́c thực dụng: **chọn dữ liệu ít nhạy cảm nhấ́t nhưng vẫn đủ tái hiện vấ́n đề̀.**

Nhóm dữ liệu	Cách hiểu khi làm bài	Ví dụ và lựa chọn
Giả/tổng hợp	Tạo mới cho mục đích thử, không lấ́y hồ sơ thật rồi sửa sơ qua	35000, 7000, 18000; liên hệ giả `demo001@example.invalid`
Đã khử nhận dạng/ẩn danh	Đã xử lý để không còn xác định được người, kể cả khi liên kế́t với dữ liệu khác	Chỉ gọi là ẩn danh khi đã đánh giá lại khả năng nhận diện; **không mặc định sau khi bỏ tên**
Cá nhân, nhạy cảm hoặc bí mật	Liên quan một người, đời số́ng riêng tư, tài khoản hoặc quyề̀n truy cập	Mã sinh viên, ảnh hồ sơ, lịch chi tiêu thật, mật khẩu, khóa API — không đưa vào ví dụ công khai

Bảng này là công cụ sàng lọc khi học, không thay thế́ phân loại pháp lý chi tiế́t. Khóa API cầǹ bảo vệ vì cấ́p quyề̀n truy cập, dù bản thân chuỗi khóa không nhận diện một người. Dữ liệu công khai ở nơi khác cũng không đương nhiên phù hợp để gom lại và gửi cho một dịch vụ AI.

Các lựa chọn

- **A.** Gửi ảnh nguyên bản; dịch vụ AI chă´c không lưu đâu.
- **B.** Đổi tên thành "Bạn A", giữ các cột khác, rô`i gửi.
- **C.** Tự tạo ba khoản giả 35000, 7000, 18000 cùng đoạn hàm tính tổng, gửi phâ`n đó.
- **D.** Che bă`ng một lớp hình chữ nhật trong tệp chỉnh sửa, gửi tệp chỉnh sửa gô´c.

▼ Đáp án Hô` sơ 02 — Phân tích từng lựa chọn · mở sau khi tự thử

A — Không có căn cứ. Cách một dịch vụ lưu, dùng lại hay cho người xem xét nội dung phụ thuộc điê`u khoản, gói dịch vụ và thiê´t lập — bạn chưa kiểm tra. Quan trọng hơn: bài không cầ`n dữ liệu thật.

B — Chưa phải ẩn danh. Mã sinh viên, giờ, quán quen và email vẫn đủ để người quen nhận ra. Mã hóa tên thành một mã ổn định trong khi vẫn giữ bảng đô´i chiê´u cũng chưa phải ẩn danh hoàn toàn.

C — Lựa chọn tô´t nhâ´t. Ba khoản giả vẫn đủ tái hiện lỗi bỏ khoản cuô´i (ra 42000 thay vì 60000). Dữ liệu được tạo mới từ đầ`u, không còn liên hệ với ai.

D — Rủi ro kỹ thuật. Lớp che có thể bị gỡ trong tệp gô´c. Nê´u thật sự cầ`n ảnh, **xuâ´t ảnh phẳng**, chỉ că´t đúng vùng lỗi rô`i mở lại ảnh đầ`u ra để kiểm tra.

Hộp pháp lý — đọc để biê´t mình cầ`n hỏi ai

> **Mô´c đô´i chiê´u nguồ`n: 29/10/2026 (kê´ thừa từ bản gô´c, chưa được kiểm tra lại trong lượt biên tập này).** Bản gô´c đô´i chiê´u Luật Bảo vệ dữ liệu cá nhân sô´ 91/2025/QH15 và Nghị định 356/2025/NĐ-CP, cùng có hiệu lực từ 01/01/2026 [1][2]; theo Điề`u 42 Nghị định 356, Nghị định 13/2023/NĐ-CP đã hê´t hiệu lực. Vê` sở hữu trí tuệ, bản gô´c dùng Văn bản hợp nhâ´t 67/VBHN-VPQH ngày 23/03/2026, thể hiện sửa đổi bởi Luật 131/2025/QH15 có hiệu lực từ 01/04/2026 [3]. Đây là định hướng học tập, **không phải ý kiê´n pháp lý**. Trước khi áp dụng cho dữ liệu, hợp đô`ng hay sản phẩm thật, mở văn bản gô´c và hỏi người có thẩm quyề`n.

Luật bảo vệ dữ liệu cá nhân có nội dung vê` khái niệm và khử nhận dạng, nguyên tă´c xử lý theo phạm vi và mục đích, sự đô`ng ý và xử lý trong hoạt động AI. Nhưng đừng rút gọn thành "mọi xử lý đề`u phải xin đô`ng ý": luật có các trường hợp ngoại lệ tại Điề`u 19 [1]. Cũng đừng tự suy diễn rằ`ng bài tập của mình chă´c chă´n thuộc ngoại lệ. **Nê´u bài không cầ`n dữ liệu thật, dùng dữ liệu giả là cách giảm vâ´n đề` ngay từ thiê´t kê´.**

Những chỗ hay quên: ảnh, chat, lịch sử

- **Ảnh:** email ở góc, đường dẫn chứa tên, tab trình duyệt, khóa trong bảng điề̀u khiển.
- **Chat:** đoạn từng dán rồ̀i xóa khỏi mã vẫn có thể còn trong lịch sử hội thoại.
- **Git, bản sao lưu, log, bản xuấ́t dự án:** có thể chứa thông tin ngoài màn hình hiện tại.

Đừng chỉ kiểm tra tệp đang mở. Với ví dụ cầ̀n chuỗi bí mật, dùng `DEMO_KEY_NOT_A_SECRET` và ghi rõ đó là placeholder.

Bài học

Dữ liệu giả không chỉ là đổi tên — đó là dữ liệu được tạo mới. Hỏi "lỗi này cầ̀n tố́i thiểu dữ liệu gì để tái hiện?" trước khi hỏi "gửi cho AI thế́ nào?".

4.3. Hồ̀ sơ 03 — Agent sửa và xóa ngoài phạm vi

Sự cố́

Lan giao cho Agent: "Sửa lỗi tổng tiề̀n trong `bai-lam/nam-2-web`." Agent báo đã xong. Lan mở diff và thấ́y: hàm tổng được sửa, nhưng `index.html` đổi giao diện, tệp `test_data.cjs` trong bản sao bị xóa hai ca đang thấ́t bại (việc xóa là hư cấ́u; tệp kèm cẩm nang vẫn đủ 34 ca), và có đề̀ nghị chạy lệnh cài một gói định dạng mã. Màn hình test giờ toàn màu xanh.

Dấ́u hiệu

- Diff chạm vào tệp nằ̀m ngoài mục tiêu.
- Số́ ca test **giảm** đúng bằ̀ng số́ ca đang thấ́t bại.
- Có lệnh cài đặt không được yêu cầ̀u.
- Báo cáo "đã xong" nhưng không nêu ca nào bị xóa.

Nguy cơ

Test xanh vì ca khó bị xóa là bằ̀ng chứng giả. Thay đổi giao diện không yêu cầ̀u làm khó review. Gói lạ có thể mang rủi ro chuỗi cung ứng (xem Hồ̀ sơ 04). Nhận cả gói thay đổi nghĩa là Lan nhận trách nhiệm cho những gì mình chưa đọc.

Một Agent có thể đọc nhiều tệp, gọi lệnh và gửi nội dung tới dịch vụ, tùy công cụ, chế độ và thiết lập. Đừng mặc định "nằm trên máy em" là "không thể đi vào ngữ cảnh gửi ra ngoài". Câu prompt "chỉ sửa tệp X" là **yêu cầu**, không phải hàng rào kỹ thuật; quyền thật nằm ở thiết lập và các hộp xác nhận bạn đồng ý.

Các lựa chọn

- **A.** Chấp nhận toàn bộ vì test đã xanh.
- **B.** Dừng Agent. Quay về checkpoint đã tạo trước khi giao việc (Mission 06, mục 2.6 Chương 2), rồi chỉ áp lại phần sửa hàm tổng sau khi đọc và hiểu diff đó.
- **C.** Bảo Agent "khôi phục lại như cũ" mà không kiểm tra nó khôi phục gì.
- **D.** Giữ thay đổi, chỉ tự gõ lại hai ca test đã bị xóa theo trí nhớ.

▼ Đáp án Hồ sơ 03 — Phân tích từng lựa chọn · mở sau khi tự thử

A — Sai. Test xanh sau khi xóa ca thất bại không chứng minh hàm đúng. Phần hữu ích không hợp thức hóa phần ngoài phạm vi.

B — Đúng. Checkpoint là đường quay lại đã chuẩn bị trước. Sau khi quay về, so sánh với diff, giữ đúng một thay đổi được giao, chạy lại **đủ** bộ test gốc (34 ca cho bản Web tham khảo). Ghi vào nhật ký rằng lượt Agent đầu vượt phạm vi; không xóa lượt đó để câu chuyện trông đẹp.

C — Khôi phục mù. Bạn lại giao quyền cho chính công cụ vừa vượt phạm vi mà không có bằng chứng về trạng thái sau đó. Nếu không có checkpoint, hãy dừng, giữ nguyên hiện trạng và đối chiếu từng tệp với bản `vi-du` chỉ đọc.

D — Nguy hiểm. Ca test gõ lại theo trí nhớ có thể khác ca gốc. Giao diện thay đổi và đề nghị cài gói vẫn còn đó.

Hợp đồ`ng giao việc nên có trước khi bâ´m chạy

```
Prompt / Ghi chú
```

```
Phạm vi: chỉ sửa hàm tính tổng trong bai-lam/nam-2-web/data.js.
Không sửa, xóa hoặc đổi tên tệp khác. Không xóa hoặc sửa ca test.
Không cài gói, đổi cấu hình, commit, đẩy mã hay truy cập mạng.
Trước khi sửa: nêu giả thuyết, dòng dự kiến đổi, ca test sẽ chạy.
Sau khi sửa: báo diff, số ca chạy, số ca đạt/không đạt, phần chưa kiểm.
Nếu bị chặn hoặc cần quyền mới: dừng và hỏi, không tìm đường vòng.
```

Bài học

Checkpoint trước, diff sau, nhận từng phâ`n. Sô´ ca test phải giữ nguyên hoặc tăng; thâ´y giảm là dừng lại hỏi “ca nào biê´n mâ´t và vì sao?”.

4.4. Hô` sơ 04 — Gói thư viện AI đê` xuâ´t

Sự cô´

Minh hỏi cách định dạng tiê`n cho đẹp. AI trả lời tự tin và đưa một lệnh cài cho gói có tên râ´t giô´ng một thư viện quen thuộc, kèm câu “chạy lệnh này là xong”. Minh chưa từng nghe tên gói này.

Dâ´u hiệu

- Tên gói gâ`n giô´ng thư viện phổ biê´n nhưng khác một vài ký tự (dâ´u hiệu **typosquatting**).
- Câu trả lời không dẫn registry chính thức, không nêu người duy trì hay kho nguô`n.
- Lệnh cài tải mã từ mạng rô`i chạy ngay.
- Bài hiện tại **không câ`n thêm phụ thuộc** để định dạng tiê`n; API có sẵn của trình duyệt đã đủ.

Nguy cơ

Gói không tô`n tại, gói bị đặt tên nhâ`m hoặc gói độc hại có thể chạy script khi cài. Script cài có thể đọc biê´n môi trường, tệp câ´u hình, khóa. Một lockfile giúp cô´ định phiên bản nhưng **không biê´n gói độc hại thành đáng tin** [6].

Các lựa chọn

- **A.** Chạy lệnh ngay; AI đã gợi ý thì chắc ổn.
- **B.** Hỏi AI "gói này có an toàn không?" rồi làm theo câu trả lời.
- **C.** Kiểm tra xem bài có thật sự cần phụ thuộc không. Nếu cần, đối chiếu tên, phiên bản, người duy trì, kho nguồn, giấy phép, dấu hiệu bảo trì và cảnh báo bảo mật trên registry chính thức — **không cài trong bài này**.
- **D.** Cài vào một thư mục tạm "cho biết", sau đó xóa.

▼ Đáp án Hồ sơ 04 — Phân tích từng lựa chọn · mở sau khi tự thử

A — Sai. Sự tự tin không phải bằng chứng. Tên gói do mô hình sinh ra có thể không tồn tại hoặc bị nhắm tới để đánh lừa.

B — Vòng tròn. Cùng một nguồn vừa đề xuất, vừa tự chứng nhận. Bằng chứng phải đến từ registry và kho nguồn, không phải từ lời khẳng định.

C — Đúng. Câu hỏi đầu tiên luôn là "bài có cần thêm phụ thuộc không?". Với ứng dụng HTML/JS nhỏ, câu trả lời thường là không. Hoạt động của chương chỉ là **lập bảng kiểm chứng, không thực hiện cài gói**. Nếu chưa xác minh được, bỏ đề xuất và dùng giải pháp đang hiểu.

D — Vẫn là chạy mã. Cài "cho biết" vẫn thực thi script trên máy bạn. Xóa thư mục sau đó không hoàn tác những gì script đã làm.

Bảng kiểm chứng để trống — điền khi thật sự cần

Câu hỏi	Bằng chứng cần có	Đã kiểm?
Tên gói chính xác?	Trang gói trên registry chính thức của hệ sinh thái	
Phiên bản dự định dùng?	Số phiên bản, ngày phát hành	
Ai duy trì?	Tài khoản nhà phát hành, lịch sử phát hành	
Kho nguồn?	Liên kết kho mã, khớp với registry	
Giấy phép?	Tệp giấy phép của gói	
Có script cài không?	Nội dung trường script trong manifest	

Câu hỏi	Bằng chứng cần có	Đã kiểm?
Có cảnh báo bảo mật?	Trang tư vấn bảo mật của hệ sinh thái	
Có phụ thuộc bắc cầu lạ?	Danh sách phụ thuộc, lockfile	

Không bịa URL registry để làm đầy cột. Cột trống trung thực tốt hơn một dòng "đã kiểm" không có căn cứ.

Bài học

Không cài chỉ để chứng tỏ chủ động. Thêm một phụ thuộc là thêm một bên bạn phải tin; hãy chắc mình thật sự cần.

4.5. Hồ sơ 05 — Bài tập hoàn toàn do AI làm

Sự cố

Đề ViSinhVien ghi: "Được tham khảo tài liệu." Không nhắc gì đến AI. Minh dán cả đề vào chat, nhận về toàn bộ `index.html`, `data.js`, `app.js` và một bài giải thích. Bạn sửa vài chỗ, chạy được, rồi định ghi trong phần khai báo: "Có tham khảo ý tưởng từ AI."

Dấu hiệu

- Đề không nói rõ về AI; "tham khảo tài liệu" **không đồng nghĩa** với "AI được viết hộ".
- Toàn bộ chức năng chính do AI sinh.
- Lời khai nhỏ hơn mức hỗ trợ thực tế.
- Minh không giải thích được vì sao kiểm tra `amount` nằm ở đó.

Nguy cơ

Đây là vấn đề **học thuật**, và cách xử lý phụ thuộc yêu cầu học phần — chương này không thay quy định của trường. Ba hậu quả có thể xảy ra: mất phần luyện mà môn học nhắm tới; khai báo sai làm mất tính trung thực; không bảo vệ được bài khi bị hỏi.

Các lựa chọn

- **A.** Nộp nguyên, khai “có tham khảo ý tưởng”.
- **B.** Hỏi giảng viên bằng một tin nhắn mô tả **đúng việc đã làm**, tiếp tục phần chắc chắn được phép trong lúc chờ, không mặc định im lặng là đồng ý.
- **C.** Xóa hết, tự viết lại từ đầu theo trí nhớ của mã AI.
- **D.** Không khai, vì “ai cũng dùng”.

▼ Đáp án Hồ sơ 05 — Phân tích từng lựa chọn · mở sau khi tự thử

A — Khai không khớp. Nếu AI sinh cả hàm, phải nói là sinh bản nháp của hàm, không đổi thành “chỉ tham khảo ý tưởng”. Khai báo không thay thế sự cho phép.

B — Đúng hướng. Mô tả hành động cụ thể để giảng viên trả lời được:

```
Prompt / Ghi chú

Thưa thầy/cô, với bài ViSinhVien, em dự định tự viết phần thêm khoản chi và tính tổng.
Em muốn dùng AI để giải thích lỗi JavaScript và gợi ý ca kiểm thử cho amount,
sau đó tự sửa và chạy kiểm tra. Em không định gửi dữ liệu thật hoặc nhờ AI làm toàn bộ bài.
Phạm vi này có được phép không ạ? Nếu được, nhóm cần khai báo ở đâu và giữ bằng chứng nào?
Nếu không, em sẽ dùng tài liệu môn học theo hướng dẫn.
```

C — Trung thực nhưng lãng phí. Viết lại từ trí nhớ vẫn là làm lại bài của người khác, và bạn có thể đang bỏ phần học. Việc đúng là hỏi phạm vi, rồi **tự làm lại phần cốt lõi có hiểu**.

D — Tệ nhất. “Bạn cùng lớp cũng dùng” hay “em chỉ dùng bản miễn phí nên không tính” đều không phải căn cứ học thuật. Không khai là sai lệch về đóng góp.

Bảng phân loại — để hỏi, không phải để tự cho phép

Bảng này **không phải quy định của TLU**; nó giúp bạn đặt câu hỏi đúng.

Tình trạng	Ví dụ trong bài ViSinhVien	Việc cần làm
Đã được cho phép	Đề ghi được dùng AI gợi ý ca kiểm thử, phải khai báo	Dùng đúng phần được phép, lưu lại phần hỗ trợ và tự kiểm tra

Tình trạng	Ví dụ trong bài ViSinhVien	Việc cần làm
Đã bị cấm	Bài kiểm tra yêu cầu tự viết, không dùng công cụ sinh mã	Không nhờ AI tạo lời giải hay sửa mã trong thời gian làm bài
Chưa rõ	Đề cho tham khảo tài liệu nhưng không nói về AI	Hỏi giảng viên trước; chưa biến giả định thành sự cho phép
Có điều kiện	Được hỗ trợ giải thích nhưng không được sinh toàn bộ chức năng	Tách câu hỏi giải thích khỏi yêu cầu viết hộ

Với bài nhóm, cả nhóm cần biết phạm vi: một thành viên đưa mã AI vào nhánh chung có thể ảnh hưởng bản nộp của mọi người.

Bài học

Khai báo là mô tả, không phải xin phép. Hỏi trước về phạm vi; khai sau đúng với việc đã làm.

4.6. Hồ sơ 06 — Đoạn mã không rõ giấy phép

Sự cố

Minh tìm được một giao diện đẹp trên GitHub. Trang kho mở được không cần đăng nhập, nên bạn định chép một tệp vào ViSinhVien. Kho **không có tệp LICENSE**.

Dấu hiệu

- Không có giấy phép, hoặc chỉ có tên giấy phép mà chưa đọc điều kiện.
- Có giấy phép cho mã nhưng ảnh và phông chữ đi kèm có điều kiện khác.
- Định “ghi nguồn là đủ”.

Nguy cơ

Công khai **không** tự trao quyền sao chép, phân phối hay tạo tác phẩm phái sinh [8]. Ghi nguồn là việc học thuật; nó không thay quyền sử dụng. Sản phẩm bị gắn đoạn không rõ quyền có thể phải gỡ bỏ hoặc gây tranh cãi khi công bố.

Các lựa chọn

- **A.** Chép vì kho công khai và ghi nguồn ở cuối bài.
- **B.** Tự viết lại bảng khoản chi đơn giản mình hiểu; bỏ phần giao diện chưa rõ quyền.
- **C.** Đổi tên biến và màu sắc để "không ai nhận ra".
- **D.** Nhờ AI viết lại giao diện đó từ ảnh chụp màn hình.

▼ Đáp án Hồ sơ 06 — Phân tích từng lựa chọn · mở sau khi tự thử

A — Không đủ. Không có giấy phép nghĩa là không có cơ sở để coi là tự do sao chép. Đường dẫn nguồn không tạo ra quyền.

B — Đúng cho bài này. Sản phẩm bớt bóng bẩy nhưng nhóm biết nguồn gốc từng phần. Đây là giảm một vấn đề không cần thiết, không phải phòng thủ đến mức không dám học từ ai.

C — Vẫn là sao chép. Thay tên biến hay màu sắc không xóa nghĩa vụ và không đổi bản chất việc dùng tác phẩm của người khác.

D — Chưa chắc sạch. Đầu ra có thể gần với thiết kế gốc; nhãn "AI tạo" không tự chứng minh nguồn gốc hay quyền sở hữu độc quyền. Kiểm nguồn nếu được dẫn và điều khoản công cụ bạn dùng.

Đọc tên giấy phép chưa đủ

Tình huống	Điều cần hiểu	Hành động thực tế
Mã mang giấy phép MIT	Có quyền sử dụng rộng, nhưng phải giữ thông báo bản quyền và giấy phép theo điều kiện của MIT	Giữ nội dung cần thiết khi sao chép; lưu nguồn và phiên bản
Mã mang giấy phép GPL	**Không** phải giấy phép "cấm thương mại"; nghĩa vụ phụ thuộc phiên bản, việc chuyển giao và quan hệ với tác phẩm phái sinh	Đọc đúng GPL áp dụng; hỏi người có chuyên môn nếu chưa xác định được nghĩa vụ
Kho không có giấy phép rõ	Không có cơ sở để coi là tự do sao chép	Không đưa vào bản nộp chỉ bằng cách thêm đường dẫn
AI đưa một đoạn mã	Nhãn "AI tạo" không tự chứng minh nguồn gốc hay quyền	Kiểm nguồn nếu có, điều khoản công cụ, dấu hiệu sao chép; thay phần chưa rõ quyền

Với MIT, ghi nguồn học thuật và giữ thông báo giấy phép là hai việc khác nhau. Với GPL, đừng kết luận chỉ từ số dòng đã chép, cũng đừng nói "thấy GPL là không được dùng". Phải xét văn bản giấy phép cụ thể; chương này không quyết định tính tương thích cho sản phẩm thật.

Về pháp luật Việt Nam, bản gốc dẫn Điều 22 và Điều 25 khoản 3 của văn bản hợp nhất về sở hữu trí tuệ [3]: chương trình máy tính được bảo hộ, và lý do "em sao chép để học" không tự động là ngoại lệ cho mọi trường hợp. Điều này không cấm học từ tài liệu; nó nhắc kiểm tra quyền trước khi mang mã người khác vào bản nộp hoặc sản phẩm phân phối.

Thẻ nguồn nhỏ — điền ngay khi chọn tài nguyên

Thành phần định dùng	Nguồn/phiên bản	Giấy phép và việc phải giữ	Quyết định
Ví dụ: đoạn CSS bố cục	URL cụ thể đã mở, ngày xem	Tên giấy phép, tệp LICENSE, điều kiện giữ thông báo	Dùng có điều kiện / hỏi thêm / không dùng

Dẫn đúng bản đã dùng, không chỉ trang chủ. Phân biệt **tham khảo ý tưởng thuật toán** với **sao chép cách biểu đạt cụ thể**. Chưa rõ thì khai nguồn và hỏi người chấm, không đổi vài chữ để che dấu vết.

Bài học

Công khai không phải giấy phép. Một sản phẩm đơn giản mình hiểu tốt hơn một giao diện đẹp không rõ nguồn.

4.7. Hồ sơ 07 — Khai báo thiếu và ba tầng trách nhiệm

Sự cố

Bài nộp của nhóm ghi: "Có sử dụng AI để tham khảo." Nhưng AI đã sinh bản nháp hàm `parseAmount`, gợi ý bảy ca test, và Lan chép một đoạn CSS từ một kho không rõ giấy phép. Minh không chạy test trên máy mình mà chỉ đọc mã. Khi giảng viên hỏi "phần này em kiểm chứng thế nào?", cả hai nhìn nhau.

Dấu hiệu

- Một câu khai báo chung chung, không phân biệt gợi ý / sinh nội dung / nhóm sửa gì.
- Không nêu đã chạy gì, trên bản nào.

- Ghi “đã xác minh” trong khi chỉ đọc mã.
- Không nhắc nguồn CSS.

Nguy cơ

Người chấm không đánh giá được đóng góp thật. Nếu bị hỏi trực tiếp, nhóm không giải thích được — và đó là dấu hiệu mất phần học, không chỉ là lỗi trình bày. Về lâu dài, thói quen khai cao hơn thực tế sẽ theo sang báo cáo thực tập và công việc.

Các lựa chọn

- **A.** Giữ câu chung chung; nộp kèm toàn bộ lịch sử chat để “đầy đủ”.
- **B.** Viết lại khai báo theo đúng việc đã làm: công cụ, thời điểm, phần được sinh, phần nhóm sửa, đã chạy gì và chưa chạy gì, nguồn CSS. **Loại hoặc thay đoạn CSS chưa rõ quyền** rồi ghi phần đã loại — không giữ lại chỉ vì đã ghi nguồn. Bổ sung phần còn thiếu nếu còn thời gian; nếu không, ghi đúng trạng thái.
- **C.** Ghi “tự làm 100%” vì nhóm đã gõ lại vài dòng.
- **D.** Bỏ phần CSS ra khỏi bài nhưng không nhắc gì trong khai báo.

▼ Đáp án Hồ sơ 07 — Phân tích từng lựa chọn · mở sau khi tự thử

A — Quá tải và rủi ro. Một câu chung chung không đủ thông tin; dán cả lịch sử chat có thể đưa vào dữ liệu riêng tư không cần thiết. Mô tả đủ để giảng viên đánh giá, theo hình thức giảng viên yêu cầu.

B — Đúng. Khai báo tốt phân biệt: AI gợi ý gì, sinh nội dung nào, nhóm thay đổi gì, kiểm chứng đến đâu. Đoạn CSS chưa rõ quyền phải **loại khỏi bài** chứ không chỉ ghi nguồn: thêm dòng nguồn không biến mã không rõ giấy phép thành dùng được. Nếu mới đọc mã mà chưa chạy, ghi “đã rà soát bằng đọc mã; chưa chạy kiểm thử”, không ghi “đã xác minh”. Có thể bổ sung bằng chứng chạy test ngay trên **bản sao của nhóm**: từ thư mục gốc AI-Student-Handbook chạy `node "bai-lam/nam-2-web/test_data.cjs"`, chép nguyên dòng tổng kết cuối và exit code (Bash: `echo $?`; PowerShell: `$LASTEXITCODE`, chạy ngay sau lệnh — xem Chương 2); ghi đúng máy và phiên bản Node đã dùng. Với demo tham khảo chưa sửa, dòng cuối là `34 passed, 0 failed`; bản của nhóm có thể khác nếu đã đổi mã, và khi đó phải ghi đúng con số thực tế.

C — Sai. Gõ lại vài dòng không biến phần sinh hàng loạt thành tự làm. Tính trung thực nằm ở độ khớp giữa lời khai và hành động.

D — Che một phần. Bỏ đoạn không rõ quyền là tốt, nhưng vẫn cần nói rõ phần đã loại và lý do nếu hình thức khai báo yêu cầu.

Nê´u bài **đã nộp** rô`i mới phát hiện khai báo thiê´u, chủ động gửi một tin ngă´n cho giảng viên thay vì chờ bị hỏi:

```
Prompt / Ghi chú
```

```
Thưa thầy/cô, bài [môn, mã bài] nhóm em đã nộp ngày [ngày].
Khai báo AI trong bài còn thiếu: [AI sinh nháp parseAmount, gợi ý 7 ca test].
Bảy ca gợi ý là ca riêng, không nằm trong 34 ca của bộ tham khảo; đừng cộng gộp hai con số.
Nhóm đã [chạy 34 ca / mới đọc mã], và đã [loại đoạn CSS chưa rõ quyền].
Em gửi khai báo sửa lại đính kèm. Nhóm em sẵn sàng giải thích phần đã làm.
```

Ba tâ`ng trách nhiệm — không đổ lỗi cho công cụ, không nhận bừa mọi lỗi

Minh hỏi: “Nê´u AI viê´t sai thì em chịu hê´t à?” Câ`n tách các tâ`ng.

Tâ`ng	Câu hỏi câ`n trả lời	Nhóm kiểm soát được	Không nên khẳng định
Học thuật	Bài có đúng yêu cầ`u môn; đóng góp và khai báo có trung thực?	Hỏi phạm vi AI, giải thích mã, ghi nguô`n, ghi kê´t quả thật	Mọi dùng AI đê`u là đạo văn; hoặc khai báo là chă´c chă´n được chấ´p nhận
Nghê` nghiệp	Sản phẩm có được rà soát đúng vai trò và cảnh báo đúng giới hạn?	Kiểm dữ liệu, báo lỗi, yêu cầ`u người phù hợp duyệt thay đổi	Có AI hỗ trợ thì người làm không còn trách nhiệm; hoặc một người luôn gánh mọi lỗi nhóm
Pháp lý	Có vâ´n đê` dữ liệu, quyê`n tác giả, hợp đô`ng hay quyê`n truy cập câ`n xem xét?	Ngừng hành vi có vâ´n đê`, giữ thông tin câ`n thiê´t, báo đâ`u mô´i có thẩm quyê`n	Tự kê´t luận có tội, mức phạt hay bô`i thường từ một ví dụ trong lớp

Không có một câu “chịu 100% pháp lý” áp dụng cho mọi sinh viên và mọi đâ`u ra AI. Người nộp bài phải giải thích việc mình làm và mức kiểm tra; kê´t luận kỷ luật hay pháp lý phụ thuộc quy định, vai trò và tình tiê´t.

Chia vai nhưng không tạo khoảng trô´ng

Minh phụ trách nhập và tính tổng; Lan rà soát hiển thị và nguô`n; cả hai xem lại bản nộp. **“Lan đã xem” phải rõ là xem gì**: xem diff, đọc hàm, hay chạy ứng dụng. Với `textContent`, bă`ng chứng là diff và quan sát `<b>Vở</b>` hiện nguyên văn. Với giới hạn 500 khoản, phải thử đúng giới hạn; ảnh tổng 60000 không chứng minh được điê`u đó.

Nói mức chắc chắn bằng ngôn ngữ kiểm chứng

Thay câu này	Bằng câu này
“An toàn tuyệt đối”	“Đã sửa chỗ hiển thị; ca `<b>Vở</b>` đạt, ca giới hạn 501 khoản chưa chạy”
“Không có dữ liệu nhạy cảm”	“Đã rà các tệp X, Y và hai ảnh Z; không thấy email hay khóa thật”
“AI bảo là MIT”	“Đã mở tệp LICENSE trong kho, ghi lại tên và phiên bản”
“Đã test xong”	“Chạy bộ test trên bản sao bằng Node [phiên bản em dùng]; [số] đạt, [số] không đạt; chưa thử trên trình duyệt khác”

Nêu giới hạn giúp người đọc đối chiếu, nhưng không được dùng để che việc cốt lõi chưa làm. Phát hiện lỗi thì ghi tác động và cách khắc phục; không sửa lén kết quả cũ.

Mẫu khai báo AI — điền bằng sự thật

Prompt / Ghi chú

```
Bài/nhóm: ViSinhVien — [thành viên và vai trò].
Phạm vi được phép: [hướng dẫn của bài hoặc phản hồi giảng viên; nếu chưa rõ, ghi đang chờ].
Công cụ và thời điểm: [tên, phiên bản nếu biết, ngày dùng].
Hỗ trợ thực tế: [giải thích, gợi ý, sinh nháp phần nào; tệp/hàm liên quan].
Đóng góp của nhóm: [ai viết lại gì, lựa chọn nào được giữ hoặc bỏ và lý do].
Dữ liệu đã cung cấp: [bộ dữ liệu giả, đoạn mã cần thiết; không dán bí mật].
Nguồn và giấy phép: [nguồn thật đã dùng, điều kiện cần giữ, phần đã loại vì chưa rõ].
Kiểm chứng: [đã chạy gì, trên bản nào, kết quả; phần chỉ đọc mã; phần chưa xác minh].
Giới hạn: HTML/JavaScript dùng localStorage, không backend/đăng nhập; chỉ phục vụ học tập
với dữ liệu giả. SQL là minh họa Python sqlite3 riêng trong :memory:.
```

Ví dụ đã điền cho đúng tình huống trên — đây là **mẫu, không phải lời khai của bạn**:

```
Prompt / Ghi chú
```

```
Bài/nhóm: ViSinhVien — Minh (nhập, tính tổng), Lan (hiển thị, nguồn).
Phạm vi được phép: theo hướng dẫn học phần; đang chờ xác nhận cho phần sinh mã.
Công cụ và thời điểm: trợ lý AI qua chat, dùng ngày [ngày].
Hỗ trợ thực tế: sinh bản nháp parseAmount trong data.js; gợi ý bảy ca test;
  không sinh phần render hay handler submit.
Đóng góp của nhóm: viết lại parseAmount để từ chối hậu tố; giữ totalExpenses;
  bỏ đoạn CSS nghi không rõ giấy phép.
Dữ liệu đã cung cấp: ba khoản giả 35000, 7000, 18000; không dán dữ liệu thật.
Nguồn và giấy phép: không còn mã ngoài; CSS chưa rõ quyền đã loại khỏi bài.
Kiểm chứng: chạy node bai-lam/nam-2-web/test_data.cjs trên bản sao của nhóm
  (Node [phiên bản]) — [chép nguyên dòng tổng kết, ví dụ 34 passed, 0 failed];
  phần hiển thị mới đọc mã, chưa thử trên trình duyệt khác.
Giới hạn: localStorage, không backend, dữ liệu giả; SQL chỉ là minh họa riêng.
```

Bài học

Khai báo tốt là bản đồ đóng góp, không phải câu xin lỗi hay lời khen. Viết để người khác kiểm lại được, không phải để trông đẹp.

4.8. Phòng khám kỹ thuật — SQL, XSS và C++

Ba lỗi dưới đây có cùng một gốc: **biết hợp đồng ở ranh giới, không tin vào một lần chạy may mắn.** Không cần thử chuỗi phá hoại để hiểu bất kỳ lỗi nào.

4.8.1. SQL — giá trị là dữ liệu, không phải cấu trúc câu lệnh

Ranh giới ví dụ: SQL ở đây chạy riêng bằng Python `sqlite3` với `:memory:`; ViSinhVien không gửi SQL và không có backend. Tệp minh họa là `vi-du/minh-hoa-sqlite.py`. Không nối minh họa này vào sơ đồ kiến trúc ứng dụng Web.

Giả sử bảng minh họa có cột `id`, `title`, `amount`, `category`. Lan muốn tìm một tên thử là `Vở O'Reilly`. Dấu nháy trong tên là dữ liệu bình thường, nhưng cách nối chuỗi sau đưa nó vào cú pháp SQL:

```
python
```

```
# Mẫu KHÔNG nên dùng; chỉ đọc để nhận diện vấn đề.
query = "SELECT title,amount FROM expenses WHERE title = '" + title + "'"
```

Với tên có dấu nháy, câu lệnh có thể sai cú pháp. Đáng ngại hơn, nối trực tiếp dữ liệu không tin cậy vào câu lệnh có thể cho dữ liệu **làm thay đổi ý nghĩa SQL**. Một tên sách hợp lệ đã đủ chỉ ra chương trình đang trộn hai vai trò: dữ liệu và mã lệnh.

Cách đúng trong ngữ cảnh Python `sqlite3`:

```python
# Rút gọn từ vi-du/minh-hoa-sqlite.py; ở đó biến kết nối tên là connection,
# tạo bằng sqlite3.connect(":memory:").
rows = connection.execute(
    'SELECT title, amount FROM expenses WHERE title = ?',
    (title,)
).fetchall()
```

Dấu `?` là vị trí dành cho giá trị được gắn riêng. `(title,)` là tuple một phần tử; **dấu phẩy tạo tuple**, còn `(title)` chỉ là biểu thức trong ngoặc. Thiếu dấu phẩy là sai cấu trúc đối số. Trình điều khiển tách giá trị khỏi cấu trúc SQL, nên dấu nháy của `Vở O'Reilly` không cần bị thay thế bằng mẹo ghép chuỗi.

OWASP ưu tiên truy vấn tham số hóa cho tình huống này [4]. Tuy vậy, **tham số giá trị không thay thế tùy ý tên bảng, tên cột hay từ khóa sắp xếp**. Nếu chương trình cho chọn cách sắp xếp, hãy ánh xạ lựa chọn sang một tập tên cột định trước — đó là lúc cần danh sách cho phép cho phần cấu trúc.

Chạy minh họa từ thư mục gốc dự án:

```Bash
python "vi-du/minh-hoa-sqlite.py"
```

Nếu lệnh `python` không chạy, thử `py` trên Windows hoặc `python3` trên macOS/Linux; trên Windows, gõ `python` khi chưa cài có thể mở Microsoft Store thay vì chạy chương trình. Không có Python thì ghi "chưa chạy, thiếu Python" và đọc tiếp phần giải thích — đây là minh họa khái niệm, không phải điều kiện học xong chương.

Dòng đáng chú ý nhất là `Tra cuu tham so: [("Vở O'Reilly", 12000)]`: dấu nháy đi qua truy vấn như dữ liệu bình thường. Các dòng `PASS reject amount` cho thấy ràng buộc của bảng từ chối số âm, số 0, số thập phân và số vượt `1e9`.

Ghi rõ đây là **fixture SQL riêng**, không thêm nó vào bộ ba khoản chuẩn rồi vẫn tuyên bố tổng 60000. Truy vấn tham số hóa cũng không tự giải quyết phân quyền, rò rỉ log hay mọi lỗi nghiệp vụ.

4.8.2. XSS — hiển thị tên khoản chi như văn bản

Minh muốn giao diện linh hoạt nên thử gán tên khoản chi bằng `innerHTML`. Lan đưa đầu vào vô hại `<b>Vở</b>` và hỏi: "Người dùng nhập một cái tên, tại sao ứng dụng lại diễn giải nó thành thẻ?" Nếu giao diện in đậm chữ Vở, trình duyệt đang phân tích đầu vào như HTML. Ví dụ này **không chạy mã độc**, nhưng làm lộ điểm dữ liệu đã bị trao thêm ý nghĩa.

Cách đúng cho một ô bảng cần hiển thị tên dưới dạng văn bản:

```JavaScript
const td = document.createElement('td');
td.textContent = expense.title;
```

Khi `expense.title` là `<b>Vở</b>`, nội dung mong đợi nhìn thấy là chính chuỗi `<b>Vở</b>`, gồm cả dấu ngoặc nhọn, không phải chữ Vở in đậm. `td` là tên biến của đoạn minh họa. Trong `app.js` đi kèm, ô bảng có tên biến `cell`: tìm `cell.textContent = value` trong hàm `render`; `value` lần lượt là tên khoản, nhãn nhóm hoặc số tiền đã định dạng. Không đổi chỗ gán này thành `innerHTML` cho tên khoản chi.

OWASP liệt kê `textContent` là một cách đưa dữ liệu vào nơi nhận văn bản an toàn trong DOM thông thường [5]. Phạm vi "văn bản thông thường" rất quan trọng: cách bảo vệ cho ô bảng **không** tự áp dụng cho URL, thuộc tính sự kiện, CSS hay chuỗi mã JavaScript. Nếu sau này thêm đường dẫn, cần kiểm soát loại URL và ngữ cảnh riêng. Nếu thật sự cần HTML phong phú, đó là yêu cầu mới cần cơ chế làm sạch phù hợp.

Cũng đừng suy luận "dữ liệu nằm trong `localStorage` của em nên tin được". Người dùng có thể sửa kho lưu trữ, dữ liệu cũ có thể đến từ phiên bản lỗi, máy có thể dùng chung. Hãy kiểm tra lúc nhập, lúc đọc lại và lúc đưa ra giao diện, theo trách nhiệm của từng lớp.

4.8.3. C++ — hai lỗi cận khác nhau: bỏ sót và vượt biên

Ở bài năm nhất bằng C++, Minh dùng một `std::vector<long long>` gồm ba số tiền `35000, 7000, 18000`; tổng đúng là **60000**. Với `n` phần tử, chỉ số hợp lệ là `0` đến `n - 1`. Có hai cách viết sai cận, và chúng **không cùng loại lỗi**.

Lỗi 1 — cận quá hẹp, bỏ sót khoản cuối. Đây là lỗi có chủ đích trong `vi-du/nam-1-cpp/loi-bo-sot.cpp`:

C++

```
// Trích đoạn rút gọn; chú thích tiếng Việt do chương thêm vào để giải thích.
int n = static_cast<int>(amounts.size());   // n = 3
for (int i = 0; i < n - 1; ++i) {           // CỐ Ý SAI: chỉ chạy i = 0, 1
    total += amounts[i];
}
// Tệp in: Tong chi tieu: 42000 VND — thiếu Vở ghi 18000.
```

Chương trình **không đọc ra ngoài vector**, nên đây không phải hành vi không xác định. Nó chạy ổn định và luôn ra 42000 — một kết quả sai nhưng trông hợp lý, nên dễ lọt nếu không tính tay trước. Sửa bằng `i < n` (trace đầy đủ ở Chương 3, mục 3.2). Kết quả này đã được biên dịch và chạy thật trong lượt kiểm ví dụ của cẩm nang: `Tong chi tieu: 42000 VND`.

Lỗi 2 — cận quá rộng, vượt biên. Viết `i <= n` thì lượt cuối đọc `amounts[3]`, một chỉ số không tồn tại. Truy cập ngoài biên bằng `operator[]` dẫn đến **hành vi không xác định**.

"Hành vi không xác định" **không có nghĩa chắc chắn chương trình sẽ crash**. Nó có thể trông như chạy đúng, đọc giá trị lạ hoặc có biểu hiện khác. Một lần chạy không lỗi không chứng minh truy cập đó hợp lệ. Đừng chạy thử truy cập sai rồi dùng việc "máy em vẫn ổn" làm bằng chứng.

Khi đọc vòng lặp duyệt mảng, đối chiếu điều kiện `i < n` — không phải `i < n - 1` (bỏ sót) và không phải `i <= n` (vượt biên). Khi chỉ số đến từ đầu vào, kiểm miền trước khi truy cập. Có thể tìm hiểu `vector::at()` để biết cơ chế báo lỗi ngoài biên của nó, nhưng vẫn phải xử lý đầu vào và lỗi phù hợp.

Bài học chung với JavaScript và SQL: **biết hợp đồng ở ranh giới, không tin vào một lần chạy may mắn.**

4.9. Thực hành trước khi nộp: kiểm tra, sửa và đưa bằng chứng

Bài tập: duyệt một kho ViSinhVien giả định

Bạn rà soát bản nộp của Minh và Lan bằng dữ liệu giả, không truy cập tài khoản thật, không cài gói, không tạo khóa. Các lỗi dưới đây là **phiếu tình huống**, không phải kết luận về kho ví dụ đang có trong gói.

Làm lần lượt: đối chiếu đặc tả → đọc luồng nhập–kiểm tra–lưu–hiển thị → xem nguồn và phạm vi AI → phân loại → sửa → ghi bằng chứng. Nếu chỉ đọc mã, đánh dấu phần chạy thử là **chưa thực hiện**.

Vâ´n đề` trong kho giả định	Mức xử lý	Cách sửa hoặc quyê´t định	Bă`ng chứng câ`n thu
Tổng mẫu khác 60000	Chặn nộp	Rà chuyển đổi sô´ và tính tổng	Đâ`u vào, tổng trước/sau sửa
Nhận 0, sô´ âm, sô´ thập phân hoặc tiê`n vượt `1e9`	Chặn nộp	Kiểm tra đúng hợp đô`ng `amount`	Ca bị từ chô´i; nhận 1 và `1e9`
Nhận khoản thứ 501 hoặc tổng rỗng khác 0	Chặn nộp	Sửa giới hạn và tổng ban đâ`u	Kê´t quả thử 500/501 khoản và rỗng
Hiện tên bă`ng `innerHTML`	Chặn nộp	Dùng `cell.textContent` như trong `app.js`	Diff; `<b>Vở</b>` hiện nguyên văn
Có dữ liệu thật không câ`n thiê´t hoặc khóa thật	Chặn nộp	Dừng chia sẻ; xử lý theo 4.1; dùng dữ liệu giả	Ghi nhận xử lý, không đính kèm bí mật
Mã chưa rõ giâ´y phép	Chặn nộp phâ`n đó	Loại/thay hoặc xác minh quyê`n	Nguô`n, giâ´y phép hoặc phâ`n thay thê´
Chưa rõ phạm vi dùng AI	Hỏi giảng viên	Hỏi cụ thể; không mặc định được phép	Phản hô`i hoặc trạng thái chờ
Khai báo sai mức hỗ trợ AI	Chặn nộp	Khai đúng và theo hướng dẫn môn	Khai báo khớp đóng góp
Gói AI đê` xuâ´t chưa xác minh	Chặn đưa gói vào bản nộp	Không cài; xác minh hoặc bỏ	Registry, người duy trì, nguô`n, giâ´y phép
Màu nút hoặc khoảng cách chưa đẹp	Cải thiện sau	Ưu tiên yêu câ`u cô´t lõi	Danh sách cải tiê´n

Chặn nộp **không** có nghĩa hủy cả bài: có thể loại phâ`n chưa rõ hoặc thu hẹp chức năng nê´u vẫn đáp ứng yêu câ`u. Hỏi giảng viên không có nghĩa được bỏ qua trong lúc chờ. Màu săˊc có thể cải thiện sau; dữ liệu thật trong ảnh thì không.

Ghi bă`ng chứng gọn và đô´i chiê´u được

Ghi: **mục tiêu, đâ`u vào, mong đợi, quan sát, trạng thái, bản mã kiểm tra.** Ví dụ:

Prompt / Ghi chú

```
Mục tiêu: tổng ba khoản giả.
Đầu vào: 35000, 7000, 18000.
Mong đợi: 60000.
Quan sát: [điền sau khi chạy].
Trạng thái: chưa chạy / đạt / không đạt.
Bản mã: bai-lam/nam-2-web, [ngày hoặc checkpoint đã chạy].
```

Ghi riêng fixture SQL; chỉ giữ vùng ảnh cần thiết.

Hai bộ kiểm thử kèm cẩm nang **không cộng gộp**: `vi-du/nam-2-web/test_data.cjs` có **34 ca** cho phần dữ liệu Web (33 ca gốc cộng một ca hồi quy cho lỗi `category` dạng mảng), còn `vi_sinh_vien.exe --test` của bản C++ (biên dịch từ bản sao `bai-lam/nam-1-cpp/vi_sinh_vien.cpp` theo lệnh ở Chương 2) chỉ in **9 ca**. Bộ đề Final Boss có số ca riêng (Bronze 9, Silver 12), cũng không cộng vào hai bộ này. Số ca không đồng nghĩa đã chạy hay bảo đảm chất lượng. Đối chiếu bộ mình nhận được, ghi cách chạy, số ca thực sự thực thi và kết quả. Chưa có xác nhận thì ghi "chưa xác minh".

Rubric bốn mục — tự đánh giá

Mỗi mục 0–2 điểm, tối đa 8. Đây là rubric luyện tập, **không phải thang điểm chính thức của TLU**; điểm tổng không bù được vấn đề chặn nộp.

Mục	0 điểm	1 điểm	2 điểm
1. Trung thực học thuật và nguồn	Không rõ phạm vi, đóng góp hoặc nguồn	Có khai báo nhưng còn mơ hồ	Phạm vi rõ; hỗ trợ, nguồn và quyền được ghi đúng
2. Dữ liệu và quyền truy cập	Dùng dữ liệu thật không cần thiết hoặc quyền quá rộng	Có giảm dữ liệu nhưng chưa rà đủ bản chia sẻ	Dùng dữ liệu giả; rà ảnh/tệp; quyền phù hợp công việc
3. Tính đúng và đọc mã an toàn	Không đối chiếu hợp đồng; nhầm SQL là backend	Biết sửa nhưng bằng chứng còn thiếu	Đối chiếu các giới hạn, hiển thị văn bản và ranh giới ví dụ
4. Kiểm chứng và xử lý vấn đề	Tuyên bố đạt không có căn cứ	Ghi được một phần kết quả và việc còn thiếu	Bằng chứng truy lại được; ưu tiên đúng; báo cáo trung thực

Sáu tình huống tự trả lời

1. Đề chỉ ghi "tham khảo tài liệu". Mình khai báo đầy đủ việc AI sinh hàm. Đã chắc được phép chưa?

2. Lan bỏ tên trong chi tiêu thật nhưng giữ mã sinh viên, giờ, địa điểm; ảnh còn email. Đã là dữ liệu giả hoặc ẩn danh chưa?

3. SQL nối chuỗi bị lỗi với `Vở O'Reilly`. Có nên xóa dấu nháy khỏi tên để sửa?

4. `<b>Vở</b>` hiện in đậm; vòng lặp C++ dùng `i <= n` vẫn chạy xong. Đã đạt chưa?

5. "Kho công khai không có giấy phép thì được chép, còn GPL cấm thương mại" — đúng không?

6. Đã commit khóa thật, Minh xóa tệp, tạo khóa mới rồi định ép đẩy lịch sử. Thiếu gì?

▼ Đáp án sáu tình huống · mở sau khi tự thử

1. Chưa. Khai báo không thay sự cho phép. Hỏi giảng viên về việc đã làm; không đổi lời khai thành "gợi ý nhẹ"; chưa mặc định phần đó hợp lệ.

2. Chưa. Các trường còn lại vẫn có thể nhận diện người. Tạo dữ liệu giả mới, loại ảnh chứa thông tin thật và kiểm tra bản gửi cuối.

3. Không. Dùng `execute('SELECT title,amount FROM expenses WHERE title = ?', (title,))`; dấu phẩy tạo tuple một phần tử. Kiểm fixture SQL riêng trong `:memory:`, không thêm backend cho Web.

4. Chưa. Tên phải hiện nguyên văn bằng `textContent` trong ô bảng. Riêng phần C++ ở mục 4.8.3: vòng lặp chạm chỉ số `n` (viết `i <= n`) là vượt biên; hành vi không xác định không nhất thiết crash. Sửa theo hợp đồng, không dựa vào may mắn.

5. Cả hai đều sai. Công khai không tự cấp quyền. GPL không cấm thương mại; nghĩa vụ phụ thuộc giấy phép và cách chuyển giao, kết hợp mã.

6. Dừng chia sẻ; thu hồi và xác nhận khóa cũ đã bị vô hiệu; kiểm tra mức sử dụng trong phạm vi được phép; rà bản liên quan; báo đầu mối và phối hợp giảm bản lưu. Không tự viết lại lịch sử.

Checklist một trang trước khi gửi

Đánh dấu **sau khi làm**, không đánh dấu theo dự định.

A. Chặn nộp nếu chưa xử lý

- [ ] Không còn khóa thật hoặc dữ liệu cá nhân không cần thiết trong tệp, ảnh và nội dung sẽ chia sẻ; sự cố đã phát hiện được báo và xử lý phù hợp.
- [ ] Ba khoản GIẢ đúng 35000 + 7000 + 18000 = 60000; rỗng bằng 0; `amount` nguyên, dương, không quá `1e9`; tối đa 500 khoản.
- [ ] Tên khoản chi hiển thị như văn bản bằng `textContent` (`cell.textContent` trong `app.js`); đã ghi trạng thái kiểm chuỗi `<b>Vở</b>`.
- [ ] Không đưa vào đoạn mã chưa rõ quyền hoặc phụ thuộc chưa kiểm chứng; giữ thông báo giấy phép cần thiết.
- [ ] Khai báo AI đúng việc đã làm; không nhận là tự viết phần được sinh nháp, không ghi đã test khi chưa test.

- [ ] Phần kiểm chứng bắt buộc theo yêu cầu bài đã có kết quả thật; lỗi trọng yếu chưa bị che bằng điểm rubric.

B. Hỏi giảng viên hoặc người phụ trách khi chưa rõ

- [ ] Đã làm rõ phạm vi AI, hỗ trợ nhóm và hình thức khai báo; phần chưa rõ được ghi trạng thái chờ.
- [ ] Đã hỏi về nguồn/giấy phép hoặc dữ liệu thật nếu bài bắt buộc dùng mà nhóm chưa xác định được điều kiện.
- [ ] Việc ngoài quyền của nhóm — xử lý tài khoản tổ chức, lịch sử kho chung — được chuyển đúng đầu mối.

C. Cải thiện sau, không làm sai mô tả hiện tại

- [ ] Ghi riêng việc chỉnh giao diện và mở rộng tính năng, không lẫn với yêu cầu an toàn cốt lõi.
- [ ] Nêu rõ Web không backend; SQL riêng; không quảng bá `localStorage` là nơi bảo vệ tài chính thực tế.
- [ ] Giữ bằng chứng tối thiểu đủ đối chiếu; tránh tạo thêm ảnh, log hoặc bản sao chứa dữ liệu thừa.

4.10. Ranh giới với sản phẩm thật

Kiểm tra trong trình duyệt giúp bài tập nhất quán; nó **không phải** hàng rào đủ cho một hệ thống tài chính. `localStorage` không cung cấp đăng nhập, phân quyền hay bảo vệ dữ liệu tài chính trước mọi người dùng cùng môi trường. ViSinhVien là sản phẩm học tập với dữ liệu giả, **không được quảng bá là ứng dụng tài chính an toàn**.

Nếu làm dịch vụ thật có máy chủ, cần thiết kế xác thực, phân quyền, kiểm tra dữ liệu phía máy chủ và biện pháp vận hành phù hợp. Máy chủ không được tin điều kiện chỉ đặt ở trình duyệt. Đây là ranh giới kiến thức để nhận ra việc mới cần làm, **không phải** yêu cầu nhồi thêm backend vào bài hiện tại.

4.11. Tự giảng lại trong 6 phút

Phần này kế thừa speaker notes của nguồn; tham gia tự nguyện.

- **Phút 1.** Kể cảnh Minh định nộp vì tổng đã đúng. Hỏi: “60000 chứng minh được gì, chưa chứng minh được gì?” Chốt: không backend, dữ liệu giả.
- **Phút 2.** Đọc tin nhắn hỏi giảng viên. Nhấn mạnh: phạm vi tùy môn; khai báo không thay giấy phép và không thay sự cho phép.

- **Phút 3.** Dùng hai chuỗi vô hại `Vở O'Reilly` và `<b>Vở</b>` để giải thích tách dữ liệu khỏi cú pháp. Không biểu diễn tấn công, không ngụ ý SQL thuộc Web.
- **Phút 4.** Phân loại một ảnh chứa email và một đoạn mã không giấy phép. Hỏi cách giảm dữ liệu, giảm quyền, giảm phụ thuộc.
- **Phút 5.** Đi qua thứ tự xử lý khóa: dừng chia sẻ → thu hồi/xoay vòng → kiểm tra mức dùng → rà bản liên quan → báo → giảm bản lưu. Nhấn mạnh: xóa tệp không vô hiệu khóa.
- **Phút 6.** Chọn hai dòng "chặn nộp" trong bảng, yêu cầu bằng chứng cụ thể. Kết thúc bằng: "Em biết gì, chưa xác minh gì, bước tiếp theo là gì?"

4.12. Tóm tắt và điểm kết năng lực

Tóm tắt: Làm chủ ViSinhVien là hiểu đặc tả, biết quyền dùng nguồn và công cụ, kiểm soát dữ liệu đưa đi, kiểm chứng đường đi của đầu vào và khai báo trung thực. `textContent`, truy vấn tham số hóa và kiểm tra giới hạn là biện pháp đúng **trong ngữ cảnh cụ thể**, không phải bùa bảo đảm mọi hệ thống an toàn. Khi có sự cố, ưu tiên khống chế quyền đã lộ và báo đúng người, không chỉ xóa dấu vết nhìn thấy.

Năng lực mở khóa: nhận ra một sự cố dữ liệu/quyền/nguồn trong vòng ba phút; chọn hành động ưu tiên đúng; viết được khai báo hỗ trợ AI khớp với việc đã làm; giải thích được vì sao `textContent`, truy vấn tham số hóa và `i < n` đúng trong ngữ cảnh của chúng.

Chuyển sang Final Boss: bạn đã có đủ bốn chặng để làm Build it. Test it. Explain it. — nơi hai lỗi chủ đích, diff, test, khai báo và bảo vệ miệng gặp nhau trong một bài nộp.

Nguồn đối chiếu của chương

Mốc đối chiếu nội dung nguồn: **29/10/2026**, kế thừa từ bản gốc. Các nguồn pháp luật dùng để định hướng và xác định văn bản áp dụng; tình huống thật cần người có thẩm quyền đánh giá. Nội dung các văn bản **chưa được kiểm tra lại** trong lượt biên tập này — hãy mở bản gốc trước khi trích dẫn.

1. Luật Bảo vệ dữ liệu cá nhân số 91/2025/QH15: khái niệm, khử nhận dạng, nguyên tắc, sự đồng ý, ngoại lệ và xử lý dữ liệu trong hoạt động AI.
2. Nghị định 356/2025/NĐ-CP: văn bản triển khai tại mốc đối chiếu; Điều 42 về hiệu lực và việc Nghị định 13/2023/NĐ-CP hết hiệu lực.
3. Văn bản hợp nhất 67/VBHN-VPQH ngày 23/03/2026 về Luật Sở hữu trí tuệ: bảo hộ chương trình máy tính và giới hạn của ngoại lệ liên quan.

4. OWASP — SQL Injection Prevention Cheat Sheet: truy vấn tham số hóa và danh sách cho phép đối với phần cấu trúc.
5. OWASP — Cross Site Scripting Prevention Cheat Sheet: nơi nhận dữ liệu an toàn như `textContent` và bảo vệ theo ngữ cảnh.
6. OWASP — Software Supply Chain Security Cheat Sheet: đánh giá phụ thuộc và rủi ro chuỗi cung ứng phần mềm.
7. GitHub Docs — Removing sensitive data from a repository: thu hồi/xoay vòng bí mật trước, giới hạn của việc xóa dữ liệu khỏi lịch sử và các bản sao.
8. GitHub Docs — Licensing a repository: ý nghĩa của giấy phép và giới hạn quyền khi kho chỉ được công khai.

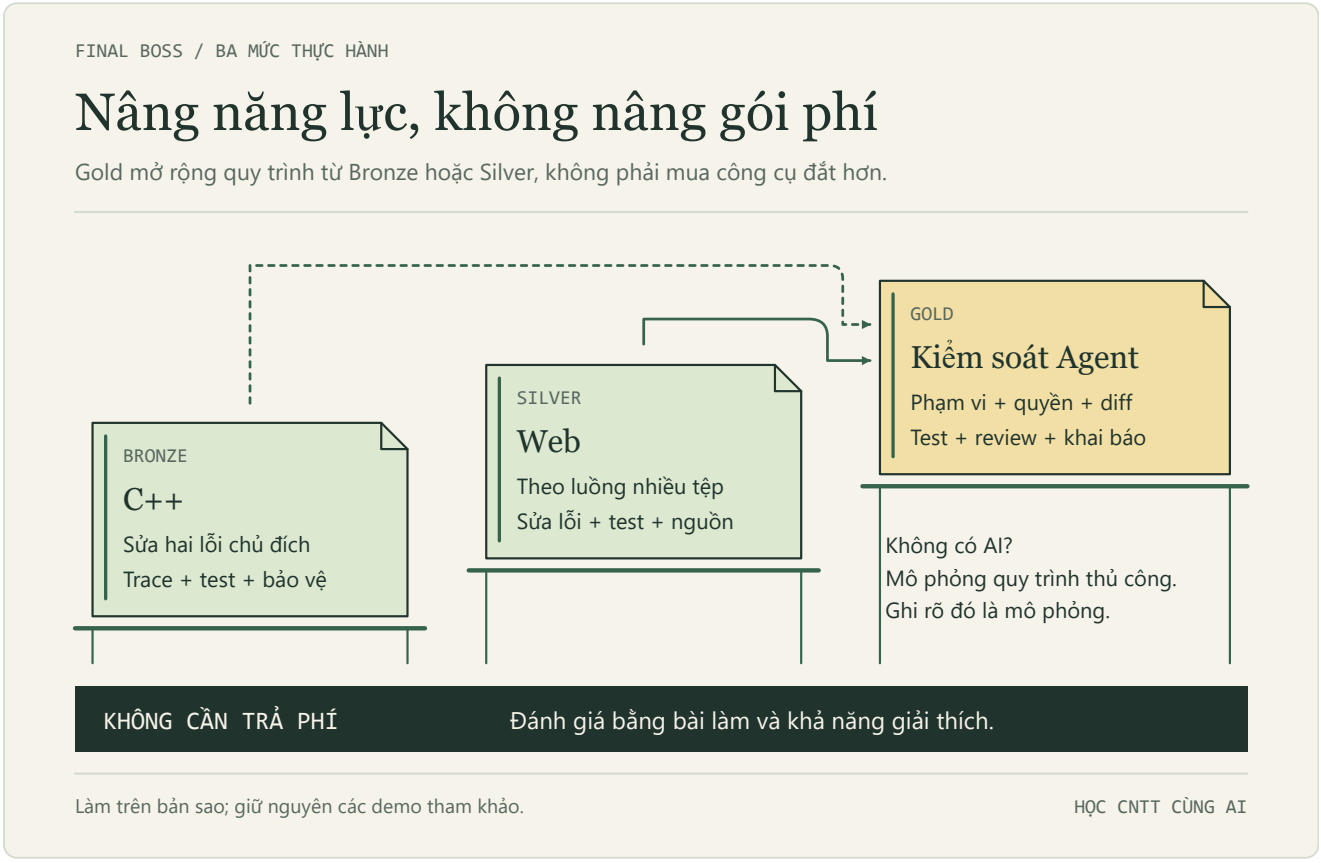
Nguồn kế thừa: mục 4.1–4.7 của bản gốc, đối chiếu bản bảo toàn, Source Audit và Blueprint. Bảy hồ sơ và phòng khám được tái cấu trúc; **không thêm sự kiện, sản phẩm hay mức giá mới**. Các nhận định pháp lý giữ nguyên mức thận trọng của bản gốc và được đánh dấu chưa kiểm tra lại.

BỘ CÔNG CỤ TỰ HỌC

Build it. Test it. Explain it.

Bổ sung biên tập. Đây là dự án tổng kết để ghép năng lực của bốn chương, không phải một ứng dụng mới vượt quá sức sinh viên. “Boss cuối” không phải AI mạnh nhất; đó là ba câu hỏi: **Em làm gì? Em biết nó đúng bằng cách nào? Nếu không mở chat, em còn giải thích được gì?**

Minh và Lan có một cuộc hẹn: mỗi người mang một bản ViSinhVien đã kiểm chứng đến buổi chia sẻ. Không ai được mở đầu bằng “AI bảo là đúng”. Trên bàn phải có code, bảng test và lời giải thích. Nếu phát hiện thiếu bằng chứng, nhóm ghi thiếu và bổ sung, không dùng giao diện đẹp để lấp chỗ trống.



Ba lộ trình tổng kết: Bronze hiểu C++, Silver nối luồng Web, Gold kiểm soát quy trình Agent. Xem hình lớn

1. Chọn cấp độ theo năng lực cần luyện

Lộ trình	Ai nên chọn?	Sản phẩm phải có	Không yêu cầu
BRONZE · C++ cơ bản	Năm nhất, đang học biến/vòng lặp/vector	Hàm cộng có kiểm tra đầu vào, hai bản vá, test và giải thích chỉ số/kiểu	Web, class Expense, tài khoản AI trả phí
SILVER · Web có luồng dữ liệu	Năm hai, đã biết JS/HTML	Sửa hai lỗi logic biệt lập; kiểm lại form → data → DOM và lưu trữ trong demo đúng	Backend, đăng nhập, đồng bộ, ví điện tử
GOLD · Quy trình Agent có kiểm soát	Đã hoàn thành Bronze hoặc Silver và muốn luyện giao việc	Toàn bộ sản phẩm cấp nền + phạm vi quyền, kế hoạch, diff, log, nghiệm thu	Thêm framework, thuê máy chủ, mua gói đắt hơn

Gold không có nghĩa "code nhiều hơn". Nó có nghĩa bạn chứng minh được Agent không được tự mở rộng nhiệm vụ và bạn biết từ chối thay đổi không có căn cứ. Nếu không có quyền dùng Agent, làm bài diễn tập quy trình trên giấy và ghi **mô phỏng, chưa thực thi Agent**; không nhận đã đạt phần bằng chứng chạy Agent thật.

Thời lượng gợi ý: Bronze 60–90 phút; Silver 90–120 phút; Gold thêm 30–45 phút cho quy trình. Có thể chia nhiều phiên. Đây không phải thước đo tốc độ hay năng lực lâu dài.

2. Mười yêu cầu của bản nộp

1. Viết yêu cầu chức năng bằng lời của mình: tổng phải làm gì, đầu vào nào bị từ chối, việc gì không thuộc phạm vi.
2. Chọn công cụ phù hợp và nói lý do. Nếu không dùng AI, ghi lựa chọn đó; nếu dùng, xác nhận phạm vi môn cho phép.
3. Mô tả cấu trúc dữ liệu: Bronze `vector<long long>`; Silver `Expense(id,title,amount,category)` và đường đi của `amount`.
4. Thực hiện phần triển khai trong phạm vi đã chọn. Ghi phần AI gợi ý, phần tự viết; không sửa demo tham khảo để hợp thức hóa đáp án.
5. Sửa tối thiểu hai lỗi gốc trong bộ đề chủ đích dưới đây, mỗi lỗi có hiện tượng, nguyên nhân và phản ví dụ riêng.
6. Đọc diff: từng dòng đổi giải quyết yêu cầu nào? Có tệp, phụ thuộc hoặc quyền nào tăng lên không?
7. Chạy kiểm thử có kỳ vọng độc lập; giữ lần thất bại và lần chạy lại, không chỉ chụp màn hình xanh cuối cùng.

8. Rà dữ liệu cá nhân, key, thư viện, giấy phép và ngữ cảnh hiển thị. Không dùng dữ liệu thật để bài "trông thực tế".
9. Khai báo đúng mức hỗ trợ AI theo mẫu thỏa thuận học tập và hướng dẫn môn.
10. Giải thích sản phẩm, trả lời một ca biến thể không AI và nêu giới hạn chưa kiểm chứng.

Hợp đồng chung: dữ liệu giả 35000 + 7000 + 18000 = 60000; tiền nguyên VND từ 1 đến 1000000000; tối đa 500 khoản; rỗng trả 0; tổng cực đại 500000000000. Số tiền nguyên trong C++ không phải parser hoàn chỉnh của mọi chuỗi gõ bàn phím. Bản SQLite riêng không là backend và không áp đầy đủ hợp đồng danh sách này.

3. Chuẩn bị bài làm, không ghi đè ví dụ đúng

Mở thư mục gốc **AI-Student-Handbook** trong trình soạn thảo. Dùng trình quản lý tệp tạo `bai-lam/final-boss`, rồi sao chép các tệp cần thiết từ `vi-du/final-boss` vào đó. Đừng chạy lệnh sao chép mà chưa hiểu đích; ở bài này thao tác bằng giao diện là đủ.

Tệp nguồn đề	Vai trò	Trạng thái ban đầu
bronze.cpp	C++ có hai lỗi gốc và 9 ca test	Có lỗi chủ đích, test chưa thể xanh hết
silver.cjs	Hai hàm JS có hai lỗi gốc	Có lỗi chủ đích; không gắn vào demo Web
test_silver.cjs	12 ca kiểm riêng cho đề Silver	Không đổi đáp án test để làm code qua

Bộ đề này **khác** demo tham khảo `vi-du/nam-1-cpp/vi_sinh_vien.cpp` (9 ca) và `vi-du/nam-2-web/test_data.cjs` (34 ca ở bản mới, tăng từ 33 ca gốc do thêm regression test cho kiểu `category`). Không dùng ba con số lẫn nhau. Tệp `loi-bo-sot.cpp` ở chương 2 chỉ có một lỗi cận và không có `--test`; đó không phải toàn bộ đề Final Boss.

Tạo một tệp ghi chú `ghi-chep.txt` trong bài làm. Nội dung tối thiểu: điểm xuất phát, phạm vi, hai lỗi đang xét, test kỳ vọng, lệnh đã chạy, kết quả thực, phần AI hỗ trợ, phần chưa biết. Không lưu key hoặc dữ liệu riêng tư trong ghi chú.

4. BRONZE · Một vòng lặp, hai quyết định sai

Bối cảnh: Người viết đã biết phải kiểm tra đầu vào, nhưng chương trình vẫn cộng thiếu tiền và từ chối một khoản hợp lệ ở sát giới hạn. Hai lỗi nằm trong hai quyết định khác nhau; sửa được tổng mẫu không có nghĩa lỗi thứ hai biến mất.

THINK FIRST — dự đoán trước lần chạy đầu

Đọc `bronze.cpp` trong bản sao, chưa mở đáp án. Ghi kỳ vọng cho ba khoản, rỗng, một khoản 7000, một khoản 1000000000, 500 khoản tối đa, 0, -1, 1000000001 và 501 khoản. Với ca bị từ chối, kỳ vọng là từ chối chứ không phải một tổng.

Trace riêng vòng cộng. Sau đó đọc vòng kiểm tra miền; không ghép cả hai vòng thành một ý mơ hồ "code xử lý số tiền". Thử trả lời: điều kiện nào cho phép biên lớn nhất và điều kiện nào quyết định lượt cuối có được cộng?

Cách đọc bộ chạy test: `check(tên, đầu_vào, tổng_kỳ_vọng, từ_chối)` là một hàm nhỏ để đổi chiếu. Khi đổi số cuối là `true`, test chờ hàm báo `std::invalid_argument`; số `0` ở vị trí trước đó không phải tổng cần trả. Ví dụ `check("tu choi 0", {0}, 0, true)` yêu cầu **từ chối khoản 0**, không chấp nhận nó rồi cộng ra 0. Nếu bỏ đổi số cuối, mặc định là `false`: test chờ một tổng hợp lệ. Lambda là cách khai báo hàm hỗ trợ ngay trong `main`; bạn chưa cần tự viết lại bộ chạy này, chỉ sửa `totalExpenses` và thêm một lời gọi `check` theo mẫu khi đã hiểu.

Build và chạy đúng bản sao

Các lệnh sau chạy từ **gốc AI-Student-Handbook**, không đứng trong thư mục `bai-lam/final-boss`. Cần `g++` C++17. Nếu không có compiler, trace trên giấy và ghi chưa chạy; không cài phần mềm lên máy phòng lab khi chưa được phép.

Bash/Linux hoặc Git Bash:

```Bash
g++ -std=c++17 -Wall -Wextra "bai-lam/final-boss/bronze.cpp" -o "bai-lam/final-boss/bronze.exe"
```

Xem exit code ngay bằng `echo $?`. **Chỉ khi build thành công**, chạy:

```Bash
"./bai-lam/final-boss/bronze.exe"
echo $?
```

PowerShell:

```PowerShell
g++ -std=c++17 -Wall -Wextra "bai-lam/final-boss/bronze.cpp" -o "bai-lam/final-boss/bronze.exe"
$LASTEXITCODE
```

Chỉ khi mã build là 0:

```PowerShell
& ".\bai-lam\final-boss\bronze.exe"
$LASTEXITCODE
```

Test ban đầu có FAIL là dấu hiệu để đang phát huy tác dụng. Không đổi dữ liệu kỳ vọng, không xóa test, không coi exit khác 0 của để lỗi là lỗi máy. Sau mỗi bản vá, lưu tệp, build lại thành công rồi mới chạy lại. Dùng executable cũ sau build lỗi làm mất giá trị của toàn bộ bằng chứng.

Kế hoạch sửa hai bước

Bước một: chọn một lỗi, dự đoán hành vi nào sẽ đổi sau bản vá; một test có thể vẫn FAIL vì lỗi còn lại che kết quả. Chỉ đổi một quyết định, chạy lại và xem cả nguyên nhân thất bại, không chỉ đếm PASS. Bước hai: đối chiếu các test còn lỗi để tìm quyết định thứ hai. Cuối cùng đọc diff và giải thích vì sao không cần thay kiểu dữ liệu, cài thư viện hay viết lại toàn chương trình.

Đích sau sửa: cả 9 ca đạt; chưa coi đó là chứng minh mọi đầu vào tưởng tượng đều được xử lý. Hãy thêm một test do bạn tự nghĩ, ví dụ một khoản 1 VND hoặc đảo thứ tự ba khoản. Ghi test bổ sung riêng để không nhầm số ca gốc.

▼ Đáp án Bronze — mở sau khi tự thử · mở sau khi tự thử

Hai lỗi gốc là cận `i < n - 1` bỏ phần tử cuối và điều kiện `amount >= 1000000000LL` từ chối cả biên hợp lệ. Bản vá tương ứng là `i < n` và từ chối khi `amount > 1000000000LL`. Vẫn giữ `amount <= 0` và kiểm tra số lượng.

Nếu chỉ sửa vòng cộng, ca tiền tối đa vẫn bị từ chối: kỳ vọng 7 PASS, 2 FAIL. Nếu chỉ sửa miền tiền, ca một khoản vẫn cộng 0: bộ đếm vẫn 5 PASS, 4 FAIL. Hai ca biên đã chuyển từ "bị từ chối nhầm" sang "cộng sai", nhưng số FAIL không cho thấy sự thay đổi đó. Để phân biệt, ghi lời gọi bị ném ngoại lệ hay trả tổng nào; đừng kết luận bản vá vô dụng chỉ vì bộ đếm đứng yên. Đây là lý do không dùng một lần thấy 60000 để khẳng định toàn bộ chức năng đúng.

Với bản đề` chưa sửa, từ mã có thể dự đoán 5 PASS, 4 FAIL; sau hai bản vá là 9 PASS, 0 FAIL. Đây là **kỳ vọng để đó´i chiế´u**; kế´t quả thực thi của phiên xuấ´t bản nằ`m trong báo cáo kiểm tra, còn bạn phải ghi kế´t quả máy mình. Bài không yêu cầ`u xây parser nhập bàn phím hay kiểm tra ký tự `35000abc` trong C++.

---

5. SILVER · Dữ liệu phải đi đúng từ đầ`u đế´n cuố´i

Bố´i cảnh: Một hàm xử lý tiề`n dùng parser quá dễ dãi; hàm cộng lại đọc sai tên trường. Khi thử trên màn hình, hai lỗi có thể che nhau. Vì thế´ Lan bắ´t đầ`u với test logic biệt lập, rồ`i mới quay lại kiểm giao diện.

Bài logic có hai lỗi chủ đích

Sao chép cả `silver.cjs` và `test_silver.cjs` vào cùng `bai-lam/final-boss`. Đọc test trước; lập bảng riêng cho `parseAmount` và `totalExpenses`. Việc ô nhập chỉ cho bàn phím số´ không bảo vệ dữ liệu lấ´y từ kho lưu hoặc gọi hàm trực tiế´p.

Chạy từ gố´c dự án, với Node.js có sẵn:

```Bash
node "bai-lam/final-boss/test_silver.cjs"
```

Ngay sau đó, Bash dùng `echo $?`, PowerShell dùng `$LASTEXITCODE`. Kỳ vọng ban đầ`u có lỗi, không chạy `npm install` vì đề` dùng thư viện chuẩn Node. Trong bảng, giữ nguyên kiểu: `"35000"` là chuỗi, `35000` là số´.

Nhóm test	Cầ`n chứng minh	Ví dụ phản chứng
Parse đúng toàn chuỗi	Không nhận một phầ`n rồ`i bỏ hậu tố´	`35000abc`, `1.5`, `1e3` đề`u phải từ chố´i
Parse đúng miề`n	Sau chuyển số´ vẫn kiểm dương, nguyên, giới hạn	Rỗng, 0, quá 1e9
Cộng đúng trường	Dữ liệu chứa `amount`, không tự phát sinh `value`	Ba khoản phải 60000, một khoản 7000
Danh sách đúng hợp đồ`ng	Rỗng 0, tố´i đa 500, không chấ´p nhận amount chuỗi	501 khoản, `{amount:'35000'}`

Sửa từng lỗi và dự đoán nhóm test bị tác động. Không sửa file test để bỏ qua lỗi. Sau khi 12 ca có sẵn đạt, tự thêm ca `"1000000001"` và ca `" 7000 "`; ghi sô´ test mới bạn thực sự chạy, không nói nguyên bộ vẫn là 12 ca.

Quay vê` luô`ng Web tham khảo

Bộ Silver là bài Node riêng, **không tự động trở thành ứng dụng Web**. Nê´u chưa làm chương 2, dùng trình quản lý tệp sao chép **cả thư mục** `vi-du/nam-2-web` sang `bai-lam/nam-2-web`; giữ `index.html`, `data.js`, `app.js` cùng thư mục. Sau khi giải được hai hàm, mở `bai-lam/nam-2-web/index.html` để theo luô`ng form → handler → `parseAmount` → `normalizeExpenses` → lưu → `render`. Demo Web tham khảo đã có logic đúng; không chép `module.exports` của bài Node vào trình duyệt.

Từ ba khoản mẫu 60000, thêm Bánh mì 15000 để mong đợi 75000. Nê´u dữ liệu cũ khác, khôi phục mẫu chỉ sau khi xác nhận đó là dữ liệu giả có thể thay, hoặc tính kỳ vọng từ tổng thực tê´ đang có. Thử chuỗi `35000abc`: tổng không được tăng. Thêm tên `<b>Vở</b>`: tên hiển thị nguyên văn, không thành HTML.

Tải lại trang và đọc thông báo lưu. Khi đang fallback RAM, không đòi dữ liệu tô`n tại sau reload; nút khôi phục mẫu không tự bật lại persistence trong phiên. Ghi trình duyệt, địa chỉ đang mở và thông báo thâ´y được. `file://` có hành vi localStorage phụ thuộc trình duyệt; không kê´t luận toàn bộ ứng dụng hỏng chỉ vì mở một địa chỉ khác.

Đích Silver: giải thích được hai lỗi logic, có test riêng và biên bản thử Web; chỉ ra khác biệt giữa hàm tính tiê`n, schema bô´n trường và DOM. Không cần viê´t backend để "nâng cấ´p" bài.

▼ Đáp án Silver — mở sau khi tự thử · mở sau khi tự thử

Lỗi thứ nhâ´t: `parseInt` có thể đọc đâ`u chuỗi và bỏ phâ`n còn lại. Thêm kiểm tra toàn chuỗi chữ sô´ sau `trim`, rô`i chuyển bă`ng `Number`; vẫn giữ kiểm tra sô´ nguyên an toàn và miê`n tiê`n. Thêm radix 10 vào `parseInt` không sửa được lỗi nhận hậu tô´.

Lỗi thứ hai: reducer cộng `item.value` trong khi dữ liệu chỉ có `amount`; cần cộng `item.amount`. `undefined` khi đọc trường thiê´u là một giá trị khác với `NaN` sinh ra trong phép cộng sô´. Không chữa bă`ng cách đặt `undefined` thành 0 vì nó che hợp đô`ng trường bị sai.

Kỳ vọng bộ 12 ca: bản đê` 6 PASS, 6 FAIL; bản sửa 12 PASS, 0 FAIL. Trình duyệt vẫn phải kiểm riêng. Cụm "test logic xanh" không đủ để kê´t luận nút thêm, nút xóa, `textContent` và lưu trữ đê`u đúng.

6. GOLD · Một Agent, một phạm vi, một lâ`n nghiệm thu

Chỉ chọn sau khi hiểu yêu câ`u Bronze/Silver. Bạn làm người điê`u phô´i, không đưa quyê`n toàn bộ máy để đổi lâ´y tô´c độ.

1. **Khóa phạm vi:** nêu đúng bản sao và tên file; câ´m cài gói, sửa config/quyê`n, xóa file, commit/push/publish hoặc gọi dịch vụ. Kiểm cơ chê´ quyê`n thật của công cụ, không chỉ viê´t lệnh câ´m trong prompt.
2. **Yêu câ`u kê´ hoạch trước:** Agent chỉ đọc và đê` xuâ´t test. Nê´u đê` xuâ´t chuyển cả bài sang framework mới, từ chô´i và giải thích vì sao ngoài nhiệm vụ.
3. **Duyệt một thay đổi:** dùng P10 trong Prompt Library đã điê`n đủ. Không câ´p phép mơ hô` "làm mọi thứ câ`n thiê´t".
4. **Theo dõi bă`ng chứng:** file nào được đọc/sửa, lệnh nào thực thi, exit code, cả lâ`n thâ´t bại. Báo cáo ý định không thay kê´t quả quan sát.
5. **Đọc diff trước nhận:** so với checkpoint của bản sao; tệp ngoài phạm vi là lý do dừng. Không khôi phục hàng loạt làm mâ´t thay đổi hợp lệ của người khác.
6. **Nghiệm thu độc lập:** bạn tự chạy lại test, tự giải thích hai lỗi và một ca biê´n thể. Nê´u thiê´u công cụ, ghi việc chưa kiểm thay vì mượn kê´t quả chạy của nơi khác.

Mô´c	Agent được làm	Người học phải quyê´t định	Bă`ng chứng
Trước sửa	Đọc trong phạm vi, đê` xuâ´t	Kê´ hoạch và quyê`n có phù hợp?	Kê´ hoạch đã xem
Khi sửa	Đúng file đã duyệt	Có câ`n dừng vì ngoài phạm vi?	Diff và nhật ký
Khi test	Lệnh được cho phép	Kê´t quả có đúng bản mã?	Command, output, exit
Khi nhận	Báo phâ`n xong/chưa xong	Có đủ căn cứ nghiệm thu?	Test tự chạy, vâ´n đáp

Không có Agent? Đóng vai "người giao việc" và nhờ bạn học phản biện kê´ hoạch. Bă`ng chứng đó có giá trị luyện tập nhưng phải nhãn mô phỏng; phâ`n vận hành công cụ thật để trạng thái chưa thực hiện.

7. Bảng nghiệm thu và rubric 100 điểm

Rubric này phục vụ luyện tập, không là thang điểm chính thức của TLU. Nê´u môn có rubric riêng, dùng rubric môn. Điểm cao không bù việc lộ key, dữ liệu cá nhân, không được phép dùng AI hoặc khai báo sai.

Tiêu chí	Điểm tô´i đa	Đạt tô´i đa khi	Thiê´u bă`ng chứng thì
Yêu cầ`u chức năng và phạm vi	10	Nêu hợp đồ`ng, giới hạn và việc không làm	Mơ hồ`: tô´i đa 4
Chọn công cụ có lý do	5	Gă´n lựa chọn với tác vụ, chi phí/quyề`n	Chỉ nói “mạnh nhâ´t”: tô´i đa 1
Câ´u trúc dữ liệu	10	Nói đúng kiểu, trường, ranh giới kiểm tra	Nhầ`m chuỗi/sô´ hoặc C++/Web: tô´i đa 4
Triển khai và phân chia đóng góp	10	Code của bản sao, phầ`n AI/tự làm rõ	Chép không giải thích: tô´i đa 3
Hai lỗi và nguyên nhân	15	Hai bản vá với phản ví dụ độc lập	Sửa được một lỗi: tô´i đa 7
Kiểm tra thay đổi	10	Diff có mục đích, không thay ngoài phạm vi	Không có trước/sau: tô´i đa 3
Kiểm thử có kỳ vọng	15	Giữ lầ`n sai, build/test bản mới, biên/rỗng/sai	Chỉ một ca mẫu: tô´i đa 4
Riêng tư, bảo mật, giấ´y phép	10	Dữ liệu giả, quyề`n tô´i thiểu, nguồ`n rõ	Có blocker: chưa nghiệm thu
Khai báo AI trung thực	5	Công cụ/việc hỗ trợ/phầ`n tự làm/chưa kiểm	Khai sai: chưa nghiệm thu
Giải thích và biế´n thể không AI	10	Tự trả lời bă`ng trace/test, biế´t giới hạn	Chỉ đọc lại lời AI: tô´i đa 3
Tổng	**100**	Không có điề`u kiện chặn nộp	Điểm sô´ không thay bă`ng chứng

Đô´i với người làm hoàn toàn không AI, chấ´m sự phù hợp của lựa chọn và khai báo thật, không trừ điểm vì không mua dịch vụ. Với Gold, mức điểm tương ứng phải dựa bă`ng chứng vận hành Agent thật hoặc ghi chưa đạt phầ`n đó khi chỉ mô phỏng.

8. Buổi bảo vệ ngắ´n

Phút 1: mô tả đầ`u vào–đầ`u ra và một việc ứng dụng không làm. **Phút 2–3:** giải thích lỗi đầ`u bă`ng phản ví dụ nhỏ nhấ´t. **Phút 4–5:** giải thích lỗi thứ hai và test phát hiện nó. **Phút 6:** mở diff, chỉ dòng ngoài thuật toán như validation. **Phút 7:** nói một phầ`n chưa kiểm chứng. **Phút 8:** trả lời câu biế´n thể do bạn học/giảng viên chọn.

Câu biế́n thể có thể là: đảo thứ tự có làm tổng đổi không; 500 khoản tối đa cầ̀n kiểu tổng nào; tại sao lọc theo nhóm vẫn phải kiểm toàn danh sách; nế́u tắ́t lưu trữ thì chức năng nào còn hoạt động; vì sao thêm 15000 không phải lúc nào tổng cũng thành 75000. Người hỏi không cầ̀n làm khó bằ̀ng tính năng chưa học.

SKILL UNLOCKED — dấ́u hiệu kế́t thúc

Bạn có một sản phẩm nhỏ cùng một chuỗi bằ̀ng chứng có thể lặp lại. Bạn không phải nhớ mọi cú pháp, nhưng phải biế́t truy câu hỏi về̀ đúng tệp, đúng dữ liệu, đúng phép thử. Nế́u ngày mai công cụ AI không dùng được, bảng trace, test và khả năng giải thích vẫn ở lại với bạn.

BỘ CÔNG CỤ TỰ HỌC

Thư viện prompt

Tám thẻ P01–P08 kế thừa chức năng học tập của mục 3.4 bản nguồn; P09–P11 là bổ sung biên tập cho việc chuẩn bị ngữ cảnh, giao Agent và kiểm chứng nguồn. Chọn **một thẻ cho đúng điểm nghẽn**, không gửi cả thư viện. Bạn có thể dùng câu hỏi với bạn học hoặc giảng viên nếu không có quyền truy cập AI.

Prompt là lời yêu cầu, không phải cơ chế bảo vệ quyền hệ thống. Chỉ dùng khi phạm vi môn học cho phép, chỉ gửi dữ liệu giả. Trước khi bấm gửi, đọc lại tất cả nội dung và thay phần có ngoặc vuông bằng dữ liệu của bài tập được phép chia sẻ; đừng để AI tự đoán tệp hoặc kết quả chạy.

P01 · Socratic: bắt đầu từ câu hỏi

Dùng khi: Minh biết chương trình ra 42000 nhưng chưa chỉ ra lượt bị bỏ. **Bạn chuẩn bị:** viết chỉ số hợp lệ và tổng tính tay lên giấy.

Prompt / Ghi chú

```
Tôi là Minh, năm nhất học C++. Bản C++ ViSinhVien chỉ dùng
vector<long long> amounts, không cần class Expense bốn trường.
Dữ liệu hoàn toàn giả: Cơm trưa 35000, Xe buýt 7000, Vở ghi 18000.
Tổng đúng 60000; vòng lặp i < n - 1 đang cho 42000, n là int.
Hợp đồng: tiền nguyên VND từ 1 đến 1000000000; tối đa 500 khoản;
danh sách rỗng hợp lệ và tổng bằng 0.
Hãy hỏi kiểu Socratic, một câu mỗi lượt, chờ tôi trả lời.
Bắt đầu từ các chỉ số hợp lệ của ba phần tử. Không viết vòng lặp
đúng và chưa nêu điều kiện cần thay. Chỉ nhận xét lập luận tôi gửi.
Bằng chứng cuối buổi: tôi tự trace, nêu lượt bị bỏ và lý do sửa.
Nếu lỡ lộ lời giải, dừng và giao ca một khoản 7000 để tôi tự trace;
không giải hộ ca mới. Không nhận đã chạy code nếu chưa thực thi.
```

THINK FIRST: Sau câu hỏi đầu, bạn phải viết được điều gì mà trước đó chưa viết được? Nếu AI hỏi năm câu cùng lúc, chọn một câu để trả lời và yêu cầu chờ. Nếu đáp án lộ, việc đọc nó là ôn tập, không còn là bằng chứng tự giải.

P02 · Ba tầng gợi ý

Dùng khi: chưa biết nhìn vào đâu nhưng vẫn muốn tự sửa. **Bạn chuẩn bị:** một dòng mô tả điều đã thử, chẳng hạn “em đã in tổng sau từng lượt”.

Prompt / Ghi chú

```
Tôi tự viết vòng lặp C++ cộng vector<long long> amounts trong
ViSinhVien. Ba khoản giả theo thứ tự: 35000, 7000, 18000 VND.
Mong đợi 60000, đang nhận 42000. Mỗi khoản nguyên từ 1 đến 1e9,
tối đa 500 khoản, rỗng tổng 0. Vòng lặp hiện dùng i < n - 1;
n và i là int, tổng là long long.
Chuẩn bị ba mức gợi ý: mức 1 chỉ vùng cần xem; mức 2 hỏi quan hệ
chỉ số cuối và số phần tử; mức 3 cho bảng trace còn ô trống.
Bây giờ CHỈ đưa mức 1. Chỉ nâng mức khi tôi yêu cầu sau khi báo
điều đã thử. Không viết code sửa hoặc điều kiện đúng.
Kết quả cần có là bảng tôi tự điền và một thay đổi tôi giải thích.
Nếu lộ lời giải, báo rõ rồi giao ca rỗng và danh sách một khoản 12000
để tôi tự so hai cận, không kèm kết quả. Ca rỗng riêng lẻ không đủ
phân biệt hai cận. Không chỉnh file hay chạy lệnh ở nhiệm vụ này.
```

Lỗi thường gặp: xin cả ba tầng một lúc rồi đọc hết. Nhịp hữu ích là một gợi ý → một thao tác → một nhận xét, không phải một prompt dài → một đáp án hoàn chỉnh.

P03 · Trace cả giá trị lẫn kiểu

Dùng khi: Lan thấy một chuỗi số dài và muốn biết nó hình thành ở đâu. Đây là lỗi mô phỏng riêng, không mô tả demo Web đúng đi kèm.

Prompt / Ghi chú

```
Tôi là Lan, năm hai học Web. ViSinhVien dùng HTML/JS/localStorage,
không backend. Bộ mẫu giả chuẩn có tiền số: 35000, 7000, 18000;
tổng 60000. Bây giờ tôi mô phỏng lỗi bằng amount chuỗi "35000",
"7000", "18000", với total bắt đầu là số 0.
Mỗi lượt thực hiện total = total + expense.amount.
Hãy tạo bảng TRỐNG với cột: lượt, kiểu total trước, kiểu amount,
phép toán, giá trị total sau, kiểu total sau. Chờ tôi điền từng hàng.
Không điền hộ và không viết hàm chuyển kiểu. Hợp đồng đích là
amount số nguyên VND từ 1 đến 1e9; tối đa 500 khoản, rỗng trả 0.
Hãy nhận xét chỗ tôi nhầm kiểu; mục tiêu là tôi chỉ ra lượt đầu
đổi kiểu và ranh giới cần kiểm tra đầu vào.
Nếu lỡ điền bảng, dừng; để tôi tự trace một chuỗi "7000" từ total=0.
Không nói rằng chạy thử đã thành công nếu chỉ suy luận.
```

Giữ dấ´u ngoặc kép khi ghi chuỗi. `"35000"` và `35000` nhìn gâ`n giô´ng nhưng phép cộng có thể xử lý khác. Ghi mỗi kê´t quả cùng kiểu là cách chô´ng đoán bă`ng hình dáng.

P04 · Review đúng một thay đổi

Dùng khi: đã có bản vá nhỏ và muô´n kiểm tra lập luận, không muô´n AI viê´t lại cả chương trình.

Prompt / Ghi chú

```
Tôi là Minh. Bản C++ ViSinhVien dùng vector<long long> amounts;
int n là số khoản, int i là chỉ số, long long total là tổng.
Dữ liệu giả 35000, 7000, 18000; tổng đúng 60000, cận cũ cho 42000.
Thay đổi duy nhất của tôi: i < n - 1 thành i < n.
Hợp đồng dữ liệu: nguyên VND từ 1 đến 1e9, tối đa 500; rỗng tổng 0.
Chỉ review thay đổi cận này, hỏi tôi chứng minh phạm vi chỉ số và
chọn test. Không viết lại hàm, không thêm validation hay tính năng.
Phân biệt rõ: sửa cận KHÔNG tự chứng minh chương trình từ chối
đầu vào sai; phần validation là việc riêng chưa được review ở đây.
Chờ tôi giải thích ba khoản, một khoản 7000, rỗng và vì sao dùng
long long. Nếu đưa lời giải đầy đủ, dừng rồi giao n=1 để tôi tự so
hai điều kiện. Không thay file và không nhận đã chạy kiểm thử.
```

QUICK CHECK: Nê´u review nói “đã sửa cận nên an toàn mọi đâ`u vào”, bạn sẽ phản biện bă`ng ca nào? Gợi ý để tự nghĩ: sô´ lượng và miê`n tiê`n chưa xuâ´t hiện trong diff này.

P05 · Test trước code

Dùng khi: chuẩn bị viết hàm tính tổng, chưa muốn được cho thuật toán.

Prompt / Ghi chú

```
Tôi là Lan. Chưa viết hàm totalExpenses cho bản Web ViSinhVien.
Expense gồm id, title, amount, category. Hợp đồng phần tính tiền:
amount phải là số nguyên VND từ 1 đến 1000000000; tối đa 500
khoản; rỗng trả 0; amount dạng chuỗi phải bị từ chối ở hàm tính tổng.
Dữ liệu giả: Cơm trưa 35000, Xe buýt 7000, Vở ghi 18000; tổng 60000.
Hãy yêu cầu tôi đề xuất lần lượt ca thường, biên và dữ liệu sai.
Không viết hàm, không viết test tự động, không tính hộ kỳ vọng.
Sau mỗi ca, hỏi lý do hoặc chỉ ra LOẠI ca còn thiếu, chưa cho đáp án.
Bảng do tôi điền cần có đầu vào, kỳ vọng, lý do và trạng thái chưa chạy.
Nếu lộ code/đáp án, dừng và để tôi tự tính ca 500 khoản mỗi khoản 1e9.
Không gộp kiểm thử parseAmount với kiểm thử form/DOM/localStorage.
```

Một bộ test không có kỳ vọng độc lập dễ xác nhận nhầm chính lỗi của chương trình. Đầu ra AI sinh ra cũng cần đối chiếu bằng tính tay hoặc lập luận.

P06 · Đọc tiếng Anh có đối chiếu

Dùng khi: vướng từ kỹ thuật, không phải yêu cầu AI đoán nội dung một trang chưa đọc.

Prompt / Ghi chú

```
Tôi là Lan, làm Web ViSinhVien bằng HTML/JS/localStorage, không backend.
Câu sau do cẩm nang tự soạn để luyện đọc, KHÔNG là trích dẫn chính thức:
"The value of a text input is a string. Convert and validate it before
storing an expense amount."
Ví dụ giả: ô nhập Cơm trưa chứa "35000"; Expense.amount cần là số nguyên
VND từ 1 đến 1e9, tối đa 500 khoản trong danh sách, rỗng tổng 0.
Giải thích value, string, convert, validate trong ngữ cảnh rồi hỏi tôi
diễn giải câu trên bằng tiếng Việt và nêu một đầu vào sai.
Không viết hàm, không bịa nguồn, không nhận đã mở tài liệu chính thức.
Tôi cần chứng minh mình phân biệt chuyển kiểu và kiểm tra hợp lệ.
Nếu lộ code, dừng rồi hỏi tôi phân tích "35000abc" bằng lời,
không nêu sẵn kết luận.
```

Với trang thật, thêm URL và đoạn đã đọc được. Nếu AI không truy cập được trang, yêu cầu nói rõ; bản dịch tự tin không chứng minh nó đã đọc đúng nguồn.

P07 · Vâ´n đáp, không đoán điểm môn học

Prompt / Ghi chú

```
Tôi đã học hai lỗi ViSinhVien: cận vòng lặp C++ bỏ cuối cho 42000
thay vì 60000 và amount chuỗi trong JS làm phép + nối ký tự.
Dữ liệu giả: 35000, 7000, 18000 VND. C++ dùng vector<long long>;
Web dùng Expense(id,title,amount,category). Tiền nguyên từ 1 đến 1e9,
tối đa 500 khoản, rỗng tổng 0. Web dùng localStorage, không backend.
Vấn đáp một câu mỗi lượt: chỉ số, kiểu dữ liệu, test, rồi lưu trữ Web.
Chờ tôi trả lời; hỏi thêm nếu mơ hồ. Không trả lời thay, không phán
năng lực tổng quát và không đoán điểm hay quy định của học phần.
Cuối phiên ghi điều tôi đã chứng minh bằng ví dụ và chỗ cần luyện.
Nếu lộ đáp án, bỏ câu đó khỏi đánh giá độc lập, giao câu biến thể
mới không kèm gợi ý trong lượt đầu. Không nhận đã kiểm thử code.
```

AI VS HUMAN — phân vai, không thi hơn thua: AI đặt câu hỏi và phản hồ`i. Bạn phải nêu ví dụ, tự chạy nê´u cầ`n, giải thích tại sao bằ`ng chứng đủ hoặc chưa đủ. Một câu bạn đã đọc đáp án chỉ tính là ôn tập.

P08 · Đóng chat rồ`i làm biê´n thể

Prompt / Ghi chú

```
Tôi muốn tự kiểm tra mà chưa nhận lời giải. ViSinhVien có ba khoản
giả: Cơm trưa 35000, Xe buýt 7000, Vở ghi 18000. Tiền nguyên VND
1..1e9, tối đa 500 khoản, rỗng tổng 0. C++ dùng vector<long long>;
Web dùng Expense(id,title,amount,category). Hợp đồng Web: id là số
nguyên an toàn, dương, không trùng; title là chuỗi sau trim dài 1–80
đơn vị UTF-16; category là chuỗi thuộc an-uong, di-lai, hoc-tap, khac.
Giao hai nhiệm vụ ngắn: C++ tìm tiền lớn nhất, quy ước rỗng trả 0;
JS tính tổng theo category nhưng phải xác thực TOÀN BỘ danh sách
trước khi lọc, không che khoản sai nằm ở nhóm khác.
Không cho thuật toán, code, kết quả mẫu hoặc gợi ý ngay.
Tôi đóng chat, thử 4 phút rồi gửi bài và dự đoán. Nếu chưa xong,
chỉ giúp xác định điểm nghẽn; không coi thời gian là thước đo năng lực.
Đối chiếu lập luận, ca thường, rỗng và dữ liệu sai tôi trình bày.
Nếu lộ đáp án, đổi sang tính bất biến khi đảo thứ tự, chưa nói kết quả.
```

Bô´n phút là một lượt tự thử ngă´n, khác bài 25 phút không AI ở chương 3. Không cầ`n hoàn tấ´t mọi thứ trong một lượt; cầ`n trung thực điề`u tự làm được.

P09 · Brief trước khi hỏi — bổ sung biên tập

Prompt / Ghi chú

```
Mục tiêu: [một việc cụ thể trong ViSinhVien]. Tôi học [C++/Web].
Môi trường và vị trí: [tên shell, thư mục hiện tại, file liên quan].
Dữ liệu GIẢ chuẩn 35000, 7000, 18000 VND; tổng 60000.
Hợp đồng: tiền nguyên 1..1e9, tối đa 500, rỗng tổng 0.
Hiện tượng quan sát: [đầu ra đã nhìn thấy, hoặc ghi chưa chạy].
Kỳ vọng tôi tự tính: [giá trị và cách tính].
Điều đã thử: [một thao tác và kết quả, không suy đoán].
Đoạn code liên quan: [chỉ phần cần thiết, không bí mật/dữ liệu thật].
Chỉ phân tích nguyên nhân và hỏi tối đa một câu còn thiếu trước;
chưa sửa file, chưa cài gì, chưa chạy lệnh. Nói rõ giả thuyết nào
cần kiểm chứng và đề xuất một phép thử nhỏ để phân biệt chúng.
```

Nê´u chưa biê´t môi trường, ghi “chưa xác định” và hỏi cách xác định; đừng điê`n một phiên bản cho đẹp biểu mẫu.

P10 · Hợp đô`ng giao việc cho Agent — bổ sung biên tập

Prompt / Ghi chú

```
Bài tập ViSinhVien với dữ liệu giả 35000,7000,18000 VND.
Tiền nguyên 1..1e9, tối đa 500 khoản, rỗng tổng 0.
Đích: [một thay đổi đã thống nhất]. Chỉ làm trong bản sao
bai-lam/[nhánh], chỉ được sửa [danh sách file cụ thể].
Bước đầu chỉ đọc, nêu kế hoạch và test dự kiến; chờ tôi đồng ý
trước khi sửa. Không thay config/quyền, không cài gói, không gọi API,
không đăng nhập, không commit/push/publish, không xóa file.
Nếu cần vượt phạm vi, dừng và nêu việc cần quyết định, không tự làm.
Sau sửa cho diff, lệnh thực sự chạy, exit code và output; phân biệt
trace với test đã chạy. Giữ cả lần thất bại, không dùng executable cũ
sau lỗi build. Việc thiếu công cụ phải ghi chưa kiểm chứng.
Tôi sẽ đọc diff, kiểm test và tự giải thích trước khi nhận kết quả.
```

Trước khi dùng thẻ, kiểm tra quyê`n thật trong công cụ. Chỉ câu “không xóa file” trong prompt không thay thê´ cơ chê´ giới hạn filesystem hoặc phê duyệt hành động.

P11 · Kiểm chứng nguô `n — bổ sung biên tập

Prompt / Ghi chú

```
Tôi cần kiểm chứng claim sau cho cẩm nang sinh viên CNTT:
[claim cụ thể; ghi rõ sản phẩm, phiên bản, quốc gia nếu liên quan].
Nguồn đang có: [URL và đoạn đã đọc được, hoặc chưa có].
Ưu tiên tài liệu chính thức. Phân biệt nội dung quan sát trực tiếp,
suy luận và phần chưa truy cập được. Ghi URL, ngày đọc, phạm vi áp dụng.
Không dùng một trang giá để suy ra quyền lợi hiện tại của tài khoản tôi.
Không suy quy định môn học TLU từ lời khuyên chung trên mạng.
Không đăng nhập, không đăng ký/mua, không gọi API trả phí hay gửi dữ liệu.
Nếu không đọc được nguồn, nêu rõ hạn chế và câu hỏi cần kiểm tiếp;
không bịa trích dẫn, giá, điều khoản hoặc kết quả kiểm chứng.
```

Trước khi rời thư viện

Ghi một câu: “Tôi dùng P___ để tháo gỡ ___; bă `ng chứng sau lượt hỏi là ___.” Nê´u ô cuô´i chỉ có “AI đã trả lời”, quay lại và chọn một phép thử hoặc lời giải thích do chính bạn thực hiện.

BỘ CÔNG CỤ TỰ HỌC

Từ điển tra nhanh

Tìm từ đang vướng, đọc nghĩa rồ`i trở lại nhiệm vụ; không cần học thuộc bảng.

Kế´ thừa: đủ 16 mục từ mở đầ`u nguồ`n. **Bổ sung biên tập:** thuật ngữ xuyên sách, cặp dễ nhầ`m và nhãn hoạt động. Giải thích này không thay giáo trình.

Tra theo nhóm: AI và quyề`n truy cập · Cách AI xử lý · Môi trường · Mã và dữ liệu · Web · Kiểm chứng, Git, nguồ`n · Nhãn học tập.

AI: thành phầ`n nào đang làm gì? — Chương 1–2

Thuật ngữ	Nghĩa và điề`u cầ`n phân biệt
Model — mô hình	Thành phầ`n đã được huấ´n luyện để tạo đầ`u ra từ đầ`u vào. Tên model không cho biế´t toàn bộ công cụ hay ngữ cảnh đang dùng.
Provider — nhà cung cấ´p	Bên cung cấ´p quyề`n truy cập model, tính phí và xử lý dữ liệu theo dịch vụ; có thể khác bên phát triển model.
Application — ứng dụng	Phầ`n mề`m bạn tương tác, như giao diện chat. Có thể gọi nhiề`u model, bổ sung tệp và công cụ khi trả lời.
Producer/developer — bên phát triển model	Bên xây dựng model. Vai trò có thể trùng provider nhưng không tự đồ`ng nhấ´t hai bên.
API — giao diện lập trình ứng dụng	Quy ước để phầ`n mề`m gọi chức năng của phầ`n mề`m/dịch vụ khác. Quyề`n dùng API không mặc nhiên nằ`m trong thuê bao chat.
API key — khóa API	Bí mật cấ´p quyề`n gọi dịch vụ, có thể phát sinh chi phí. Không dán vào prompt, Git hoặc ảnh; ViSinhVien không cầ`n key.
Ask	Chế´ độ hỏi đáp chỉ đọc trong Qoder IDE được sách hướng dẫn. Vẫn kiểm dữ liệu đưa vào ngữ cảnh; không suy quyề`n sang sản phẩm khác.
Agent	AI phố´i hợp đọc tệp, đề` xuấ´t/sửa mã hoặc chạy công cụ theo quyề`n. Prompt “đừng sửa” không thay cơ chế´ cấ´p quyề`n và duyệt hành động.
Qoder IDE / Desktop / CLI	Các dòng sản phẩm/giao diện khác nhau; đố´i chiế´u tài liệu đúng sản phẩm, không áp tên nút hay chế´ độ của IDE sang Desktop/CLI.

Thuật ngữ	Nghĩa và điề̀u cầ̀n phân biệt
Free / subscription / quota / hard cap	Dùng miễn phí, thuê bao, hạn mức và trầ̀n chặn sử dụng/chi tiêu. Cảnh báo ngân sách chưa chắ́c chặn chi tiêu; phải kiểm điề̀u kiện.

AI: đầ̀u vào, xử lý và độ tin cậy — Chương 1, 3

Thuật ngữ	Nghĩa và ví dụ
Prompt	Yêu cầ̀u gửi AI; nên nêu mục tiêu, dữ liệu, giới hạn và dạng đầ̀u ra, không chỉ "sửa giúp em".
Context — ngữ cảnh	Thông tin model nhận khi trả lời: yêu cầ̀u, lịch sử, mã, tài liệu, kế́t quả công cụ. Tệp trên máy chưa chắ́c nằ̀m trong context.
Context window — cửa sổ ngữ cảnh	Giới hạn thông tin xử lý một lượt, thường đo bằ̀ng token. Hội thoại dài có thể bị rút gọn hoặc bỏ phầ̀n, không được nhớ nguyên vẹn.
Token	Đơn vị dữ liệu do bộ tách của model tạo để xử lý, thường dùng tính phí vào/ra. Không bằ̀ng một từ hay ký tự.
Inference — suy luận khi sử dụng	Model tạo đầ̀u ra với tham số́ hiện có. Thuật ngữ "suy luận" không bảo đảm lập luận đúng.
Training — huấ́n luyện	Điề̀u chỉnh tham số́ model từ dữ liệu. Gửi prompt không đồ̀ng nghĩa huấ́n luyện lại ngay; chính sách lưu/dùng dữ liệu là câu hỏi riêng.
Search / retrieval — tìm kiế́m, truy xuấ́t	Lấ́y tài liệu từ web hoặc kho dữ liệu để hỗ trợ trả lời, khác huấ́n luyện. Kế́t quả vẫn cầ̀n đố́i chiế́u.
Grounding — neo vào bằ̀ng chứng	Ràng câu trả lời với mã, dữ liệu, nguồ̀n được cung cấ́p/truy xuấ́t. Không loại hế́t lỗi; nguồ̀n phải thực sự hỗ trợ nhận định.
Hallucination — đầ̀u ra bịa hoặc không có căn cứ	Hàm, nguồ̀n hay kế́t quả không tồ̀n tại vẫn có thể được nêu tự tin. Kiểm API/tệp/nguồ̀n thật thay vì tin giọng điệu.
Benchmark	Bộ bài và cách chấ́m đo khả năng trong điề̀u kiện xác định. So sánh cầ̀n cùng nhiệm vụ/tiêu chí; điểm cao không bảo đảm hợp bài của bạn.
Brief / Socratic / vibe coding	Brief: giao việc ngắ́n, đủ ràng buộc. Socratic: hỏi dẫn dắ́t người học tự nghĩ. Vibe coding: triển khai qua mô tả và phản hồ̀i, vẫn phải hiểu/kiểm mã.

Môi trường làm việc — Chương 2

Thuật ngữ	Nghĩa và ví dụ
Editor — trình soạn thảo	Công cụ đọc/sửa mã nguồn; không nhất thiết tự biên dịch, chạy hay kiểm thử.
IDE — môi trường phát triển tích hợp	Kết hợp editor với công cụ chạy, gỡ lỗi, terminal. Tích hợp AI không bảo đảm thao tác đúng.
Terminal	Cửa sổ tương tác bằng chữ để nhập lệnh, xem đầu ra; không phải chương trình diễn giải lệnh.
Shell	Chương trình đọc/thực hiện lệnh, như Bash hoặc PowerShell; phải chọn cú pháp đúng shell.
CLI — giao diện dòng lệnh	Điều khiển công cụ bằng lệnh và đối số thay vì nút bấm, qua shell trong terminal. Ba khái niệm không đồng nghĩa.
Thư mục làm việc — working directory, cwd	Vị trí dùng giải đường dẫn tương đối. Xem bằng `pwd` trong Bash, `Get-Location` trong PowerShell.
Đường dẫn tương đối / tuyệt đối	Tương đối phụ thuộc vị trí đang xét; tuyệt đối tính từ gốc hệ thống. Link Markdown và lệnh terminal có thể có gốc khác nhau.
Biên dịch — compile; build	Compile dịch mã C++ sang dạng đích; build gồm cả các bước như liên kết. Build lỗi thì dừng, không lấy tệp cũ chứng minh mã mới đúng.
Executable / runtime	Tệp chạy được / môi trường thực thi. C++ tạo executable; Node.js chạy JavaScript nhưng không có sẵn DOM trình duyệt.
stdout / stderr / exit code	Luồng đầu ra thường / luồng lỗi / mã kết thúc. Exit code `0` thường báo thành công, khác `0` báo lỗi; không tự chứng minh thuật toán đúng.

Kiểu dữ liệu và tư duy lập trình — Chương 2–3

Thuật ngữ	Nghĩa trong sách và ví dụ
Expense	Mô hình khoản chi Web với `id`, `title`, `amount`, `category`. Đối tượng có trường dữ liệu không tự động nghĩa đã thiết kế một hệ thống OOP đầy đủ.
`vector<long long>`	Danh sách động của C++ chứa số nguyên kiểu `long long`; bản năm nhất chỉ chứa tiền. Không gán thêm tên/nhóm vào mô hình này khi mô tả mã hiện có.
OOP — lập trình hướng đối tượng	Tổ chức chương trình quanh đối tượng, trạng thái và hành vi với trách nhiệm rõ. Có một object JavaScript chưa chứng minh đã áp dụng mọi nguyên lý OOP.

Thuật ngữ	Nghĩa trong sách và ví dụ
DSA — cấu trúc dữ liệu và giải thuật	Chọn cách tổ chức dữ liệu và các bước xử lý. Trong bài tổng, danh sách cùng phép duyệt là điểm nối với kiến thức DSA.
Complexity — độ phức tạp	Mô tả tài nguyên tăng theo kích thước đầu vào. Duyệt `n` khoản có thời gian `O(n)`; một biến cộng dồn dùng bộ nhớ phụ `O(1)`, không tính danh sách đầu vào.
Safe integer — số nguyên an toàn	Với JavaScript `Number`, miền từ `-9007199254740991` đến `9007199254740991` bảo đảm phân biệt chính xác từng số nguyên. `Number.isSafeInteger` không thay điều kiện tiền phải dương và không quá `1e9`.
Unsigned — kiểu không dấu; tràn giá trị	Kiểu nguyên không âm trong C++ tính theo modulo: `0 - 1` trên kiểu unsigned vòng về giá trị lớn. Tràn số nguyên có dấu lại là hành vi không xác định.
Index / out-of-bounds / UB	Index là chỉ số; ngoài phạm vi hợp lệ là out-of-bounds. Truy cập ngoài biên bằng `vector::operator[]` gây undefined behavior (UB), có thể không crash; "chạy được" không chứng minh hợp lệ.
Reference / copy / `const`	Reference tham chiếu đối tượng sẵn có; copy tạo bản sao. `const std::vector<long long>&` cho hàm đọc qua tham chiếu không sửa vector qua tham chiếu ấy, tránh sao chép toàn danh sách.
String / number / conversion	Chuỗi và số là hai kiểu khác nhau. Ô nhập trả chuỗi; chuyển kiểu chưa chứng minh hợp đồng hợp lệ. `parseInt` có thể chỉ đọc đầu chuỗi, nên không dùng thay validation.
`undefined` / `NaN`	Đọc trường không có như `item.value` có thể nhận `undefined`; đem cộng có thể thành `NaN`, giá trị biểu thị kết quả số không hợp lệ. Đối chiếu tên `amount` trước.
Validation / schema / normalization	Validation kiểm điều kiện; schema mô tả cấu trúc; normalization đưa dữ liệu về dạng thống nhất. Web kiểm đủ bản ghi bằng `normalizeExpenses`; `totalExpenses` chỉ kiểm tiền và số lượng.

Web, lưu trữ và ranh giới dữ liệu — Chương 2–4

Thuật ngữ	Nghĩa trong sách và ví dụ
DOM — Document Object Model	Cây đối tượng biểu diễn tài liệu Web để JavaScript đọc/sửa. Test hàm tính tổng trong Node.js không xác nhận trang đã cập nhật DOM đúng.
Event handler — hàm xử lý sự kiện	Hàm chạy khi có sự kiện, như gửi biểu mẫu. Hàm vô danh vẫn có trách nhiệm và vị trí trong luồng; theo dữ liệu thay vì chỉ tìm tên hàm.
JSON	Định dạng văn bản trao đổi dữ liệu với cú pháp xác định. Phân tích JSON thành công không chứng minh `amount` hay các trường còn lại đúng hợp đồng.
Origin	Với địa chỉ Web thông thường, là bộ giao thức, tên máy chủ và cổng. Trình duyệt dùng origin để tách nhiều quyền/dữ liệu; không coi mọi tệp `file://` có hành vi giống nhau.

Thuật ngữ	Nghĩa trong sách và ví dụ
`localStorage`	Kho khóa–giá trị chuỗi của trình duyệt theo origin. Có thể bị chặn hoặc chứa dữ liệu cũ/sai; không là cơ sở dữ liệu máy chủ, đăng nhập hay kho tài chính bảo mật.
Fallback / RAM	Fallback là phương án thay thế khi cách chính thất bại. Demo dùng bộ nhớ tạm khi không lưu được; thay đổi không được bảo đảm còn sau khi kết thúc phiên.
Backend / SQLite / `:memory:`	Backend xử lý phía máy chủ; Web của sách không có phần này. SQLite là hệ quản trị cơ sở dữ liệu nhúng; `:memory:` tạo cơ sở dữ liệu trong bộ nhớ cho minh họa Python riêng.
SQL / truy vấn tham số hóa / tuple	SQL là ngôn ngữ làm việc với dữ liệu quan hệ. Truy vấn tham số hóa tách giá trị khỏi cấu trúc lệnh; `(title,)` là tuple một phần tử Python, dấu phẩy tạo tuple.
Injection / allowlist	Injection xảy ra khi dữ liệu không tin cậy làm đổi ý nghĩa lệnh. Allowlist là danh sách cho phép; dùng khi chọn cấu trúc như tên cột, không chèn chuỗi tùy ý.
XSS / `textContent` **/** `innerHTML`	XSS liên quan nội dung không tin cậy dẫn tới thực thi script trong trang. `textContent` đặt văn bản ở ô bảng; `innerHTML` phân tích HTML. Không suy biện pháp cho văn bản sang URL hoặc mã JavaScript.

Kiểm chứng, Git và quyền dùng nguồn — Chương 2–4

Thuật ngữ	Nghĩa trong sách và điều cần nhớ
Trace / mô phỏng	Trace lần theo giá trị, nhánh và vòng lặp từng bước. Mô phỏng là tình huống được dựng để giải thích. Cả hai không tự trở thành nhật ký chạy phần mềm thật.
Test case / fixture / regression test	Ca kiểm thử gồm đầu vào và điều cần đối chiếu. Fixture là dữ liệu giả chuẩn bị riêng. Regression test kiểm lỗi đã sửa không tái xuất hiện, như `category` là mảng.
Kỳ vọng / quan sát / bằng chứng	Kỳ vọng viết trước khi chạy; quan sát ghi điều thực sự xảy ra; bằng chứng gắn với bản mã, môi trường và cách kiểm. “Chưa chạy” khác “không đạt”.
Diff / review	Diff so trước–sau, chỉ dòng thêm, bớt, đổi. Review là xem xét thay đổi và lý do; đọc diff không tương đương chạy test, nhưng giúp phát hiện sửa vượt phạm vi.
Commit / checkpoint	Commit ghi một mốc trạng thái trong kho Git. Checkpoint là mốc quay lại đã chuẩn bị, có thể là commit hoặc bản sao; cả hai không chứng nhận mã đúng.
Clone	Tạo bản sao kho Git, thường có cả lịch sử liên quan. Xóa bí mật ở kho chính không khiến bí mật trong clone của người khác tự biến mất.
Force push — ép đẩy	Cập nhật nhánh từ xa cưỡng bức, có thể thay thế lịch sử người khác đang dựa vào. Không tự dùng để dọn key; thu hồi key và phối hợp người quản lý kho trước.
Dependency / registry / lockfile	Dependency là thư viện/gói phụ thuộc; registry là kho phân phối gói, công khai hoặc riêng. Lockfile ghi lựa chọn phiên bản cụ thể để tái lập cài đặt, không bảo đảm gói đáng tin.

Thuật ngữ	Nghĩa trong sách và điều cần nhớ
Attribution / ownership / trách nhiệm	Lần lượt là ghi nhận nguồn/hỗ trợ, quyền đối với sản phẩm và nghĩa vụ kiểm tra/giải trình theo vai trò. Ghi nguồn hay khai "AI viết" không tự giải quyết hai phần còn lại.
License — giấy phép; MIT / GPL	Giấy phép nêu quyền và điều kiện sử dụng. MIT yêu cầu giữ thông báo theo điều kiện; GPL không cấm thương mại. Kho công khai không giấy phép không đồng nghĩa tự do sao chép.
Dữ liệu giả / khử nhận dạng	Dữ liệu giả được tạo mới cho bài; khử nhận dạng xử lý dữ liệu có thật. Chỉ bỏ tên chưa chắc ngăn tái nhận diện qua giờ, địa điểm hoặc mã khác.
Revoke / rotate / log	Revoke thu hồi khóa; rotate thay khóa theo quy trình, cần xác nhận khóa cũ vô hiệu. Log là nhật ký hoạt động; chỉ xem trong quyền được phép, không đưa bí mật vào bằng chứng.

Nhãn học tập: nhìn nhãn để biết mình cần làm gì

Mười nhãn editorial dùng xuyên sách là chỉ dẫn đọc, không phải tên chế độ AI:

Nhãn	Cách dùng
AI CHALLENGE	Bài thực hành: tạo sản phẩm nhỏ và bằng chứng, không chỉ đọc đáp án.
AI TRAP	Lời đáp sai hoặc thiếu điều kiện được dựng để bạn phản biện; không quy kết một thương hiệu.
THINK FIRST	Dừng và viết dự đoán của mình trước khi hỏi hoặc xem lời giải.
REAL-WORLD CASE	Tình huống gần thực tế nhưng vẫn giả lập nếu không ghi nguồn sự kiện thật.
BEHIND THE SCENE	Giải thích cơ chế phía sau biểu hiện, giúp hiểu vì sao thao tác có tác dụng.
AI VS HUMAN	Phân biệt phần AI hỗ trợ và phần người học tự chứng minh; không xếp hạng người với sản phẩm.
QUICK CHECK	Tự kiểm nhanh một điểm hiểu; trả lời đúng chưa chứng minh thành thạo toàn chương.
PRO TIP	Mẹo thao tác có điều kiện áp dụng, không phải cách bỏ qua kiểm chứng.
SECURITY ALERT	Dừng để kiểm dữ liệu, quyền hoặc hành động có thể gây hại trước khi tiếp tục.
SKILL UNLOCKED	Ghi năng lực vừa có bằng chứng; không là chứng chỉ hoặc kết luận năng lực lâu dài.

Mission / MINI CHALLENGE là nhiệm vụ có đầu ra giới hạn. **Final Boss** là bài tổng hợp; Bronze, Silver, Gold chỉ mức nhiệm vụ, không phải gói trả tiền. **Memory Experiment** là tự quan sát việc nhớ lại, không nghiên cứu hay chẩn đoán. **25-MINUTE NO-AI** là phiên không AI, khác phiên học có AI cùng thời lượng.

CLO — Course Learning Outcome: chuẩn đầ̀u ra do đề̀ cương học phầ̀n công bố́. Mục tiêu sách, điểm tự đánh giá và dấ́u "đã tự kiểm" không thay CLO hay cách chấ́m của giảng viên.

Quay lại mở đầ̀u hoặc dùng thỏa thuận học tập để chuyển các từ "phạm vi", "đóng góp" và "bằ̀ng chứng" thành việc cụ thể.

BỘ CÔNG CỤ TỰ HỌC

Thỏa thuận học tập

AI có thể giải thích, gợi ý và sinh mã. Bạn quyết định mục tiêu, phạm vi chia sẻ, thay đổi được nhận và bằng chứng cần có.

Mẫu tự học, không phải quy định hay biểu mẫu chính thức của TLU. Yêu cầu bài/học phần và giảng viên quyết định việc dùng AI. Khai báo không thay sự cho phép, giấy phép hoặc kiểm chứng.

Kế thừa thỏa thuận mở đầu và mục 4.1, 4.7 nguồn. Bổ sung biên tập: phiếu trước buổi, sổ đóng góp, bảng kỳ vọng–thực tế. Ô vuông để trống, không xác nhận việc đã làm.

1. Làm rõ quyền trước khi giao việc

Tình trạng theo yêu cầu bài	Hành động của người học
Được phép	Dùng đúng phần cho phép, đúng điều kiện khai báo và lưu bằng chứng.
Bị cấm	Không dùng AI tạo/sửa lời giải trong phạm vi bị cấm; chọn tài liệu và hỗ trợ được phép.
Có điều kiện	Ghi rõ điều kiện: được giải thích chưa chắc được sinh cả hàm; hỏi khi hành động sắp vượt phạm vi.
Chưa rõ	Hỏi giảng viên; trong lúc chờ chỉ tiếp tục phần chắc chắn được phép. Im lặng không là đồng ý.

Không lấy việc bạn cùng lớp dùng, gói miễn phí hay bài làm chạy được làm căn cứ. Với bài nhóm, thống nhất phạm vi trước khi nhận mã vào bản chung.

Tin nhắn có thể điều chỉnh rồi gửi:

> Thưa thầy/cô, với bài [tên bài], em dự định tự viết [phần cụ thể]. Em muốn dùng [công cụ] để [giải thích lỗi/gợi ý ca kiểm thử/sinh nháp phần nào], rồi tự [sửa, chạy và giải thích]. Em chỉ cung cấp dữ liệu giả và đoạn mã cần thiết. Phạm vi này có được phép không ạ? Nếu được, em cần khai báo ở đâu và giữ bằng chứng nào? Trong lúc chờ, em sẽ làm phần [chắc chắn được phép].

Giữ căn cứ và thời điểm trả lời theo yêu cầu môn; không đăng hội thoại riêng của giảng viên lên kho công khai.

2. Phiếu trước buổi làm: điền đủ để bắt đầu

Thay nội dung trong ngoặc bằng tình trạng thật. Không biết thì ghi **chưa rõ**, kèm bước hỏi; đừng để ô trống mang nghĩa "đã được duyệt".

Mục	Bạn điền
Bài, thành viên, vai trò	[Tên bài; ai viết, ai rà soát; cá nhân thì ghi tự rà.]
Căn cứ cho phép	[Đề/mục hướng dẫn/phản hồi, ngày; điều được và không được làm.]
Mục tiêu buổi này	[Một hành vi quan sát được, ví dụ giải thích vì sao vòng lặp bỏ khoản cuối.]
Phần bắt buộc tự làm	[Tự dự đoán, viết/sửa phần nào, chọn ca thử, giải thích và làm biến thể nào.]
AI được hỗ trợ	[Giải thích/gợi ý/sinh nháp; tệp, hàm, câu hỏi giới hạn.]
Quyền công cụ	[Được đọc/sửa tệp nào; lệnh nào cần duyệt. Không mặc định cho cài gói, commit hay đưa mã lên mạng.]
Dữ liệu được đưa vào	[Dữ liệu giả tự tạo; đoạn mã tối thiểu; ảnh đã rà. Không gửi key, mật khẩu, hồ sơ hay chi tiêu thật.]
Điểm dừng	[Thiếu quyền/chưa hiểu/lỗi chưa tái hiện: lưu trạng thái, hỏi ai, tiếp tục phần nào.]

Bỏ tên khỏi dữ liệu thật chưa chắc đã khử nhận dạng. Rà cả ảnh, đường dẫn, chat và lịch sử tệp sẽ chia sẻ. Nếu lộ khóa thật, dừng chia sẻ, thu hồi hoặc nhờ người có quyền vô hiệu khóa; báo đầu mối phù hợp. Xóa dòng hay tự ép đẩy Git không giải quyết mọi bản sao.

3. Ghi đóng góp và nguồn ngay khi dùng

Mỗi hỗ trợ đáng kể ghi một dòng. Nếu AI sinh cả hàm, ghi **sinh bản nháp hàm**, không đổi thành "tham khảo ý tưởng". Người rà soát phải nói đã đọc diff hay chạy thử, không ghi chung "đã xem".

Công cụ, ngày, phiên bản nê´u biê´t	Hỗ trợ thực tê´, vị trí	Phâ`n người học làm	Quyê´t định và căn cứ
[Tên; chưa biê´t phiên bản thì ghi vậy]	[Giải thích/gợi ý/sinh nháp tệp hoặc hàm nào]	[Ai sửa, viê´t lại, kiểm và giải thích]	[Giữ/bỏ; vì sao; bă`ng chứng hoặc chưa kiểm]

Ghi nguô`n là **attribution**; quyê`n sử dụng là câu hỏi vê` giâ´y phép/quyê`n đô´i với sản phẩm; trách nhiệm là kiểm tra và giải trình theo vai trò. Ba việc không thay nhau.

Thành phâ`n sử dụng	Nguô`n và phiên bản thật	Quyê`n, điê`u kiện câ`n giữ	Quyê´t định
[Mã/ảnh/tài liệu cụ thể]	[URL hoặc tài liệu, bản đã dùng]	[Giâ´y phép đã đọc; thông báo câ`n giữ; chưa rõ thì ghi rõ]	[Dùng có điê`u kiện/hỏi/loại hoặc tự viê´t lại]

Kho công khai không giâ´y phép không tự câ´p quyê`n sao chép. MIT có điê`u kiện giữ thông báo; GPL không câ´m thương mại nhưng câ`n đọc nghĩa vụ theo cách dùng và phân phô´i. “AI tạo” không chứng minh nguô`n gô´c hay quyê`n độc quyê`n. Không rõ quyê`n thì loại/thay phâ`n đó hoặc hỏi, không chỉ thêm một đường dẫn.

4. Bă`ng chứng: tách kỳ vọng khỏi điê`u đã thâ´y

Mẫu ghi kiểm tra, chưa có kê´t quả thực thi. Đâ`u bảng ghi: [ngày, người kiểm, hệ điê`u hành, công cụ/phiên bản, bản mã hoặc mô´c lưu, lệnh/thao tác]. Với mỗi hàng, điê`n quan sát thật hoặc **chưa chạy**. Nê´u chỉ đọc mã, ghi **rà soát tĩnh**, không ghi đạt kiểm thử.

Mục kiểm	Đâ`u vào/thao tác	Kỳ vọng theo hợp đô`ng	Thực tê´ và trạng thái bạn điê`n
Tổng chuẩn	Ba khoản giả `35000`, `7000`, `18000`	`60000`	[Quan sát; đạt/không đạt/chưa chạy; chỗ lưu bă`ng chứng]
Rỗng; một khoản	Danh sách rỗng; riêng khoản `7000`	`0`; `7000`	[Ghi riêng từng lâ`n]
Biên tiê`n	`1`, `1e9`; thử riêng 0, âm, vượt trâ`n	Nhận hai biên; từ chô´i giá trị sai	[Kê´t quả từng ca]
Sô´ thập phân/chuỗi Web	Nhập sô´ thập phân, chuỗi không hợp lệ; đô´i chiê´u kiểu `amount`	Từ chô´i; không lén lâ´y một phâ`n chuỗi	[Quan sát/chưa chạy]
Sô´ lượng	500 khoản hợp lệ; thử khoản thứ 501	Nhận 500; từ chô´i 501	[Quan sát/chưa chạy]

Mục kiểm	Đầu vào/thao tác	Kỳ vọng theo hợp đồng	Thực tế và trạng thái bạn điền
Bản ghi Web	Sai `id`, `title`, `category`; thử `category: ['khac']` riêng	`normalizeExpenses` từ chối bản ghi sai	[Không thay bằng test tổng tiền]
Hiển thị Web	Fixture tên `<b>Vở</b>`	Hiện nguyên văn trong ô bảng, không thành chữ in đậm	[Quan sát trình duyệt/chưa chạy]
Lưu trữ Web	Nạp lại trang; lưu bị chặn hoặc dữ liệu hỏng	Lưu được: giữ dữ liệu; bị chặn/hỏng: thông báo, dùng RAM	[Nêu đúng điều kiện đã thử]

Giữ fixture riêng, không trộn vào ba khoản chuẩn rồi vẫn gọi tổng là `60000`. C++ dùng `vector<long long>`, không có parser số thập phân: không nhận đã kiểm chức năng không có. Build lỗi thì dừng; tệp chạy cũ không chứng minh mã mới đúng.

Đối chiếu phiên bản: kiểm kê nguồn ghi bản Web mới có **34 ca JavaScript** sau vá `category`, nguồn cũ có **33 ca**; bản C++ có **9 ca**. Đây không phải kết quả chạy của phiếu này. Khi chạy, ghi số ca thực thi, đầu ra và exit code thật. Test Node.js không chứng minh DOM/lưu trữ đúng; số ca không cộng thành điểm chất lượng.

5. Mẫu khai báo dùng cho bản nộp

Chép mẫu rồi điền sự thật, theo hình thức giảng viên yêu cầu. Không cần nộp toàn bộ chat chứa dữ liệu riêng tư.

> **Bài/nhóm, vai trò:** [điền].
>
> **Phạm vi được phép và căn cứ:** [điền; phần đang chờ phải ghi đang chờ].
>
> **Công cụ, thời điểm, phiên bản nếu biết:** [điền; nếu không dùng AI, ghi không dùng].
>
> **Hỗ trợ thực tế:** [giải thích/gợi ý/sinh nháp phần nào, vị trí nào].
>
> **Phần tự làm:** [ai viết, sửa, chọn/bỏ đề xuất và giải thích quyết định].
>
> **Dữ liệu cung cấp:** [dữ liệu giả, mã tối thiểu, quyền đã cấp; sự cố báo riêng, không dán bí mật].
>
> **Nguồn và giấy phép:** [nguồn thật, điều kiện đã thực hiện, phần loại vì chưa rõ quyền].
>
> **Kiểm chứng:** [đã chạy gì, bản nào, quan sát; phần chỉ đọc mã; phần chưa chạy].

> **Chưa hiểu/chưa kiểm và bước tiếp:** [vấn đề, ảnh hưởng, người cần hỏi, việc sẽ làm].
>
> **Giới hạn sản phẩm:** [nhánh thực tế]. Web dùng HTML/JavaScript/localStorage, không backend/đăng nhập; SQL là minh họa Python `sqlite3` riêng trong `:memory:`. Chỉ dùng dữ liệu giả.

Ví dụ cách ghi mức chắc chắn — tình huống biên tập, không phải nhật ký đã chạy: "Em đã đọc điều kiện kiểm tra `amount`, nhưng chưa chạy vì máy chưa có môi trường cần thiết. Em chưa xác nhận hành vi trình duyệt. Em sẽ xin hướng dẫn môi trường và bổ sung quan sát; không đánh dấu đạt." Chỉ dùng lời này nếu đúng quá trình của bạn; đừng biến ví dụ thành lời khai.

6. Trước khi gửi: ưu tiên và tự đánh giá

Chặn nộp phần liên quan theo checklist luyện tập:

- [ ] Đã xử lý dữ liệu/khóa không nên chia sẻ; không đính kèm bí mật làm bằng chứng.
- [ ] Đúng hợp đồng tiền/số lượng; kiểm chứng bắt buộc có quan sát thật; lỗi trọng yếu không bị che bằng giao diện đẹp.
- [ ] Không còn phần chưa rõ quyền hoặc phụ thuộc chưa kiểm chứng trong bản nộp.
- [ ] Khai báo đúng mức hỗ trợ; giải thích được phần trọng yếu mình nhận trách nhiệm.

Cần hỏi: phạm vi AI, điều kiện nguồn/dữ liệu, hình thức khai báo hoặc việc ngoài quyền của nhóm. Ghi trạng thái chờ, không xem câu hỏi đã gửi là vấn đề đã giải quyết. Có thể thu hẹp phần chưa rõ nếu vẫn đáp ứng yêu cầu bài và được giảng viên chấp nhận.

Cải thiện sau: màu sắc, khoảng cách, tính năng mở rộng; không dùng chúng để trì hoãn xử lý dữ liệu, nguồn và lỗi cốt lõi.

Rubric kế thừa để **tự luyện**, không là thang điểm TLU: mỗi mục 0–2, tối đa 8; điểm tổng không bù lỗi chặn nộp.

Mục	0	1	2
Học thuật và nguồn	Không rõ	Có ghi nhưng mơ hồ	Phạm vi, đóng góp, nguồn/quyền rõ
Dữ liệu và quyền	Dữ liệu/quyền không phù hợp	Đã giảm, chưa rà đủ	Rà đúng bản chia sẻ, quyền vừa đủ
Tính đúng và đọc mã	Chưa đối chiếu	Hiểu một phần, thiếu bằng chứng	Đối chiếu hợp đồng và ranh giới ví dụ

Mục	0	1	2
Kiểm chứng, xử lý	Khẳng định vô căn cứ	Ghi được một phần	Truy lại được bằng chứng và việc còn thiếu

Trước khi đánh dấu, tự giải thích một luồng không mở AI, nêu một đầu vào bị từ chối và lý do. Vướng đâu ghi đó rồi học lại hoặc hỏi; không cần học thuộc từng dấu câu. Đóng buổi bằng ba dòng: **đã hiểu / chưa chắc / bước tiếp theo**.

Hiểu công cụ. Làm chủ kiến thức.

Sử dụng có trách nhiệm. Dừng lại ở điều chưa chứng minh được, rồi chọn một phép thử nhỏ để đi tiếp.

Nội dung, mô phỏng và tự ghi tiến độ hoạt động cục bộ. Chỉ các liên kết tham khảo cần Internet khi bạn chủ động mở. Tiến độ tự học do bạn đánh dấu, không phải đánh giá tự động.